

StoreHub — Complete Codebase Documentation

Combined single-file version of all documentation in this folder. See individual files (01-13 + README) for the same content split by module.

StoreHub — Codebase Documentation

This is module-by-module developer documentation for the StoreHub ERP codebase, generated **directly from the current source code** at `/home/user/Myproject` (backend: Spring Boot / Java under `backend/src/main/java/com/storehub`; frontend: React + TypeScript + Vite under `frontend/src`). Nothing here was invented — every class name, method name, URL, table name, and validation rule quoted in these documents was read directly from the actual files. Where something was asked for but not found in the code, the module docs say so explicitly: `NOT FOUND IN CURRENT CODEBASE`.

This documentation is read-only. No application source file was modified, refactored, or renamed while producing it.

How this was produced

The codebase was scanned to identify every controller (46), service (62), entity (~120 incl. enums), repository (58), and DTO (180) on the backend, and every page/component/API-client/type file on the frontend. These were grouped into 13 functional modules based on how the actual code is organized (not a guessed taxonomy), and each module was documented independently and in depth, including full UI → Function → API → Controller → DTO → Service → Repository → Database → Response → UI traces for every important operation, plus a "Manual Changes" section explaining exactly how to extend or modify that module yourself.

Module Index

#	Module	File
1	Authentication & Authorization	01-authentication-authorization.md
2	User & Employee Management	02-user-employee-management.md
3	Store / Multi-Branch (Multi-Store)	03-store-multi-branch.md
4	Lookup / Simple Masters (Country, State, City, Zone, Currency, Nationality, Unit, Item Group, HSN)	04-lookup-masters.md
5	Party / Customer / Supplier	05-party-customer-supplier.md
6	Product / Item Master & Inventory	06-product-inventory.md
7	Purchase (Order, Bill, Kacchi Purchase, Payment, Debit Note)	07-purchase.md
8	Sales (Order, Bill, Kacchi Sale, POS, Receipt, Credit Note)	08-sales.md
9	Accounting Core Engine (Chart of Accounts, Journal, Voucher Numbering, Ledger)	09-accounting-core.md
10	Accounting Reports	10-accounting-reports.md
11	GST	11-gst.md
12	Expense, Cash Management, Day Closing, Payment Method, Financial Year, Audit Trail	12-expense-cash-fy-audit.md
13	Alerts, Global Search & Dashboard	13-alerts-search-dashboard.md

Each module file follows the same 15-section structure: Overview, Frontend, API Calls, Backend (Controllers/DTOs/Services/Repository), Database, Validation, Authentication/Authorization, Permissions,

Transaction Handling, Error Handling, Audit Flow, Important Side Effects, Dependencies on Other Modules, Key Operation Flows, and Manual Changes.

Complete Project Structure

```
Myproject/
├── backend/                                     Spring Boot 3 / Java (Maven)
│   └── src/main/java/com/storehub/
│       ├── controller/    46 REST controllers – one per module/sub-area
│       ├── service/       62 services – business logic + 15 helper/enum-like classes
│       ├── entity/        ~120 JPA entities + enums
│       ├── repository/    58 Spring Data JPA repositories
│       ├── dto/           180 request/response DTOs
│       ├── exception/     24 custom exceptions + GlobalExceptionHandler
│       ├── security/      JWT auth (JwtUtil, JwtAuthenticationFilter, UserPrincipal,
│       CustomUserDetailsService)
│       ├── config/        SecurityConfig and other Spring config
│       └── util/          GstinValidator, BarcodeUtil, SecurityUtil, etc.
│   └── src/test/java/com/storehub/service/      backend automated test suites (per phase/step)
├── frontend/                                   React 18 + TypeScript + Vite
│   └── src/
│       ├── pages/         ~105 page components, organized by module folder
│       └── accounting/, accounting/reports/, admin/, gst/, inventory/, masters/, purchases/,
sales/
    ├── components/       shared UI building blocks + a few module-specific dialogs
    ├── layout/           MainLayout, Sidebar, Topbar, GlobalSearch, StoreSwitcher, nav-items.ts
    ├── masters/          MasterCrudPage.tsx (shared generic master-page component)
    └── ui/               shadcn/ui-style primitives (Button, Table, Dialog, Select, ...)
    ├── api/              31 API client files (one per module/entity), all built on axios.ts
    ├── types/            26 TypeScript type files matching backend DTOs
    ├── context/          AuthContext.tsx (auth/session/permissions/store-selection state)
    ├── hooks/            useUnsavedChangesGuard.ts
    └── App.tsx           all route definitions + route guards
└── docs/codebase/       ← this documentation set
```

Complete API List

Grouped by controller, with each controller's base `@RequestMapping` path. Method + sub-path combine with the base to form the full URL (e.g. `AccountController` base `/api/accounts` + `GET /{id}` = `GET /api/accounts/{id}`). Full request/response shapes and `@PreAuthorize` values are in each module's own doc — this is the master index.

Controller	Base Path	Endpoints (Method + sub-path)	Module Doc
AuthController	/api/auth	POST /register · POST /login · POST /logout · GET /me · GET /my-stores · PUT /current-store	01
PermissionController	/api/permissions	GET (list all permissions)	01
RoleController	/api/roles	GET (list roles) · GET /{role}/permissions	01

Controller	Base Path	Endpoints (Method + sub-path)	Module Doc
UserController	/api/users	GET · GET /{id} · GET /{id}/effective-permissions · POST · PUT /{id} · PATCH /{id}/status · POST /{id}/reset-password · GET /{id}/stores · PUT /{id}/stores · PUT /me/password	02
EmployeeController	/api/masters/employees	GET · GET /{id} · POST /generate-code · POST · PUT /{id} · PATCH /{id}/activate · PATCH /{id}/deactivate · GET /{id}/stores · PUT /{id}/stores	02
StoreController	/api/masters/stores	GET · GET /{id} · GET /{id}/gst-context · POST /generate-code · POST · PUT /{id} · PATCH /{id}/activate · PATCH /{id}/deactivate	03
StockTransferController	/api/stock-transfers	GET · GET /{id} · POST · PATCH /{id}/approve · PATCH /{id}/dispatch · PATCH /{id}/receive · PATCH /{id}/cancel	03
StoreComparisonController	/api/reports/store-comparison	GET	03
CountryController	/api/masters/countries	GET · GET /{id} · POST · PUT /{id} · PATCH /{id}/activate · PATCH /{id}/deactivate	04
StateController	/api/masters/states	(same CRUD shape as Country)	04
CityController	/api/masters/cities	(same CRUD shape)	04
ZoneController	/api/masters/zones	(same CRUD shape)	04
CurrencyController	/api/masters/currencies	(same CRUD shape)	04
NationalityController	/api/masters/nationalities	(same CRUD shape)	04
UnitController	/api/masters/units	(same CRUD shape)	04
ItemGroupController	/api/masters/item-groups	(same CRUD shape)	04
HsnController	/api/masters/hsn	(same CRUD shape, plus nested tax-rate rows in the request body)	04
PartyController	/api/masters/parties	GET · GET /{id} · POST · PUT /{id} · PATCH /{id}/activate · PATCH /{id}/deactivate	05
CustomerController	/api/customers	GET · POST (no update/deactivate endpoints — see module 05)	05
SupplierController	/api/suppliers	GET · GET /{id} · GET /{id}/purchases · POST · PUT /{id} · PATCH /{id}/activate · PATCH /{id}/deactivate	05
ProductController	/api/products	GET · GET /{id} · GET /barcode/{barcode} · POST /generate-sku · POST /generate-barcode · POST · PUT /{id} · PATCH /{id}/activate · PATCH /{id}/deactivate · GET /export · POST /import	06
CategoryController	/api/categories	GET · GET /{id} · POST · PUT /{id} · PATCH /{id}/activate · PATCH /{id}/deactivate	06
InventoryController	/api/inventory	GET · GET /summary · GET /history · GET /{id} · GET /{id}/history · POST /adjust · GET /export	06

Controller	Base Path	Endpoints (Method + sub-path)	Module Doc
PurchaseController	/api/purchases	GET · GET /{id} · POST · PUT /{id} · POST /{id}/post · PATCH /{id}/cancel · DELETE /{id}	07
PurchaseOrderController	/api/purchase-orders	GET · GET /{id} · POST · PUT /{id} · PATCH /{id}/status · PATCH /{id}/cancel	07
PaymentController	/api/payments	GET · GET /{id} · GET /outstanding/{supplierId} · POST · DELETE /{id}	07
PurchaseSummaryController	/api/purchase-summary	GET	07
DebitNoteController	/api/debit-notes	GET · GET /{id} · POST · POST /{id}/post · PATCH /{id}/cancel	07
SaleController	/api/sales	GET · GET /{id} · POST · PUT /{id} · POST /{id}/post · PATCH /{id}/cancel · DELETE /{id}	08
SalesOrderController	/api/sales-orders	GET · GET /{id} · POST · PUT /{id} · PATCH /{id}/status · PATCH /{id}/cancel	08
ReceiptController	/api/receipts	GET · GET /{id} · GET /outstanding/{customerId} · POST · DELETE /{id}	08
SalesSummaryController	/api/sales-summary	GET	08
CreditNoteController	/api/credit-notes	GET · GET /{id} · POST · POST /{id}/post · PATCH /{id}/cancel	08
AccountController	/api/accounts	GET · GET /groups · GET /{id} · POST · PUT /{id}	09
AccountingController	/api/accounting/journals	GET · GET /{id} · POST	09
AccountingReportController	/api/accounting/reports	GET /dashboard · /day-book · /account-ledger/{accountId} · /trial-balance · /cash-book · /bank-book · /party-ledger · /receivable · /payable · /outstanding/customers · /outstanding/suppliers · /profit-loss · /balance-sheet · /account-summary · /expense-summary · /income-summary · /health-check	10
GstReportController	/api/gst-reports	GET /gstr1 · /purchase · /gstr3b · /output · /input · /hsn · /tax-rate · /liability · /reconciliation	11
BusinessGstConfigController	/api/masters/business-gst-config	GET · PUT	11
ExpenseController	/api/expenses	GET · GET /reports/summary · GET /{id} · POST · PUT /{id} · POST /{id}/post · PATCH /{id}/cancel	12
ExpenseCategoryController	/api/masters/expense-categories	GET · GET /active · GET /{id} · POST · PUT /{id} · PATCH /{id}/activate · PATCH /{id}/deactivate	12
CashTransactionController	/api/cash-transactions	GET · GET /{id} · POST · POST /{id}/post · PATCH /{id}/cancel	12
DayClosingController	/api/day-closing	GET /summary · GET /history · POST /close	12
PaymentMethodController	/api/payment-methods	GET · POST · PUT /{id} · PATCH /{id}/activate · PATCH /{id}/deactivate	12
FinancialYearController	/api/financial-years	GET · GET /current · GET /{id} · GET /{id}/summary · POST · PATCH /{id}/open · PATCH /{id}/close · PATCH /{id}/mark-current	12

Controller	Base Path	Endpoints (Method + sub-path)	Module Doc
AuditLogController	/api/audit-logs	GET	12
AlertController	(explicit path on the method)	GET /api/alerts	13
GlobalSearchController	(explicit path on the method)	GET /api/search	13

Note: all 9 lookup-master controllers (Country/State/City/Zone/Currency/Nationality/Unit/ItemGroup/Hsn) share the exact same 6-endpoint CRUD shape: GET (list), GET /{id}, POST (create), PUT /{id} (update), PATCH /{id}/activate, PATCH /{id}/deactivate — this is the MasterCrudPage.tsx pattern, see module 04.

Complete Database / Table List

64 JPA entities map to 64 tables (from each entity's @Table(name = "...")). Grouped by owning module:

Authentication/User/Employee/Store — users, employees, employee_code_sequence, employee_stores, stores, store_code_sequence, user_stores

Lookup Masters — countries, states, cities, zones, currencies, nationalities, units, item_groups, hsn_codes, hsn_tax_rates

Party/Customer/Supplier — parties, party_addresses, customers, suppliers

Product/Inventory — products, categories, item_sku_sequence, item_barcode_sequence, inventory, stock_history

Purchase — purchases, purchase_items, purchase_orders, purchase_order_items, payments, payment_allocations, supplier_ledger_entries, purchase_gst_entries, debit_notes, debit_note_items

Sales — sales, sale_items, sales_orders, sales_order_items, receipts, receipt_allocations, customer_ledger_entries, gst_entries, credit_notes, credit_note_items

Accounting Core — accounts, account_groups, journal_headers, journal_details, voucher_sequences

GST — gst_transactions, business_gst_config

Expense/Cash/FY/Audit — expenses, expense_categories, cash_transactions, cash_ledger_entries, day_closings, payment_methods, financial_years, audit_logs

Stock Transfer (Multi-Store) — stock_transfers, stock_transfer_items

Accounting Reports module — owns **no tables**; it is pure read-aggregation over the tables above (see module 10).

Every _id/entity-name-typed column is a FK to the corresponding table above; store_id appears on 13+ transactional tables as part of the Multi-Store design (see module 03) — the module docs give the exact FK list per table.

Complete Frontend Route List

All routes are defined in `frontend/src/App.tsx`. Every route (except `/login`, `/register`) sits under `ProtectedRoute` (requires a valid session) and `MainLayout` (sidebar shell). Additional guard wrappers are noted.

Path	Page Component	Guard
<code>/login</code>	<code>Login.tsx</code>	none (public)
<code>/register</code>	<code>Register.tsx</code>	none (public)
<code>/dashboard</code>	<code>Dashboard.tsx</code>	<code>ProtectedRoute</code>
<code>/alerts</code>	<code>AlertsCenter.tsx</code>	<code>ProtectedRoute</code>
<code>/unauthorized</code>	<code>Unauthorized.tsx</code>	<code>ProtectedRoute</code>
<code>/users</code>	<code>Users.tsx</code>	<code>AdminRoute</code>
<code>/admin/financial-years</code>	<code>FinancialYears.tsx</code>	<code>AdminRoute</code>
<code>/admin/audit-trail</code>	<code>AuditTrail.tsx</code>	<code>AdminRoute</code>
<code>/admin/roles-permissions</code>	<code>RolesAndPermissions.tsx</code>	<code>AdminRoute</code>
<code>/purchases</code> + 15 sub-routes (orders, bills, payments, kacchi, debit-notes — list/new/:id/:id/edit)	<code>PurchaseHub.tsx</code> + <code>purchases/*</code>	<code>ManagerRoute(+PURCHASE_USER)</code>
<code>/sales</code> , <code>/sales/pos</code>	<code>SalesHub.tsx</code> , <code>Pos.tsx</code>	<code>ProtectedRoute</code> (no manager gate)
<code>/sales/orders</code> , <code>/sales/bills</code> , <code>/sales/kacchi</code> (list/new/:id)	<code>sales/*</code>	<code>ProtectedRoute</code>
<code>/sales/orders/:id/edit</code> , <code>/sales/bills/:id/edit</code> , <code>/sales/kacchi/:id/edit</code>	<code>sales/*Form.tsx</code>	<code>ManagerRoute(+SALES_USER)</code> — edit only
<code>/sales/receipts</code> (list/new/:id)	<code>Receipts.tsx</code> + <code>ReceiptForm/Detail</code>	<code>ProtectedRoute</code>
<code>/sales/credit-notes</code> (list/new/:id)	<code>CreditNotes.tsx</code> + <code>Form/Detail</code>	<code>ProtectedRoute</code>
<code>/accounting</code> + 15 sub-routes (accounts, journals, trial-balance, expenses, cash-transactions, payment-methods, day-closing, reports/*)	<code>accounting/*</code>	<code>ManagerRoute(+ACCOUNTANT)</code>
<code>/accounting/journals/new</code>	<code>JournalEntryForm.tsx</code>	<code>AdminRoute</code> (nested inside the <code>ACCOUNTANT</code> gate)
<code>/gst-reports</code> + 9 sub-routes	<code>gst/*</code>	<code>ManagerRoute(+ACCOUNTANT)</code>
<code>/products</code>	<code>Products.tsx</code>	<code>ProtectedRoute</code>
<code>/products/new</code> , <code>/products/:id/edit</code>	<code>ProductForm.tsx</code>	<code>ManagerRoute</code>
<code>/inventory</code>	<code>Inventory.tsx</code>	<code>ProtectedRoute</code>
<code>/inventory/stock-transfers</code> (list/new/:id)	<code>inventory/StockTransfer*.tsx</code>	<code>ProtectedRoute</code>
<code>/suppliers</code> (list/new/:id/edit)	<code>Suppliers.tsx</code> , <code>SupplierForm.tsx</code>	<code>ManagerRoute</code>

Path	Page Component	Guard
/masters + 14 sub-routes (stores, currencies, countries, states, cities, zones, nationalities, units, item-groups, hsn, employees, parties, business-gst-config, expense-categories)	MastersDashboard.tsx + masters/*	ManagerRoute
/customers , /payments , /reports	ComingSoon.tsx	ProtectedRoute (disabled placeholder modules — see DISABLED_MODULES in App.tsx)
* (catch-all)	redirects to /dashboard	—

ManagerRoute = ADMIN or STORE_MANAGER (plus any extraRoles listed). AdminRoute = ADMIN only. Full guard implementation is in module 01.

Important Classes & Services (Central / Common)

These are used by **multiple** modules and are the highest-leverage files in the codebase — changing them has wide blast radius.

Class	File	Role
AuditService	backend/.../service/AuditService.java	Central audit-log writer (log(...) , 8-arg and 9-arg overloads). Called from ~18+ services across nearly every module for create/update/status-change events. Writes to audit_logs .
AccountingService	backend/.../service/AccountingService.java	The double-entry posting engine. Every transactional module (Sale, Purchase, Receipt, Payment, Expense, CashTransaction, CreditNote, DebitNote) posts through this rather than writing journal rows directly. 8 distinct caller modules found.
VoucherNumberService	backend/.../service/VoucherNumberService.java	Centralized FY-aware document numbering for every voucher type. 11 distinct caller modules found.
LedgerService	backend/.../service/LedgerService.java	Records Customer/Supplier ledger entries and Expense credit/reversal entries. 9 distinct caller modules found.
StoreAccessService	backend/.../service/StoreAccessService.java	The single authorization choke point for the ALL_STORES vs ASSIGNED_STORES model. Every store-scoped read/write across every module resolves its storeId through here.
InventoryService.applyMovement	backend/.../service/InventoryService.java	The only path by which stock is ever mutated. Called by Purchase, Sale, Credit Note, Debit Note, Stock Transfer.
GstCalculationService	backend/.../service/GstCalculationService.java	Central tax-split (CGST/SGST/IGST) calculation logic, reused by Sale/Purchase/Notes/Expense.
GstTransactionSyncService	backend/.../service/GstTransactionSyncService.java	Syncs/reverses gst_transactions rows whenever Sale/Purchase/CreditNote/DebitNote/Expense post or cancel.

Class	File	Role
PermissionService / RolePermissions	backend/.../service/PermissionService.java , RolePermissions.java	The single source of truth for which Role has which Permission; backs every @PreAuthorize("hasAuthority('PERM_...')") and UserPrincipal.getAuthorities().
FinancialYearService	backend/.../service/FinancialYearService.java	resolveForDate / resolveOpenForPosting gate almost every posting flow; auto-seeds a FY covering "today" at startup.
GlobalExceptionHandler	backend/.../exception/GlobalExceptionHandler.java	Maps every custom + framework exception to the standard ApiError JSON shape and HTTP status across the whole API surface.
SecurityConfig + JwtAuthenticationFilter + JwtUtil + UserPrincipal	backend/.../config/ , backend/.../security/	The entire authentication/authorization plumbing every request passes through.
AuthContext.tsx	frontend/src/context/AuthContext.tsx	Frontend session state: current user, permissions set, myStores / switchStore , hasPermission() — read by nearly every page/guard.
axios.ts	frontend/src/api/axios.ts	The single configured axios instance every api/*.ts client is built on (base URL, auth header injection, interceptors).
MasterCrudPage.tsx	frontend/src/components/masters/MasterCrudPage.tsx	The shared generic component every simple master page (9 of them) is built on top of.
ProtectedRoute / AdminRoute / ManagerRoute	frontend/src/components/*.tsx	Frontend route guards every protected route in App.tsx is wrapped in.

Module Dependencies

Read as "the row depends on the column" (row's code calls into column's services/data).

Module ↓ depends on →	Auth	User/ Emp	Store	Masters	Party	Product/ Inv	Purchase	Sales	Acct Core	Acct Reports	GST	Expense/ Cash/FY/ Audit
User/ Employee	✓		✓									
Store	✓	✓				✓						
Party/ Customer/ Supplier				✓								
Product/ Inventory	✓		✓									
Purchase	✓		✓		✓	✓			✓		✓	✓ (FY)
Sales	✓		✓		✓	✓			✓		✓	✓ (FY)
Accounting Core			✓									✓ (FY)
Accounting Reports							✓	✓	✓			✓

Module ↓ depends on →	Auth	User/ Emp	Store	Masters	Party	Product/ Inv	Purchase	Sales	Acct Core	Acct Reports	GST	Expense/ Cash/FY/ Audit
GST			✓		✓		✓	✓				✓ (Expense)
Expense/ Cash/FY/ Audit			✓		✓				✓		✓	
Alerts/ Search/ Dashboard						✓	✓	✓	✓		✓	

Nearly everything depends on Auth (JWT/permissions) and **Store** (storeId resolution) — they are not shown as columns receiving many checks above only because the table would be all-✓; treat both as implicit universal dependencies. **Accounting Core and GST are depended on by every transactional module but depend on almost nothing themselves** (only Financial Year) — they are the highest fan-in, lowest fan-out modules, i.e. the most sensitive to break.

Critical Files

Files where a change ripples across many modules — treat edits here with extra care and re-check every module doc's "Dependencies on Other Modules" section before changing:

1. **AccountingService.java** — every financial posting in the system goes through it.
2. **VoucherNumberService.java** — every voucher-numbered document depends on its exact algorithm/format.
3. **StoreAccessService.java** — every store-scoped authorization decision in the app.
4. **InventoryService.java** (**applyMovement** , **getCurrentStock**) — the only legitimate way stock changes; bypassing it anywhere would desync `inventory/stock_history`.
5. **RolePermissions.java** — the single source of truth for every `@PreAuthorize("hasAuthority('PERM_...')")` check in the app; a typo here silently changes access control everywhere.
6. **GlobalExceptionHandler.java** — the error-shape contract every frontend `parseApiError` call relies on.
7. **SecurityConfig.java** / **JwtAuthenticationFilter.java** — the entire request pipeline's authentication.
8. **Permission.java (enum)** — adding/removing a value here requires touching `RolePermissions.java`, every relevant `@PreAuthorize`, `UserPrincipal.getAuthorities()`, and the frontend `Role/permission` types.
9. **App.tsx** — the single frontend routing table; every new page must be registered here or it's unreachable.
10. **axios.ts** — every single frontend API call goes through this one configured instance.
11. **MasterCrudPage.tsx** — 9 master pages share this component; a breaking prop change breaks all 9 at once.

Manual Change Quick Reference

I want to...	Primary files to touch	Also check
Add a field to an existing entity/master	Entity class + its DTO(s) + service create/update methods + frontend type + frontend form	The module's own "Manual Changes" section for exact line-level guidance
Add a brand-new simple lookup master	New entity+status enum, repository, DTO(s), service, controller, frontend type, <code>mastersApi.ts</code> export, new page via <code>MasterCrudPage.tsx</code> , <code>MastersDashboard.tsx</code> card, route in <code>App.tsx</code>	Module 04's step-by-step checklist
Add a new Role or Permission	<code>Role.java</code> / <code>Permission.java</code> enums, <code>RolePermissions.java</code> , relevant <code>@PreAuthorize</code> values, <code>UserPrincipal.getAuthorities()</code> , frontend <code>Role</code> /permission types	Module 01
Add a new Store-scoped field to a module	That entity's <code>store</code> FK, its create-request DTO's <code>storeId</code> , its service calling <code>storeAccessService.resolveEffectiveStoreId</code> , its list/report queries calling <code>resolveViewableStoreId</code>	Module 03
Change stock movement logic	<code>InventoryService.applyMovement</code> / <code>StockMovementType</code> enum	Every caller: Purchase, Sale, Credit/Debit Note, Stock Transfer services (module 06's caller list)
Add a new voucher/document type	<code>VoucherType</code> / <code>VoucherDocType</code> enums, <code>VoucherNumberService</code> , the owning module's service	Module 09
Change GST calculation/split logic	<code>GstCalculationService.java</code>	Every module that calls it (Sale, Purchase, Notes, Expense) — module 11
Add a new accounting report	New service method, <code>AccountingReportController</code> endpoint, frontend api export + page + route	Module 10
Change JWT expiry/claims or login flow	<code>JwtUtil.java</code> , <code>AuthService.java</code> , <code>SecurityConfig.java</code>	Module 01
Add a new Permission-gated frontend action	Backend <code>@PreAuthorize</code> , <code>RolePermissions.java</code> , frontend <code>hasPermission()</code> check in the component	Modules 01 + the relevant feature module
Fix the Alert Center 400-on-no-current-FY bug	<code>AlertService.java</code> 's call into <code>FinancialYearService.getCurrent()</code>	Module 13
Enforce <code>REPORT_VIEW</code> / <code>REPORT_EXPORT</code> server-side (currently unenforced)	Add <code>@PreAuthorize</code> to <code>AccountingReportController</code>	Module 10
Enforce <code>GST_VIEW</code> / <code>GST_REPORT</code> / <code>GST_EXPORT</code> server-side (currently unenforced)	Add <code>@PreAuthorize</code> to <code>GstReportController</code>	Module 11

Known Gaps & Inconsistencies Found During Documentation

These are real, code-grounded observations surfaced while writing this documentation — not opinions, and not exhaustive. Each is cited in its owning module's doc with exact file/method references.

- **Permission definitions that exist but aren't enforced server-side:** `MASTER_VIEW` (module 04), `REPORT_VIEW/REPORT_EXPORT` (module 10 — `AccountingReportController` has zero `@PreAuthorize` annotations), `GST_VIEW/GST_REPORT/GST_EXPORT` (module 11 — only `GST_CONFIG` is actually checked). These permissions are defined in `RolePermissions.java` and granted to roles, but nothing currently reads them.
- **Frontend/backend permission mismatch:** `ManagerRoute` gates the entire `/masters/*` frontend route tree to `ADMIN/STORE_MANAGER` only, so a role like `ACCOUNTANT` or `SALES_USER` that legitimately holds `MASTER_VIEW` on the backend can never reach those pages through the UI (module 04).
- **Missing `@PreAuthorize` on some controllers/endpoints:** `CustomerController` (module 05, any authenticated user can create/list customers), `AccountController/AccountingController` gate on `hasRole('ADMIN')` directly rather than the `ACCOUNT_VIEW/JOURNAL_*` permissions defined for them (module 09), `DebitNoteController`'s GET endpoints and all of `PurchaseSummaryController` (module 07), `EmployeeController` has no class-level guard unlike `UserController` (module 02).
- **Audit trail gaps:** `PurchaseOrderService`, `SalesOrderService`, `ExpenseCategoryService`, `PaymentMethodService`, and the entire Accounting Core engine (`AccountingService/AccountService/LedgerService/VoucherNumberService`) make no `AuditService.log(...)` calls — confirmed by `grep`, not just absence in the sampled files.
- **Party master is effectively dormant:** it has zero incoming FKs from any transactional entity and is only used by its own master page — `Customer/Supplier` (separate entities) are what `Sales/Purchase` actually reference (module 05).
- **Real bug:** `AlertController`'s `/api/alerts` will 400 entirely whenever there is no "current" Financial Year, because `AlertService` calls `FinancialYearService.getCurrent()` (which throws rather than returning null/empty) without a guard (module 13).
- **Frontend/backend enum drift:** `frontend/src/types/auditLog.ts`'s `AuditAction` union is missing `DAY_CLOSE`, which exists in the backend enum (module 12); `frontend/src/types/accounting.ts`'s `VoucherType` union and the Journals filter omit `EXPENSE` and `CASH_TRANSACTION`, which the backend enum has (module 09).
- **Dead/unwired code paths:** no frontend caller was found for `userApi.changeOwnPassword` or for the `Employee assignStores/getAssignedStores` endpoints, despite the backend supporting them (module 02); no frontend page calls `purchaseApi.cancel` (soft cancel) — the UI's "Delete" action always uses the hard-delete endpoint instead (module 07); `PurchaseStatus.PENDING` is defined and set as the JPA default but no code path ever assigns it after creation (module 07).
- **GST reports never send `storeId`:** every backend GST report endpoint and query supports a `storeId` filter (Multi-Store), but none of the 9 frontend GST report pages actually pass one (module 11).

None of these were fixed as part of this documentation task, per the read-only instruction — they are flagged here for your awareness and are each documented in more depth in their owning module file.

Authentication & Authorization Module

1. Overview

This module handles account registration, login, session identity (JWT), and permission-based authorization for the whole StoreHub application. It exists to answer two questions on every request: "who is this?" and "what is this user allowed to do?"

- **Identity/login** is handled by `AuthController / AuthService` (`backend/src/main/java/com/storehub/controller/AuthController.java`, `backend/src/main/java/com/storehub/service/AuthService.java`): register a new `User`, verify credentials on login, and issue a signed JWT.
- **Authorization** is a fixed, code-defined `Role → Permission` map (`RolePermissions.java`), not a database table. Every request's Spring Security `Authentication` carries both a legacy `ROLE <name>` authority and a `PERM <name>` authority per permission the user's role has (`UserPrincipal.getAuthorities()`), and controllers across the whole app gate endpoints with `@PreAuthorize("hasAuthority('PERM_...')")`.
- **Read-only catalog endpoints** (`PermissionController`, `RoleController`) expose the permission catalog and role → permission mapping for the admin "Roles & Permissions" screen; they are browsing endpoints only — nothing here lets an admin edit a role's permissions at runtime.

2. Frontend

Files and Components

File	Component/Purpose
<code>frontend/src/pages/Login.tsx</code>	Login form: email + password, calls <code>AuthContext.login</code> .
<code>frontend/src/pages/Register.tsx</code>	Registration form: name/email/mobile/password/role (<code>STORE_MANAGER</code> or <code>STAFF</code> only), calls <code>AuthContext.register</code> .
<code>frontend/src/context/AuthContext.tsx</code>	Global auth state: current <code>user</code> , <code>permissions</code> set, <code>myStores</code> ; hydrates from <code>localStorage</code> + <code>/auth/me</code> on load; exposes <code>login</code> , <code>register</code> , <code>logout</code> , <code>switchStore</code> , <code>hasPermission</code> .
<code>frontend/src/api/authApi.ts</code>	Axios wrapper for <code>/api/auth/*</code> endpoints.
<code>frontend/src/api/roleApi.ts</code>	Axios wrapper for <code>/api/roles</code> and <code>/api/permissions</code> .
<code>frontend/src/api/userApi.ts</code>	Includes <code>getEffectivePermissions(id)</code> used by <code>AuthContext</code> to populate the permission set (backed by <code>UserController</code> , outside this module's own controllers but part of the auth flow).
<code>frontend/src/components/ProtectedRoute.tsx</code>	Route guard: redirects to <code>/login</code> if <code>user</code> is null (after <code>loading</code> resolves).
<code>frontend/src/components/AdminRoute.tsx</code>	Route guard: only <code>user.role === 'ADMIN'</code> passes, else redirect to <code>/unauthorized</code> .
<code>frontend/src/components/ManagerRoute.tsx</code>	Route guard: <code>ADMIN / STORE_MANAGER</code> plus an optional <code>extraRoles</code> list pass, else redirect to <code>/unauthorized</code> .
<code>frontend/src/pages/Unauthorized.tsx</code>	Static "Access denied" page shown by the route guards.
<code>frontend/src/pages/admin/RolesAndPermissions.tsx</code>	Admin-only, read-only browser: lists roles (<code>roleApi.list()</code>) and, per selected role, its permissions grouped by module (<code>roleApi.getPermissions(role)</code>).

File	Component/Purpose
frontend/src/types/user.ts	Role, User, AuthResponse, LoginPayload, RegisterPayload, RoleInfo, ApiErrorResponse, etc.
frontend/src/utils/apiError.ts	parseApiError() — turns an axios error into a message + per-field fieldErrors map for inline form display.
frontend/src/App.tsx	Route wiring: /login, /register are public; everything else sits under <ProtectedRoute>, with <AdminRoute> / <ManagerRoute> nested for role-restricted branches.

Frontend Flow

Login:

1. User submits Login.tsx's form → handleSubmit calls useAuth().login({ email, password }).
2. AuthContext.login calls authApi.login(payload) → POST /api/auth/login.
3. On success: JWT stored as localStorage['storehub_token'], user JSON as localStorage['storehub_user'], setUser(res.data.user), then loadPermissions(user) and loadMyStores(user) are fired (not awaited) to populate permissions and myStores.
4. Login.tsx navigates to /dashboard on success; on failure, parseApiError fills error and fieldErrors for inline display.

Registration:

1. User submits Register.tsx's form → client first checks form.password !== form.confirmPassword locally (shows confirmPassword: 'Passwords do not match' without a network call).
2. handleSubmit calls useAuth().register(form) → AuthContext.register calls authApi.register(payload) → POST /api/auth/register.
3. On success, Register.tsx shows a success alert and setTimeouts to navigate('/login') after 1500ms (registration does **not** log the user in or return a token — see DTOs below).
4. On failure, parseApiError fills error/fieldErrors (e.g. backend's mobile: "Mobile number must be 10 digits").

AuthContext hydration on load (useEffect in AuthProvider):

1. Reads storehub_token and storehub_user from localStorage.
2. If both present: optimistically setUser(parsed), then fires loadPermissions(parsed) and loadMyStores(parsed) immediately (using the possibly-stale cached user), and in parallel calls authApi.me() (GET /api/auth/me, authenticated via the stored JWT) to refresh the user from the server.
3. On me() success: setUser(res.data), refresh localStorage['storehub_user'], reload permissions from the fresh user.
4. On me() failure (e.g. expired/invalid token → 401): clears both localStorage keys, resets user/permissions/myStores to empty — effectively logs the user out client-side.
5. loadMyStores: calls authApi.getMyStores() (GET /api/auth/my-stores); if the user has exactly one accessible store and no currentStoreId yet, it silently calls authApi.setCurrentStore(id) to auto-select it.
6. loading is set to false once this resolves; ProtectedRoute shows a spinner until then.

3. API Calls

Frontend function (file:function)	HTTP Method	URL	Request body shape	Response shape
authApi.ts:login	POST	/api/auth/login	LoginPayload { email, password }	AuthResponse { token, user: User }

Frontend function (file:function)	HTTP Method	URL	Request body shape	Response shape
authApi.ts:register	POST	/api/auth/register	RegisterPayload { firstName, lastName, email, mobile, password, confirmPassword, role }	User (the created UserResponse)
authApi.ts:logout	POST	/api/auth/logout	none	{ message: string }
authApi.ts:me	GET	/api/auth/me	none (JWT in Authorization header)	User
authApi.ts:getMyStores	GET	/api/auth/my-stores	none	Store[]
authApi.ts:setCurrentStore	PUT	/api/auth/current-store	{ storeId: number }	User
roleApi.ts:list	GET	/api/roles	none	RoleInfo[] { name, description, active, permissionCount }
roleApi.ts:getPermissions	GET	/api/roles/{role}/permissions	none	PermissionsByModule (Record<module, PermissionEntry[]>)
roleApi.ts:permissionApi.list	GET	/api/permissions	none	PermissionsByModule
userApi.ts:getEffectivePermissions	GET	/api/users/{id}/effective-permissions	none	Permission[] (string names, used by AuthContext.hasPermission)

4. Backend

Controllers

AuthController — backend/src/main/java/com/storehub/controller/AuthController.java, @RequestMapping("/api/auth"), no class-level @PreAuthorize (endpoints are permitAll() or protected via SecurityConfig's anyRequest().authenticated()):

Method	Annotation	Signature	Calls
register	@PostMapping("/register")	register(@Valid @RequestBody RegisterRequest request) → ResponseEntity<UserResponse> (201)	authService.register(request)
login	@PostMapping("/login")	login(@Valid @RequestBody LoginRequest request) → ResponseEntity<AuthResponse>	authService.login(request)
logout	@PostMapping("/logout")	logout() → ResponseEntity<Map<String,String>>	none — stateless JWT, purely client-side t static success message
getCurrentUser	@GetMapping("/me")	getCurrentUser(@AuthenticationPrincipal UserPrincipal principal) → ResponseEntity<UserResponse>	authService.getCurrentUser(princip

Method	Annotation	Signature	Calls
getMyStores	@GetMapping("/my-stores")	getMyStores(@AuthenticationPrincipal UserPrincipal principal) → ResponseEntity<List<StoreResponse>>	userService.getAccessibleStores(principal)
setCurrentStore	@PutMapping("/current-store")	setCurrentStore(@AuthenticationPrincipal UserPrincipal principal, @Valid @RequestBody SetCurrentStoreRequest request) → ResponseEntity<UserResponse>	userService.setCurrentStore(principal, request.getStoreId())

/register and /login are reachable unauthenticated because SecurityConfig permits /api/auth/**; /me, /my-stores, /current-store still require a valid JWT since the filter chain requires authentication for everything not explicitly permitted, and rely on @AuthenticationPrincipal being populated by JwtAuthenticationFilter.

PermissionController — backend/src/main/java/com/storehub/controller/PermissionController.java, @RequestMapping("/api/permissions"), class-level @PreAuthorize("hasAuthority('PERM_PERMISSION_VIEW')") :

Method	Annotation	Signature	Calls
list	@GetMapping (inherits class @PreAuthorize)	list() → ResponseEntity<Map<String, List<PermissionResponse>>>	Permission.values() mapped to PermissionResponse.fromPermission, grouped by PermissionResponse::getModule

RoleController — backend/src/main/java/com/storehub/controller/RoleController.java, @RequestMapping("/api/roles"), class-level @PreAuthorize("hasAuthority('PERM_ROLE_VIEW')") :

Method	Annotation	Signature	Calls
list	@GetMapping (inherits class @PreAuthorize)	list() → ResponseEntity<List<RoleResponse>>	Role.values() mapped to RoleResponse.fromRole
permissionsFor	@GetMapping("/{role}/permissions") (inherits class @PreAuthorize)	permissionsFor(@PathVariable Role role) → ResponseEntity<Map<String, List<PermissionResponse>>>	RolePermissions.forRole(role) mapped/grouped

Per the class Javadoc: "Roles are a fixed code enum here (see Role), not a DB table, so there is nothing to create/rename/delete — only RolePermissions ... is out of scope for this step's read API."

DTOs

LoginRequest — backend/src/main/java/com/storehub/dto/LoginRequest.java

- email: String — @NotBlank(message = "Email is required"), @Email(message = "Email must be valid")
- password: String — @NotBlank(message = "Password is required")

RegisterRequest — backend/src/main/java/com/storehub/dto/RegisterRequest.java

- firstName: String — @NotBlank(message = "First name is required")
- lastName: String — @NotBlank(message = "Last name is required")
- email: String — @NotBlank(message = "Email is required"), @Email(message = "Email must be valid")
- mobile: String — @NotBlank(message = "Mobile number is required"), @Pattern(regexp = "^[0-9]{10}\$", message = "Mobile number must be 10 digits")
- password: String — @NotBlank(message = "Password is required"), @Size(min = 6, message = "Password must be at least 6 characters")

- confirmPassword: String — @NotBlank(message = "Confirm password is required")
- role: Role — @NotNull(message = "Role is required")

AuthResponse — backend/src/main/java/com/storehub/dto/AuthResponse.java

- token: String
- user: UserResponse

UserResponse — backend/src/main/java/com/storehub/dto/UserResponse.java

- id: Long, firstName: String, lastName: String, email: String, mobile: String, role: Role, status: UserStatus
- employeeId: Long, employeeName: String, employeeCode: String (from linked Employee, else null)
- mustChangePassword: boolean, lastLogin: LocalDateTime
- allStoresAccess: boolean — computed as RolePermissions.has(user.getRole(), Permission.STORE_ACCESS_ALL)
- currentStoreId: Long, currentStoreName: String, currentStoreCode: String
- createdAt: LocalDateTime, updatedAt: LocalDateTime
- No validation annotations (response DTO); fromEntity(User user) is the mapping factory.

PermissionResponse — backend/src/main/java/com/storehub/dto/PermissionResponse.java

- name: String, module: String; fromPermission(Permission permission) maps permission.name() / permission.getModule().

RoleResponse — backend/src/main/java/com/storehub/dto/RoleResponse.java

- name: String, description: String, active: boolean, permissionCount: int; fromRole(Role role) maps from RoleDescriptions.of(role) and RolePermissions.forRole(role).size(). active is always hardcoded true (no soft-disable of roles exists).

UserCreateRequest (used by UserController, module-adjacent) — backend/src/main/java/com/storehub/dto/UserCreateRequest.java

- firstName, lastName — @NotBlank
- email — @NotBlank, @Email
- mobile — @NotBlank, @Pattern(regexp = "^[0-9]{10}\$", message = "Mobile number must be 10 digits")
- password — @NotBlank, @Size(min = 6, message = "Password must be at least 6 characters")
- role: Role — @NotNull
- status: UserStatus (optional, no annotation)
- employeeId: Long (optional)

UserUpdateRequest — backend/src/main/java/com/storehub/dto/UserUpdateRequest.java

- firstName, lastName — @NotBlank
- email — @NotBlank, @Email
- mobile — @NotBlank, @Pattern(regexp = "^[0-9]{10}\$", message = "Mobile number must be 10 digits")
- role: Role — @NotNull
- status: UserStatus — @NotNull(message = "Status is required")
- employeeId: Long (optional, "null clears the link" per comment)

Services

AuthService — backend/src/main/java/com/storehub/service/AuthService.java (@Service, depends on UserRepository, PasswordEncoder, JwtUtil, AuditService):

- **register(RegisterRequest request)** — @Transactional. Business rules enforced, in order:
 1. if (!request.getPassword().equals(request.getConfirmPassword())) throw new BadRequestException("Password and confirm password do not match");
 2. if (userRepository.existsByEmail(request.getEmail())) throw new

```
DuplicateEmailException(request.getEmail());
3. if (request.getRole() == Role.ADMIN) throw new BadRequestException("Public registration cannot
create an ADMIN account"); — the codebase's exact ADMIN self-registration guard.
4. Builds a User with status(UserStatus.ACTIVE) and BCrypt-encoded password
(passwordEncoder.encode(...)), saves via userRepository.save(user), returns
UserResponse.fromEntity(saved). Note this method never logs the user in and never calls
AuditService (no audit log is written for registration).
```

- **login(LoginRequest request)** — @Transactional.

```
1. User user = userRepository.findByEmail(request.getEmail()).orElse(null);
2. If user == null OR !passwordEncoder.matches(request.getPassword(), user.getPassword()): logs a
failed-login audit entry (AuditAction.LOGIN, description "Failed login attempt for " +
request.getEmail() + ": invalid credentials") and throws InvalidCredentialsException("Invalid
email or password").
3. If user.getStatus() == UserStatus.INACTIVE: logs a failed-login audit entry (description "Failed
login attempt for " + user.getEmail() + ": account is inactive") and throws
InvalidCredentialsException("Your account is inactive. Please contact an administrator").
4. On success: user.setLastLogin(LocalDateTime.now()); userRepository.save(user);, logs a
success audit entry (description "User " + user.getEmail() + " logged in"), generates the JWT via
jwtUtil.generateToken(user.getEmail(), user.getRole().name()), returns
AuthResponse{ token, user: UserResponse.fromEntity(user) }.
```

- **getCurrentUser(String email)** — not transactional.

```
userRepository.findByEmail(email).orElseThrow(() -> new InvalidCredentialsException("User not
found")) -> UserResponse.fromEntity(user). BACKS GET /api/auth/me.
```

PermissionService — backend/src/main/java/com/storehub/service/PermissionService.java (@Service, no repository dependency — pure logic over RolePermissions):

```
- hasPermission(User user, Permission permission) ->
user != null && RolePermissions.has(user.getRole(), permission).
- currentUserHasPermission(Permission permission) -> hasPermission(SecurityUtil.currentUserOrNull(),
permission).
- effectivePermissions(Role role) -> RolePermissions.forRole(role).
- Per its Javadoc, this exists for the handful of checks that need a plain Java call rather than a
@PreAuthorize SpEL expression (admin-safety checks, the effective-permissions endpoint, self-service
guards) — but the primary enforcement mechanism everywhere else remains
@PreAuthorize("hasAuthority('PERM_...')") backed by the same RolePermissions map.
```

RolePermissions (not a Spring bean — final class with static methods) — backend/src/main/java/com/storehub/service/RolePermissions.java:

```
- forRole(Role role) -> BY_ROLE.getOrDefault(role, Collections.emptySet()).
- has(Role role, Permission permission) -> forRole(role).contains(permission).
- build() constructs the fixed Map<Role, Set<Permission>> (see section 8 below for full contents). This is
described in its own Javadoc as "the single source of truth" and "the one place authorization logic is
ever duplicated from."
```

RoleDescriptions (also static-only, not a bean) — backend/src/main/java/com/storehub/service/RoleDescriptions.java: of(Role role) returns a hardcoded human-readable description per role, "display only, not authorization."

Repository

UserRepository — `backend/src/main/java/com/storehub/repository/UserRepository.java` (extends `JpaRepository<User, Long>`), methods used by this module:

- `findByEmail(String email): Optional<User>` — used by `AuthService.login`, `getCurrentUser`, and `CustomUserDetailsService.loadUserByUsername`.
- `existsByEmail(String email): boolean` — used by `AuthService.register` for the duplicate-email check.
- (Also present, used elsewhere in the user-management module: `findByEmployeeId`, `existsByEmployeeId`, `existsByEmployeeIdAndIdNot`, `countByRoleAndStatus`, and a JPQL `search(...)` query.)

5. Database

Tables

users (entity `User`, `backend/src/main/java/com/storehub/entity/User.java`, `@Table(name = "users")`):

Column	Type/Constraint
<code>id</code>	<code>@Id @GeneratedValue(strategy = GenerationType.IDENTITY)</code>
<code>first_name</code>	<code>@Column(name = "first_name", nullable = false, length = 50)</code>
<code>last_name</code>	<code>@Column(name = "last_name", nullable = false, length = 50)</code>
<code>email</code>	<code>@Column(nullable = false, unique = true, length = 100)</code>
<code>mobile</code>	<code>@Column(nullable = false, length = 15)</code>
<code>password</code>	<code>@Column(nullable = false)</code> (BCrypt hash)
<code>role</code>	<code>@Enumerated(EnumType.STRING) @Column(nullable = false, length = 20)</code>
<code>status</code>	<code>@Enumerated(EnumType.STRING) @Column(nullable = false, length = 20)</code> (ACTIVE / INACTIVE)
<code>employee_id</code>	<code>@ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "employee_id", unique = true)</code> — nullable, unique (at most one <code>User</code> per <code>Employee</code>)
<code>must_change_password</code>	<code>@Column(name = "must_change_password", nullable = false)</code> , default <code>false</code>
<code>last_login</code>	<code>@Column(name = "last_login")</code> — nullable
<code>current_store_id</code>	<code>@ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "current_store_id")</code> — nullable
<code>created_at</code>	<code>@Column(name = "created_at", nullable = false, updatable = false)</code> — set in <code>@PrePersist onCreate()</code>
<code>updated_at</code>	<code>@Column(name = "updated_at", nullable = false)</code> — refreshed in <code>@PreUpdate onUpdate()</code>

`@PrePersist onCreate()` also defaults `status` to `UserStatus.ACTIVE` if null (belt-and-braces alongside `AuthService.register` explicitly setting it).

`Role` and `Permission` (`backend/src/main/java/com/storehub/entity/Role.java`, `Permission.java`) are plain Java enums — not database tables — per their own Javadoc.

Relationships

- `User.role` — enum column, no FK (not a separate `roles` table).
- `User.employee` → `Employee` — `@ManyToOne`, nullable, unique on `employee_id` (one `Employee` has at most one `User` login).

- `User.currentStore` → `Store` — `@ManyToOne`, nullable; validated by `StoreAccessService` (per the entity's own Javadoc) before ever being set, not trusted blindly from the frontend request.

6. Validation

Bean Validation (Jakarta):

- `LoginRequest.email` — `@NotBlank("Email is required")`, `@Email("Email must be valid")`.
- `LoginRequest.password` — `@NotBlank("Password is required")`.
- `RegisterRequest` — see full field list in section 4 DTOs; notably `mobile` requires `^[0-9]{10}$` ("Mobile number must be 10 digits") and `password` requires `@Size(min = 6, ...)` ("Password must be at least 6 characters").
- `UserCreateRequest` / `UserUpdateRequest` — same `mobile` pattern and `password` size rule (create only; update has no password field).

Custom business validation (in `AuthService`, thrown as exceptions, not annotations):

- Password/confirm-password mismatch on register → `BadRequestException("Password and confirm password do not match")`.
- Duplicate email on register → `DuplicateEmailException` (message: "A user with email '" + email + "' already exists").
- Public self-registration attempting `Role.ADMIN` → `BadRequestException("Public registration cannot create an ADMIN account")`.
- Login with unknown email or wrong password → `InvalidCredentialsException("Invalid email or password")` (deliberately the same message for both cases, so failed login never reveals whether the email exists).
- Login while `UserStatus.INACTIVE` → `InvalidCredentialsException("Your account is inactive. Please contact an administrator")`.
- `getCurrentUser` on a since-deleted user → `InvalidCredentialsException("User not found")`.

7. Authentication / Authorization

JWT issuance — `JwtUtil.generateToken(String email, String role)`

(`backend/src/main/java/com/storehub/security/JwtUtil.java`): builds a JWT with `setSubject(email)`, a single custom claim `claim("role", role)`, `setIssuedAt(now)`, `setExpiration(now + expirationMs)`, signed HS256 with a `SecretKey` built from `${jwt.secret}` (`Keys.hmacShaKeyFor(secret.getBytes(UTF_8))`). `expirationMs` comes from `${jwt.expiration-ms}`, both injected via `@Value` in the constructor.

JWT validation — `JwtUtil.isTokenValid(token)` parses the claims and checks

`claims.getExpiration().after(new Date())`, catching `JwtException` / `IllegalArgumentException` as invalid. `extractEmail` / `extractRole` read the subject/role claim respectively (note: the `role` claim is extractable but is **not** what drives authorization at request time — see below).

Filter chain wiring — `JwtAuthenticationFilter` (`OncePerRequestFilter`) reads the `Authorization: Bearer`

`<token>` header; if `jwtUtil.isTokenValid(token)`, extracts the email, and if no `Authentication` is already set in the `SecurityContextHolder`, loads a fresh `UserDetails` via `CustomUserDetailsService.loadUserByUsername(email)` and sets a `UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities())` into the context. `SecurityConfig` (`backend/src/main/java/com/storehub/config/SecurityConfig.java`) registers this filter with `.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)`, disables CSRF, sets `SessionCreationPolicy.STATELESS`, and authorizes requests with:

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()
    .requestMatchers("/actuator/health").permitAll()
    .anyRequest().authenticated())
```

`@EnableMethodSecurity` on `SecurityConfig` is what makes `@PreAuthorize` annotations on controllers take effect.

Authority derivation — `UserPrincipal.getAuthorities()` (`backend/src/main/java/com/storehub/security/UserPrincipal.java`) is rebuilt fresh from the database `User` on every request (via `CustomUserDetailsService`, called per-request by the filter above — never from a claim baked into the JWT itself). It returns:

- `ROLE_<user.role.name(>)` (the legacy authority every pre-existing `hasRole(...)` / `hasAnyRole(...)` check used), plus
- one `PERM_<permission.name(> SimpleGrantedAuthority` per entry in `RolePermissions.forRole(user.getRole())`.

Because this is derived per-request from the current DB row rather than the JWT payload, an ADMIN changing a user's role takes effect on that user's very next request, even with an old-but-still-valid token in hand (explicitly called out in the class Javadoc).

`UserPrincipal` also implements the `UserDetails` account-status flags: `isEnabled()` returns `user.getStatus() == UserStatus.ACTIVE`, so an INACTIVE user's existing token, if authenticated, would still be rejected downstream by any check relying on `isEnabled()` (Spring Security's own auth flow), though the primary INACTIVE gate in this codebase is the explicit check in `AuthService.login`.

@PreAuthorize / hasAuthority usage in this module: `PermissionController` (class-level `hasAuthority('PERM_PERMISSION_VIEW')`), `RoleController` (class-level `hasAuthority('PERM_ROLE_VIEW')`). `AuthController` has none — it relies on `SecurityConfig`'s path-level `permitAll()` / `authenticated()` instead.

8. Permissions

Source: `RolePermissions.java` (`build()` method). Full permission catalog is in `Permission.java`, grouped by module there; the module value only affects UI grouping (`PermissionResponse.getModule()`), not authorization.

Role	Permissions (EnumSet contents)
ADMIN	<code>EnumSet.allOf(Permission.class)</code> — every permission in the system.
STORE_MANAGER	All of ADMIN's set minus : <code>USER_VIEW</code> , <code>USER_CREATE</code> , <code>USER_EDIT</code> , <code>USER_DEACTIVATE</code> , <code>ROLE_VIEW</code> , <code>ROLE_MANAGE</code> , <code>PERMISSION_VIEW</code> , <code>FY_MANAGE</code> , <code>AUDIT_VIEW</code> , <code>GST_CONFIG</code> , <code>STORE_CREATE</code> , <code>STORE_EDIT</code> , <code>STORE_ASSIGN</code> , <code>STORE_ACCESS_ALL</code> .
ACCOUNTANT	<code>DASHBOARD_VIEW</code> ; <code>PARTY_VIEW</code> , <code>ITEM_VIEW</code> , <code>MASTER_VIEW</code> , <code>EMPLOYEE_VIEW</code> , <code>STORE_VIEW</code> ; <code>SALES_VIEW</code> , <code>RECEIPT_VIEW</code> , <code>CREDIT_NOTE_VIEW</code> ; <code>PURCHASE_VIEW</code> , <code>PAYMENT_VIEW</code> , <code>DEBIT_NOTE_VIEW</code> ; <code>ACCOUNT_VIEW</code> , <code>JOURNAL_VIEW</code> , <code>JOURNAL_CREATE</code> , <code>JOURNAL_POST</code> ; <code>EXPENSE_VIEW</code> , <code>EXPENSE_CREATE</code> , <code>EXPENSE_POST</code> , <code>EXPENSE_CANCEL</code> ; <code>REPORT_VIEW</code> , <code>REPORT_EXPORT</code> , <code>CASH_MANAGE</code> ; <code>GST_VIEW</code> , <code>GST_REPORT</code> , <code>GST_EXPORT</code> , <code>GST_CONFIG</code> ; <code>INVENTORY_VIEW</code> .
SALES_USER	<code>DASHBOARD_VIEW</code> ; <code>SALES_VIEW</code> , <code>SALES_CREATE</code> , <code>SALES_EDIT</code> , <code>SALES_POST</code> ; <code>RECEIPT_VIEW</code> , <code>RECEIPT_CREATE</code> , <code>RECEIPT_POST</code> ; <code>CREDIT_NOTE_VIEW</code> ; <code>POS_ACCESS</code> ; <code>PARTY_VIEW</code> , <code>ITEM_VIEW</code> , <code>MASTER_VIEW</code> , <code>STORE_VIEW</code> .

Role	Permissions (EnumSet contents)
PURCHASE_USER	DASHBOARD_VIEW; PURCHASE_VIEW, PURCHASE_CREATE, PURCHASE_EDIT, PURCHASE_POST; PAYMENT_VIEW, PAYMENT_CREATE, PAYMENT_POST; DEBIT_NOTE_VIEW; PARTY_VIEW, ITEM_VIEW, MASTER_VIEW, STORE_VIEW.
INVENTORY_USER	DASHBOARD_VIEW; ITEM_VIEW, INVENTORY_VIEW, INVENTORY_ADJUST; STOCK_TRANSFER_VIEW, STOCK_TRANSFER_CREATE, STOCK_TRANSFER_DISPATCH, STOCK_TRANSFER_RECEIVE; MASTER_VIEW, STORE_VIEW.
STAFF	DASHBOARD_VIEW, POS_ACCESS; SALES_VIEW, SALES_CREATE; RECEIPT_VIEW, RECEIPT_CREATE; ITEM_VIEW, PARTY_VIEW, MASTER_VIEW, STORE_VIEW.

Permissions relevant specifically to this module's own screens: PERMISSION_VIEW (gates GET /api/permissions), ROLE_VIEW (gates GET /api/roles and GET /api/roles/{role}/permissions) — both held only by ADMIN (they are in the ADMIN-minus set removed from STORE_MANAGER). STORE_ACCESS_ALL (also ADMIN-only) drives UserResponse.allStoresAccess.

9. Transaction Handling

- AuthService.register(RegisterRequest) — @Transactional.
- AuthService.login(LoginRequest) — @Transactional (covers both the lastLogin update/save and the audit log write inside the same transaction as the login check).
- AuthService.getCurrentUser(String) — not annotated (read-only, single repository call).
- AuditService.log(...) (the overload actually taking storeId) is @Transactional, with **default propagation** (not REQUIRES_NEW) — per its own Javadoc, "an audit entry for an action that later rolls back should roll back with it," i.e. it deliberately joins the caller's existing transaction rather than committing independently.
- PermissionService, RolePermissions, RoleDescriptions have no transactional methods (no persistence involved).

10. Error Handling

GlobalExceptionHandler (backend/src/main/java/com/storehub/exception/GlobalExceptionHandler.java) maps this module's exceptions as follows, always returning an ApiError { timestamp, status, error, message, path, fieldErrors } body (fieldErrors omitted from JSON when null, via @JsonInclude(NON_NULL)):

Exception	HTTP Status	Message source	fieldErrors
MethodArgumentNotValidException (bean validation failure on @Valid DTOs)	400 BAD_REQUEST	"Validation failed"	populated from ex.getBindingResult().getFieldErrors(), keyed by DTO field name
DuplicateEmailException	409 CONFLICT	exception message	none
InvalidCredentialsException	401 UNAUTHORIZED	exception message	none
BadCredentialsException (Spring Security's own)	401 UNAUTHORIZED	hardcoded "Invalid email or password"	none
BadRequestException	400 BAD_REQUEST	exception message	none

Exception	HTTP Status	Message source	fieldErrors
<code>AccessDeniedException</code> (from <code>@PreAuthorize</code> failures)	403 FORBIDDEN	hardcoded "You do not have permission to access this resource"	none
any other <code>Exception</code>	500 <code>INTERNAL_SERVER_ERROR</code>	hardcoded "An unexpected error occurred"	none (also logged via <code>log.error()</code>)

This is the map the frontend's `parseApiError` (`frontend/src/utils/apiError.ts`) relies on: it prefers `fieldErrors` (rendering one line per field, e.g. "Mobile: Mobile number must be 10 digits") and falls back to the top-level `message` when `fieldErrors` is empty, per this project's CLAUDE.md debugging-rule pattern.

11. Audit Flow

Yes — `AuthService.login` calls `AuditService.log(...)` directly (`backend/src/main/java/com/storehub/service/AuthService.java`, lines 64-78):

```
// failed login, unknown email or wrong password
auditService.log(AuditAction.LOGIN, "AUTH", "User", user != null ? user.getId() : null,
    null, null, null, "Failed login attempt for " + request.getEmail() + ": invalid
credentials");

// failed login, inactive account
auditService.log(AuditAction.LOGIN, "AUTH", "User", user.getId(), null,
    null, null, "Failed login attempt for " + user.getEmail() + ": account is inactive");

// successful login
auditService.log(AuditAction.LOGIN, "AUTH", "User", user.getId(), null,
    null, null, "User " + user.getEmail() + " logged in");
```

Each call logs: `action = AuditAction.LOGIN`, `module = "AUTH"`, `entityType = "User"`, `entityId` = the user's id (or `null` if the email didn't match any user), `documentNumber = null`, `oldValue = null`, `newValue = null`, and a human-readable `description`. `AuditService.log` (the 8-arg overload, `backend/src/main/java/com/storehub/service/AuditService.java`) delegates to the 9-arg `@Transactional` version with `storeId = null`, which additionally captures `userId/username` from `SecurityUtil.currentUserOrNull()` (typically `null / "System"` for a login attempt, since there's no authenticated principal yet), plus `ipAddress` (from `X-Forwarded-For` or `request.getRemoteAddr()`) and `userAgent` (from the `User-Agent` header).

`AuthService.register` does **not** call `AuditService` — no audit log is written for a new registration.

12. Important Side Effects

- **Login updates `lastLogin`:** `AuthService.login` sets `user.setLastLogin(LocalDateTime.now())` and saves it on every successful login (not on failed attempts).

- **Registration auto-assigns status:** `AuthService.register` always sets `status(UserStatus.ACTIVE)` — a newly self-registered account is active immediately, no email verification or admin approval step exists in this codebase.
- **Registration does not log the user in:** it returns a `UserResponse` (201 Created), not an `AuthResponse` /token; `Register.tsx` explicitly redirects to `/login` after success rather than storing a session.
- **JWT has no server-side revocation:** `/api/auth/logout` (`AuthController.logout`) is a no-op that returns a static success message — it does not blacklist the token server-side. "Logout" is purely the frontend discarding `localStorage['storehub_token'] / storehub_user`. A previously issued token remains valid (and accepted by `JwtAuthenticationFilter`) until its `expirationMs` lapses, regardless of client-side logout.
- **JWT expiry affects the whole app:** since `SecurityConfig` requires authentication for `anyRequest()` beyond `/api/auth/**` and `/actuator/health`, every other module's API calls fail with 401 once the token in `{jwt.expiration-ms}` expires; `AuthContext`'s `me()` failure handler is what clears local session state client-side in that case.
- **Role/permission changes take effect immediately, not on next login:** because `UserPrincipal.getAuthorities()` is derived fresh per-request from the DB `User.role` (not a JWT claim), changing a user's role via user management is live on their very next API call, even with their existing token still valid.
- **Single-store auto-selection:** `AuthContext.loadMyStores` silently calls `authApi.setCurrentStore` when the user has exactly one accessible store and no `currentStoreId` set yet — a side effect of the login/hydration flow, not something the user explicitly requested.
- **Same generic message for "no such email" and "wrong password":** `InvalidCredentialsException("Invalid email or password")` is thrown identically in both cases, a deliberate anti-enumeration side effect of `AuthService.login`'s logic.

13. Dependencies on Other Modules

This module depends on:

- `Employee` entity (`User.employee` FK) — optional link surfaced in `UserResponse.employeeId/employeeName/employeeCode`.
- `Store` entity (`User.currentStore` FK, and `Store`-scoped `getMyStores / setCurrentStore` endpoints on `AuthController`) — multi-store access is validated elsewhere (`StoreAccessService`, per `User.java`'s own Javadoc) but surfaced through this module's `AuthResponse / UserResponse`.
- `AuditService / AuditLogRepository / Store` (via `AuditService.storeRepository`) — for writing login audit entries.
- `UserService` — `AuthController` delegates `getMyStores / setCurrentStore` to `UserService.getAccessibleStores / setCurrentStore` (not itself part of this module's ground-truth files, but directly called from `AuthController`).

What depends on this module: effectively every other module in the application. Every other controller's `@PreAuthorize("hasAuthority('PERM_...')")` check depends on `RolePermissions / UserPrincipal.getAuthorities()` defined here; every authenticated request depends on `JwtAuthenticationFilter / JwtUtil / CustomUserDetailsService`; every frontend route beyond `/login` and `/register` depends on `ProtectedRoute / AdminRoute / ManagerRoute` and `AuthContext.user / hasPermission`.

14. Key Operation Flows

Login:

```
UI (Login.tsx:Login/handleSubmit) → Function (AuthContext.tsx:login) → API (POST /api/auth/login) → Controller (AuthController.login) → DTO (LoginRequest) → Service (AuthService.login) → Repository (UserRepository.findByEmail) → Database (users) → Response (AuthResponse{token, user: UserResponse}) → UI (AuthContext sets user/token in state + localStorage, Login.tsx navigates to /dashboard)
```

Register:

```
UI (Register.tsx:Register/handleSubmit) → Function (AuthContext.tsx:register) → API (POST /api/auth/register) → Controller (AuthController.register) → DTO (RegisterRequest) → Service (AuthService.register) → Repository (UserRepository.existsByEmail, UserRepository.save) → Database (users) → Response (UserResponse, HTTP 201) → UI (Register.tsx shows success alert, redirects to /login after 1500ms)
```

Session hydration on app load (not strictly "login"/"register" but the third core flow of this module):

```
UI (AuthContext.tsx:AuthProvider useEffect) → API (GET /api/auth/me, Bearer token from localStorage) → Controller (AuthController.getCurrentUser) → Service (AuthService.getCurrentUser) → Repository (UserRepository.findByEmail) → Database (users) → Response (UserResponse) → UI (AuthContext refreshes user + permissions, or clears session on 401)
```

Password Change/Reset: NOT FOUND IN CURRENT CODEBASE within this module's ground-truth files — frontend/src/types/user.ts declares PasswordChangePayload/AdminPasswordResetPayload and userApi.ts exposes changeOwnPassword/resetPassword (PUT /api/users/me/password, POST /api/users/{id}/reset-password), but the controller/service backing these live in the User Management module (UserController/UserService), not in AuthController/AuthService/PermissionController/RoleController covered here.

15. Manual Changes

Add a new field to registration/login:

- Add the field to RegisterRequest (backend/src/main/java/com/storehub/dto/RegisterRequest.java) with its validation annotation, and to LoginRequest (backend/src/main/java/com/storehub/dto/LoginRequest.java) if it belongs to login.
- Add the same field to the User entity (backend/src/main/java/com/storehub/entity/User.java) with its @Column, and set it in AuthService.register's User.builder() call.
- Surface it on the response side in UserResponse (backend/src/main/java/com/storehub/dto/UserResponse.java, both the field and fromEntity).
- Add the form field to Register.tsx (state in the form object, <Input>, and error display) and/or Login.tsx.
- Add it to RegisterPayload/LoginPayload/User in frontend/src/types/user.ts.
- Affected: a DB migration/schema change for the new users column; any admin UserCreateRequest/UserUpdateRequest DTOs and their forms (UserFormModal.tsx, per CLAUDE.md) if the field should also be editable by an admin.

Change JWT expiry or claims:

- Expiry: change the jwt.expiration-ms property (application config, e.g. application.yml/application.properties) consumed by JwtUtil's constructor (backend/src/main/java/com/storehub/security/JwtUtil.java).
- Claims: add to JwtUtil.generateToken(String email, String role) (add another .claim("x", value) call before .compact()), then add a corresponding extractX(token) helper if the claim needs to be read back. Note authorization does **not** currently trust the role claim at request time (it's re-derived from

the DB via `CustomUserDetailsService/UserPrincipal`), so a new claim used for authorization would be a deliberate architecture change, not just a JWT tweak.

- Affected: nothing else in the authorization path needs to change since authorities are DB-derived per request, but any code relying on `JwtUtil.extractRole` (if any exists elsewhere) would need review.

Change password validation rules:

- Update `@Size(min = 6, ...)` on `RegisterRequest.password` (`backend/src/main/java/com/storehub/dto/RegisterRequest.java`) and the equivalent field on `UserCreateRequest` (`backend/src/main/java/com/storehub/dto/UserCreateRequest.java`) — keep both in sync, since they currently duplicate the same rule.
- Update the frontend's `minLength={6}` on `Register.tsx` 's password `<Input>` to match.
- Any password-change/reset DTOs (outside this module's read files, per section 14) would also need the same rule applied for consistency.

Add a new Permission or Role:

- New Permission: add the enum constant (with its module string) to `Permission.java` (`backend/src/main/java/com/storehub/entity/Permission.java`), then add it to every Role's `EnumSet` in `RolePermissions.build()` (`backend/src/main/java/com/storehub/service/RolePermissions.java`) that should carry it. `ADMIN` gets it automatically via `EnumSet.allOf(Permission.class)`.
- New Role: add the enum constant to `Role.java` (`backend/src/main/java/com/storehub/entity/Role.java`), add its entry to `RolePermissions.build()` 's map, and add a description in `RoleDescriptions.build()` (`backend/src/main/java/com/storehub/service/RoleDescriptions.java`).
- Affected: (1) `RolePermissions.java` — the authorization source of truth; (2) `UserPrincipal.getAuthorities()` needs no code change (it already iterates `RolePermissions.forRole(...)` generically) but any `@PreAuthorize("hasAuthority('PERM...')` annotation that should now reference the new permission must be added/updated on the relevant controller; (3) frontend `Role` type in `frontend/src/types/user.ts` (add the new literal to the `Role` union); (4) `Register.tsx` 's `REGISTERABLE_ROLES` array if the new role should be self-registerable; (5) `ManagerRoute / AdminRoute` usages in `App.tsx` (the `extraRoles` arrays) if the new role should access existing manager-gated routes.

Change which roles can self-register:

- Backend gate: `AuthService.register` 's single check `if (request.getRole() == Role.ADMIN)` throw new `BadRequestException(...)` (`backend/src/main/java/com/storehub/service/AuthService.java`) — currently the only backend restriction is "not ADMIN"; every other role is backend-permitted to self-register.
- Frontend gate: `Register.tsx` 's `REGISTERABLE_ROLES` constant (`frontend/src/pages/Register.tsx` , lines 14-17) currently limits the dropdown to `STORE_MANAGER` and `STAFF` only — this is a UI-only restriction; a request crafted directly against `POST /api/auth/register` with, say, `role: "ACCOUNTANT"` would currently succeed since the backend does not block it. Any tightening of self-registerable roles should be enforced in `AuthService.register` (backend), not just in `Register.tsx` (frontend-only restrictions are bypassable).

Every one of these changes should be cross-checked against: `RolePermissions.java` (authorization source of truth), `UserPrincipal.getAuthorities()` (only if authority-derivation logic itself changes, not for adding within existing structure), the specific `@PreAuthorize` annotations on affected controllers, and `frontend/src/types/user.ts` 's `Role / Permission` types plus any frontend route guards (`AdminRoute.tsx` , `ManagerRoute.tsx`) or role-gated UI (`Register.tsx` , `RolesAndPermissions.tsx`) that assume a fixed role/permission set.

User & Employee Management Module

1. Overview

This module covers two related but distinct record types in StoreHub:

- **User** (`com.storehub.entity.User` , table `users`) — a login account: email + password, a `Role` , a `UserStatus` (ACTIVE/INACTIVE), and optional links to an `Employee` and to a `Store` (`currentStore`).
- **Employee** (`com.storehub.entity.Employee` , table `employees`) — a master/HR record (name, mobile, designation, department, address, city/state, joining date) that need not have any login at all.

A `User` may optionally be linked to one `Employee` (`User.employee` , unique FK `employee_id`), and an `Employee` may optionally have one `User` login pointed at it. The link is enforced unique from the `User` side only — `Employee` itself carries no FK back to `User` ; the reverse lookup is done via `UserRepository.findByEmployeeId` .

Two controllers/services back this module:

- `UserController` / `UserService` — login-account CRUD, status, password management, admin-safety rules, store assignment (`/api/users/**`).
- `EmployeeController` / `EmployeeService` — HR master CRUD, employee-code generation, informational store tagging (`/api/masters/employees/**`).

The module never physically deletes a `User` or `Employee` — only ever flips `status` between `ACTIVE` / `INACTIVE` , per the code comment on `UserService` : "Never physically deletes a user (spec section 52) — only ever ACTIVE/INACTIVE, so historical audit/transaction references stay valid."

2. Frontend

Files and Components

File	Role
<code>frontend/src/pages/Users.tsx</code>	User Management page: search/filter/paginate users, table with actions (View, Edit, Reset Password, Store Access, Activate/Deactivate). Hosts <code>UserFormModal</code> , <code>UserViewModal</code> , <code>ResetPasswordDialog</code> , and <code>StoreAccessDialog</code> (store-assignment dialog, not in this module's required file set but wired from here).
<code>frontend/src/components/UserFormModal.tsx</code>	Add/Edit User modal. Fields: firstName, lastName, email, mobile, password (add-mode only), role, status, employeeId (dropdown of ACTIVE employees). Surfaces backend <code>fieldErrors</code> inline per field.
<code>frontend/src/components/UserViewModal.tsx</code>	Read-only detail view: mobile, role, status, linked employee, mustChangePassword, lastLogin, createdAt/updatedAt.
<code>frontend/src/components/ResetPasswordDialog.tsx</code>	ADMIN-initiated password reset for another user: newPassword + confirmPassword only (no current password).
<code>frontend/src/api/userApi.ts</code>	Axios wrapper for all <code>/api/users/**</code> endpoints.
<code>frontend/src/types/user.ts</code>	<code>Role</code> , <code>UserStatus</code> , <code>User</code> , <code>UserCreatePayload</code> , <code>UserUpdatePayload</code> , <code>AdminPasswordResetPayload</code> , <code>PasswordChangePayload</code> , <code>PagedResponse<T></code> , <code>ApiErrorResponse</code> .
<code>frontend/src/pages/masters/EmployeeMaster.tsx</code>	Employee master CRUD page, built on the generic <code>MasterCrudPage<Employee, EmployeePayload></code> component (list/create/edit/activate/deactivate), with a "Generate Code" button calling <code>employeeApi.generateCode()</code> .

File	Role
frontend/src/api/mastersApi.ts (employeeApi export)	Axios wrapper for /api/masters/employees/** (list, getByld, generateCode, create, update, activate, deactivate).
frontend/src/types/masters.ts (Employee, EmployeePayload)	Employee response/request shapes for the frontend.
frontend/src/utils/apiError.ts	parseApiError — turns an axios error's ApiErrorResponse into a display message + fieldErrors map, used by every form above.

Frontend Flow

List users — Users.tsx loadUsers() calls userApi.list({ search, role, status, page, size }) on mount and whenever page / roleFilter / statusFilter change, or the search form is submitted; renders into the table with TableSkeleton while loading and EmptyState when empty.

Create user — "Add User" button opens UserFormModal in mode: 'add' with EMPTY_FORM (role defaults STAFF, status ACTIVE). On submit, Users.tsx handleFormSubmit calls userApi.create(...) with employeeId coerced from '' to null. Validation errors thrown by the API surface as error / fieldErrors inside the modal (caught in UserFormModal.handleSubmit); success shows a toast, closes the modal, and reloads the list.

Edit user — "Edit" menu item opens the same modal in mode: 'edit' pre-filled from the selected User (password field is omitted/not sent). Submit calls userApi.update(id, ...).

Deactivate/reactivate user — "Deactivate"/"Activate" menu item calls toggleStatus(user), which calls userApi.updateStatus(user.id, newStatus) (flips ACTIVE↔INACTIVE) and reloads on success; errors are toasted via parseApiError.

Reset password — "Reset Password" menu item opens ResetPasswordDialog for that user; submit calls userApi.resetPassword(user.id, { newPassword, confirmPassword }); success toast tells the admin the user must change it at next login.

Assign stores — "Store Access" menu item opens StoreAccessDialog (component file not included in this module's read set, but its API calls are userApi.getAssignedStores(id) / userApi.assignStores(id, storeIds)), replace-all semantics.

Self password change — userApi.changeOwnPassword(payload) exists in userApi.ts (PUT /users/me/password) but no calling UI component was found under the read files for this module — NOT FOUND IN CURRENT CODEBASE (the calling page, if any, lives outside this module's file set).

Employee — list — EmployeeMaster.tsx passes fetchList=(({search,status,page,size}) => employeeApi.list(...)) into the generic MasterCrudPage, which handles the paging/search UI itself.

Employee — create — MasterCrudPage's create flow calls employeeApi.create with the form built in renderForm; a "Generate Code" icon button calls employeeApi.generateCode() and fills employeeCode into the form before submit (does not auto-save).

Employee — edit — MasterCrudPage's edit flow calls employeeApi.update(id, payload); toFormValues maps an Employee response back into EmployeePayload for the form.

Employee — deactivate/activate — MasterCrudPage wires activate={employeeApi.activate} / deactivate={employeeApi.deactivate}.

Employee — assign stores — `EmployeeController` exposes `GET/PUT /api/masters/employees/{id}/stores`, but `employeeApi` in `mastersApi.ts` does **not** export `assignStores/getAssignedStores` — NOT FOUND IN CURRENT CODEBASE (backend endpoint exists, no frontend caller wired in the read files).

3. API Calls

Function	HTTP Method	URL	Request shape	Response shape
<code>userApi.list</code>	GET	<code>/users</code>	query params <code>search?</code> , <code>role?</code> , <code>status?</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<User></code>
<code>userApi.getById</code>	GET	<code>/users/{id}</code>	—	<code>User</code>
<code>userApi.create</code>	POST	<code>/users</code>	<code>UserCreatePayload</code>	<code>User</code>
<code>userApi.update</code>	PUT	<code>/users/{id}</code>	<code>UserUpdatePayload</code>	<code>User</code>
<code>userApi.updateStatus</code>	PATCH	<code>/users/{id}/status</code>	<code>{ status: UserStatus }</code>	<code>User</code>
<code>userApi.resetPassword</code>	POST	<code>/users/{id}/reset-password</code>	<code>AdminPasswordResetPayload</code> (<code>{ newPassword, confirmPassword }</code>)	<code>User</code>
<code>userApi.changeOwnPassword</code>	PUT	<code>/users/me/password</code>	<code>PasswordChangePayload</code> (<code>{ currentPassword, newPassword, confirmPassword }</code>)	<code>void (204)</code>
<code>userApi.getEffectivePermissions</code>	GET	<code>/users/{id}/effective-permissions</code>	—	<code>Permission[]</code>
<code>userApi.getAssignedStores</code>	GET	<code>/users/{id}/stores</code>	—	<code>number[]</code>
<code>userApi.assignStores</code>	PUT	<code>/users/{id}/stores</code>	<code>{ storeIds: number[] }</code>	<code>User</code>
<code>employeeApi.list</code>	GET	<code>/masters/employees</code>	query params via <code>buildParams: search?</code> , <code>status?</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<Employee></code>
<code>employeeApi.getById</code>	GET	<code>/masters/employees/{id}</code>	—	<code>Employee</code>
<code>employeeApi.generateCode</code>	POST	<code>/masters/employees/generate-code</code>	—	<code>{ employeeCode: string }</code>
<code>employeeApi.create</code>	POST	<code>/masters/employees</code>	<code>EmployeePayload</code>	<code>Employee</code>
<code>employeeApi.update</code>	PUT	<code>/masters/employees/{id}</code>	<code>EmployeePayload</code>	<code>Employee</code>
<code>employeeApi.activate</code>	PATCH	<code>/masters/employees/{id}/activate</code>	—	<code>Employee</code>

Function	HTTP Method	URL	Request shape	Response shape
employeeApi.deactivate	PATCH	/masters/employees/{id}/deactivate	—	Employee

Note: axios instance is `api` from `./axios`, and all URLs above are relative to its base (`/api`), e.g. `userApi.list` really hits `/api/users`.

4. Backend

Controllers

UserController — `@RequestMapping("/api/users")`, class-level `@PreAuthorize("hasRole('ADMIN')")` (every endpoint below requires ADMIN unless a method-level override is noted):

Method	Endpoint	Notes
<code>getUsers</code>	GET <code>/api/users</code>	params <code>search</code> , <code>role</code> , <code>status</code> , <code>page</code> (default 0), <code>size</code> (default 10)
<code>getUserById</code>	GET <code>/api/users/{id}</code>	
<code>getEffectivePermissions</code>	GET <code>/api/users/{id}/effective-permissions</code>	comment: "Troubleshooting/access review (spec section 39)"
<code>createUser</code>	POST <code>/api/users</code>	<code>@Valid @RequestBody UserCreateRequest</code> ; returns 201
<code>updateUser</code>	PUT <code>/api/users/{id}</code>	<code>@Valid @RequestBody UserUpdateRequest</code>
<code>updateStatus</code>	PATCH <code>/api/users/{id}/status</code>	<code>@Valid @RequestBody UserStatusUpdateRequest</code>
<code>resetPassword</code>	POST <code>/api/users/{id}/reset-password</code>	<code>@Valid @RequestBody AdminPasswordResetRequest</code> ; comment: "never requires or reveals the existing one"
<code>getAssignedStores</code>	GET <code>/api/users/{id}/stores</code>	ADMIN-only via class-level guard
<code>assignStores</code>	PUT <code>/api/users/{id}/stores</code>	<code>@Valid @RequestBody AssignStoresRequest</code>
<code>changeOwnPassword</code>	PUT <code>/api/users/me/password</code>	method-level <code>@PreAuthorize("isAuthenticated()")</code> overrides the class-level ADMIN guard — reachable by every authenticated role, not just ADMIN; returns 204

EmployeeController — `@RequestMapping("/api/masters/employees")`, no class-level `@PreAuthorize` (each endpoint sets its own):

Method	Endpoint	@PreAuthorize
<code>list</code>	GET <code>/api/masters/employees</code>	none (open to any authenticated caller reachable by Spring Security filter chain)
<code>getById</code>	GET <code>/api/masters/employees/{id}</code>	none

Method	Endpoint	@PreAuthorize
generateCode	POST /api/masters/employees/generate-code	hasAuthority('PERM_EMPLOYEE_CREATE')
create	POST /api/masters/employees	hasAuthority('PERM_EMPLOYEE_CREATE') ; returns 201
update	PUT /api/masters/employees/{id}	hasAuthority('PERM_EMPLOYEE_EDIT')
activate	PATCH /api/masters/employees/{id}/activate	hasAuthority('PERM_EMPLOYEE_EDIT')
deactivate	PATCH /api/masters/employees/{id}/deactivate	hasAuthority('PERM_EMPLOYEE_EDIT')
getAssignedStores	GET /api/masters/employees/{id}/stores	hasAuthority('PERM_EMPLOYEE_VIEW')
assignStores	PUT /api/masters/employees/{id}/stores	hasAuthority('PERM_EMPLOYEE_EDIT')

(`PERM_<name>` authorities are granted by `UserPrincipal.getAuthorities()` , which adds `"PERM_" + permission.name()` for every `Permission` the user's `Role` carries per `RolePermissions`.)

DTOs

UserCreateRequest

```
@NotBlank(message = "First name is required") firstName
@NotBlank(message = "Last name is required") lastName
@NotBlank(message = "Email is required") @Email(message = "Email must be valid") email
@NotBlank(message = "Mobile number is required") @Pattern(regexp = "[0-9]{10}$", message = "Mobile
number must be 10 digits") mobile
@NotBlank(message = "Password is required") @Size(min = 6, message = "Password must be at least 6
characters") password
@NotNull(message = "Role is required") role
status // no validation, optional – defaults to ACTIVE in service
employeeId // optional – links this login to an existing Employee record (spec section 7)
```

UserUpdateRequest

```
@NotBlank(message = "First name is required") firstName
@NotBlank(message = "Last name is required") lastName
@NotBlank(message = "Email is required") @Email(message = "Email must be valid") email
@NotBlank(message = "Mobile number is required") @Pattern(regexp = "[0-9]{10}$", message = "Mobile
number must be 10 digits") mobile
@NotNull(message = "Role is required") role
@NotNull(message = "Status is required") status
employeeId // optional – null clears the link
```

(no `password` field — password is never updated here)

UserStatusUpdateRequest

```
@NotNull(message = "Status is required") status
```

AdminPasswordResetRequest

```
@NotBlank(message = "New password is required") @Size(min = 6, message = "Password must be at least 6 characters") newPassword
@NotBlank(message = "Confirm password is required") confirmPassword
```

PasswordChangeRequest

```
@NotBlank(message = "Current password is required") currentPassword
@NotBlank(message = "New password is required") @Size(min = 6, message = "Password must be at least 6 characters") newPassword
@NotBlank(message = "Confirm password is required") confirmPassword
```

AssignStoresRequest

```
@NotNull(message = "storeIds is required (use an empty array to clear all store access)")
storeIds // Set<Long>
```

UserResponse fields: id, firstName, lastName, email, mobile, role, status, employeeId, employeeName, employeeCode, mustChangePassword, lastLogin, allStoresAccess, currentStoreId, currentStoreName, currentStoreCode, createdAt, updatedAt. allStoresAccess is computed via RolePermissions.has(user.getRole(), Permission.STORE_ACCESS_ALL), not stored on the entity.

EmployeeRequest

```
@NotBlank(message = "Employee code is required") employeeCode
@NotBlank(message = "Employee name is required") name
@NotBlank(message = "Mobile number is required") @Pattern(regexp = "[0-9]{10}$", message = "Mobile number must be 10 digits") mobile
>Email(message = "Email must be valid") email // optional (no @NotBlank)
designation, department, address, cityId, stateId, joiningDate, notes // no validation annotations
```

EmployeeResponse fields: id, employeeCode, name, mobile, email, designation, department, address, cityId, cityName, stateId, stateName, joiningDate, status, notes, linkedUserId, linkedUserEmail, createdAt, updatedAt. linkedUserId/linkedUserEmail are populated by the caller passing a User found via UserRepository.findById(id) — Employee entity itself has no such field.

GenerateEmployeeCodeResponse: { employeeCode: String }.

PagedResponse<T>: content: List<T>, page, size, totalElements, totalPages, built via PagedResponse.fromPage(Page<T>).

Services

UserService

- getUsers(search, role, status, page, size) — delegates to UserRepository.search(...), sorted by createdAt descending, maps to UserResponse.
- getUserById(id) — findUserOrThrow + map.
- createUser(request) (@Transactional) — throws DuplicateEmailException if existsByEmail; resolves optional employeeId via resolveEmployee(employeeId, null); builds User with passwordEncoder.encode(request.getPassword()), status defaults to UserStatus.ACTIVE if request status is null; saves; audit-logs CREATE.
- updateUser(id, request) (@Transactional) — throws DuplicateEmailException if email changed to one that already exists (case-insensitive compare on current email); re-resolves employeeId excluding this user's own id; **admin-safety guard**: if (user.getRole() == Role.ADMIN && request.getRole() != Role.ADMIN) guardNotLastActiveAdmin(...) and

```

if (user.getRole() == Role.ADMIN && request.getStatus() == UserStatus.INACTIVE)
guardNotLastActiveAdmin(...); audit-logs PERMISSION_CHANGE if role changed, else UPDATE .
- updateStatus(id, request) (@Transactional) — same admin-safety guard for ADMIN → INACTIVE; audit-
logs UPDATE with old/new status.
- adminResetPassword(id, request) (@Transactional) — throws BadRequestException("New password and
confirm password do not match") if mismatched; encodes and sets password; sets mustChangePassword =
true; audit-logs UPDATE with description "Password reset for user ... by an administrator" (comment:
"Never log the password itself — only that a reset happened").
- changeOwnPassword(currentUserEmail, request) (@Transactional) — mismatch check same as above;
looks up user by currentUserEmail (throws BadRequestException("User not found") if absent); throws
BadRequestException("Current password is incorrect") if !passwordEncoder.matches(currentPassword,
user.getPassword()); sets new password, mustChangePassword = false; audit-logs UPDATE .
- effectivePermissions(id) — findUserOrThrow then RolePermissions.forRole(user.getRole()).
- assignStores(id, storeIds) (@Transactional) — delegates to StoreAccessService.assignStores; audit-
logs UPDATE with count of stores.
- getAssignedStoreIds(id) — findUserOrThrow (404 guard) then
StoreAccessService.getAssignedStoreIds(id) .
- setCurrentStore(currentUserEmail, storeId) (@Transactional) — validates via
StoreAccessService.assertStoreAccess; audit-logs UPDATE under module "AUTH" .
- getAccessibleStores(currentUserEmail) — returns every store the caller may act on.
- resolveEmployee(employeeId, excludeUserId) (private) — if employeeId null, returns null; else looks up
Employee (404 MasterNotFoundException if missing); checks existsByEmployeeIdAndIdNot /
existsByEmployeeId depending on create vs update; throws BadRequestException("Employee '" + name + "'
is already linked to another user login") if already linked.
- guardNotLastActiveAdmin(user, message) (private) — exact condition: "Throws if user is the only
ACTIVE ADMIN in the system". If user.getStatus() != ACTIVE, returns (no-op — an already-inactive
admin can be freely changed). Else counts userRepository.countByRoleAndStatus(Role.ADMIN,
UserStatus.ACTIVE); if <= 1, throws BadRequestException(message) .
- findUserOrThrow(id) (private) — UserRepository.findById or throw UserNotFoundException(id) .

```

EmployeeService

```

- search(search, status, page, size) — sorted by name ascending; maps each Employee to
EmployeeResponse.fromEntity(e, userRepository.findById(e.getId()).orElse(null)) (N+1 reverse
lookup per row).
- getById(id) — findOrThrow + same reverse-lookup mapping.
- generateCode() — delegates entirely to EmployeeCodeGeneratorService.generateNext(); comment: "The
ONLY place an auto-generated employee code is produced (spec section 5) — a manually-entered code
bypasses this."
- create(request) (@Transactional) — three uniqueness checks in order: existsByEmployeeCodeIgnoreCase
→ BadRequestException("An employee with code '...' already exists"); existsByMobile →
BadRequestException("An employee with mobile number '...' already exists"); if email present,
existsByEmailIgnoreCase → BadRequestException("An employee with email '...' already exists").
Resolves optional City/State via cityService.findOrThrow / stateService.findOrThrow. Builds and saves;
audit-logs CREATE under module "MASTER".
- update(id, request) (@Transactional) — same three uniqueness checks but scoped with ...AndIdNot
and only re-checked when the value actually changed; audit-logs UPDATE .
- setStatus(id, status) (@Transactional) — comment: "Deactivating an employee never touches a
linked User account (spec section 33) — that is a separate, explicit action." Sets status, saves, audit-
logs UPDATE with old/new status values.
- assignStores(id, storeIds) (@Transactional) — comment: "purely informational (spec section 10),
distinct from StoreAccessService.assignStores which is the actual login access control on User."
Replace-all: employeeStoreRepository.deleteByEmployeeId(id) then re-inserts one EmployeeStore row per

```

id (404 MasterNotFoundException("Store", storeId) if a store doesn't exist); audit-logs UPDATE .

- getAssignedStoreIds(id) — findOrThrow then maps EmployeeStoreRepository.findById(id) to store ids.
- findOrThrow(id) (public) — EmployeeRepository.findById or throw MasterNotFoundException("Employee", id) .

EmployeeCodeGeneratorService — "The ONLY place an auto-generated employee code is produced ... Format: EMP-000001 ." generateNext() runs @Transactional(propagation = Propagation.REQUIRES_NEW) : calls sequenceRepository.ensureRowExists(1L) (native upsert-if-absent), then sequenceRepository.lockForUpdate(1L) (@Lock(PESSIMISTIC_WRITE)) — throws IllegalStateException if the row is somehow still missing. Loops: increments lastNumber, formats as "EMP-" + String.format("%06d", number), and re-checks employeeRepository.existsByEmployeeCodeIgnoreCase(candidate) in a do...while loop to skip any number a manually-typed code already occupies. Saves the sequence row and returns the candidate. Comment: "mirrors SkuGeneratorService exactly ... two simultaneous 'Generate Code' calls can never collide", and "A generated code is never reused even if the caller never saves an employee with it (gaps are fine, duplicates are not)."

Repository

UserRepository: findByEmail, existsByEmail, findByEmployeeId, existsByEmployeeId, existsByEmployeeIdAndIdNot, countByRoleAndStatus, and a JPQL @Query search(search, role, status, pageable) doing case-insensitive LIKE on firstName/lastName/email plus optional role/status equality filters.

EmployeeRepository: existsByEmployeeCodeIgnoreCase, existsByEmployeeCodeIgnoreCaseAndIdNot, existsByMobile, existsByMobileAndIdNot, existsByEmailIgnoreCase, existsByEmailIgnoreCaseAndIdNot, and a JPQL @Query search(search, status, pageable) doing case-insensitive LIKE on name/employeeCode and plain LIKE on mobile, plus optional status equality.

EmployeeCodeSequenceRepository: lockForUpdate(id) (@Lock(PESSIMISTIC_WRITE) JPQL select), ensureRowExists(id) (native INSERT ... ON DUPLICATE KEY UPDATE id = id, @Modifying).

EmployeeStoreRepository: findByEmployeeId(employeeId), deleteByEmployeeId(employeeId) .

5. Database

Tables

```
users (from User entity @Table(name = "users")):
| Column | Notes |
|---|---|
| id | PK, IDENTITY |
| first_name | nullable=false, length=50 |
| last_name | nullable=false, length=50 |
| email | nullable=false, unique=true, length=100 |
| mobile | nullable=false, length=15 |
| password | nullable=false (bcrypt hash) |
| role | enum string, nullable=false, length=20 |
| status | enum string, nullable=false, length=20 |
| employee_id | FK → employees.id, unique=true (nullable) |
| must_change_password | nullable=false, default false |
| last_login | nullable |
```

```
| current_store_id | FK → stores.id, nullable |
| created_at | nullable=false, updatable=false |
| updated_at | nullable=false |
```

```
employees (@Table(name = "employees")):
| Column | Notes |
|---|---|
| id | PK, IDENTITY |
| employee_code | nullable=false, unique=true, length=30 |
| name | nullable=false, length=100 |
| mobile | nullable=false, length=10 |
| email | length=100, nullable |
| designation | length=100, nullable |
| department | length=100, nullable |
| address | TEXT, nullable |
| city_id | FK → cities.id, nullable |
| state_id | FK → states.id, nullable |
| joining_date | nullable |
| status | enum string, nullable=false, length=20 |
| notes | TEXT, nullable |
| created_at | nullable=false, updatable=false |
| updated_at | nullable=false |
```

employee_code_sequence (@Table(name = "employee_code_sequence")): id (PK, always 1), last_number (nullable=false) — single global counter row.

```
employee_stores (@Table(name = "employee_stores", uniqueConstraints =
@UniqueConstraint(name="uk_employee_store", columnNames={"employee_id","store_id"})): id (PK,
IDENTITY), employee_id (FK → employees.id, nullable=false), store_id (FK → stores.id,
nullable=false), created_at (nullable=false, updatable=false).
```

```
user_stores (referenced dependency, @Table(name = "user_stores", uniqueConstraints =
@UniqueConstraint(name="uk_user_store", columnNames={"user_id","store_id"})): id, user_id (FK →
users.id), store_id (FK → stores.id), created_at.
```

Relationships

- `User.employee` → `Employee` (@ManyToOne, `employee_id` unique): at most one User per Employee, enforced by the DB unique constraint **and** pre-checked in `UserService.resolveEmployee` for a clean 400 instead of a raw DB constraint violation.
- `Employee` → `User`: no FK on the `Employee` side; the reverse link is resolved at read time via `UserRepository.findById(employeeId)`.
- `User.currentStore` → `Store` (@ManyToOne, `current_store_id`, nullable): the store the user is currently working in; validated through `StoreAccessService` before being set, never trusted directly from a request.
- `User` ↔ `Store` via `UserStore` (`user_stores`): explicit ASSIGNED_STORES access rows, managed by `StoreAccessService.assignStores` / `UserService.assignStores`. This is the actual, backend-enforced login access control.
- `Employee` ↔ `Store` via `EmployeeStore` (`employee_stores`): purely informational tagging of which store(s) an employee is associated with, managed by `EmployeeService.assignStores` — explicitly **not** an access-control mechanism (see code comment on `EmployeeService.assignStores`).
- `Employee.city` / `Employee.state` → `City` / `State` (Master module dependency, not documented in depth here).

6. Validation

- Mobile (User and Employee): `^[0-9]{10}$`, message "Mobile number must be 10 digits" — applied to `UserCreateRequest.mobile`, `UserUpdateRequest.mobile`, `EmployeeRequest.mobile`.
- Email: `@Email` (`UserCreateRequest.email`, `UserUpdateRequest.email` both also `@NotBlank`; `EmployeeRequest.email` is `@Email` only — optional).
- Password: `@Size(min = 6)` on `UserCreateRequest.password`, `AdminPasswordResetRequest.newPassword`, `PasswordChangeRequest.newPassword`.
- Required fields (`@NotBlank` / `@NotNull`): first/last name, email, mobile, role (create/update), status (`UserUpdateRequest`, `UserStatusUpdateRequest`), employeeCode/name/mobile (`EmployeeRequest`), storeIds (`AssignStoresRequest`, allows empty array but not null), currentPassword/newPassword/confirmPassword (`PasswordChangeRequest`), newPassword/confirmPassword (`AdminPasswordResetRequest`).
- Cross-field password match: `newPassword.equals(confirmPassword)` checked manually in `UserService.adminResetPassword` and `changeOwnPassword` (not a bean-validation annotation) — throws `BadRequestException("New password and confirm password do not match")`.
- Uniqueness (service-layer, not annotation-based): `User.email` unique (`DuplicateEmailException`); `Employee.employeeCode`, `Employee.mobile`, `Employee.email` all unique (`BadRequestException` with specific messages); one User per Employee (`BadRequestException`).
- Bean Validation triggers `MethodArgumentNotValidException`, handled globally.

7. Authentication / Authorization

Endpoint	Requirement
All <code>/api/users/**</code> except <code>me/password</code>	<code>hasRole('ADMIN')</code> (class-level <code>@PreAuthorize</code> on <code>UserController</code>)
<code>PUT /api/users/me/password</code>	<code>isAuthenticated()</code> (method-level override — any logged-in role)
<code>GET/POST /api/masters/employees</code> , <code>GET /api/masters/employees/{id}</code>	No explicit <code>@PreAuthorize</code> (reachable by any authenticated caller passing the global security filter chain)
<code>POST /api/masters/employees/generate-code</code> , <code>POST /api/masters/employees</code>	<code>hasAuthority('PERM_EMPLOYEE_CREATE')</code>
<code>PUT /api/masters/employees/{id}</code> , <code>PATCH ../activate</code> , <code>PATCH ../deactivate</code> , <code>PUT ../{id}/stores</code>	<code>hasAuthority('PERM_EMPLOYEE_EDIT')</code>
<code>GET /api/masters/employees/{id}/stores</code>	<code>hasAuthority('PERM_EMPLOYEE_VIEW')</code>

Depends on the Authentication & Authorization module for `UserPrincipal`, JWT filter, `Role/Permission` enums and `RolePermissions` — referenced here, not re-documented.

8. Permissions

Relevant `Permission` enum values (module "Security" unless noted) and which roles hold them per `RolePermissions.build()`:

Permission	ADMIN	STORE_MANAGER	ACCOUNTANT	SALES_USER	PURCHASE_USER	INVENTORY_USER	ST
<code>USER_VIEW</code> / <code>USER_CREATE</code> / <code>USER_EDIT</code> / <code>USER_DEACTIVATE</code>	yes	no (explicitly removed)	no	no	no	no	no

Permission	ADMIN	STORE_MANAGER	ACCOUNTANT	SALES_USER	PURCHASE_USER	INVENTORY_USER	STAFF
EMPLOYEE_VIEW (module "Master")	yes	yes	yes	no	no	no	no
EMPLOYEE_CREATE / EMPLOYEE_EDIT (module "Master")	yes	yes	no	no	no	no	no
STORE_ASSIGN	yes	no (explicitly removed)	no	no	no	no	no
STORE_ACCESS_ALL	yes	no (explicitly removed)	no	no	no	no	no
ROLE_VIEW / ROLE_MANAGE / PERMISSION_VIEW	yes	no	no	no	no	no	no
AUDIT_VIEW	yes	no (explicitly removed)	no	no	no	no	no

(ADMIN holds `EnumSet.allOf(Permission.class)`; STORE_MANAGER is a copy of ADMIN's set with USER_, ROLE_, PERMISSION_VIEW, FY_MANAGE, AUDIT_VIEW, GST_CONFIG, and STORE_CREATE/STORE_EDIT/STORE_ASSIGN/STORE_ACCESS_ALL explicitly removed. ACCOUNTANT/SALES_USER/PURCHASE_USER/INVENTORY_USER/STAFF are each an explicit `EnumSet.of(...)` that includes EMPLOYEE_VIEW only for ACCOUNTANT among the non-admin/non-manager roles, per the table's EMPLOYEE_VIEW list above — SALES_USER/PURCHASE_USER/INVENTORY_USER/STAFF do not include EMPLOYEE_VIEW.)

9. Transaction Handling

- UserService: `createUser`, `updateUser`, `updateStatus`, `adminResetPassword`, `changeOwnPassword`, `assignStores`, `setCurrentStore` are all `@Transactional` (default propagation). Read methods (`getUsers`, `getUserById`, `effectivePermissions`, `getAssignedStoreIds`, `getAccessibleStores`) are not annotated.
- EmployeeService: `create`, `update`, `setStatus`, `assignStores` are `@Transactional`. `search`, `getById`, `generateCode`, `getAssignedStoreIds`, `findOrThrow` are not.
- EmployeeCodeGeneratorService.`generateNext()` runs `@Transactional(propagation = Propagation.REQUIRES_NEW)` deliberately — code comment: "running in its own *REQUIRES_NEW* transaction so the single counter row is locked/released independently of the caller's transaction and two simultaneous 'Generate Code' calls can never collide."
- AuditService.`log(...)` uses the **default** propagation (joins the caller's transaction), not *REQUIRES_NEW*, per its class comment: "an audit entry for an action that later rolls back should roll back with it." A logging failure itself is caught internally and only logged (`log.warn()`), never rethrown, so it cannot fail the surrounding business transaction.

10. Error Handling

All handled centrally by `GlobalExceptionHandler` (`@RestControllerAdvice`), building a uniform `ApiError` { `timestamp`, `status`, `error`, `message`, `path`, `fieldErrors` } (via `buildResponse`):

Exception	HTTP Status
<code>MethodArgumentNotValidException</code> (bean validation)	400, <code>message="Validation failed"</code> , <code>fieldErrors</code> populated <code>field→message</code> from <code>ex.getBindingResult().getFieldErrors()</code>

Exception	HTTP Status
DuplicateEmailException	409 Conflict
UserNotFoundException	404
MasterNotFoundException (covers Employee/Store/City/State not-found in this module)	404
BadRequestException (admin-safety guard, password mismatch, uniqueness on Employee, employee-already-linked, current-password-incorrect, etc.)	400
AccessDeniedException (e.g. StoreAccessService.assertStoreAccess)	403, fixed message "You do not have permission to access this resource"
Any other Exception	500, "An unexpected error occurred", full stack trace logged server-side

ApiError uses @JsonInclude(JsonInclude.Include.NON_NULL), so fieldErrors is omitted from the JSON entirely when null (only present on validation failures).

11. Audit Flow

Every mutation in this module writes through AuditService.log(...). Exact call sites:

UserService.createUser :

```
auditService.log(AuditAction.CREATE, "ADMIN", "User", saved.getId(), null,
    null, null, "User " + saved.getEmail() + " created with role " + saved.getRole());
```

UserService.updateUser (role changed):

```
auditService.log(AuditAction.PERMISSION_CHANGE, "ADMIN", "User", saved.getId(), null,
    oldRole, saved.getRole().name(), "User " + saved.getEmail() + " role changed from " +
    oldRole + " to " + saved.getRole());
```

(role unchanged):

```
auditService.log(AuditAction.UPDATE, "ADMIN", "User", saved.getId(), null,
    null, null, "User " + saved.getEmail() + " updated");
```

UserService.updateStatus :

```
auditService.log(AuditAction.UPDATE, "ADMIN", "User", saved.getId(), null,
    oldStatus.name(), saved.getStatus().name(), "User " + saved.getEmail() + " status changed to
    " + saved.getStatus());
```

UserService.adminResetPassword :

```
// Never log the password itself – only that a reset happened.
auditService.log(AuditAction.UPDATE, "ADMIN", "User", saved.getId(), null,
    null, null, "Password reset for user " + saved.getEmail() + " by an administrator");
```

UserService.changeOwnPassword :

```
auditService.log(AuditAction.UPDATE, "ADMIN", "User", user.getId(), null,
    null, null, "User " + user.getEmail() + " changed their own password");
```

UserService.assignStores :

```
auditService.log(AuditAction.UPDATE, "ADMIN", "User", id, null,
    null, null, "Store access for user " + user.getEmail() + " set to " + storeIds.size() + "
store(s)");
```

UserService.setCurrentStore :

```
auditService.log(AuditAction.UPDATE, "AUTH", "User", saved.getId(), store.getStoreCode(),
    null, null, "User " + saved.getEmail() + " switched current store to " +
store.getStoreCode());
```

EmployeeService.create :

```
auditService.log(AuditAction.CREATE, "MASTER", "Employee", saved.getId(), saved.getEmployeeCode(),
    null, null, "Employee " + saved.getEmployeeCode() + " (" + saved.getName() + ") created");
```

EmployeeService.update :

```
auditService.log(AuditAction.UPDATE, "MASTER", "Employee", saved.getId(), saved.getEmployeeCode(),
    null, null, "Employee " + saved.getEmployeeCode() + " updated");
```

EmployeeService.setStatus :

```
auditService.log(AuditAction.UPDATE, "MASTER", "Employee", saved.getId(), saved.getEmployeeCode(),
    oldStatus.name(), status.name(), "Employee " + saved.getEmployeeCode() + " status changed to
" + status);
```

EmployeeService.assignStores :

```
auditService.log(AuditAction.UPDATE, "MASTER", "Employee", id, employee.getEmployeeCode(),
    null, null, "Store assignment for employee " + employee.getEmployeeCode() + " set to " +
storeIds.size() + " store(s)");
```

All entries are persisted via `AuditLogRepository`, capturing `userId/username` of the actor (via `SecurityUtil.currentUserOrNull()`), `action`, `module`, `entityType`, `entityId`, optional `store`, `documentNumber`, `oldValue/newValue`, `ipAddress` (from `X-Forwarded-For` or `remoteAddr`), `userAgent`, and `description`.

12. Important Side Effects

- **Password hashing:** every password write (`createUser`, `adminResetPassword`, `changeOwnPassword`) runs through `passwordEncoder.encode(...)` — plaintext is never stored.
- **mustChangePassword flag:** set `true` by `adminResetPassword` (forces the user to change it at next login); set `false` by `changeOwnPassword` once the user sets their own new password. Not touched by `createUser/updateUser` (defaults to `false` via `@Builder.Default`).
- **Employee↔User link enforcement:** `UserService.resolveEmployee` checks `existsByEmployeeId/existsByEmployeeIdAndIdNot` before allowing a link — prevents two `User`s pointing at the same

`Employee`. The reverse direction (`Employee → User`) is never stored, only computed at read time via `UserRepository.findById`.

- **LastLogin**: field exists on `User` but is set outside this module (in the Authentication module's login flow) — NOT FOUND IN CURRENT CODEBASE within the files read for this module.
- **Admin-safety (last active ADMIN)**: enforced in `UserService.guardNotLastActiveAdmin`, triggered from `updateUser` (role change away from ADMIN, or status change to INACTIVE while role is ADMIN) and `updateStatus` (status change to INACTIVE while role is ADMIN). Only counts/blocks when the target user is currently ACTIVE; an already- INACTIVE admin can be freely edited.
- **Employee deactivation never cascades to the linked User**: explicit code comment on `EmployeeService.setStatus` — deactivating an `Employee` leaves any linked `User` login untouched; the two must be deactivated independently.
- **Replace-all store assignment**: both `StoreAccessService.assignStores` (`User→Store`, login access) and `EmployeeService.assignStores` (`Employee→Store`, informational) delete all existing rows for the id and re-insert from the given set — no incremental add/remove semantics.
- **Stale current-store cleanup**: `StoreAccessService.assignStores` clears `user.currentStore` if the new assignment no longer includes it (and the user isn't ALL_STORES).
- **Employee code collision guard**: `EmployeeCodeGeneratorService.generateNext()` loops past any number a manually-entered code already occupies, so a hand-typed `EMP-000005` can never collide with a subsequently generated code.

13. Dependencies on Other Modules

- **Authentication & Authorization module**: `Role`, `Permission`, `RolePermissions`, `UserPrincipal`, `SecurityUtil`, `PermissionService`, JWT filter. This module reads `Role/Permission` and calls `RolePermissions.forRole/RolePermissions.has`, but does not redefine or re-document that mapping's internals beyond what's listed in section 8.
- **Store module**: `Store`, `StoreRepository`, `StoreService`, `StoreAccessService`, `UserStore/StoreRepository`. `UserService` delegates all store-access-list logic to `StoreAccessService` (`assignStores`, `getAssignedStoreIds`, `assertStoreAccess`, `getAccessibleStoreIds`) rather than reimplementing it. `User.currentStore` and `EmployeeStore` also depend on `Store`.
- **City/State masters**: `EmployeeService` depends on `CityService.findOrThrow` / `StateService.findOrThrow` to resolve `Employee.city/Employee.state`.
- **Audit module**: `AuditService`, `AuditAction`, `AuditLogRepository` — every mutation logs through here (section 11).

14. Key Operation Flows

Create User: `UserFormModal` (mode add) → `Users.tsx handleFormSubmit` → `userApi.create(payload)` → `POST /api/users` → `UserController.createUser (@Valid UserCreateRequest)` → `UserService.createUser` (checks `existsByEmail`, resolves optional `employeeId` via `resolveEmployee`, encodes password, builds `User`, `userRepository.save`, `auditService.log(CREATE,...)`) → `UserRepository.save` → `users` table INSERT → `UserResponse.fromEntity(saved)` → 201 JSON → `Users.tsx` toasts success, closes modal, reloads list.

Edit User: `UserFormModal` (mode edit, pre-filled) → `userApi.update(id, payload)` → `PUT /api/users/{id}` → `UserController.updateUser (@Valid UserUpdateRequest)` → `UserService.updateUser` (email-change/duplicate check, `resolveEmployee` excluding self, admin-safety guards on role/status change, field updates, `save`, conditional `PERMISSION_CHANGE/UPDATE` audit log) → `users` table UPDATE → `UserResponse` → UI toast + reload.

Deactivate/Reactivate User: dropdown action → `toggleStatus` → `userApi.updateStatus(id, {status})` → `PATCH /api/users/{id}/status` → `UserController.updateStatus (@Valid UserStatusUpdateRequest)` →

`UserService.updateStatus` (admin-safety guard if `role==ADMIN && status==INACTIVE`, sets status, save, UPDATE audit log with old/new status) → `users` table UPDATE → `UserResponse` → UI toast + reload.

Admin Password Reset: `ResetPasswordDialog` → `userApi.resetPassword(id, {newPassword, confirmPassword})` → `POST /api/users/{id}/reset-password` → `UserController.resetPassword (@Valid AdminPasswordResetRequest)` → `UserService.adminResetPassword` (match check, `findUserOrThrow`, `passwordEncoder.encode`, `mustChangePassword=true`, save, UPDATE audit log without the password) → `users` table UPDATE → `UserResponse` → UI toast telling admin the user must change it at next login.

Self Password Change: caller not found among the read frontend files (`userApi.changeOwnPassword` exists but has no known UI trigger in this module's file set) → `PUT /api/users/me/password` (`isAuthenticated()`) → `UserController.changeOwnPassword (@AuthenticationPrincipal UserPrincipal, @Valid PasswordChangeRequest)` → `UserService.changeOwnPassword` (match check, `findByEmail`, `passwordEncoder.matches(currentPassword,...)` check, encode new password, `mustChangePassword=false`, save, UPDATE audit log) → `users` table UPDATE → 204 No Content.

Create Employee: `EmployeeMaster.tsx` (via `MasterCrudPage`, optionally pre-filled via "Generate Code" → `employeeApi.generateCode()` → `POST /api/masters/employees/generate-code` → `EmployeeController.generateCode (PERM_EMPLOYEE_CREATE)` → `EmployeeService.generateCode()` → `EmployeeCodeGeneratorService.generateNext()`, its own `REQUIRES_NEW` transaction, row-locks `employee_code_sequence`, increments, dedupes against `employees.employee_code`) → form submit → `employeeApi.create(payload)` → `POST /api/masters/employees` → `EmployeeController.create (@Valid EmployeeRequest, PERM_EMPLOYEE_CREATE)` → `EmployeeService.create` (uniqueness checks on code/mobile/email, resolves City/State, builds and saves Employee, CREATE audit log) → `employees` table INSERT → `EmployeeResponse.fromEntity(saved)` → 201 JSON → UI toast + list reload.

Edit Employee: `MasterCrudPage` edit form (`toFormValues` maps `Employee` → `EmployeePayload`) → `employeeApi.update(id, payload)` → `PUT /api/masters/employees/{id}` (`PERM_EMPLOYEE_EDIT`) → `EmployeeService.update` (uniqueness re-checks only for changed code/mobile/email, resolves City/State, field updates, save, UPDATE audit log) → `employees` table UPDATE → `EmployeeResponse` → UI reload.

Deactivate Employee: `MasterCrudPage` deactivate action → `employeeApi.deactivate(id)` → `PATCH /api/masters/employees/{id}/deactivate` (`PERM_EMPLOYEE_EDIT`) → `EmployeeController.deactivate` → `EmployeeService.setStatus(id, EmployeeStatus.INACTIVE)` (no cascade to any linked User; UPDATE audit log with old/new status) → `employees` table UPDATE → `EmployeeResponse.fromEntity(saved, userRepository.findById(id).orElse(null))` → UI reload.

15. Manual Changes

- **Add a new field to User:** add the `@Column` to `backend/src/main/java/com/storehub/entity/User.java`; add it to `UserCreateRequest`/`UserUpdateRequest` (with any `jakarta.validation` annotations needed) and to `UserResponse` (+ `fromEntity` mapping); a DB migration is expected wherever this project's schema migrations live (not read in this module — check the migration tool/folder used by the Store/Masters modules for the convention, e.g. Flyway/Liquibase under `backend/src/main/resources`); update `UserFormModal.tsx` (`UserFormValues`, form fields) and `frontend/src/types/user.ts` (`User`, `UserCreatePayload`, `UserUpdatePayload`); if the field should render in the read-only view, also update `UserViewModal.tsx`. If it participates in the users search/filter, extend `UserRepository.search` and `UserController.getUsers`/`UserService.getUsers`.
- **Add a new field to Employee:** same pattern against `Employee.java`, `EmployeeRequest`/`EmployeeResponse` (+ `fromEntity`), a DB migration, and `EmployeeMaster.tsx` (`EMPTY`, `toFormValues`,

`renderForm, renderView`) plus `frontend/src/types/masters.ts` (`Employee`, `EmployeePayload`). If it should be searchable, extend `EmployeeRepository.search`.

- **Change password rules:** minimum length is `@Size(min = 6)` on `UserCreateRequest.password`, `AdminPasswordResetRequest.newPassword`, `PasswordChangeRequest.newPassword` — change all three together to keep create/admin-reset/self-change consistent. The confirm-password match check is hand-written in `UserService.adminResetPassword` and `UserService.changeOwnPassword` (not a bean-validation annotation) — update both if match logic changes. Hashing algorithm is whatever `PasswordEncoder` bean is configured (Authentication module) — not overridden here.
- **Change `employeeCode` generation format:** the only place is `EmployeeCodeGeneratorService.format(long number)` (`PREFIX = "EMP-" + %06d`). Changing the prefix or digit-width there is sufficient — `EmployeeService.generateCode()` and the frontend's "Generate Code" button need no changes. Be aware the `do...while` uniqueness loop in `generateNext()` depends on `EmployeeRepository.existsByEmployeeCodeIgnoreCase`, so any format change should keep candidates checkable by that same method.
- **Add a new admin-safety rule** (beyond "never leave zero active ADMINS"): add the new guard inside `UserService` alongside `guardNotLastActiveAdmin`, and call it from every user-mutating method that could violate it (`updateUser`, `updateStatus`, and any new endpoint that changes role/status) — the codebase's existing pattern is to enforce this in the service layer, not the controller, so a single guard method covers all admin-facing endpoints. Also consider whether `assignStores` / store-access changes should be constrained similarly (currently they are not).
- **Modules affected by any User/Employee schema change:** Authentication module (`User.role/status` feed `UserPrincipal` /JWT claims), Store module (`User.currentStore`, `UserStore`, `EmployeeStore` all FK into `users/employees`), Audit module (every mutation here writes an `AuditLog` row referencing `entityType "User" / "Employee"` and `entityId`), and the Masters dashboard/navigation that lists `EmployeeMaster.tsx` under `/masters/employees` (see `frontend/src/pages/MastersDashboard.tsx`, `App.tsx` route table — not modified as part of this documentation task).

Store / Multi-Branch Module

1. Overview

The Store (aka Branch/Warehouse) module gives StoreHub multi-location support. It is deliberately **not** a new top-level module — per `StoreController`'s own doc comment, it "lives under Master → Store/Branch" (Multi-Store spec section 1/4/68). A `Store` (`backend/src/main/java/com/storehub/entity/Store.java`) represents an operational location — `StoreType` is `STORE`, `WAREHOUSE`, or `HEAD_OFFICE` — that every store-attributed transaction (Sale, Purchase, Inventory, Expense, CashTransaction, etc.) is recorded against.

ALL_STORES vs ASSIGNED_STORES authorization model (from `StoreAccessService`'s class doc, spec section 11): every user has exactly one of two access modes, decided centrally and never re-derived ad hoc in a controller:

- **ALL_STORES** — granted by the `Permission.STORE_ACCESS_ALL` permission (ADMIN holds it by default; see `RolePermissions`). Such a user can act on every store, active or inactive.
- **ASSIGNED_STORES** — backed by explicit `UserStore` join rows (one row per user-store pair). A user with neither `STORE_ACCESS_ALL` nor any `UserStore` rows has **zero** store access.

`StoreAccessService` is explicitly called out as "the single centralized store-authorization service" so that store-access checks are never scattered across dozens of controllers (mirroring `PermissionService`'s role for permissions), and it "never trusts a frontend-supplied storeId" — every requested store id is validated against one of the two access sources before use.

Default Store historical-data anchor. `StoreService.DEFAULT_STORE_CODE = "MAIN"` is the store code of an auto-created "Main Store" (`StoreService.getOrCreateDefaultStore()`), described as "the single anchor store that pre-Multi-Store historical data (Sales/Purchases/Inventory recorded before Stores existed) is backfilled onto, and the fallback every store-aware flow uses until a real store is selected" (spec section 76). It is idempotent (created once, found by `storeCodeIgnoreCase` thereafter) and is also the fallback `StoreAccessService.resolveEffectiveStoreId` and `InventoryService.applyMovement` (overloaded, no-storeId variant) use when there is no authenticated user in context (a system-triggered call).

Stock Transfer workflow states. `StockTransferStatus` (`DRAFT` → `APPROVED` → `DISPATCHED` → `RECEIVED`, or `CANCELLED` from `DRAFT / APPROVED`) models a store-to-store movement. Per `StockTransferService`'s class doc: stock only ever moves through the same centralized `InventoryService.applyMovement` every other store-attributed voucher uses — a `TRANSFER_OUT` deduction from the source store at `DISPATCHED`, and a `TRANSFER_IN` addition to the destination store at `RECEIVED`. `DRAFT / APPROVED` are "purely an authorization stage with no stock impact." Cancelling after `DISPATCHED` is rejected because it "would need a receive-back movement, which is out of scope for this workflow."

2. Frontend

Files and Components

File	Purpose
<code>frontend/src/pages/masters/StoreMaster.tsx</code>	CRUD page for Store master (list/create/edit/activate/deactivate), built on the generic <code>MasterCrudPage</code> .
<code>frontend/src/components/layout/StoreSwitcher.tsx</code>	Topbar dropdown to switch <code>currentStore</code> ; renders <code>null</code> when <code>myStores.length <= 1</code> .

File	Purpose
frontend/src/components/StoreAccessDialog.tsx	ADMIN dialog to assign a user's ASSIGNED_STORES list (checkbox list of active stores).
frontend/src/pages/inventory/StockTransfers.tsx	List/search stock transfers, with row actions (Approve/Dispatch/Receive/Cancel) gated by hasPermission.
frontend/src/pages/inventory/StockTransferForm.tsx	Create a new stock transfer (DRAFT).
frontend/src/pages/inventory/StockTransferDetail.tsx	View one transfer, run lifecycle actions, and see a status timeline.
frontend/src/pages/accounting/reports/StoreComparison.tsx	Store Comparison report — sales/purchases/stock KPIs per store for a date range.
frontend/src/api/storeApi.ts	Axios wrapper for /api/masters/stores*.
frontend/src/api/stockTransferApi.ts	Axios wrapper for /api/stock-transfers*.
frontend/src/api/storeComparisonApi.ts	Axios wrapper for /api/reports/store-comparison.
frontend/src/types/store.ts	Store, StorePayload, StoreGstContext, StoreType types.
frontend/src/types/stockTransfer.ts	StockTransfer, StockTransferItem, StockTransferStatus, payload types.
frontend/src/types/storeComparison.ts	StoreComparisonRow, StoreComparisonResponse types.
frontend/src/context/AuthContext.tsx	Holds myStores, switchStore(), and the single-store auto-select-on-login effect (store-relevant parts only — see Auth module doc for the rest).
frontend/src/api/userApi.ts	getAssignedStores(id) / assignStores(id, storeIds) used by StoreAccessDialog.

Frontend Flow

Store CRUD (StoreMaster.tsx): uses the shared MasterCrudPage<Store, StorePayload> component. The form has two tabs ("Basic Info", "Location & GST"). A "Generate Code" button next to Store Code calls storeApi.generateCode() and fills the field with the backend-generated STR-000001-style code; the field remains freely editable (manual entry bypasses the generator). GSTIN input uppercases on change. Field-level errors from the backend (fieldErrors) are surfaced per-field via invalid/inline <p> messages (matches the CLAUDE.md pattern). Deactivate/Activate call PATCH ../activate|deactivate instead of a delete (Store is never hard-deleted).

Store switching: StoreSwitcher.tsx reads myStores / user.currentStoreId / switchStore from AuthContext. It renders nothing if the user has 0–1 accessible stores. Clicking a store in the dropdown calls switchStore(storeId) → authApi.setCurrentStore(storeId) → PUT /api/auth/current-store; on success the returned UserResponse (with updated currentStoreId / currentStoreName) replaces user in context and localStorage.

Store access assignment: StoreAccessDialog (opened from the Users admin page, not itself listed as a route file here) loads all ACTIVE stores plus the target user's currently assigned store ids (userApi.getAssignedStores), lets the ADMIN toggle checkboxes, and on Save calls userApi.assignStores(user.id, Array.from(selected)) — replace-all semantics.

Stock transfer create: StockTransferForm.tsx preselects fromStoreId from user.currentStoreId if set. Client-side validate() checks: fromStore required, toStore required, fromStore ≠ toStore, at least one item, every item with a productId must have quantity > 0. On submit, calls

`stockTransferApi.create(...)`; on success navigates to the detail page. Backend `fieldErrors` populate per-field messages (`fromStoreId` , `toStoreId` , `transferDate` , etc.) alongside a top-level `error` alert.

Stock transfer approve/dispatch/receive/cancel: Both `StockTransfers.tsx` (row dropdown) and `StockTransferDetail.tsx` (page-level buttons) gate each action on both the transfer's current `status` (client-side state machine mirror) and the matching `hasPermission('STOCK_TRANSFER_*')` check, then call the corresponding `stockTransferApi` method and reload. Cancel goes through a `ConfirmDialog` first in the detail page (list page cancels directly from the dropdown without a confirm step).

Store comparison report viewing: `StoreComparison.tsx` defaults `fromDate` / `toDate` to first-of-month/ today, fetches on mount and on "Apply", and renders per-store KPI cards (stores compared, total sales, total purchases, total stock units — summed client-side from the row list) plus a full table (Sales, Sales Count, Purchases, Purchase Count, Stock Units, Low Stock, Out of Stock), with badge highlighting for non-zero Low Stock (`warning`) / Out of Stock (`destructive`).

3. API Calls

Function	HTTP Method	URL	Request shape	Response shape
<code>storeApi.list</code>	GET	<code>/api/masters/stores</code>	query: <code>search?</code> , <code>status?</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<StoreResponse></code>
<code>storeApi.getById</code>	GET	<code>/api/masters/stores/{id}</code>	—	<code>StoreResponse</code>
<code>storeApi.generateCode</code>	POST	<code>/api/masters/stores/generate-code</code>	—	<code>GenerateStoreCodeResponse { storeCode }</code>
<code>storeApi.create</code>	POST	<code>/api/masters/stores</code>	<code>StoreRequest</code>	<code>StoreResponse</code> (201)
<code>storeApi.update</code>	PUT	<code>/api/masters/stores/{id}</code>	<code>StoreRequest</code>	<code>StoreResponse</code>
<code>storeApi.activate</code>	PATCH	<code>/api/masters/stores/{id}/activate</code>	—	<code>StoreResponse</code>
<code>storeApi.deactivate</code>	PATCH	<code>/api/masters/stores/{id}/deactivate</code>	—	<code>StoreResponse</code>
<code>storeApi.gstContext</code>	GET	<code>/api/masters/stores/{id}/gst-context</code>	—	<code>StoreGstContextResponse</code>

Function	HTTP Method	URL	Request shape	Response shape
authApi.getMyStores (used by AuthContext)	GET	/api/auth/my-stores	—	StoreResponse[]
authApi.setCurrentStore (used by AuthContext / switchStore)	PUT	/api/auth/current-store	SetCurrentStoreRequest { storeId }	UserResponse
userApi.getAssignedStores	GET	/api/users/{id}/stores	—	number[] (store ids)
userApi.assignStores	PUT	/api/users/{id}/stores	AssignStoresRequest { storeIds: number[] }	UserResponse
employeeApi.getAssignedStores (Employee module, same pattern)	GET	/api/employees/{id}/stores	—	number[]
employeeApi.assignStores	PUT	/api/employees/{id}/stores	AssignStoresRequest { storeIds }	EmployeeResponse
stockTransferApi.list	GET	/api/stock-transfers	query: search?, status?, fromDate?, toDate?, storeId?, page, size	PagedResponse<StockTransferResponse>
stockTransferApi.getById	GET	/api/stock-transfers/{id}	—	StockTransferResponse
stockTransferApi.create	POST	/api/stock-transfers	StockTransferRequest	StockTransferResponse (201)
stockTransferApi.approve	PATCH	/api/stock-transfers/{id}/approve	—	StockTransferResponse
stockTransferApi.dispatch	PATCH	/api/stock-transfers/{id}/dispatch	—	StockTransferResponse
stockTransferApi.receive	PATCH	/api/stock-transfers/{id}/receive	—	StockTransferResponse

Function	HTTP Method	URL	Request shape	Response shape
stockTransferApi.cancel	PATCH	/api/stock-transfers/{id}/cancel	—	StockTransferResponse
storeComparisonApi.compare	GET	/api/reports/store-comparison	query: fromDate, toDate (ISO dates)	StoreComparisonResponse

4. Backend

Controllers

StoreController (/api/masters/stores) — GET endpoints unguarded (doc comment: "every authenticated user needs to read the store list to populate a store picker"), mutations require `STORE_*` permissions:

- GET / — search(search?, status?, page=0, size=10) — no @PreAuthorize.
- GET /{id} — getById(id) — no @PreAuthorize.
- GET /{id}/gst-context — gstContext(id) — no @PreAuthorize.
- POST /generate-code — @PreAuthorize("hasAuthority('PERM_STORE_CREATE')").
- POST / — create(StoreRequest) — @PreAuthorize("hasAuthority('PERM_STORE_CREATE')") — 201.
- PUT /{id} — update(id, StoreRequest) — @PreAuthorize("hasAuthority('PERM_STORE_EDIT')").
- PATCH /{id}/activate — @PreAuthorize("hasAuthority('PERM_STORE_EDIT')").
- PATCH /{id}/deactivate — @PreAuthorize("hasAuthority('PERM_STORE_EDIT')").

StockTransferController (/api/stock-transfers):

- GET / — search(search?, status?, fromDate?, toDate?, storeId?, page=0, size=20) — @PreAuthorize("hasAuthority('PERM_STOCK_TRANSFER_VIEW')").
- GET /{id} — getById(id) — @PreAuthorize("hasAuthority('PERM_STOCK_TRANSFER_VIEW')").
- POST / — create(StockTransferRequest) — @PreAuthorize("hasAuthority('PERM_STOCK_TRANSFER_CREATE')") — 201.
- PATCH /{id}/approve — @PreAuthorize("hasAuthority('PERM_STOCK_TRANSFER_APPROVE')").
- PATCH /{id}/dispatch — @PreAuthorize("hasAuthority('PERM_STOCK_TRANSFER_DISPATCH')").
- PATCH /{id}/receive — @PreAuthorize("hasAuthority('PERM_STOCK_TRANSFER_RECEIVE')").
- PATCH /{id}/cancel — @PreAuthorize("hasAuthority('PERM_STOCK_TRANSFER_CANCEL')").

StoreComparisonController (/api/reports/store-comparison):

- GET / — compare(fromDate, toDate) — no @PreAuthorize; class doc: "every authenticated user may call this; the rows returned are already scoped to their own accessible stores."

Related endpoints in other controllers (store-assignment surface, listed here since they are part of this module's flow):

- UserController (/api/users, class-level @PreAuthorize("hasRole('ADMIN')")): GET /{id}/stores → getAssignedStores; PUT /{id}/stores → assignStores(id, AssignStoresRequest). Both inherit the class-level ADMIN-only guard; no additional method-level @PreAuthorize (i.e. not gated on PERM_STORE_ASSIGN specifically, only on the ADMIN role via the class guard).
- EmployeeController (/api/employees): GET /{id}/stores → getAssignedStores (no method-level @PreAuthorize shown for the GET); PUT /{id}/stores → assignStores — @PreAuthorize("hasAuthority('PERM_EMPLOYEE_EDIT')").

- AuthController (/api/auth): GET /my-stores → getMyStores (no @PreAuthorize, any authenticated user); PUT /current-store → setCurrentStore(SetCurrentStoreRequest) (no @PreAuthorize, any authenticated user — self-service, backend-validated via StoreAccessService.assertStoreAccess inside UserService.setCurrentStore).

DTOs

DTO	Fields	Validation
StoreRequest	storeCode, storeName, storeType, legalName, address, countryId, stateId, cityId, zoneId, pincode, phone, email, gstin	@NotBlank storeCode, @NotBlank storeName, @NotNull storeType, @Email email, @Pattern on gstin: ^\$\\^[0-9]{2}[A-Za-z]{5}[0-9]{4}[A-Za-z]{1}[1-9A-Za-z]{1}Z[0-9A-Za-z]{1}\$ (blank or valid 15-char GSTIN).
StoreResponse	id, storeCode, storeName, storeType, legalName, address, countryId, countryName, stateId, stateName, stateCode, cityId, cityName, zoneId, zoneName, pincode, phone, email, gstin, status, createdAt, updatedAt	Built via StoreResponse.fromEntity(Store).
GenerateStoreCodeResponse	storeCode	none (pure output DTO).
StoreGstContextResponse	storeId, gstin, legalName, stateCode, source ("STORE" or "BUSINESS")	none.
SetCurrentStoreRequest	storeId	@NotNull(message = "storeId is required").
AssignStoresRequest	storeIds: Set<Long>	@NotNull(message = "storeIds is required (use an empty array to clear all store access)").
StockTransferItemRequest	productId, quantity, notes	@NotNull productId, @NotNull @Min(1) quantity, notes unvalidated.
StockTransferItemResponse	id, productId, productName, sku, quantity, notes	Built via fromEntity(StockTransferItem).
StockTransferRequest	transferDate, fromStoreId, toStoreId, remarks, items: List<StockTransferItemRequest>	@NotNull transferDate, @NotNull fromStoreId, @NotNull toStoreId, @NotEmpty @Valid items (cascades into item-level validation).
StockTransferResponse	id, transferNumber, transferDate, fromStoreId, fromStoreName, fromStoreCode, toStoreId, toStoreName, toStoreCode, status, remarks, items, createdBy, createdAt, approvedBy, approvedAt, dispatchedBy, dispatchedAt, receivedBy, receivedAt, cancelledBy, cancelledAt, cancellationReason	Built via fromEntity(StockTransfer).
StoreComparisonRow	storeId, storeName, storeCode, totalSales, salesCount, totalPurchases, purchaseCount, totalStockUnits, lowStockCount, outOfStockCount	none (server-computed).

DTO	Fields	Validation
StoreComparisonResponse	fromDate, toDate, rows: List<StoreComparisonRow>	none.

Services

StoreService — Store CRUD. `search`, `getById`, `generateCode()` (delegates to `StoreCodeGeneratorService.generateNext()` — "the ONLY place an auto-generated store code is produced"), `create` (rejects duplicate `storeCode` case-insensitively via `existsByStoreCodeIgnoreCase`; rejects invalid GSTIN via `GstinValidator.isValid`; uppercases/trims stored GSTIN), `update` (same duplicate/GSTIN checks, excluding self via `existsByStoreCodeIgnoreCaseAndIdNot`), `setStatus(id, status)` (used for activate/deactivate — never a hard delete: "Never physically deletes a store... status is set to INACTIVE instead, so historical transactions referencing it remain valid"), `resolveGstContext(id)` (returns the store's own GSTIN as `source="STORE"` if set, else falls back to `BusinessGstConfigService.get()` as `source="BUSINESS"`), `findOrThrow(id)` (throws `MasterNotFoundException`), `getOrCreateDefaultStore()` (idempotent Default Store creation/lookup, `storeCode="MAIN"`, `storeType=HEAD_OFFICE`, `status=ACTIVE`).

StoreAccessService — the centralized authorization choke point. Every public method:

- `hasAllStoresAccess(User user)` — returns `permissionService.hasPermission(user, Permission.STORE_ACCESS_ALL)`. No throw.
- `getAccessibleStoreIds(User user)` — returns `List.of()` if `user == null`; if `hasAllStoresAccess`, returns every `Store` id in the system (active + inactive); otherwise returns the ids from the user's `UserStore` rows (`userStoreRepository.findByUserId`). No throw.
- `hasStoreAccess(User user, Long storeId)` — returns `false` if `user == null`; `true` if `ALL_STORES`; `false` if `storeId == null` (a null `storeId` means "no store on the record" — only possible on a pre-Default-Store-migration leftover row; even an `ALL_STORES` user only "can still see it", this method itself returns `false` for a null `storeId` regardless); otherwise `userStoreRepository.existsByUserIdAndStoreId(...)`. No throw.
- `assertStoreAccess(User user, Long storeId)` — calls `hasStoreAccess`; throws `AccessDeniedException("You do not have access to this store")` (→ HTTP 403) if it returns `false`. "The one place that decision is ever made."
- `resolveEffectiveStoreId(User user, Long requestedStoreId)` — resolves which store a **new transaction** belongs to. If `requestedStoreId != null`: calls `assertStoreAccess` then returns it (throws 403 if not accessible). Else if `user == null`: returns `storeService.getOrCreateDefaultStore().getId()` (system-triggered call fallback). Else if `user.getCurrentStore() != null`: asserts access to it and returns its id. Else: throws `BadRequestException("A store must be selected for this transaction")` (→ 400).
- `resolveViewableStoreId(User user, Long requestedStoreId)` — resolves which store id a list/summary/export view should filter by. If `requestedStoreId != null`: asserts access, returns it. Else if `user == null` or `hasAllStoresAccess(user)`: returns `null` (meaning "aggregate/unfiltered across every store"). Else if `user.getCurrentStore() != null`: returns its id. Else if the user's `getAccessibleStoreIds` has exactly one entry: returns it. Else: throws `BadRequestException` with "You do not have access to any store" (if empty) or "Please select a store" (if 2+ and none selected).
- `assignStores(Long userId, Set<Long> storeIds)` — `@Transactional`. Loads the `User` (throws `UserNotFoundException` if missing), deletes all existing `UserStore` rows for that user (`userStoreRepository.deleteByUserId`), then inserts one `UserStore` row per id in `storeIds` (throws `MasterNotFoundException("Store", storeId)` for any invalid store id). Replace-all semantics. If the user's `currentStore` is no longer in the new assignment and they aren't `ALL_STORES`, `currentStore` is cleared (`setCurrentStore(null)`) and saved — "rather than leave a stale selection the user can no longer actually use."
- `getAssignedStoreIds(Long userId)` — returns the raw list of store ids from `UserStore` rows for that user (ignores `ALL_STORES` — this is specifically the assignment list, not the effective access list). No throw.

StoreCodeGeneratorService — `generateNext()`, `@Transactional(propagation = REQUIRES_NEW)`. Ensures the single counter row exists (`ensureRowExists`, native `upsert`), locks it (`lockForUpdate`, `PESSIMISTIC_WRITE`), increments `lastNumber`, formats as `STR- + 6-digit zero-padded number`, loops while `storeRepository.existsByStoreCodeIgnoreCase(candidate)` (skips any number a manually-entered code already claimed), saves, returns. Runs in its own transaction "so the single counter row is locked/released independently of the caller's transaction."

StockTransferService — `search` (resolves `storeId` via `storeAccessService.resolveViewableStoreId`), `getId` (via `assertEitherStoreAccess` — visible if the user can access EITHER `fromStore` or `toStore`, else throws `AccessDeniedException`), `create` (asserts access to `fromStoreId`; rejects `fromStoreId == toStoreId` with `BadRequestException`; resolves both stores active via `resolveActiveStore` — throws `BadRequestException` if either is `INACTIVE`; builds items, each requiring the product not be `INACTIVE`; status starts `DRAFT`; assigns `financialYearId` via `financialYearService.resolveForDate`; generates `transferNumber` via `voucherNumberService.next(VoucherDocType.STOCK_TRANSFER, transferDate)` after the initial save), `approve` (asserts access to `fromStore`; requires current status `DRAFT` via `requireStatus`, else `BadRequestException`; sets `APPROVED`, `approvedBy`, `approvedAt`), `dispatch` (asserts access to `fromStore`; requires `APPROVED`; for each item calls `inventoryService.applyMovement(productId, fromStoreId, -quantity, TRANSFER_OUT, STOCK_TRANSFER, transferId, reason)`; sets `DISPATCHED`), `receive` (asserts access to `toStore`; requires `DISPATCHED`; for each item calls `applyMovement(productId, toStoreId, +quantity, TRANSFER_IN, STOCK_TRANSFER, transferId, reason)`; sets `RECEIVED`), `cancel` (asserts access to `fromStore`; only allowed from `DRAFT` or `APPROVED`, else `BadRequestException` explaining a `DISPATCHED / RECEIVED` transfer "already moved stock and cannot be cancelled"; sets `CANCELLED + cancelledBy/At + cancellationReason`).

StoreComparisonService — `compare(fromDate, toDate)`: gets `accessibleStoreIds` for the current user via `storeAccessService.getAccessibleStoreIds`, loads those `Store` rows, sorts by name, and for each store builds a `StoreComparisonRow` from per-store aggregate queries: `saleRepository.sumAndCountByDateRangeAndStore`, `purchaseRepository.sumAndCountByDateRangeAndStore`, `inventoryRepository.sumCurrentStock`, `inventoryRepository.countLowStock`, `inventoryRepository.countOutOfStock` — "one query per store per metric — never fetched in bulk and summed client-side."

Repository

`StoreRepository` extends `JpaRepository<Store, Long>`:

- `findByStoreCodeIgnoreCase(String)`
- `existsByStoreCodeIgnoreCase(String)`
- `existsByStoreCodeIgnoreCaseAndIdNot(String, Long)`
- `findByStatus(StoreStatus)`
- `search(search, status, Pageable)` — JPQL: matches `storeName / storeCode` (case-insensitive `LIKE`) when `search` is non-null/non-empty, and `status` when non-null.

`StoreCodeSequenceRepository` extends `JpaRepository<StoreCodeSequence, Long>`:

- `lockForUpdate(Long id)` — `@Lock(PESSIMISTIC_WRITE)` JPQL select by id.
- `ensureRowExists(Long id)` — `@Modifying` native `INSERT ... ON DUPLICATE KEY UPDATE id = id`.

`UserStoreRepository` extends `JpaRepository<UserStore, Long>`:

- `findByUserId(Long)`
- `existsByUserIdAndStoreId(Long, Long)`
- `deleteByUserId(Long)`
- `countByStoreId(Long)`

StockTransferRepository extends JpaRepository<StockTransfer, Long> :

- search(search, status, fromDate, toDate, storeId, Pageable) — JPQL: transferNumber case-insensitive LIKE when search set; status equality when set; transferDate >= fromDate / <= toDate when set; storeId matches **either** fromStore.id or toStore.id when set — "a store-scoped user sees a transfer if it touches any store they can access."

5. Database

Tables

Entity	@Table name	Key columns
Store	stores	id (IDENTITY), store_code (unique, len 30), store_name (len 150), store_type (enum string, len 20), legal_name (len 200), address (TEXT), country_id/state_id/city_id/zone_id (FK), pincode (len 10), phone (len 15), email (len 100), gstin (len 15), status (enum string, len 20), created_at, updated_at.
UserStore	user_stores	id, user_id (FK, not null), store_id (FK, not null), created_at. Unique constraint uk_user_store on (user_id, store_id).
StockTransfer	stock_transfers	id, transfer_number (unique, len 30), transfer_date, from_store_id (FK, not null), to_store_id (FK, not null), status (enum string, VARCHAR(15)), remarks (TEXT), financial_year_id, created_by/created_at, approved_by/approved_at, dispatched_by/dispatched_at, received_by/received_at, cancelled_by/cancelled_at, cancellation_reason (len 255).
StockTransferItem	stock_transfer_items	id, transfer_id (FK, not null), product_id (FK, not null), quantity (not null), notes (len 255).
StoreCodeSequence	store_code_sequence	id (always 1 — single global counter row), last_number (not null).

Schema is Hibernate-managed (spring.jpa.hibernate.ddl-auto=update in application.properties); no separate SQL migration files were found for these tables.

Relationships

- **Store ↔ User:** many-to-many via UserStore (user_stores.user_id / user_stores.store_id), representing ASSIGNED_STORES. Additionally User.currentStore is a direct @ManyToOne FK (users.current_store_id → stores.id) holding the user's active/selected store.
- **Store ↔ StockTransfer:** two FKs from stock_transfers to stores — from_store_id and to_store_id, both optional = false.
- **Store ↔ Inventory:** inventory.store_id (FK, part of the unique constraint uk_inventory_product_store ON (product_id, store_id)).
- **Store ↔ Employee:** many-to-many via EmployeeStore (employee_stores.employee_id / employee_stores.store_id, unique constraint uk_employee_store) — the Employee-module mirror of UserStore.
- **Store-attributed transaction tables** — every entity below has a direct @ManyToOne Store mapped to a store_id FK column (found via grep across backend/src/main/java/com/storehub/entity):
 - Sale.store → sales.store_id
 - SalesOrder.store → sales_orders.store_id
 - Purchase.store → purchases.store_id
 - PurchaseOrder.store → purchase_orders.store_id

- `Payment.store` → `payments.store_id`
- `Receipt.store` → `receipts.store_id`
- `Expense.store` → `expenses.store_id`
- `CashTransaction.store` → `cash_transactions.store_id`
- `JournalHeader.store` → `journal_headers.store_id` (nullable — null for a manual JOURNAL voucher)
- `GstTransaction.store` → `gst_transactions.store_id` (nullable — null for pre-migration data)
- `AuditLog.store` → `audit_logs.store_id` (nullable — null for a global/system-level action)
- `StockHistory.store` → `stock_history.store_id`
- `Inventory.store` → `inventory.store_id`
- **CreditNote / DebitNote** do **not** carry their own `store_id` column — `CreditNote.sourceSale` and `DebitNote.sourcePurchase` reference the parent `Sale / Purchase`, which itself carries the store. (NOT FOUND IN CURRENT CODEBASE: a direct `store / storeId` field on `CreditNote` or `DebitNote`.)

6. Validation

- `StoreRequest`: `storeCode / storeName` required (`@NotBlank`); `storeType` required (`@NotNull`); `email` must be `@Email`; `gstIn` optional but if non-blank must match `^[0-9]{2}[A-Za-z]{5}[0-9]{4}[A-Za-z]{1}[1-9A-Za-z]{1}Z[0-9A-Za-z]{1}$` (standard 15-char GSTIN format) via `@Pattern`.
- `StoreService.create / update` additionally re-validate GSTIN with `GstinValidator.isValid(...)` (a second, service-level check beyond the DTO `@Pattern`) and reject a duplicate `storeCode` (case-insensitive) with `BadRequestException`.
- `SetCurrentStoreRequest.storeId`: `@NotNull`.
- `AssignStoresRequest.storeIds`: `@NotNull` (an empty `Set` is valid and explicitly documented as the way to clear all store access).
- `StockTransferRequest`: `transferDate`, `fromStoreId`, `toStoreId` all `@NotNull`; `items` must be `@NotEmpty` and each item is validated via cascading `@Valid`.
- `StockTransferItemRequest`: `productId` `@NotNull`; `quantity` `@NotNull @Min(1)`.
- Service-level validation in `StockTransferService.create`: source and destination store must differ (`BadRequestException`); both stores must be `ACTIVE` (`resolveActiveStore` throws otherwise); every product must not be `INACTIVE`.
- Status-transition validation lives in `StockTransferService.requireStatus / cancel` — throws `BadRequestException` describing the required prior state and the transfer's actual current state.
- All 400-level validation failures from `@Valid / DTO` annotations surface as `MethodArgumentNotValidException`, converted by `GlobalExceptionHandler.handleValidation` into `ApiError.fieldErrors` (a `Map<String, String>` keyed by DTO field name).

7. Authentication / Authorization

Every store-scoping decision in this module funnels through `StoreAccessService` (see section 4 for exact per-method behavior). The model:

- **ALL_STORES**: a user holding `Permission.STORE_ACCESS_ALL` (checked via `PermissionService.hasPermission`, which reads `RolePermissions.has(user.getRole(), permission)` — a fixed, code-defined `Role → Set<Permission>` map, not a DB table). `StoreAccessService.hasAllStoresAccess` is the single check. An `ALL_STORES` user: sees every store (active+inactive) from `getAccessibleStoreIds`; passes `hasStoreAccess` for any `storeId`; `resolveViewableStoreId` returns `null` for them when no store is explicitly requested (meaning "aggregate/unfiltered").
- **ASSIGNED_STORES**: a user without `STORE_ACCESS_ALL` is scoped to exactly the stores with a `UserStore` row for them. `getAccessibleStoreIds` returns those ids; `hasStoreAccess` checks

`userStoreRepository.existsByUserIdAndStoreId`; a requested `storeId` outside that set always fails `assertStoreAccess` with `AccessDeniedException` → HTTP 403.

- A user with **neither** has zero store access — `getAccessibleStoreIds` returns an empty list, `hasStoreAccess` always returns `false`.
- Endpoint-level authorization is a second, separate layer on top: Spring Security `@PreAuthorize("hasAuthority('PERM_...')")` checks (backed by the same `RolePermissions` map, surfaced as granted authorities) gate whether the caller may call the endpoint at all (e.g. `PERM_STORE_CREATE`, `PERM_STOCK_TRANSFER_APPROVE`); `StoreAccessService` then gates which store(s) within that endpoint the caller may touch. Both layers must pass.
- A frontend-supplied `storeId` (e.g. on `StockTransferRequest.fromStoreId`, or a list/report's `storeId` filter param) is never trusted directly — every code path resolves it through `assertStoreAccess`, `resolveEffectiveStoreId`, or `resolveViewableStoreId` before use.

8. Permissions

From `Permission.java` and `RolePermissions.java`:

Permission	Module	ADMIN	STORE_MANAGER	ACCOUNTANT	SALES_USER	PURCHASE_USER	INVENTORY_USER
STORE_VIEW	Master	✓	✓	✓	✓	✓	✓
STORE_CREATE	Master	✓	X (explicitly removed)	X	X	X	X
STORE_EDIT	Master	✓	X (explicitly removed)	X	X	X	X
STORE_ASSIGN	Security	✓	X (explicitly removed)	X	X	X	X
STORE_ACCESS_ALL	Security	✓	X (explicitly removed)	X	X	X	X
STOCK_TRANSFER_VIEW	Inventory	✓	✓	X	X	X	✓
STOCK_TRANSFER_CREATE	Inventory	✓	✓	X	X	X	✓
STOCK_TRANSFER_APPROVE	Inventory	✓	✓	X	X	X	X
STOCK_TRANSFER_DISPATCH	Inventory	✓	✓	X	X	X	✓
STOCK_TRANSFER_RECEIVE	Inventory	✓	✓	X	X	X	✓
STOCK_TRANSFER_CANCEL	Inventory	✓	✓	X	X	X	X

Note: `STORE_MANAGER`'s permission set is computed as "every permission `ADMIN` has, minus an explicit exclusion set" (`RolePermissions.build()`), and that exclusion set explicitly includes `STORE_CREATE`, `STORE_EDIT`, `STORE_ASSIGN`, `STORE_ACCESS_ALL` — commented as "Creating/editing Store records and assigning users to stores, or bypassing store scoping entirely, are structural/admin-tier decisions... `STORE_MANAGER` works within their assigned store(s), never manages the store list itself."

`INVENTORY_USER` deliberately excludes `STOCK_TRANSFER_APPROVE` / `STOCK_TRANSFER_CANCEL` (can create/dispatch/receive but not approve or cancel). Only `ADMIN` and `STORE_MANAGER` hold every `STOCK_TRANSFER_*` permission.

9. Transaction Handling

- `StoreService.create`, `update`, `setStatus`, `getOrCreateDefaultStore` are all `@Transactional`.
- `StoreCodeGeneratorService.generateNext()` runs in its **own** transaction (`@Transactional(propagation = Propagation.REQUIRES_NEW)`) so the pessimistic row lock on the single counter row is acquired/

released independently of the caller's transaction, preventing two concurrent "Generate Code" calls from colliding.

- `StoreAccessService.assignStores` is `@Transactional` (delete-then-insert of `UserStore` rows, plus a possible `currentStore` clear, all atomic).
- `StockTransferService.search/getById` are `@Transactional(readOnly = true)`; `create`, `approve`, `dispatch`, `receive`, `cancel` are `@Transactional` — each status transition (and, for `dispatch/receive`, the associated `InventoryService.applyMovement` calls per item) commits atomically with the transfer's own status/timestamp update.
- `StoreComparisonService.compare` is `@Transactional(readOnly = true)` — one read-only transaction spans all the per-store aggregate queries.
- `InventoryService.applyMovement` (called from `dispatch/receive`) is itself `@Transactional`; since it's called within the already-`@Transactional` `dispatch/receive` methods, all item movements plus the transfer status update share one transaction (default `REQUIRED` propagation) — a mid-loop failure rolls back the whole `dispatch/receive`.

10. Error Handling

Mapped by `GlobalExceptionHandler` (`backend/src/main/java/com/storehub/exception/GlobalExceptionHandler.java`):

Exception	HTTP Status	Notes
<code>MethodArgumentNotValidException</code>	400	<code>fieldErrors</code> map populated from <code>BindingResult</code> field errors — used for all <code>@Valid</code> DTO failures (<code>StoreRequest</code> , <code>StockTransferRequest</code> , <code>SetCurrentStoreRequest</code> , <code>AssignStoresRequest</code> , etc.).
<code>BadRequestException</code>	400	Used for: duplicate store code, invalid GSTIN, inactive store/product in a transfer, same from/to store, missing current-store selection, invalid status transition, "no store access"/"please select a store". Message only, no <code>fieldErrors</code> .
<code>MasterNotFoundException</code>	404	Thrown by <code>StoreService.findOrThrow</code> and <code>StoreAccessService.assignStores</code> (invalid store id in the set).
<code>StockTransferNotFoundException</code>	404	Thrown by <code>StockTransferService.findOrThrow</code> ; simple <code>RuntimeException</code> subclass with message "Stock transfer not found with id: " + id.
<code>AccessDeniedException</code> (Spring Security)	403	Thrown by <code>StoreAccessService.assertStoreAccess</code> and <code>StockTransferService.assertEitherStoreAccess</code> ; response message is a generic "You do not have permission to access this resource" (the specific "You do not have access to this store" / "...this stock transfer" message thrown by the service is not echoed to the client — the handler overwrites it with the generic text).
Exception (catch-all)	500	Fallback handler for anything unmapped.

11. Audit Flow

Exact `AuditService.log(...)` call sites:

- `StoreService.create` — `AuditAction.CREATE`, module "MASTER", entityType "Store", entityId = new store id, documentNumber = store code, description "Store " + code + " (" + name + ") created".

- `StoreService.update` — `AuditAction.UPDATE`, `"MASTER" / "Store"`, description `"Store " + code + " updated"`.
- `StoreService.setStatus` — `AuditAction.UPDATE`, `"MASTER" / "Store"`, `oldValue/newValue = old/new` `StoreStatus` name, description `"Store " + code + " status changed to " + status"`.
- `StoreService.getOrCreateDefaultStore` (only on first creation) — `AuditAction.CREATE`, `"MASTER" / "Store"`, description `"Default store 'MAIN' auto-created to hold pre-Multi-Store historical data"`.
- `StockTransferService.create` — `AuditAction.CREATE`, module `"INVENTORY"`, entityType `"StockTransfer"`, documentNumber = transfer number, description `"Stock transfer " + number + " created: " + fromCode + " -> " + toCode`, **storeId = fromStore.id** (the overloaded `log(..., storeId)` variant).
- `StockTransferService.approve` — `AuditAction.UPDATE`, `"INVENTORY" / "StockTransfer"`, description `"...approved"`, `storeId = fromStore.id`.
- `StockTransferService.dispatch` — `AuditAction.UPDATE`, `"INVENTORY" / "StockTransfer"`, description `"...dispatched: stock deducted from " + fromStoreCode`, `storeId = fromStore.id`.
- `StockTransferService.receive` — `AuditAction.UPDATE`, `"INVENTORY" / "StockTransfer"`, description `"...received: stock added to " + toStoreCode`, `storeId = toStore.id`.
- `StockTransferService.cancel` — `AuditAction.CANCEL`, `"INVENTORY" / "StockTransfer"`, description `"Stock transfer cancelled: " + transferNumber`, `storeId = fromStore.id`.
- `UserService.assignStores` (store-access assignment) — `AuditAction.UPDATE`, module `"ADMIN"`, entityType `"User"`, description `"Store access for user " + email + " set to " + storeIds.size() + " store(s)"`.
- `UserService.setCurrentStore` — `AuditAction.UPDATE`, module `"AUTH"`, entityType `"User"`, documentNumber = new store code, description `"User " + email + " switched current store to " + storeCode"`.

Every `AuditService.log` call is wrapped internally in a try/catch (per `AuditService`'s own implementation) so an audit-logging failure never blocks the underlying operation.

12. Important Side Effects

- **Default Store historical-data-safety fallback:** `StoreService.getOrCreateDefaultStore()` (code `"MAIN"`) is the anchor every pre-Multi-Store transaction was backfilled onto, and is the fallback both `StoreAccessService.resolveEffectiveStoreId` (when `user == null`) and `InventoryService.applyMovement`'s legacy no-storeId overload use, so that call sites not yet migrated to pass an explicit `storeId` keep working unchanged.
- **Single-store auto-select on login:** lives in `frontend/src/context/AuthContext.tsx` (not in this module's own backend/frontend files, but directly store-relevant) — after login, `authApi.getMyStores()` is called; if the result has exactly one store **and** the user has no `currentStoreId` yet, `authApi.setCurrentStore(that store's id)` is called silently so a single-store business never has to manually pick "the only store that exists." A multi-store user with none selected is left alone (the `StoreSwitcher` is available in the Topbar).
- **Stock only moves at DISPATCHED/RECEIVED, never at DRAFT/APPROVED:** `StockTransferService.dispatch()` is the only point stock leaves the `fromStore` (`TRANSFER_OUT`, negative delta), and `receive()` is the only point stock lands in the `toStore` (`TRANSFER_IN`, positive delta). `create()` / `approve()` never touch `InventoryService` — they are "purely an authorization/paperwork stage."
- **Cancel is only possible before stock moves:** `cancel()` is rejected outside `DRAFT / APPROVED` specifically because reversing a `DISPATCHED` transfer would require a compensating receive-back movement, which this workflow does not implement.

- **Store deactivation never rewrites history:** `StoreService.setStatus(id, INACTIVE)` only flips the `status` column; it does not touch any historical `Sale / Purchase / Inventory / StockTransfer` row that already references the store. An `INACTIVE` store is, however, blocked from being used as either side of a **new** stock transfer (`StockTransferService.resolveActiveStore`).
- **StoreAccessService.assignStores can silently clear a user's current store:** if the new assignment no longer includes the user's `currentStore` and they are not `ALL_STORES`, `currentStore` is set to `null` as part of the same transaction, so their next transaction requires re-selecting a store (`resolveEffectiveStoreId` would otherwise throw `BadRequestException`).

13. Dependencies on Other Modules

Modules that depend on Store (every entity with a direct `store / storeId` FK, confirmed by `grep` across `backend/src/main/java/com/storehub/entity`): `Sale`, `SalesOrder`, `Purchase`, `PurchaseOrder`, `Payment`, `Receipt`, `Expense`, `CashTransaction`, `JournalHeader`, `GstTransaction`, `AuditLog`, `StockHistory`, `Inventory`, plus the store-assignment tables `UserStore` and `EmployeeStore`. `CreditNote / DebitNote` depend on `Store` only indirectly, through their `sourceSale / sourcePurchase` reference.

Modules Store depends on:

- **Inventory** — `StockTransferService.dispatch / receive` call `InventoryService.applyMovement(...)` directly to move stock; this module never duplicates stock-mutation logic. (`InventoryService` itself falls back to `StoreService.getOrCreateDefaultStore()` in its legacy no-storeId overload.)
- **User** — `StoreAccessService / UserService` depend on `User / UserRepository / UserStoreRepository` for access assignment (`assignStores`, `getAssignedStoreIds`) and current-store selection (`setCurrentStore`).
- **Employee** — `EmployeeService / EmployeeController` reuse the exact same `AssignStoresRequest` DTO and replace-all pattern for `EmployeeStore` (a parallel table to `UserStore`), though `EmployeeService.assignStores` itself is out of scope here (belongs to the `Employee` module).
- **Permission/Role** — `StoreAccessService.hasAllStoresAccess` depends on `PermissionService / RolePermissions` for the `STORE_ACCESS_ALL` check.
- **Country/State/City/Zone (masters)** — `StoreService` resolves these as optional FK fields on `Store` via `CountryService / StateService / CityService / ZoneService`.
- **BusinessGstConfig** — `StoreService.resolveGstContext` falls back to `BusinessGstConfigService.get()` when the store has no GSTIN override.
- **Sale/Purchase/Inventory repositories** — `StoreComparisonService` reads aggregate queries from `SaleRepository`, `PurchaseRepository`, `InventoryRepository` to build its report rows.
- **VoucherNumberService / FinancialYearService** — `StockTransferService.create` uses these to assign `transferNumber` and `financialYearId`.

14. Key Operation Flows

Create Store

`StoreMaster.tsx` (form submit) → `storeApi.create(payload)` → `POST /api/masters/stores` (`@PreAuthorize` `PERM_STORE_CREATE`) → `StoreController.create(StoreRequest)` → `StoreService.create(request)` (checks `existsByStoreCodeIgnoreCase`, validates GSTIN via `GstinValidator`, resolves optional `Country/State/City/Zone` FKs, builds `Store` with `status` defaulted to `ACTIVE` via `@PrePersist`) → `storeRepository.save(store)` → `stores` table INSERT → `auditService.log(CREATE, "MASTER", "Store", ...)` → `StoreResponse.fromEntity(saved)` → 201 response → `MasterCrudPage` closes the form and refreshes the list.

Switch Current Store

`StoreSwitcher.tsx` dropdown item click → `switchStore(storeId)` (in `AuthContext`) → `authApi.setCurrentStore(storeId)` → `PUT /api/auth/current-store` (`SetCurrentStoreRequest { storeId }`),

no @PreAuthorize — any authenticated user) → AuthController.setCurrentStore → UserService.setCurrentStore(username, storeId) → storeAccessService.assertStoreAccess(user, storeId) (throws 403 if not accessible) → storeRepository.findById(storeId) (throws MasterNotFoundException if missing) → user.setCurrentStore(store) → userRepository.save(user) → users.current_store_id UPDATE → auditService.log(UPDATE, "AUTH", "User", ...) → UserResponse.fromEntity(saved) → frontend replaces user in context + localStorage.

Assign User to Stores

StoreAccessDialog Save → userApi.assignStores(userId, storeIds) → PUT /api/users/{id}/stores (class-level @PreAuthorize hasRole('ADMIN'), AssignStoresRequest { storeIds }) → UserController.assignStores → UserService.assignStores(id, storeIds) → storeAccessService.assignStores(id, storeIds) (@Transactional: userStoreRepository.deleteByUserId(id) then one userStoreRepository.save(UserStore) insert per id, validating each id via storeRepository.findById — throws MasterNotFoundException on an invalid id; clears currentStore if it fell outside the new set) → user_stores table DELETE+INSERTs (+ possible users.current_store_id UPDATE) → auditService.log(UPDATE, "ADMIN", "User", ...) → UserResponse.fromEntity(...) → dialog closes, parent list reloads.

Stock Transfer Lifecycle

- 1. Create (DRAFT):** StockTransferForm.tsx submit → stockTransferApi.create(payload) → POST /api/stock-transfers (PERM_STOCK_TRANSFER_CREATE, StockTransferRequest) → StockTransferController.create → StockTransferService.create (storeAccessService.assertStoreAccess(user, fromStoreId); rejects fromStoreId == toStoreId; resolveActiveStore for both stores — 400 if either INACTIVE; builds StockTransfer with status=DRAFT, items validated against ProductRepository/ProductStatus; financialYearService.resolveForDate; two saves — first to get an id, second after voucherNumberService.next(...) assigns transferNumber) → stockTransferRepository.save → stock_transfers/stock_transfer_items INSERT → auditService.log(CREATE, "INVENTORY", ...) → StockTransferResponse.fromEntity → 201 → frontend navigates to the detail page.
- 2. Approve:** detail/list action → stockTransferApi.approve(id) → PATCH /{id}/approve (PERM_STOCK_TRANSFER_APPROVE) → StockTransferService.approve (asserts access to fromStore; requireStatus(DRAFT) else 400) → sets APPROVED / approvedBy / approvedAt → save → audit log → response.
- 3. Dispatch (stock deducted):** → PATCH /{id}/dispatch (PERM_STOCK_TRANSFER_DISPATCH) → StockTransferService.dispatch (asserts access to fromStore; requireStatus(APPROVED)) → for each item: inventoryService.applyMovement(productId, fromStoreId, -quantity, TRANSFER_OUT, STOCK_TRANSFER, transferId, reason) → Inventory.currentStock UPDATE at the source store (throws BadRequestException "Insufficient stock..." if it would go negative) → sets DISPATCHED / dispatchedBy / dispatchedAt → save → audit log → response.
- 4. Receive (stock credited):** → PATCH /{id}/receive (PERM_STOCK_TRANSFER_RECEIVE) → StockTransferService.receive (asserts access to toStore; requireStatus(DISPATCHED)) → for each item: inventoryService.applyMovement(productId, toStoreId, +quantity, TRANSFER_IN, STOCK_TRANSFER, transferId, reason) → Inventory.currentStock UPDATE (or insert) at the destination store → sets RECEIVED / receivedBy / receivedAt → save → audit log → response.
- 5. Cancel (only from DRAFT/APPROVED):** → PATCH /{id}/cancel (PERM_STOCK_TRANSFER_CANCEL) → StockTransferService.cancel (asserts access to fromStore; if status is not DRAFT / APPROVED, throws BadRequestException naming the current status) → sets CANCELLED / cancelledBy / cancelledAt / cancellationReason → save → auditService.log(CANCEL, ...) → response. No InventoryService call — no stock had moved yet.

Throughout, the frontend (StockTransfers.tsx / StockTransferDetail.tsx) gates each action button on both the transfer's live status and hasPermission('STOCK_TRANSFER_*'), then calls load() / getById()

again after every action to refresh the displayed state and timeline from the authoritative backend response.

15. Manual Changes

- **Add a new Store field:** add the column to `Store.java` (entity), add it to `StoreRequest / StoreResponse` DTOs (with any `@NotBlank / @Pattern` /etc. validation), wire it into `StoreService.create / update` (builder/setters) and `StoreResponse.fromEntity`, and add the corresponding form field to `StoreMaster.tsx` (`StorePayload` in `frontend/src/types/store.ts`, `EMPTY` default, `toFormValues`, `renderForm`, `renderView`). Since `ddl-auto=update` is in effect, no manual migration is needed, but double-check column length/nullability annotations match intent.
- **Change the ALL_STORES/ASSIGNED_STORES logic:** everything lives in `StoreAccessService` (`backend/src/main/java/com/storehub/service/StoreAccessService.java`) — this is the single place to modify. Any change to `hasAllStoresAccess`, `hasStoreAccess`, `resolveEffectiveStoreId`, or `resolveViewableStoreId` immediately affects every module that calls it (`Sale`, `Purchase`, `Inventory`, `Expense`, `CashTransaction`, `StockTransfer`, `StoreComparison`, etc.) — grep for `storeAccessService`. call sites before changing method signatures or semantics.
- **Add a new Stock Transfer status:** add the enum constant to `StockTransferStatus.java`; add a new service method (or extend an existing one) in `StockTransferService` following the `requireStatus(transfer, REQUIRED_STATUS, "actionPastTense")` pattern; add a new `@PreAuthorize - guarded` endpoint to `StockTransferController` plus a new `STOCK_TRANSFER_*` constant in `Permission.java` and assign it to the right roles in `RolePermissions.java`; add the corresponding `stockTransferApi` method (`frontend/src/api/stockTransferApi.ts`) and wire buttons into `StockTransfers.tsx / StockTransferDetail.tsx` with the new status guard and `statusVariant()` badge color.
- **Change the store code format:** modify `StoreCodeGeneratorService.PREFIX / format(long)` only (`backend/src/main/java/com/storehub/service/StoreCodeGeneratorService.java`) — this is documented as "the ONLY place an auto-generated store code is produced," so no other file needs to change. Manually-entered codes bypass this entirely and are only checked for uniqueness in `StoreService.create / update`.
- **Change which roles get STORE_ACCESS_ALL:** edit `RolePermissions.build()` (`backend/src/main/java/com/storehub/service/RolePermissions.java`) — add/remove `Permission.STORE_ACCESS_ALL` from the relevant role's `EnumSet`. This is a single centralized map; no controller or service needs to change since every check reads through `PermissionService / StoreAccessService`.
- **Add store scoping to a new module:** (a) add a `store / @ManyToOne @JoinColumn(name = "store_id")` field to the new entity; (b) add `storeId` to its create-request DTO; (c) in its service's create method, call `storeAccessService.resolveEffectiveStoreId(currentUser, request.getStoreId())` to get the authorized store id rather than trusting the request directly; (d) in its list/search method, call `storeAccessService.resolveViewableStoreId(currentUser, requestedStoreId)` to scope reads; (e) if it mutates stock, call `inventoryService.applyMovement(productId, storeId, delta, ...)` with the resolved store id (never the legacy no-storeId overload) — following the exact pattern `StockTransferService` uses.

Lookup / Simple Masters Module

1. Overview

StoreHub has nine "simple lookup masters" — **Country, State, City, Zone, Currency, Nationality, Unit, Item Group, HSN** — that all follow the exact same CRUD shape:

- List with search + status filter + pagination
- Create
- Edit (full replace via `PUT`)
- Activate / Deactivate (soft status toggle, `PATCH`; no hard delete exists anywhere in this module)

Every one of these masters is a plain JPA `@Entity` with an `id`, a name-ish unique field, a `status` enum (`ACTIVE / INACTIVE`), and `createdAt / updatedAt` timestamps auto-set by `@PrePersist / @PreUpdate`. Four of the nine (State, City, Nationality, Zone) also carry a `ManyToOne` reference to another master in this same group (State→Country, City→State, Nationality→Country, Zone→Country/State optionally) to model a lookup hierarchy.

On the frontend, eight of the nine masters (Country, State, City, Zone, Currency, Nationality, Unit, Item Group) are built by simply configuring the shared generic component `frontend/src/components/masters/MasterCrudPage.tsx` with entity-specific columns, a form renderer, and the four CRUD callbacks — there is no bespoke list/detail/dialog code per master.

HSN is the one exception. `HsnMaster.tsx` does **not** use `MasterCrudPage` — it is a hand-written ~480-line page that duplicates most of `MasterCrudPage`'s list/search/pagination/dialog/discard-confirm logic, because HSN needs an extra capability none of the others do: each HSN code owns a **child collection of tax rates** (`HsnTaxRate`, table `hsn_tax_rates`) with its own repeatable add/remove row UI inside the create/edit form, and a read-only "View" dialog showing the full tax-rate history table. On the backend this shows up as `Hsn` having a `@OneToMany` (mappedBy = "hsn", cascade = ALL, orphanRemoval = true) list of `HsnTaxRate`, and `HsnService.update()` fully replacing the child rows (`clearTaxRates()` + `applyTaxRates()`) rather than diffing them.

2. Frontend

The Shared Component: `MasterCrudPage.tsx`

File: `frontend/src/components/masters/MasterCrudPage.tsx`

`MasterCrudPage<T extends MasterRow, TPayload>` is a generic React function component parameterized by:

- `T extends MasterRow` — the entity/response shape as returned by the API. `MasterRow` requires at minimum `{ id: number; status: string }`.
- `TPayload` — the create/update request-body shape (the form's state type).

Props (`MasterCrudPageProps<T, TPayload>`):

Prop	Type	Purpose
<code>icon</code>	<code>LucideIcon</code>	Icon shown in <code>PageHeader</code> and the empty state
<code>title</code>	<code>string</code>	Singular display name, e.g. "Country" — used in headings, toasts, dialog titles
<code>description</code>	<code>string</code>	Subtitle text under the page header

Prop	Type	Purpose
itemLabel	string	Plural label used in toast/error/empty-state text, e.g. "countries"
searchPlaceholder	string	Placeholder text for the search input
columns	MasterColumn<T>[]	{ header, render(item): ReactNode, className? } [] — defines the table's data columns (Status and Actions columns are always appended automatically)
emptyValues	TPayload	The blank/default payload used when opening the "Add" dialog
toFormValues	(item: T) => TPayload	Converts a fetched row into edit-form values when opening "Edit"
renderForm	(values, setValues, fieldErrors) => ReactNode	Renders the create/edit form body inside the shared Dialog
renderView?	(item: T) => ReactNode	Optional — if provided, adds a "View" dropdown action and a read-only detail Dialog (none of the 9 masters that use MasterCrudPage currently pass this — HSN needs its own view and bypasses MasterCrudPage entirely instead)
fetchList	(params: {search, status, page, size}) => Promise<{data: {content: T[]; totalPages: number}}>	List/search call
create	(payload: TPayload) => Promise<any>	Create call
update	(id: number, payload: TPayload) => Promise<any>	Update call
activate	(id: number) => Promise<any>	Activate call
deactivate	(id: number) => Promise<any>	Deactivate call
extraFilterSlot?	ReactNode	Extra filter control rendered next to the search bar (e.g. State/Country dropdown filters)
reloadToken?	unknown	Any value; changing it re-triggers load() (used so an extra filter, e.g. countryFilter, forces a reload)
backTo?	string (default /masters)	Back-button target
backLabel?	string (default Back to Masters)	Back-button label

Internal state and behavior:

- items, loading, search, statusFilter, page, totalPages drive the list/table/pagination.
- load() calls fetchList({search, status: statusFilter, page, size: PAGE_SIZE=10}), sets items / totalPages, and on failure shows a toast via parseApiError(err, "Failed to load {itemLabel} list").
- useEffect reloads on [page, statusFilter, reloadToken] — note: typing in the search box does **not** auto-reload; the user must submit the search form (handleSearchSubmit, which resets page to 0 and calls load()).
- formModal state ({show, mode: 'add'|'edit', item}) drives a single shared Dialog used for both create and edit.
- openAdd() resets values to emptyValues and snapshots it (initialValuesSnapshot, JSON-stringified) for dirty-checking.
- openEdit(item) calls toFormValues(item) to seed values and snapshots it the same way.
- isFormDirty() compares the current JSON.stringify(values) against the snapshot; requestCloseForm() opens a ConfirmDialog ("Discard unsaved changes?") only if dirty, otherwise closes immediately.

- `handleSubmit()` : calls `create(values)` (mode `add`) or `update(item.id, values)` (mode `edit`), shows a success toast (`"{title} created successfully"` / `"{title} updated successfully"`), closes the dialog and calls `load()` . On error, calls `parseApiError` and sets both the top-level `error` (shown in an `Alert`) and `fieldErrors` (passed into `renderForm` for per-field `invalid/feedback` rendering).
- `toggleStatus(item)` : calls `deactivate(item.id)` if `item.status === 'ACTIVE'` , else `activate(item.id)` ; shows a toast; reloads the list. Exposed via a dropdown menu item labeled "Deactivate" (destructive variant) or "Activate".
- Renders: `BackButton` → `PageHeader` (title/description/"Add {title}" button) → `search/status-filter Card` → `data Table` (or `TableSkeleton` while loading, or `EmptyState` when empty) with a per-row dropdown (`View?` , `Edit` , `Activate/Deactivate`) → `Pagination` → the `create/edit Dialog` → the `discard-confirm ConfirmDialog` → the optional `view Dialog` .

Example configuration — `CountryMaster.tsx` (`frontend/src/pages/masters/CountryMaster.tsx`):

```
<MasterCrudPage<Country, CountryPayload>
  icon={Globe}
  title="Country"
  columns={[
    { header: 'Name', render: (c) => <span className="font-medium">{c.name}</span> },
    { header: 'Code', render: (c) => c.code },
  ]}
  emptyValues={{ name: '', code: '' }}
  toFormValues={(c) => ({ name: c.name, code: c.code })}
  renderForm={(values, setValues, fieldErrors) => (/* Name + Code inputs, each with
    invalid={!fieldErrors.name} and a <p> showing fieldErrors.name/code */) }
  fetchList=(({ search, status, page, size }) => countryApi.list({ search, status, page, size }))
  create={countryApi.create}
  update={countryApi.update}
  activate={countryApi.activate}
  deactivate={countryApi.deactivate}
/>
```

The dependent masters (State, City, Nationality) additionally fetch their parent master's active list in a `useEffect` (e.g. `StateMaster` loads `countryApi.list({status:'ACTIVE', size:1000})` into a `countries` state array), render a `<Select>` of that parent in `renderForm`, and use `extraFilterSlot` + `reloadToken` to add a parent-scoped filter dropdown above the table (e.g. "All Countries" filter on `StateMaster` , which is passed as `countryFilter` into both `fetchList`'s params and `reloadToken`).

Files and Components

Master	Frontend page	Wires to entity	Wires to API object	Uses MasterCrudPage ?
Country	<code>frontend/src/pages/masters/CountryMaster.tsx</code>	Country	<code>countryApi</code>	Yes
State	<code>frontend/src/pages/masters/StateMaster.tsx</code>	<code>StateMaster</code> (frontend type; backend entity <code>State</code>)	<code>stateApi</code> (+ <code>countryApi</code> for the parent dropdown)	Yes
City	<code>frontend/src/pages/masters/CityMaster.tsx</code>	City	<code>cityApi</code> (+ <code>stateApi</code> for the parent dropdown)	Yes
Zone	<code>frontend/src/pages/masters/ZoneMaster.tsx</code>	Zone	<code>zoneApi</code> (+ <code>countryApi</code> , <code>stateApi</code> for optional parent dropdowns)	Yes

Master	Frontend page	Wires to entity	Wires to API object	Uses MasterCrudPage ?
Currency	frontend/src/pages/masters/CurrencyMaster.tsx	Currency	currencyApi	Yes
Nationality	frontend/src/pages/masters/NationalityMaster.tsx	Nationality	nationalityApi (+ countryApi)	Yes
Unit	frontend/src/pages/masters/UnitMaster.tsx	Unit	unitApi	Yes
Item Group	frontend/src/pages/masters/ItemGroupMaster.tsx	ItemGroup	itemGroupApi	Yes
HSN	frontend/src/pages/masters/HsnMaster.tsx	Hsn (+ child HsnTaxRate)	hsnApi	No — bespoke page

Shared infra: frontend/src/components/masters/MasterCrudPage.tsx (generic CRUD shell), frontend/src/pages/MastersDashboard.tsx (landing grid with per-master record counts), frontend/src/api/mastersApi.ts (per-master *Api objects), frontend/src/types/masters.ts (per-master interface X / XPayload types).

Frontend Flow

Generic flow (all MasterCrudPage -based masters):

1. Page mounts → useEffect([page, statusFilter, reloadToken]) fires load() → fetchList() → table renders items, or TableSkeleton while loading, or EmptyState if empty.
2. **Create:** "Add {title}" button → openAdd() seeds emptyValues into the shared Dialog → user fills renderForm fields → submit → create(values) → on success, toast + close + load(); on 400, parseApiError fills error (top Alert) and fieldErrors (inline per-field <p> under each Input, using invalid={!!fieldErrors.x}).
3. **Edit:** row's "..." menu → "Edit" → openEdit(item) seeds toFormValues(item) → same dialog/submit path but calls update(item.id, values).
4. **Activate/Deactivate:** row's "..." menu → toggleStatus(item) → calls activate/deactivate directly (no confirmation dialog, no form) → toast → load().
5. Search box requires explicit "Search" submit (resets to page 0); the Status dropdown and any extraFilterSlot (parent-master filter) reload immediately on change.

HSN's extra tax-rate row flow (HsnMaster.tsx, self-contained, no MasterCrudPage):

- HsnPayload.taxRates: HsnTaxRate[] is part of the same form state as hsnCode/description.
- emptyTaxRate() seeds one blank row (taxPercent: 0, effectiveFrom: today) whenever "Add HSN Code" is opened, or whenever an edited HSN happens to have zero rows.
- Inside the form, a "Tax Rates" section renders one row per taxRates[i] with Tax %/CGST %/SGST %/IGST %/Effective-From inputs plus a per-row delete (Trash2) button (disabled when only one row remains) and an "Add Rate" button (addTaxRateRow) that appends another emptyTaxRate().
- updateTaxRateRow(idx, field, value) updates one field of one row immutably; numeric fields are parsed with Number(value) (empty string → undefined), effectiveFrom stays a raw string.
- On submit, the whole taxRates array is sent as-is inside HsnPayload to hsnApi.create/hsnApi.update — the backend fully replaces the child rows (see §4 Services).
- The list table shows only the **latest** rate's taxPercent per HSN code (latestRate() sorts taxRates by effectiveFrom descending and takes the first); the "View" dialog shows the full tax-rate history table (Tax %/CGST/SGST/IGST/Cess/Effective From) for the selected HSN.

3. API Calls

All 9 masters, from `frontend/src/api/mastersApi.ts`. Every `list()` call sends `{search, status, page, size}` (plus a parent-id filter where noted) as query params via `buildParams()`, which drops empty/undefined values.

Master	Function	Method	URL	Request shape	Response shape
Country	<code>countryApi.list</code>	GET	<code>/masters/countries</code>	query: <code>search?</code> , <code>status?</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<Country></code>
	<code>countryApi.getById</code>	GET	<code>/masters/countries/{id}</code>	—	<code>Country</code>
	<code>countryApi.create</code>	POST	<code>/masters/countries</code>	<code>CountryPayload {name, code}</code>	<code>Country</code>
	<code>countryApi.update</code>	PUT	<code>/masters/countries/{id}</code>	<code>CountryPayload</code>	<code>Country</code>
	<code>countryApi.activate</code>	PATCH	<code>/masters/countries/{id}/activate</code>	—	<code>Country</code>
	<code>countryApi.deactivate</code>	PATCH	<code>/masters/countries/{id}/deactivate</code>	—	<code>Country</code>
State	<code>stateApi.list</code>	GET	<code>/masters/states</code>	query: <code>search?</code> , <code>status?</code> , <code>countryId?</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<StateMaster></code>
	<code>stateApi.getById</code>	GET	<code>/masters/states/{id}</code>	—	<code>StateMaster</code>
	<code>stateApi.create</code>	POST	<code>/masters/states</code>	<code>StatePayload {name, code?, countryId}</code>	<code>StateMaster</code>
	<code>stateApi.update</code>	PUT	<code>/masters/states/{id}</code>	<code>StatePayload</code>	<code>StateMaster</code>
	<code>stateApi.activate / deactivate</code>	PATCH	<code>/masters/states/{id}/activate /deactivate</code>	—	<code>StateMaster</code>
City	<code>cityApi.list</code>	GET	<code>/masters/cities</code>	query: <code>search?</code> , <code>status?</code> , <code>stateId?</code> , <code>countryId?</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<City></code>
	<code>cityApi.getById</code>	GET	<code>/masters/cities/{id}</code>	—	<code>City</code>
	<code>cityApi.create</code>	POST	<code>/masters/cities</code>	<code>CityPayload {name, stateId}</code>	<code>City</code>
	<code>cityApi.update</code>	PUT	<code>/masters/cities/{id}</code>	<code>CityPayload</code>	<code>City</code>

Master	Function	Method	URL	Request shape	Response shape
	cityApi.activate / deactivate	PATCH	/masters/ cities/{id}/ activate / deactivate	—	City
Zone	zoneApi.list	GET	/masters/ zones	query: search?, status?, page, size	PagedResponse<Zone>
	zoneApi.getById	GET	/masters/ zones/{id}	—	Zone
	zoneApi.create	POST	/masters/ zones	ZonePayload {name, code?, countryId?, stateId?}	Zone
	zoneApi.update	PUT	/masters/ zones/{id}	ZonePayload	Zone
	zoneApi.activate / deactivate	PATCH	/masters/ zones/{id}/ activate / deactivate	—	Zone
Currency	currencyApi.list	GET	/masters/ currencies	query: search?, status?, page, size	PagedResponse<Currency>
	currencyApi.getById	GET	/masters/ currencies/ {id}	—	Currency
	currencyApi.create	POST	/masters/ currencies	CurrencyPayload {name, code, symbol, decimalPlaces}	Currency
	currencyApi.update	PUT	/masters/ currencies/ {id}	CurrencyPayload	Currency
	currencyApi.activate / deactivate	PATCH	/masters/ currencies/ {id}/activate /deactivate	—	Currency
Nationality	nationalityApi.list	GET	/masters/ nationalities	query: search?, status?, countryId?, page, size	PagedResponse<Nationality>
	nationalityApi.getById	GET	/masters/ nationalities/ {id}	—	Nationality
	nationalityApi.create	POST	/masters/ nationalities	NationalityPayload {name, countryId}	Nationality
	nationalityApi.update	PUT	/masters/ nationalities/ {id}	NationalityPayload	Nationality
	nationalityApi.activate / deactivate	PATCH	/masters/ nationalities/ {id}/activate /deactivate	—	Nationality

Master	Function	Method	URL	Request shape	Response shape
Unit	unitApi.list	GET	/masters/ units	query: search?, status?, page, size	PagedResponse<Unit>
	unitApi.getById	GET	/masters/ units/{id}	—	Unit
	unitApi.create	POST	/masters/ units	UnitPayload {name, symbol}	Unit
	unitApi.update	PUT	/masters/ units/{id}	UnitPayload	Unit
	unitApi.activate / deactivate	PATCH	/masters/ units/{id}/ activate / deactivate	—	Unit
Item Group	itemGroupApi.list	GET	/masters/item- groups	query: search?, status?, page, size	PagedResponse<ItemGroup>
	itemGroupApi.getById	GET	/masters/item- groups/{id}	—	ItemGroup
	itemGroupApi.create	POST	/masters/item- groups	ItemGroupPayload {name, description?}	ItemGroup
	itemGroupApi.update	PUT	/masters/item- groups/{id}	ItemGroupPayload	ItemGroup
	itemGroupApi.activate / deactivate	PATCH	/masters/item- groups/{id}/ activate / deactivate	—	ItemGroup
HSN	hsnApi.list	GET	/masters/hsn	query: search?, status?, page, size	PagedResponse<Hsn>
	hsnApi.getById	GET	/masters/hsn/ {id}	—	Hsn
	hsnApi.create	POST	/masters/hsn	HsnPayload {hsnCode, description?, taxRates: HsnTaxRate[]}	Hsn
	hsnApi.update	PUT	/masters/hsn/ {id}	HsnPayload	Hsn
	hsnApi.activate / deactivate	PATCH	/masters/hsn/ {id}/activate /deactivate	—	Hsn

PagedResponse<T> = { content: T[]; page: number; size: number; totalElements: number; totalPages: number }. All URLs above are relative to the axios instance's baseUrl (/api — see frontend/src/api/axios.ts), i.e. the full path is /api/masters/...

4. Backend

Controllers

All 9 controllers follow the identical shape: `list` (GET, no `@PreAuthorize`), `getById` (GET, no `@PreAuthorize`), `create` / `update` / `activate` / `deactivate` (all `@PreAuthorize`-guarded). Base path is `/api/masters/...` (from class-level `@RequestMapping`).

Master	Controller	Base path	create / update / activate / deactivate permission
Country	CountryController	/api/masters/countries	hasAuthority('PERM_MASTER_MANAGE')
State	StateController	/api/masters/states	hasAuthority('PERM_MASTER_MANAGE')
City	CityController	/api/masters/cities	hasAuthority('PERM_MASTER_MANAGE')
Zone	ZoneController	/api/masters/zones	hasAuthority('PERM_MASTER_MANAGE')
Currency	CurrencyController	/api/masters/currencies	hasAuthority('PERM_MASTER_MANAGE')
Nationality	NationalityController	/api/masters/nationalities	hasAuthority('PERM_MASTER_MANAGE')
Unit	UnitController	/api/masters/units	hasAuthority('PERM_MASTER_MANAGE')
Item Group	ItemGroupController	/api/masters/item-groups	hasAuthority('PERM_MASTER_MANAGE')
HSN	HsnController	/api/masters/hsn	hasAuthority('PERM_HSN_EDIT') (different permission from the other 8)

Exact endpoints per controller (identical pattern, `{base}` = the base path above):

- GET `{base}?search=&status=&page=0&size=10` → `list(...)`
- GET `{base}/{id}` → `getById(id)`
- POST `{base}` (`@Valid @RequestBody {X}Request`) → `create(...)`, returns 201 CREATED
- PUT `{base}/{id}` (`@Valid @RequestBody {X}Request`) → `update(...)`
- PATCH `{base}/{id}/activate` → `setStatus(id, {X}Status.ACTIVE)`
- PATCH `{base}/{id}/deactivate` → `setStatus(id, {X}Status.INACTIVE)`

Extra query params beyond `search/status/page/size`: `StateController.list` also takes `countryId`; `CityController.list` also takes `stateId` and `countryId`; `NationalityController.list` also takes `countryId`.

DTOs

Master	Request DTO (validation)	Response DTO (extra fields beyond id/status/timestamps)
Country	CountryRequest: name @NotBlank, code @NotBlank	CountryResponse: name, code
State	StateRequest: name @NotBlank, code (no constraint), countryId @NotNull	StateResponse: name, code, countryId, countryName
City	CityRequest: name @NotBlank, stateId @NotNull	CityResponse: name, stateId, stateName, countryId, countryName (country is resolved transitively via <code>city.getState().getCountry()</code>)

Master	Request DTO (validation)	Response DTO (extra fields beyond id/status/timestamps)
Zone	ZoneRequest: name @NotBlank, code (no constraint), countryId / stateId (both optional, no constraint)	ZoneResponse: name, code, countryId, countryName, stateId, stateName (country/state name are null when not set)
Currency	CurrencyRequest: name @NotBlank, code @NotBlank, symbol @NotBlank, decimalPlaces @NotNull @Min(0) @Max(4)	CurrencyResponse: name, code, symbol, decimalPlaces
Nationality	NationalityRequest: name @NotBlank, countryId @NotNull	NationalityResponse: name, countryId, countryName
Unit	UnitRequest: name @NotBlank, symbol @NotBlank	UnitResponse: name, symbol
Item Group	ItemGroupRequest: name @NotBlank, description (no constraint)	ItemGroupResponse: name, description
HSN	HsnRequest: hsnCode @NotBlank, description (no constraint), taxRates: List<HsnTaxRateRequest> @Valid (defaults to empty list)	HsnResponse: hsnCode, description, taxRates: List<HsnTaxRateResponse>
HSN tax rate	HsnTaxRateRequest: taxPercent @NotNull, cgstPercent / sgstPercent / igstPercent / cessPercent (no constraint), effectiveFrom @NotNull	HsnTaxRateResponse: id, taxPercent, cgstPercent, sgstPercent, igstPercent, cessPercent, effectiveFrom

All list endpoints return `PagedResponse<T> { content, page, size, totalElements, totalPages }` (backend/src/main/java/com/storehub/dto/PagedResponse.java), built via `PagedResponse.fromPage(springDataPage)`.

Validation failures on `create/update` (any `@Valid` violation) are caught globally by `GlobalExceptionHandler.handleValidation (MethodArgumentNotValidException)` and returned as `400 ApiError` with a `fieldErrors: Map<fieldName, message>` populated from each DTO field's own `message=` text above (e.g. "Country name is required").

Services

All 9 services (`CountryService`, `StateService`, `CityService`, `ZoneService`, `CurrencyService`, `NationalityService`, `UnitService`, `ItemGroupService`, `HsnService` in `backend/src/main/java/com/storehub/service/`) share the same shape: `search(...)`, `getById`, `create (@Transactional)`, `update (@Transactional)`, `setStatus (@Transactional)`, and a `findOrThrow(id)` helper that throws `MasterNotFoundException(entityName, id) → mapped by GlobalExceptionHandler to 404`.

Per-master business rules:

- **Country** (`CountryService`): uniqueness on `name` (case-insensitive) and `code` (case-insensitive), each checked with `existsBy...IgnoreCase` on `create` and `existsBy...IgnoreCaseAndIdNot` on `update` (self-exclusion), each throwing `BadRequestException` with a message naming the exact duplicate value, e.g. "A country named '{name}' already exists".
- **State** (`StateService`): injects `CountryService` to `findOrThrow` the parent country. Rejects `create/update` with `BadRequestException` if the resolved country's `status == CountryStatus.INACTIVE` — **except** on `update`, this rejection is skipped if the country being set is the same country the state already belonged to (`!country.getId().equals(oldCountryId)`), i.e. you cannot move a state onto an inactive country, but an already-inactive-country state can still be edited without being forced to switch countries. Uniqueness is scoped to the country: `existsByNameIgnoreCaseAndCountryId(name, countryId)` (and `...AndIdNot` on `update`), so the same state name is allowed under two different countries.

- **City** (CityService): identical pattern to State but one level down — validates the parent State (via StateService.findOrThrow) is not INACTIVE (same same-parent exception on update), and uniqueness is existsByNameIgnoreCaseAndStateId (scoped per state).
- **Zone** (ZoneService): country / state are **both optional** — resolved only if (request.getCountryId() != null) / if (request.getStateId() != null) (no inactive-parent check for zones, unlike State/City/Nationality). Uniqueness on name only (global, case-insensitive), via existsByNameIgnoreCase(AndIdNot) .
- **Currency** (CurrencyService): uniqueness on both code and name (case-insensitive), each with the same create/update pattern as Country.
- **Nationality** (NationalityService): parent Country required and must not be INACTIVE (same same-parent-on-update exception as State/City). Uniqueness on name only, global (not scoped to country) — existsByNameIgnoreCase(AndIdNot) .
- **Unit** (UnitService): uniqueness on both name and symbol (case-insensitive), same pattern as Country/Currency.
- **Item Group** (ItemGroupService): uniqueness on name only. description is free text with no uniqueness or format constraint.
- **HSN** (HsnService): uniqueness on hsnCode (case-insensitive), same create/update pattern. **Tax-rate handling** (the distinguishing behavior):
 - create() : builds the Hsn, then calls applyTaxRates(hsn, request.getTaxRates()), which iterates every HsnTaxRateRequest in the payload, builds a new HsnTaxRate entity from it, and calls hsn.addTaxRate(rate) (which both appends to the list and sets the back-reference rate.setHsn(this), required because the FK is on HsnTaxRate.hsn_id). cascade = ALL on the @OneToMany means saving the Hsn also persists every HsnTaxRate row in one transaction.
 - update() : **full replace**, not a diff/merge — calls hsn.clearTaxRates() (which nulls out each existing rate's hsn back-reference and clears the Java list; combined with orphanRemoval = true this causes Hibernate to DELETE every previously-stored hsn_tax_rates row for that HSN on flush) and then applyTaxRates(hsn, request.getTaxRates()) again to re-insert whatever the request body currently contains. **Consequence**: any tax-rate row not resent in the taxRates array on an update is permanently deleted, including its id and history — there is no per-row create/update/delete distinction on the wire, the frontend always sends the complete current array.
- No uniqueness or overlap validation exists across effectiveFrom dates within one HSN's tax rates — multiple rates with the same or overlapping effectiveFrom are accepted; the frontend's latestRate() picks the one with the lexicographically-latest effectiveFrom string for list-table display.

Repository

Standard Spring Data JpaRepository<Entity, Long> interfaces in backend/src/main/java/com/storehub/repository/, each with hand-written existsBy... uniqueness checks and one @Query-annotated search(...) method (JPQL, case-insensitive LIKE on the searchable text field(s), :status IS NULL OR ... = :status to make status optional):

Repository	existsBy... methods	search(...) params	Joins
CountryRepository	existsByNameIgnoreCase , existsByNameIgnoreCaseAndIdNot , existsByCodeIgnoreCase , existsByCodeIgnoreCaseAndIdNot	search, status	—
StateRepository	existsByNameIgnoreCaseAndCountryId , existsByNameIgnoreCaseAndCountryIdAndIdNot	search, countryId, status	JOIN s.country c

Repository	existsBy... methods	search(...) params	Joins
CityRepository	existsByNameIgnoreCaseAndStateId , existsByNameIgnoreCaseAndStateIdAndIdNot	search, stateId, countryId, status	JOIN ci.state s JOIN s.country c
ZoneRepository	existsByNameIgnoreCase , existsByNameIgnoreCaseAndIdNot	search, status	—
CurrencyRepository	existsByCodeIgnoreCase(AndIdNot) , existsByNameIgnoreCase(AndIdNot)	search, status	—
NationalityRepository	existsByNameIgnoreCase(AndIdNot)	search, countryId, status	JOIN n.country c
UnitRepository	existsByNameIgnoreCase(AndIdNot) , existsBySymbolIgnoreCase(AndIdNot)	search, status	—
ItemGroupRepository	existsByNameIgnoreCase(AndIdNot)	search, status	—
HsnRepository	existsByHsnCodeIgnoreCase(AndIdNot)	search, status (search matches hsnCode OR description)	—

All `search()` queries sort by `Sort.by("name").ascending()` at the service layer
(`Sort.by("hsnCode").ascending()` for HSN specifically, since it has no `name` field).

5. Database

Tables

Entity	@Table name	Key columns (from entity @Column)
Country	countries	id, name (unique, len 100), code (unique, len 10), status (len 20), created_at, updated_at
State	states	id, name (len 100), code (len 10, nullable), country_id (FK, not null), status, created_at, updated_at
City	cities	id, name (len 100), state_id (FK, not null), status, created_at, updated_at
Zone	zones	id, name (unique, len 100), code (len 20, nullable), country_id (FK, nullable), state_id (FK, nullable), status, created_at, updated_at
Currency	currencies	id, name (unique, len 100), code (unique, len 10), symbol (len 5), decimal_places, status, created_at, updated_at
Nationality	nationalities	id, name (unique, len 100), country_id (FK, not null), status, created_at, updated_at
Unit	units	id, name (unique, len 50), symbol (unique, len 10), status, created_at, updated_at
ItemGroup	item_groups	id, name (unique, len 100), description (len 255, nullable), status, created_at, updated_at
Hsn	hsn_codes	id, hsn_code (unique, len 20), description (len 255, nullable), status, created_at, updated_at

Entity	@Table name	Key columns (from entity @Column)
HsnTaxRate	hsn_tax_rates	id, hsn_id (FK, not null), tax_percent (decimal 5,2, not null), cgst_percent, sgst_percent, igst_percent, cess_percent (all decimal 5,2, nullable), effective_from (date, not null), created_at (no updated_at — rows are replace-only, never individually edited in place)

Note the entity class is `Hsn` but the physical table is `hsn_codes`, not `hsns` or `hsn`.

Relationships

Within this module:

- `State.country_id` → `countries.id` (@ManyToOne required)
- `City.state_id` → `states.id` (@ManyToOne required)
- `Zone.country_id` → `countries.id` (@ManyToOne **optional**)
- `Zone.state_id` → `states.id` (@ManyToOne **optional**)
- `Nationality.country_id` → `countries.id` (@ManyToOne required)
- `HsnTaxRate.hsn_id` → `hsn_codes.id` (@ManyToOne required; owning side of `Hsn.taxRates` @OneToMany(cascade=ALL, orphanRemoval=true))

Downstream — other modules that reference these masters by FK (grepped across `backend/src/main/java/com/storehub/entity`):

This master	Referenced by	FK column	Relationship
Country	Store	store.country_id	@ManyToOne
	Party	party.country_id	@ManyToOne
State	Store	store.state_id	@ManyToOne
	Party	party.state_id	@ManyToOne
	Employee	employee.state_id	@ManyToOne
	PartyAddress	party_addresses.state_id	@ManyToOne
	BusinessGstConfig	business_gst_config.state_id	@ManyToOne
City	Store	store.city_id	@ManyToOne
	Party	party.city_id	@ManyToOne
	Employee	employee.city_id	@ManyToOne
	PartyAddress	party_addresses.city_id	@ManyToOne
Zone	Store	store.zone_id	@ManyToOne
Unit	Product	product.purchase_unit_id and product.sale_unit_id (two separate FKs , not one)	@ManyToOne
ItemGroup	Product	product.item_group_id	@ManyToOne
Hsn	Product	product.hsn_id	@ManyToOne
Currency	(none found)	—	Currency has no downstream FK reference anywhere in the entity package — it is a standalone lookup not yet wired into Product / Store / Party /anything else (NOT FOUND IN CURRENT CODEBASE)

This master	Referenced by	FK column	Relationship
Nationality	(none found)	—	Same — no other entity has a <code>nationality_id</code> FK (NOT FOUND IN CURRENT CODEBASE)

6. Validation

- Field-level (Bean Validation, enforced by `@Valid @RequestBody` on every `create/update` controller method): see the DTO table in §4 — `@NotBlank` / `@NotNull` on required fields, `@Min(0)` / `@Max(4)` on `Currency.decimalPlaces`. A violation throws `MethodArgumentNotValidException`, caught globally and returned as `400` with `fieldErrors`.
- Business-rule validation (in the service layer, throwing `BadRequestException` → `400`, message-only, no `fieldErrors` map — the field this refers to is implicit from the message text, not a DTO field name):
- Duplicate name/code/symbol checks (all 9 masters, case-insensitive, self-excluded on update) — see §4 Services for the exact scope of each (global vs. parent-scoped).
- Inactive-parent checks: `State/City/Nationality` reject a `create/update` that would attach to an `INACTIVE` parent (Country for `State/Nationality`, `State` for `City`), with an exception carved out for editing a record without changing its existing (already inactive) parent.
- No cross-field validation exists beyond the above (e.g. no check that `HsnTaxRate.cgstPercent` + `sgstPercent` sums to `taxPercent`, and no check preventing duplicate/overlapping `effectiveFrom` dates within one HSN's tax rates — `NOT FOUND IN CURRENT CODEBASE`).

7. Authentication / Authorization

- **Baseline:** `SecurityConfig.filterChain` requires `anyRequest().authenticated()` for everything except `/api/auth/**` and `/actuator/health` — so every one of these endpoints, including the unguarded `list/getById`, requires a valid JWT (`JwtAuthenticationFilter`).
- **list / getById:** no `@PreAuthorize` on any of the 9 controllers — any authenticated user, regardless of role/permission, can view. (In practice every `Role` in `RolePermissions` that has any master-related access also has `MASTER_VIEW`, but that permission is never actually checked by these two endpoints — it is enforced only implicitly by every role granting it.)
- **create / update / activate / deactivate:** `@PreAuthorize("hasAuthority('PERM_MASTER_MANAGE')")` on all 8 non-HSN controllers; **HSN uniquely uses** `@PreAuthorize("hasAuthority('PERM_HSN_EDIT')")` **instead** — a different permission, so a role that has `MASTER_MANAGE` but not `HSN_EDIT` (or vice versa) would be able to manage the other 8 masters but not HSN, or vice versa. In the current `RolePermissions` mapping, only `ADMIN` and `STORE_MANAGER` hold `MASTER_MANAGE`; `HSN_EDIT` is not granted to any role in `RolePermissions.build()` except implicitly via `ADMIN`'s `EnumSet.allOf(Permission.class)` and `STORE_MANAGER`'s copy of it (`HSN_EDIT` is not in `STORE_MANAGER`'s removal list, so `STORE_MANAGER` retains it too).
- `PERM_<name>` authorities are minted per-request from `UserPrincipal.getAuthorities()`, which iterates `RolePermissions.forRole(user.getRole())` and adds a `SimpleGrantedAuthority("PERM_" + permission.name())` per permission the user's role has (backend/src/main/java/com/storehub/security/UserPrincipal.java).
- **Frontend route gate** (`frontend/src/App.tsx`, `frontend/src/components/ManagerRoute.tsx`): all `/masters/*` routes, including all 9 of these master pages, are wrapped in `<Route element={<ManagerRoute />}>` with no `extraRoles`, so `ManagerRoute` only allows `user.role` to be `ADMIN` or `STORE_MANAGER` — any other authenticated role is redirected to `/unauthorized` before it can even reach the page, **regardless of what** `MASTER_VIEW` / `MASTER_MANAGE` / `HSN_EDIT` **the backend would otherwise allow them**. This means the backend's more granular view-only

access for ACCOUNTANT/SALES_USER/PURCHASE_USER/INVENTORY_USER/STAFF (all of whom hold MASTER_VIEW per RolePermissions) is currently unreachable through this UI — those roles can never navigate to a master page at all, only ADMIN/STORE_MANAGER can.

8. Permissions

From `backend/src/main/java/com/storehub/entity/Permission.java` and `backend/src/main/java/com/storehub/service/RolePermissions.java`:

- MASTER_VIEW (module "Master") — doc comment on the enum literal itself states: "Simple lookup masters not individually named by the spec (Currency, Country, State, City, Zone, Nationality, Unit, Item Group, Payment Method, Business GST Config, Expense Category, Supplier, Customer)." Held by ADMIN, STORE_MANAGER, ACCOUNTANT, SALES_USER, PURCHASE_USER, INVENTORY_USER, STAFF — i.e. every role. (Not actually enforced by any @PreAuthorize in this module — see §7.)
- MASTER_MANAGE (module "Master") — governs create/update/activate/deactivate for the 8 non-HSN masters. Held by ADMIN and STORE_MANAGER only.
- HSN_VIEW / HSN_EDIT (module "Master") — HSN_EDIT governs HSN's create/update/activate/deactivate specifically (see §7); HSN_VIEW is defined but **not referenced by any @PreAuthorize in HsnController** (NOT FOUND IN CURRENT CODEBASE as an enforced check — HSN's list/getById are unguarded, same as the other 8). Both held by ADMIN and (via its EnumSet.copyOf(admin) minus an explicit removal list that does not name HSN_VIEW / HSN_EDIT) STORE_MANAGER.

9. Transaction Handling

- Every mutating service method (create, update, setStatus) across all 9 services is annotated @Transactional (Spring, class-level import org.springframework.transaction.annotation.Transactional, method-level use).
- search/getById (read paths) are **not** annotated @Transactional — they run in default (non-transactional, or DataSource-autocommit) read mode. Lazy-loaded relations (State.country, City.state, Zone.country/state, Nationality.country) are still safely dereferenced inside XxxResponse.fromEntity(...) because that mapping happens inside the (still-open, request-scoped, OSIV-backed — spring.jpa.open-in-view default) Hibernate session; no explicit @Transactional(readOnly = true) is used anywhere in this module (NOT FOUND IN CURRENT CODEBASE).
- HSN's create/update transaction is the one case where multiple entities are written atomically in one call: saving the parent Hsn (via hsnRepository.save(hsn)) cascades (CascadeType.ALL) to insert/delete every HsnTaxRate child row in the same transaction — if any part fails, the whole HSN + its tax rates roll back together.
- No manual/programmatic transaction management (TransactionTemplate, @Transactional(propagation=...), etc.) is used anywhere in this module — plain declarative @Transactional only.

10. Error Handling

Centralized in `backend/src/main/java/com/storehub/exception/GlobalExceptionHandler.java` (@RestControllerAdvice), producing a uniform ApiError { timestamp, status, error, message, path, fieldErrors? } body (fieldErrors is omitted from JSON via @JsonInclude(NON_NULL) when null):

Exception	Thrown by	HTTP status	fieldErrors ?
MethodArgumentNotValidException	Spring, from <code>@Valid</code> DTO failures on <code>create / update</code>	400 BAD_REQUEST	Yes — one entry per invalid field, DTO field name → its <code>message=</code>
BadRequestException	Service-layer business rules (duplicate name/ code, inactive parent)	400 BAD_REQUEST	No — message only
MasterNotFoundException	<code>findOrThrow(id)</code> in every service, for a nonexistent <code>id</code> on <code>getById / update / activate / deactivate</code>	404 NOT_FOUND	No — message is <code>"{EntityName} not found with id: {id}"</code>
AccessDeniedException	Spring Security, when <code>@PreAuthorize</code> fails	403 FORBIDDEN	No — fixed message <code>"You do not have permission to access this resource"</code>
Any other unhandled Exception	—	500 INTERNAL_SERVER_ERROR	No — fixed generic message; full exception is logged server-side via <code>log.error(...)</code>

On the frontend, `frontend/src/utils/apiError.ts`'s `parseApiError(err, fallbackMessage)` turns this into a `{message, fieldErrors}` pair: if the response carries `fieldErrors`, it builds one combined sentence (`"Field Name: reason | Field Name2: reason2"`, field names humanized by inserting spaces before capitals) for the top-level `Alert`, while also returning the raw `fieldErrors` map so `MasterCrudPage`'s `renderForm` callback can mark each `Input` invalid and show its own inline `<p>` message. If the request never reached the server at all (`!err.response`), it instead returns a fixed `"Could not reach the server..."` message with no field errors.

11. Audit Flow

NOT FOUND IN CURRENT CODEBASE. None of the 9 services in this module (`CountryService`, `StateService`, `CityService`, `ZoneService`, `CurrencyService`, `NationalityService`, `UnitService`, `ItemGroupService`, `HsnService`) import or call `AuditService` — confirmed by grepping every file that references `AuditService` across `backend/src/main/java/com/storehub`, which returns 21 files (e.g. `ProductService`, `InventoryService`, `SaleService`, `UserService`, ...) and none of the 9 master services. Create/update/activate/deactivate on these lookup masters produce **no audit log entry** (`AuditLog` / `audit_log` table is untouched by this module).

12. Important Side Effects

- **No hard delete anywhere.** There is no `DELETE` endpoint or repository `deleteById` call exposed for any of these 9 masters — the only way to "remove" a record from active use is `PATCH .../deactivate`, which just flips `status` to `INACTIVE`. Deactivated records remain fully queryable (via `?status=INACTIVE` or omitting the filter) and are never physically removed.
- **Deactivating a parent does not cascade-deactivate or block existing children.** E.g. deactivating a `Country` does not touch its `States`, and an existing `State` already pointing at that now-inactive `Country` is left as-is — the inactive-parent check in `StateService` / `CityService` /

`NationalityService` only fires on a new create or on changing a record's parent via update, never retroactively.

- **HSN update fully replaces the tax-rate child rows** (see §4 Services) — any row omitted from the `taxRates` array on a PUT is hard-deleted (`orphanRemoval = true`), including its database `id` and its `created_at`. There is no soft-delete or history retention for individual tax-rate rows; the only "history" that exists is whatever rows the caller chooses to keep resending.
- `Currency` and `Nationality` are the two masters in this group with zero downstream FK consumers found anywhere else in the codebase (§5) — creating/deactivating/editing them has no effect on any other module's data today.
- Activating/deactivating any of these 8 non-HSN masters or HSN itself does **not** validate or affect records elsewhere that already reference it by FK (e.g. deactivating a `Unit` that a `Product.saleUnit` still points to does not warn, block, or update that `Product` — NOT FOUND IN CURRENT CODEBASE).

13. Dependencies on Other Modules

Consumers of these 9 masters, grepped from `backend/src/main/java/com/storehub/entity` (FK) and cross-referenced with frontend master pages using the same `*Api` clients:

This master	Consuming module / entity	How
Country, State, City	Store (<code>Store</code> entity / Store master)	<code>store.country_id</code> , <code>store.state_id</code> , <code>store.city_id</code> — a Store's address is built from these three masters
Zone	Store	<code>store.zone_id</code> — a Store can be assigned a distribution/sales zone
Country, State, City	Party (<code>Party</code> entity — Supplier/Customer master)	<code>party.country_id</code> , <code>party.state_id</code> , <code>party.city_id</code> for the party's primary address
State, City	PartyAddress (<code>Party</code> 's multi-address child table)	<code>party_addresses.state_id</code> , <code>party_addresses.city_id</code> — each of a party's additional addresses can independently pick a state/city
State, City	Employee (<code>Employee</code> master)	<code>employee.state_id</code> , <code>employee.city_id</code> for the employee's address
State	BusinessGstConfig (seller GST registration singleton)	<code>business_gst_config.state_id</code> , used for GST place-of-supply determination
Unit	Product (Item Master)	<code>product.purchase_unit_id</code> and <code>product.sale_unit_id</code> — a product's purchase and sale units are independently selectable from the <code>units</code> table
ItemGroup	Product	<code>product.item_group_id</code> — a product's classification group
Hsn	Product	<code>product.hsn_id</code> — a product's HSN code, which in turn determines its <code>HsnTaxRate</code> history for GST calculations
Country, State, City, Zone, Currency, Nationality, Unit, Item Group	MastersDashboard.tsx	Reads each master's <code>totalElements</code> via <code>list({size: 1})</code> to show a record count on its dashboard card

No module outside this one writes to any of these 9 masters' tables — all mutation goes through the 9 services above.

14. Key Operation Flows

Country — Create → Edit → Activate/Deactivate

Create:

1. UI: `CountryMaster.tsx` → "Add Country" → `openAdd()` (from `MasterCrudPage`) opens the shared `Dialog` with `emptyValues = {name:'', code:''}`.
2. User fills Name/Code → submits → `MasterCrudPage.handleSubmit` → `create(values)`, which is `countryApi.create`.
3. API: `countryApi.create(payload)` → `POST /api/masters/countries` with body `CountryPayload {name, code}`.
4. Controller: `CountryController.create(@Valid @RequestBody CountryRequest request)` — `@PreAuthorize("hasAuthority('PERM_MASTER_MANAGE')")` checked first by Spring Security; then `@Valid` runs `CountryRequest`'s `@NotBlank` checks on `name/code`.
5. DTO → Service: `countryService.create(request)`.
6. Service: `CountryService.create` — `countryRepository.existsByNameIgnoreCase(name)` and `existsByCodeIgnoreCase(code)`; if either is true, throws `BadRequestException("A country named '...' already exists") / "...code '...' already exists"`. Otherwise builds `Country.builder().name(...).code(...).build()` (status left null so `@PrePersist` defaults it to `ACTIVE`) and calls `countryRepository.save(country)`.
7. Repository/Database: Hibernate `INSERT INTO countries (name, code, status, created_at, updated_at) VALUES (...)` (`status='ACTIVE'`, both timestamps = now).
8. Response: `CountryResponse.fromEntity(saved)` → controller wraps it `ResponseEntity.status(201).body(...)`.
9. UI: `MasterCrudPage` gets the resolved promise → `toast.success("Country created successfully")` → closes dialog → `load()` refetches the list, showing the new row with an `ACTIVE` green `Badge`.

Edit: row `"..."` → "Edit" → `openEdit(item)` seeds the form from `toFormValues(c) = {name: c.name, code: c.code}` → submit → `update(item.id, values) = countryApi.update(id, payload)` → `PUT /api/masters/countries/{id}` → `CountryController.update (@PreAuthorize PERM_MASTER_MANAGE, @Valid)` → `CountryService.update(id, request): findOrThrow(id)` (404 `MasterNotFoundException` if missing) → re-checks name/code uniqueness with `...AndIdNot(value, id)` (skipped entirely if the value didn't change, via the `!country.getName().equalsIgnoreCase(...)` guard) → mutates the managed entity's `name/code` → `save()` → `@PreUpdate` bumps `updated_at` → same `CountryResponse` shape back → UI toasts "Country updated successfully" and reloads.

Activate/Deactivate: row `"..."` → "Deactivate" (shown when `status==='ACTIVE'`) or "Activate" → `toggleStatus(item)` → `deactivate(item.id) / activate(item.id) = countryApi.deactivate / activate(id)` → `PATCH /api/masters/countries/{id}/deactivate (or /activate)` → `CountryController.deactivate / activate (@PreAuthorize PERM_MASTER_MANAGE)` → `countryService.setStatus(id, CountryStatus.INACTIVE|ACTIVE)` → `findOrThrow` → `country.setStatus(...)` → `save()` (no uniqueness/business check at all on this path) → `CountryResponse` back → UI toasts and reloads, row's `Badge` flips to `muted / INACTIVE` or `success / ACTIVE`.

HSN — Create → Edit → Activate/Deactivate (child tax-rate pattern)

Create:

1. UI: `HsnMaster.tsx` (its own page, not `MasterCrudPage`) → "Add HSN Code" → `openAdd()` seeds `EMPTY = {hsnCode:'', description:'', taxRates:[emptyTaxRate()]}` (one blank rate row, `taxPercent:0`, `effectiveFrom = today`).
2. User fills HSN Code/Description, then one or more tax-rate rows (Tax %/CGST %/SGST %/IGST %/Effective From), using "Add Rate"/trash-icon to grow/shrink `values.taxRates` — all client-side state, no network call per row.

3. Submit → `handleSubmit` → `hsnApi.create(values)` → `POST /api/masters/hsn` with body `HsnPayload {hsnCode, description, taxRates: HsnTaxRate[]}`.
4. Controller: `HsnController.create` — `@PreAuthorize("hasAuthority('PERM_HSN_EDIT')")` (not `MASTER_MANAGE`, unlike the other 8) → `@Valid` validates `HsnRequest.hsnCode` `@NotBlank` and, via `@Valid` cascading onto `taxRates`, each `HsnTaxRateRequest.taxPercent/effectiveFrom` `@NotNull`.
5. Service: `HsnService.create(request)` — `hsnRepository.existsByHsnCodeIgnoreCase(...)` uniqueness check (`BadRequestException` if duplicate) → builds `Hsn.builder().hsnCode(...).description(...).build()` → `applyTaxRates(hsn, request.getTaxRates())`: for each `HsnTaxRateRequest`, builds an `HsnTaxRate` and calls `hsn.addTaxRate(rate)` (appends to the in-memory list **and** sets `rate.setHsn(hsn)` so the FK is populated) → `hsnRepository.save(hsn)`.
6. Database: because `Hsn.taxRates` is `@OneToMany(cascade = CascadeType.ALL)`, one `save()` call produces both `INSERT INTO hsn_codes (...)` and one `INSERT INTO hsn_tax_rates (hsn_id, tax_percent, cgst_percent, sgst_percent, igst_percent, cess_percent, effective_from, created_at)` per tax-rate row, all in the same transaction (`@Transactional` on the service method).
7. Response: `HsnResponse.fromEntity(saved)` maps `taxRates` via `HsnTaxRateResponse::fromEntity` for each child row, including its generated `id`.
8. UI: toast "HSN code created successfully", dialog closes, list reloads; the new row shows the first rate's `taxPercent` in the "Current Tax %" column (via `latestRate()` picking the max `effectiveFrom`).

Edit (the child-replace step): "Edit" → `openEdit(item)` seeds `taxRates: item.taxRates.length > 0 ? item.taxRates.map(r => ({...r})) : [emptyTaxRate()]` (shallow-copies existing rows, including their `ids`, so they can be edited in place client-side) → user edits/add/removes rows → submit → `hsnApi.update(id, values)` → `PUT /api/masters/hsn/{id}` → `HsnController.update ( PERM_HSN_EDIT )` → `HsnService.update(id, request): findOrThrow` → uniqueness re-check on `hsnCode (AndIdNot)` → sets `hsnCode/description` → `hsn.clearTaxRates()` (nulls each existing `HsnTaxRate.hsn` and empties the Java list — combined with `orphanRemoval=true` this schedules a `DELETE FROM hsn_tax_rates WHERE hsn_id = ?` for every row that was there) → `applyTaxRates(hsn, request.getTaxRates())` again, which re-inserts a brand-new `HsnTaxRate` row (new auto-generated `id`) for every entry currently in the submitted array — even ones that look unchanged from the client's perspective get a new database row/id. → `save()`. Net effect: the request body's `taxRates` array is the single source of truth for that HSN's entire tax-rate history after every update; nothing is preserved that wasn't resent.

Activate/Deactivate: identical mechanics to Country's (`toggleStatus` → `hsnApi.activate/deactivate(id)` → `PATCH /api/masters/hsn/{id}/activate|deactivate, PERM_HSN_EDIT` → `HsnService.setStatus` → `save()`), and does **not** touch `taxRates` at all — deactivating an HSN code leaves its tax-rate rows completely untouched in the database.

15. Manual Changes

To add a brand-new simple master following this exact pattern (e.g. a hypothetical "Warehouse Type" master with just a unique `name`), touch these files in this order:

1. **Entity:** `backend/src/main/java/com/storehub/entity/WarehouseType.java` — copy the shape of `ItemGroup.java` (`id`, unique `name`, `status`, `createdAt/updatedAt`, `@PrePersist/@PreUpdate` with `status` defaulting to `ACTIVE`). Set `@Table(name = "warehouse_types")`.
2. **Status enum:** `backend/src/main/java/com/storehub/entity/WarehouseTypeStatus.java` — `enum { ACTIVE, INACTIVE }`, copy `ItemGroupStatus.java`.
3. **Repository:** `backend/src/main/java/com/storehub/repository/WarehouseTypeRepository.java` — extends `JpaRepository<WarehouseType, Long>` with `existsByNameIgnoreCase/existsByNameIgnoreCaseAndIdNot` and a `@Query`-based `search(search, status, pageable)`, copy `ItemGroupRepository.java`.

4. **DTOs:** backend/src/main/java/com/storehub/dto/WarehouseTypeRequest.java (@NotBlank name) and WarehouseTypeResponse.java (fromEntity static factory), copy ItemGroupRequest / ItemGroupResponse .
5. **Service:** backend/src/main/java/com/storehub/service/WarehouseTypeService.java — search / getById / create / update / setStatus / findOrThrow , copy ItemGroupService.java (or CountryService.java if it needs a parent-master FK, or HsnService.java if it needs a child collection).
6. **Controller:** backend/src/main/java/com/storehub/controller/WarehouseTypeController.java — @RequestMapping("/api/masters/warehouse-types") , copy ItemGroupController.java exactly, including @PreAuthorize("hasAuthority('PERM_MASTER_MANAGE')") on the 4 mutating endpoints (reuse MASTER_MANAGE / MASTER_VIEW unless the new master needs its own dedicated permission the way HSN does with HSN_EDIT — if so, add a new Permission enum constant and grant it to the appropriate roles in RolePermissions.build()).
7. **Frontend type:** add WarehouseType / WarehouseTypePayload interfaces to frontend/src/types/masters.ts , copying the ItemGroup / ItemGroupPayload block.
8. **Frontend API client:** add a warehouseTypeApi export to frontend/src/api/mastersApi.ts , copying the itemGroupApi block (list / getById / create / update / activate / deactivate).
9. **Frontend page:** frontend/src/pages/masters/WarehouseTypeMaster.tsx — copy ItemGroupMaster.tsx (or CountryMaster.tsx / StateMaster.tsx if a parent-master dropdown is needed), configuring <MasterCrudPage<WarehouseType, WarehouseTypePayload>> with the new icon/title/columns/ renderForm / api callbacks.
10. **MastersDashboard card:** add one entry to the CARDS array in frontend/src/pages/MastersDashboard.tsx (key, name, description, icon, path: '/masters/warehouse-types', loadCount: () => warehouseTypeApi.list({size:1}).then(r => r.data.totalElements)).
11. **App.tsx route:** add import WarehouseTypeMaster from './pages/masters/WarehouseTypeMaster'; and <Route path="/masters/warehouse-types" element={<WarehouseTypeMaster />} /> inside the existing <Route element={<ManagerRoute />}> block that wraps /masters/* (around line 259 of frontend/src/App.tsx).
12. **Nav:** individual masters are **not** listed in the sidebar directly — frontend/src/components/layout/nav-items.ts has exactly one entry for the whole module, { label: 'Masters', path: '/masters', icon: Landmark, allowedRoles: ['ADMIN', 'STORE_MANAGER'] }, which links to MastersDashboard's card grid (step 10's CARDS array is what actually surfaces the new master to users). No nav change is needed for a new simple master beyond step 10, as long as it's reached via a dashboard card. (allowedRoles here mirrors ManagerRoute's ADMIN / STORE_MANAGER gate — if that ever changes, update both in sync.)

To add a field to an existing master (e.g. adding phoneCode to Country): add the @Column to Country.java → add it to CountryRequest (with any @NotBlank / @NotNull / etc. if required) and CountryResponse (+ its fromEntity mapping) → add it to CountryService.create / update (setting it on the builder/entity) → add it to the frontend Country / CountryPayload interfaces in types/masters.ts → add the corresponding input to CountryMaster.tsx's renderForm (and to its toFormValues / emptyValues if it's editable) and, if it should be visible in the list, to its columns array.

To change HSN's tax-rate handling (e.g. to preserve tax-rate history instead of full-replace-on-update): the only place to change is HsnService.update() in backend/src/main/java/com/storehub/service/HsnService.java — replace the hsn.clearTaxRates(); applyTaxRates(hsn, request.getTaxRates()); full-replace with per-row diff logic (match incoming HsnTaxRateRequest's that carry an existing id against hsn.getTaxRates() to update in place, add new ones without an id, and only remove ones genuinely dropped from the request). The DTOs (HsnTaxRateRequest / HsnTaxRateResponse) already carry id end-to-end (frontend HsnTaxRate.id?: number in types/masters.ts , sent back as-is on edit via openEdit's item.taxRates.map(r => ({...r}))), so no frontend or DTO shape change would be needed for that specific improvement — only the service-layer replace logic and (if row-level immutability of createdAt / effectiveFrom matters) possibly adding an updatedAt column to HsnTaxRate / hsn_tax_rates .

Party / Customer / Supplier Module

1. Overview

Contrary to the "Party is a generic superset used by Sale/Purchase, Customer/Supplier are legacy" assumption, **the actual code shows the opposite**: `Party` is an **isolated, standalone master** with no foreign-key relationship to any transactional entity anywhere in the backend, while `Customer` and `Supplier` are the entities that are actually wired into every transaction (Sale, Purchase, Orders, Payments, Receipts, Notes, Ledgers).

Evidence:

- `Party` (`backend/src/main/java/com/storehub/entity/Party.java` , table `parties`) has a `partyType` enum (`SUPPLIER` , `CUSTOMER` , `BOTH`), its own GSTIN/PAN, city/state/country FKs, a `dealsInCategories` many-to-many to `Category` , and a one-to-many to `PartyAddress` (additional addresses). It is managed via `/api/masters/parties` (`PartyController` → `PartyService` → `PartyRepository`).
- **Grep across the whole entity package for private `Party`** returns exactly one hit outside `Party` itself: `PartyAddress.java` (`private Party party;` , the address's own owning-side FK). **No other entity** (`Sale` , `Purchase` , `SalesOrder` , `PurchaseOrder` , `Payment` , `Receipt` , `CreditNote` , `DebitNote` , `CustomerLedgerEntry` , `SupplierLedgerEntry` , etc.) references `Party` . `PartyService` / `PartyRepository` are also not referenced by any other backend service (`grep "partyRepository"` across `com.storehub` only matches `PartyService.java`).
- **Grep for private `Customer` / private `Supplier`** across `entity/` shows these are the ones actually used downstream:
 - `Sale.java:44` → `private Customer customer;`
 - `SalesOrder.java:37` → `private Customer customer;`
 - `Receipt.java:37` → `private Customer customer;`
 - `CreditNote.java:52` → `private Customer customer;`
 - `CustomerLedgerEntry.java:35` → `private Customer customer;`
 - `Purchase.java:34` → `private Supplier supplier;`
 - `PurchaseOrder.java:37` → `private Supplier supplier;`
 - `Payment.java:37` → `private Supplier supplier;`
 - `Expense.java:62` → `private Supplier supplier;`
 - `DebitNote.java:43` → `private Supplier supplier;`
 - `SupplierLedgerEntry.java:36` → `private Supplier supplier;`
- On the frontend, `partyApi` (`frontend/src/api/mastersApi.ts`) is imported **only** by `pages/MastersDashboard.tsx` and `pages/masters/PartyMaster.tsx` . It is never imported by any Sales/Purchase form. In contrast, `supplierApi` / `customerApi` (and their quick-add modals) are wired directly into `SalesBillForm.tsx` , `SalesOrderForm.tsx` , `KacchiSaleForm.tsx` , `PurchaseBillForm.tsx` , `PurchaseOrderForm.tsx` , `KacchiPurchaseForm.tsx` .

Conclusion: `Customer` and `Supplier` are the two entities that actually back every sales/purchase/accounting transaction in this codebase. `Party` is a separate, self-contained "Party Master" (under `Masters` → `Parties`) that models a generic business party (supplier, customer, or both, with GSTIN/PAN, categories, multiple addresses) but as of the current codebase is **not consumed by any other module** — it has no downstream FK usage at all. It functions as a dormant/parallel master, not a superset that Sale/Purchase resolve against. `Customer` and `Supplier` also have **no relationship to each other or to `Party`** at the entity/FK level — they are three fully independent JPA entities/tables that happen to model overlapping real-world concepts.

Also notable: `PartyLedgerService.java` (accounting ledger, not part of this module's own service set) resolves ledger party names by looking up `CustomerRepository / SupplierRepository` directly (an `AccountingPartyType` discriminator), **not** `PartyRepository` — reinforcing that the accounting subsystem treats Customer/Supplier, not Party, as the real party masters.

2. Frontend

Files and Components

File	Role
<code>frontend/src/pages/masters/PartyMaster.tsx</code>	Full CRUD page for Party master (list/create/edit/activate/deactivate), built on the generic <code>MasterCrudPage</code> component. Tabs: Basic Info, Location & Tax, Categories & Addresses.
<code>frontend/src/pages/Suppliers.tsx</code>	Supplier list page: search, status filter, sort, activate/deactivate, links to <code>SupplierForm</code> , opens <code>SupplierViewModal</code> .
<code>frontend/src/pages/SupplierForm.tsx</code>	Create/edit form for Supplier (routed as <code>/suppliers/new</code> and <code>/suppliers/:id/edit</code>).
<code>frontend/src/components/SupplierViewModal.tsx</code>	Read-only detail modal for a supplier, including its purchase history (GET <code>/api/suppliers/{id}/purchases</code>).
<code>frontend/src/components/SupplierQuickAddModal.tsx</code>	Minimal inline "Add Supplier" modal (name, mobile, email, address) used from Purchase forms.
<code>frontend/src/components/CustomerQuickAddModal.tsx</code>	Minimal inline "Add Customer" modal (firstName, lastName, mobile, email, state) used from Sale forms.
<code>frontend/src/api/supplierApi.ts</code>	Axios wrapper for <code>/api/suppliers</code> .
<code>frontend/src/api/customerApi.ts</code>	Axios wrapper for <code>/api/customers</code> .
<code>frontend/src/api/mastersApi.ts</code> (<code>partyApi</code>)	Axios wrapper for <code>/api/masters/parties</code> .
<code>frontend/src/types/supplier.ts</code>	<code>Supplier</code> , <code>SupplierPayload</code> , <code>SupplierStatus</code> , <code>SupplierPurchaseSummary</code> types.
<code>frontend/src/types/sale.ts</code>	<code>Customer</code> , <code>CustomerCreatePayload</code> , <code>CustomerStatus</code> types (Customer type lives with Sale, not with a "customer" file).
<code>frontend/src/types/masters.ts</code> (lines ~204-262)	<code>Party</code> , <code>PartyType</code> , <code>PartyAddress</code> , <code>PartyPayload</code> types.
There is no <code>Customers.tsx</code> / <code>CustomerForm.tsx</code> / <code>CustomerViewModal.tsx</code> page.	NOT FOUND IN CURRENT CODEBASE

Frontend Flow

Party (`PartyMaster.tsx` , via `MasterCrudPage`):

- List: `partyApi.list({search, status, page, size})` → paginated table (Code, Name, Mobile, Type badge).
- Create/Edit: tabbed form (Basic Info / Location & Tax / Categories & Addresses) submitted through `partyApi.create` / `partyApi.update`; per-field errors shown via `fieldErrors` (`invalid prop` + `<p>` feedback), matching the `fieldErrors-map` pattern from `Register.tsx / UserFormModal.tsx`.
- Deactivate/Activate: `partyApi.deactivate` / `partyApi.activate` (PATCH), presumably wired through `MasterCrudPage`'s generic action buttons.
- Deals-in categories: checkbox grid populated from `categoryApi.list()`.

- Additional addresses: repeatable rows (label, address line, pincode; city/state selects also present in payload but not exposed in the visible inputs beyond label/line/pincode in the current UI code).

Supplier (`Suppliers.tsx` + `SupplierForm.tsx`):

- List: `supplierApi.list({search, status, page, size, sortBy, sortDir})`, with Name/City/Newest sort options and a status filter, gated by `canManage ( user.role === 'ADMIN' | 'STORE_MANAGER' )` for Add/Edit/Deactivate/Activate actions; View is available to everyone.
- Create/Edit: `SupplierForm.tsx` at `/suppliers/new` and `/suppliers/:id/edit`; client-side validation duplicates backend rules (`name` required, `mobile` exactly `^[0-9]{10}$`, `pincode` exactly `^[0-9]{6}$` if present) before submit; on 400, `parseApiError` extracts `message` + `fieldErrors` and populates per-field `invalid` /feedback text.
- View: `SupplierViewModal.tsx`, opened from the row's dropdown; lazily loads `supplierApi.getPurchaseHistory(id)`.
- Deactivate: confirmation `Dialog` ("It will no longer be selectable for new purchases, but existing purchase history will keep showing it") → `supplierApi.deactivate`.
- Activate: `supplierApi.activate`, no confirmation dialog.

Customer: there is no dedicated Customer master page in the frontend (`NOT FOUND IN CURRENT CODEBASE` for list/edit/deactivate UI). The only Customer UI is the **quick-add modal** used from Sales.

Quick-Add flow (used from Sales/Purchase forms, e.g. `SalesBillForm.tsx`, `PurchaseBillForm.tsx`, and their Order/Kacchi variants):

- `SupplierQuickAddModal`: minimal fields (name, mobile, email, address) → `supplierApi.create(...)` → `onCreated(supplier)` callback lets the parent form immediately select the new supplier without leaving the page.
- `CustomerQuickAddModal`: minimal fields (firstName, lastName, mobile, email, state) → `customerApi.create(...)` → `onCreated(customer)`. Note: this modal has **no per-field fieldErrors handling** (unlike `SupplierQuickAddModal`) — it only shows the top-level `parseApiError(...).message` in an `Alert`, so a 400 from the backend (e.g. a duplicate or malformed field) is not surfaced field-by-field here, only as a generic banner message.

3. API Calls

Function	Method	URL	Request shape	Response shape
<code>partyApi.list</code>	GET	<code>/api/masters/parties</code>	query: <code>search?</code> , <code>partyType?</code> , <code>status?</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<Party></code>
<code>partyApi.getById</code>	GET	<code>/api/masters/parties/{id}</code>	—	<code>Party</code>
<code>partyApi.create</code>	POST	<code>/api/masters/parties</code>	<code>PartyPayload</code> (<code>partyCode</code> , <code>partyName</code> , <code>mobile</code> , <code>partyType</code> , ...)	<code>Party</code> (201)
<code>partyApi.update</code>	PUT	<code>/api/masters/parties/{id}</code>	<code>PartyPayload</code>	<code>Party</code>

Function	Method	URL	Request shape	Response shape
partyApi.activate	PATCH	/api/ masters/ parties/ {id}/ activate	—	Party
partyApi.deactivate	PATCH	/api/ masters/ parties/ {id}/ deactivate	—	Party
supplierApi.list	GET	/api/ suppliers	query: search?, status?, page, size, sortBy, sortDir	PagedResponse<Supplier>
supplierApi.getById	GET	/api/ suppliers/ {id}	—	Supplier
supplierApi.getPurchaseHistory	GET	/api/ suppliers/ {id}/ purchases	—	SupplierPurchaseSummary[]
supplierApi.create	POST	/api/ suppliers	SupplierPayload (name, mobile, contactPerson?, email?, address?, city?, state?, pincode?, gstNumber?, notes?)	Supplier (201)
supplierApi.update	PUT	/api/ suppliers/ {id}	SupplierPayload	Supplier
supplierApi.activate	PATCH	/api/ suppliers/ {id}/ activate	—	Supplier
supplierApi.deactivate	PATCH	/api/ suppliers/ {id}/ deactivate	—	Supplier
customerApi.list	GET	/api/ customers	— (no pagination/search/ filter params)	Customer[]
customerApi.create	POST	/api/ customers	CustomerCreatePayload (firstName, lastName?, mobile, email?, state?)	Customer (201)

There is no `customerApi.update`, `customerApi.getById`, `customerApi.activate/deactivate`, or `/api/customers/{id}/...` route in the frontend or backend — confirmed by `CustomerController.java` only exposing `GET /api/customers` and `POST /api/customers`.

4. Backend

Controllers

- `PartyController (/api/masters/parties)`: `list` (GET, unauthenticated-beyond-JWT — no `@PreAuthorize`), `getById` (GET, same), `create` (`PERM_PARTY_CREATE`), `update` (`PERM_PARTY_EDIT`), `activate/deactivate` (`PERM_PARTY_EDIT`).
- `SupplierController (/api/suppliers)`: `getSuppliers`, `getSupplierById`, `getSupplierPurchaseHistory` — all unguarded beyond authentication; `createSupplier`, `updateSupplier`, `activateSupplier`, `deactivateSupplier` — all require `PERM_MASTER_MANAGE` (there is no dedicated `SUPPLIER_*` permission).
- `CustomerController (/api/customers)`: `getAllCustomers`, `createCustomer` — **neither method has `@PreAuthorize`**; any authenticated user can list or create customers (see §7/§8).

DTOs (fields + validation)

- **PartyRequest**: `partyCode` (`@NotBlank`), `partyName` (`@NotBlank`), `contactPerson`, `mobile` (`@NotBlank`, `@Pattern("^[0-9]{10}$")`), `email` (`@Email`), `partyType` (`@NotNull`), `gstNumber` (no annotation — validated in `PartyService` via `GstinValidator.isValid`, not `@Pattern`), `panNumber`, `address`, `cityId/stateId/countryId` (Long FKs), `pincode`, `notes`, `dealsInCategoryIds` (`Set<Long>`), `addresses` (`@Valid List<PartyAddressRequest>`).
- **PartyAddressRequest**: `label` (`@NotBlank`), `addressLine` (`@NotBlank`), `cityId`, `stateId`, `pincode`.
- **PartyResponse**: mirrors the entity plus resolved `cityName/stateName/countryName`, `dealsInCategoryNames`, and a list of `PartyAddressResponse`.
- **SupplierCreateRequest / SupplierUpdateRequest** (identical field sets): `name` (`@NotBlank`), `contactPerson`, `mobile` (`@NotBlank`, `@Pattern("^[0-9]{10}$")`), `email` (`@Email`), `address`, `city`, `state` (both **free-text strings**, not FK IDs — unlike Party's `cityId/stateId`), `pincode` (`@Pattern("^[0-9]{6}$")`), `gstNumber` (`@Pattern` for full 15-char GSTIN format `^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$`), `notes`.
- **SupplierResponse**: mirrors the entity 1:1 including `status`, `createdAt`, `updatedAt`.
- **CustomerCreateRequest**: `firstName` (`@NotBlank`), `lastName`, `mobile` (`@NotBlank` **only — no `@Pattern` digit-count check**, unlike Supplier/Party), `email` (`@Email`), `state` (free text; this is the GST "Place of Supply" anchor field — see §12).
- **CustomerResponse**: mirrors the entity.
- GSTIN/state relevance: Party validates GSTIN via `GstinValidator.isValid(...)` (service-layer, not a DTO annotation) and stores structured `stateId/countryId` FKs; Supplier validates GSTIN via a DTO-level regex and stores `state` as free text; Customer has **no GSTIN field at all** and stores `state` as free text used purely for tax-mode suggestion, not for GST registration purposes.

Services (uniqueness checks, deals-in categories, addresses)

- **PartyService**:
- Uniqueness: `partyCode` unique case-insensitively (`existsByPartyCodeIgnoreCase[AndIdNot]`) → `BadRequestException`. GSTIN, if supplied, validated with `GstinValidator.isValid` and unique case-insensitively (`existsByGstNumberIgnoreCase[AndIdNot]`) → `BadRequestException`.
- `dealsInCategories`: resolved from `dealsInCategoryIds` via `CategoryService.findCategoryOrThrow` into a `Set<Category>` (many-to-many), rebuilt in full on every create/update.
- Additional addresses: on update, `party.clearAddresses()` (orphan-removal wipe) then `applyAddresses(...)` rebuilds the full list from the request — i.e. addresses are fully replaced, not diffed/merged.
- `setStatus(id, PartyStatus)` toggles `ACTIVE/INACTIVE` (used for both activate and deactivate).

- **CustomerService**: **no uniqueness checks at all** — `createCustomer` does not check for a duplicate mobile or email before insert (contrast with **Supplier** and **Party**, both of which enforce uniqueness). No update or status-change method exists.
- **SupplierService**:
- Uniqueness: `email` (if non-blank) unique case-insensitively (`existsByEmailIgnoreCase[AndIdNot]`); `gstNumber` (if non-blank) validated via `GstinValidator.isValid` **and** unique case-insensitively (`existsByGstNumberIgnoreCase[AndIdNot]`) — both → `BadRequestException`. No uniqueness check on `mobile`.
- `searchSuppliers` whitelists sortable fields to `{"name", "city", "createdAt"}`, defaulting to `"name"` for anything else.
- `getPurchaseHistory(supplierId)` looks up the supplier (404 if missing) then `purchaseRepository.findBySupplierIdOrderByCreatedAtDesc(...)`.
- `setStatus` toggles `ACTIVE/INACTIVE`.

Repository

- `PartyRepository` extends `JpaRepository<Party, Long>`: `existsByPartyCodeIgnoreCase[AndIdNot]`, `existsByGstNumberIgnoreCase[AndIdNot]`, and a JPQL `search(search, partyType, status, pageable)` matching `partyName / partyCode / mobile` (LIKE, case-insensitive) plus optional `partyType / status` filters.
- `PartyAddressRepository` extends `JpaRepository<PartyAddress, Long>`: no custom methods (addresses are always accessed through the owning `Party`).
- `CustomerRepository` extends `JpaRepository<Customer, Long>`: **no custom query methods at all** — just the inherited CRUD (`findAll`, `findById`, `save`, ...). No search/filter/sort support.
- `SupplierRepository` extends `JpaRepository<Supplier, Long>`: `existsByEmailIgnoreCase[AndIdNot]`, `existsByGstNumberIgnoreCase[AndIdNot]`, and a JPQL `search(search, status, pageable)` matching `name / mobile / email` (LIKE, case-insensitive) plus optional `status` filter.

5. Database

Tables

- `parties` (entity `Party`) — PK `id`, unique `party_code`.
- `party_addresses` (entity `PartyAddress`) — PK `id`, FK `party_id` (NOT NULL).
- `party_deals_in_categories` — join table for `Party` ↔ `Category` (`party_id`, `category_id`).
- `customers` (entity `Customer`) — PK `id`.
- `suppliers` (entity `Supplier`) — PK `id`; note column `phone` maps to Java field `mobile` (`@Column(name = "phone")`).

Relationships

- `Party 1—* PartyAddress` (`mappedBy = "party"`, `CascadeType.ALL`, `orphanRemoval = true`): additional/alternate addresses beyond the party's primary `address / city / state / country / pincode` fields.
- `Party` — `Category` via `party_deals_in_categories` (deals-in categories).
- `Party` also has `ManyToOne` FKs to `City`, `State`, `Country` (masters) for its primary address — `PartyAddress` rows each independently reference their own `City/State`.
- `Party` itself has **no incoming FK from any other entity** (see §1 for the full grep evidence) — it is a leaf/standalone master.
- Downstream dependents of `Customer` (grepped across the entire `entity/` package for `private Customer`):

- `Sale.customer`, `SalesOrder.customer`, `Receipt.customer`, `CreditNote.customer`, `CustomerLedgerEntry.customer`.
- Downstream dependents of `Supplier` (grepped for `private Supplier`):
- `Purchase.supplier`, `PurchaseOrder.supplier`, `Payment.supplier`, `Expense.supplier`, `DebitNote.supplier`, `SupplierLedgerEntry.supplier`.

6. Validation

- **Mobile:** Party and Supplier both enforce `^[0-9]{10}$` via `@Pattern` at the DTO layer (consistent with the project-wide 10-digit rule). **Customer does not** — `CustomerCreateRequest.mobile` only has `@NotBlank`, so e.g. an 11-digit or non-numeric mobile is accepted by the backend for Customer creation. This is a real, code-confirmed inconsistency versus the documented 10-digit mobile rule for the rest of the app.
- **Pincode:** Supplier enforces `^[0-9]{6}$` via `@Pattern`. Party's `PartyRequest.pincode` and `PartyAddressRequest.pincode` have **no format annotation** — any string is accepted at the DTO layer.
- **GSTIN:** Supplier enforces the full 15-char GSTIN shape via DTO `@Pattern`. Party validates GSTIN in `PartyService` via `GstinValidator.isValid(...)` (service-layer, not annotation-based) — same effective format, different enforcement point. Customer has no GSTIN field.
- **Email:** `@Email` on Party, Supplier (create+update), and Customer.
- **Uniqueness** (service-layer, not annotation-based): Party — `partyCode` and `gstNumber` (if present) unique. Supplier — `email` (if present) and `gstNumber` (if present) unique. Customer — **none**.
- All field errors from `@Valid` failures are captured by `GlobalExceptionHandler.handleValidation` (on `MethodArgumentNotValidException`) into `ApiError.fieldErrors` (`Map<fieldName, message>`); service-thrown `BadRequestException`s (uniqueness/GSTIN-format failures) return only a top-level `message` with `fieldErrors: null` — the frontend forms for Supplier and Party surface these via the top-level `Alert`, since there's no field to attach them to individually.

7. Authentication / Authorization

All `/api/**` routes require an authenticated (JWT-bearing) request (`SecurityConfig.anyRequest().authenticated()`, with only `/api/auth/**` and `/actuator/health` permitted anonymously). Beyond that base requirement:

- Party: `list/getById` need only authentication; `create/update/activate/deactivate` additionally need `PERM_PARTY_CREATE` or `PERM_PARTY_EDIT`.
- Supplier: `list/getById/getPurchaseHistory` need only authentication; all mutating endpoints need `PERM_MASTER_MANAGE`.
- Customer: **every** endpoint (`list`, `create`) needs only authentication — there is no permission check at all on the Customer controller.

8. Permissions

From `RolePermissions.java` / `Permission.java`

(`backend/src/main/java/com/storehub/entity/Permission.java` and `.../service/RolePermissions.java`):

- `PARTY_VIEW` ("Master" group) — granted to multiple roles (appears in at least 4 role permission sets alongside `ITEM_VIEW`, `MASTER_VIEW`, `STORE_VIEW`, etc.) but is **never actually checked** by `PartyController` — its GET endpoints have no `@PreAuthorize` at all, so `PARTY_VIEW` is currently a defined-but-unenforced permission for read access.
- `PARTY_CREATE`, `PARTY_EDIT` ("Master" group) — enforced via `@PreAuthorize("hasAuthority('PERM_PARTY_CREATE')")` / `PERM_PARTY_EDIT` on Party's mutating endpoints.

- `MASTER_MANAGE` — the single permission gating all Supplier mutations (create/update/activate/deactivate); there is no `SUPPLIER_VIEW` / `SUPPLIER_CREATE` / `SUPPLIER_EDIT`.
- `CUSTOMER_VIEW` / `CUSTOMER_CREATE` / etc.: NOT FOUND IN CURRENT CODEBASE — no Customer-specific permission exists, and the Customer controller checks no permission at all.

9. Transaction Handling

- `PartyService.create`, `.update`, `.setStatus` are `@Transactional` (address replacement + category resolution happen atomically with the party save).
- `CustomerService.createCustomer` is `@Transactional`.
- `SupplierService.createSupplier`, `.updateSupplier`, `.setStatus` are `@Transactional`.
`searchSuppliers`, `getSupplierById`, `getPurchaseHistory` are read-only (no `@Transactional`, plain repository calls).

10. Error Handling

- `PartyNotFound` → `MasterNotFoundException("Party", id)` (generic, entity-name-parameterized) → `GlobalExceptionHandler.handleMasterNotFound` → HTTP 404.
- `SupplierNotFoundException(id)` → `handleSupplierNotFound` → HTTP 404.
- `CustomerNotFoundException(id)` → `handleCustomerNotFound` → HTTP 404 (thrown by `CustomerService.findCustomerOrThrow`, but note this method is **never called by CustomerController** — it exists only for other services, e.g. potentially Sales, to resolve a customer FK; `CustomerController` itself has no `getById` endpoint that could trigger it).
- Uniqueness/GSTIN-format violations → `BadRequestException` → `handleBadRequest` → HTTP 400, top-level message only.
- `@Valid` DTO failures → `MethodArgumentNotValidException` → `handleValidation` → HTTP 400 with `fieldErrors` map.
- Unhandled exceptions fall through to `handleGeneral` → HTTP 500 with a generic message (and full stack trace logged server-side).

11. Audit Flow

NOT FOUND IN CURRENT CODEBASE — `PartyService`, `CustomerService`, and `SupplierService` contain **no calls to `AuditService`** (confirmed by grepping `Party|Customer|Supplier` inside `AuditService.java` — zero matches — and grepping `AuditService|auditService` inside `PartyService.java` — zero matches). Creates, updates, activations, and deactivations of Party/Customer/Supplier records are not written to the audit log by this module, unlike modules such as User Management that do call `AuditService` explicitly.

12. Important Side Effects

- **State-based GST tax-mode auto-suggestion:** `Customer.state` (free text) and `Supplier.state` (free text) are the fields that drive the seller/counterparty "Place of Supply" comparison in the Sales and Purchase bill forms. `frontend/src/utils/gst.ts` exports `suggestTaxMode(sellerState, partyState)`, documented as mirroring the backend's `GstCalculationService#suggestTaxMode` — it returns `'INTRA_STATE'` if the two state strings match case-insensitively (trimmed), `'INTER_STATE'` if they differ, or `null` if either is missing. `SalesBillForm.tsx` calls `suggestTaxMode(sellerState, customer?.state)` to pre-select `taxMode` (line ~208); `PurchaseBillForm.tsx` calls `suggestTaxMode(sellerState, supplier?.state)` analogously. This is explicitly a **pre-select hint**

only — the backend does not re-derive or trust this value; the user's dropdown selection remains authoritative for the actual GST calculation.

- Because `Customer.state / Supplier.state` are free-text strings (not FK'd to the `State` master the way `Party's stateId` is), a typo or inconsistent casing/naming between the seller's configured state and a customer's/supplier's entered state can silently defeat the auto-suggestion (it just returns `null` or the wrong mode) — there is no dropdown constraining these fields to the canonical state list.
- Deactivating a Supplier does not cascade or block anything at the DB level — `SupplierService.setStatus` is a plain field flip; the frontend's confirmation copy ("no longer selectable for new purchases, but existing purchase history will keep showing it") is a UI-level convention, not something enforced by a DB constraint or service-layer check in this module (whatever excludes inactive suppliers from new-purchase pickers lives in the Purchase module, not here).

13. Dependencies on Other Modules

- **Sales module** (`Sale.customer`, `SalesOrder.customer`, `KacchiSaleForm.tsx`, `SalesBillForm.tsx`, `SalesOrderForm.tsx`): resolves/selects `Customer`, uses `CustomerQuickAddModal` and `customerApi`.
- **Purchase module** (`Purchase.supplier`, `PurchaseOrder.supplier`, `KacchiPurchaseForm.tsx`, `PurchaseBillForm.tsx`, `PurchaseOrderForm.tsx`): resolves/selects `Supplier`, uses `SupplierQuickAddModal` and `supplierApi`.
- **Payments/Receipts**: `Payment.supplier`, `Receipt.customer`.
- **Credit/Debit Notes**: `CreditNote.customer`, `DebitNote.supplier`.
- **Expenses**: `Expense.supplier` (an expense can optionally be attributed to a supplier).
- **Accounting / Ledgers**: `CustomerLedgerEntry.customer`, `SupplierLedgerEntry.supplier`; `PartyLedgerService` (accounting ledger reporting) resolves ledger-row party display names via `CustomerRepository / SupplierRepository` directly (keyed by an `AccountingPartyType` discriminator), independent of `PartyService / PartyRepository`.
- **GST**: tax-mode auto-suggestion (see §12) consumes `Customer.state / Supplier.state` from the Sales/Purchase forms.
- **Masters dashboard** (`MastersDashboard.tsx`) and **Categories** (`categoryApi`, for `dealsInCategoryIds`) are the only consumers of `Party / partyApi`.

14. Key Operation Flows

Create Customer (via quick-add, the only creation path that exists):

UI (`CustomerQuickAddModal.tsx`, form fields `firstName/lastName/mobile/email/state`) → `customerApi.create(payload)` → `POST /api/customers` → `CustomerController.createCustomer(@Valid CustomerCreateRequest)` → Spring validates `@NotBlank firstName`, `@NotBlank mobile` (no digit-pattern check), `@Email email` → `CustomerService.createCustomer(request)` → builds `Customer.builder()...build()` (no uniqueness check) → `customerRepository.save(customer)` → `@PrePersist` sets `createdAt/updatedAt` and defaults `status = ACTIVE` → `CustomerResponse.fromEntity(customer)` → `HTTP 201` → `modal's onCreated(customer)` callback lets the parent Sale form select the new customer immediately.

Create Supplier (full form, `SupplierForm.tsx`, `isEdit = false`):

UI (`name/contactPerson/mobile/email/address/city/state/pincode/gstNumber/notes`) → client-side `validate()` (`name` required, `mobile` `^[0-9]{10}$`, `pincode` `^[0-9]{6}$` if present) → `supplierApi.create(payload)` → `POST /api/suppliers` (requires `PERM_MASTER_MANAGE`) → `SupplierController.createSupplier(@Valid SupplierCreateRequest)` → DTO validation (`@NotBlank name`, `@Pattern mobile`, `@Email email`, `@Pattern pincode`, `@Pattern gstNumber`) → `SupplierService.createSupplier(request)`: checks `email` uniqueness (if present) and, if `gstNumber`

present, validates via `GstinValidator.isValid` then checks GSTIN uniqueness → both failures throw `BadRequestException` (400, top-level message) → builds `Supplier.builder()...build()` (blank email/GST normalized to `null`) → `supplierRepository.save(...)` → `@PrePersist` sets timestamps, defaults `status = ACTIVE` → `SupplierResponse.fromEntity(...)` → HTTP 201 → frontend `toast.success(...)`, `navigate('/suppliers')`.

Deactivate Supplier:

UI (`Suppliers.tsx` row dropdown → "Deactivate", `canManage` only) → confirmation `Dialog` → `handleDeactivateConfirm` → `supplierApi.deactivate(id)` → PATCH `/api/suppliers/{id}/deactivate` (requires `PERM_MASTER_MANAGE`) → `SupplierController.deactivateSupplier` → `SupplierService.setStatus(id, SupplierStatus.INACTIVE)` → `findSupplierOrThrow(id)` (404 via `SupplierNotFoundException` if missing) → `supplier.setStatus(INACTIVE)` → `supplierRepository.save(...)` → `@PreUpdate` bumps `updatedAt` → `SupplierResponse.fromEntity(...)` → HTTP 200 → frontend `toast.success(...)`, `loadSuppliers()` refresh; the row now shows the `INACTIVE` badge and an "Activate" action instead.

Quick-Add-from-another-form flow (e.g. adding a new customer while creating a Sale, in `SalesBillForm.tsx` / `SalesOrderForm.tsx` / `KacchiSaleForm.tsx`):

Sale form renders a "+ Add Customer" trigger → opens `CustomerQuickAddModal` (`show=true`) → user fills `firstName/mobile/(lastName/email/state optional)` → submit → `customerApi.create({firstName, lastName, mobile, email, state})` → POST `/api/customers` → same backend path as "Create Customer" above (`CustomerController` → `CustomerService` → `CustomerRepository`, no uniqueness check, no permission check) → HTTP 201 `CustomerResponse` → modal calls `onCreated(customer)` → the Sale form's handler (in the parent page) adds this customer to its in-memory customer list/select and auto-selects it, and also feeds `customer.state` into `suggestTaxMode(sellerState, customer.state)` to pre-select the sale's `taxMode` — all without the user leaving the Sale form. The Supplier equivalent (`SupplierQuickAddModal` from Purchase forms) follows the identical shape against POST `/api/suppliers`, except it does surface `fieldErrors` per-field (unlike the Customer modal) since `SupplierCreateRequest` has stricter validation that is more likely to trigger 400s (e.g. the GSTIN pattern, though GST isn't collected in the quick-add form itself, so in practice only `mobile/email` pattern failures surface here).

15. Manual Changes

- **Add a new field to Customer:** add the column to `backend/src/main/java/com/storehub/entity/Customer.java`, add it to `CustomerCreateRequest` / `CustomerResponse` (`backend/src/main/java/com/storehub/dto/`), wire it through `CustomerService.createCustomer`'s builder, and add the input to `CustomerQuickAddModal.tsx` and the `Customer` / `CustomerCreatePayload` types in `frontend/src/types/sale.ts`. There is no edit form to update, since none exists; if update support is later added, `CustomerController` / `CustomerService` would need new `PUT / update` methods first (they don't currently exist).
- **Add a new field to Supplier:** add the column to `Supplier.java`; add it to `SupplierCreateRequest` and `SupplierUpdateRequest` (keep both in sync — they are currently duplicated, not shared via a base class) and `SupplierResponse`; wire it through `SupplierService.createSupplier / updateSupplier`; add the input to `SupplierForm.tsx` (and optionally `SupplierQuickAddModal.tsx` if it should be quick-addable) and `SupplierViewModal.tsx`; update `Supplier` / `SupplierPayload` in `frontend/src/types/supplier.ts`.
- **Add a new field to Party:** add the column to `Party.java`; add to `PartyRequest` / `PartyResponse`; wire through `PartyService.create / update`; add the input to the appropriate tab in `PartyMaster.tsx`'s `renderForm / renderView`; update `Party` / `PartyPayload` in `frontend/src/types/masters.ts`.
- **Change GSTIN validation:** for Party, edit `backend/src/main/java/com/storehub/util/GstinValidator.java` (`isValid(...)`), used by `PartyService`. For Supplier, edit the `@Pattern` regex directly on `SupplierCreateRequest.gstNumber` and `SupplierUpdateRequest.gstNumber` (must be

changed in both files identically since they are not shared). Customer has no GSTIN field to change.

- **Change state validation / add a state dropdown for Customer or Supplier:** today `Customer.state` and `Supplier.state` are unconstrained free-text strings (no `@Pattern`, no FK to the `State` master). To constrain them, either add a `@Pattern`/enum-style validation at the DTO level, or — for stronger consistency with `Party` (which already uses `stateId`/`State` FK) — change the field to a `stateId` FK the way `PartyRequest`/`Party` already do, which would additionally require updating `frontend/src/utils/gst.ts#suggestTaxMode` (currently compares raw strings) and every call site of it (`SalesBillForm.tsx`, `PurchaseBillForm.tsx`, and their `Order/Kacchi` variants) plus the seller's own state source, since the whole GST tax-mode suggestion pipeline depends on string equality of these two free-text fields today.
- **Add a new "deals-in category":** this concept exists **only on Party** (`dealsInCategoryIds` / `dealsInCategoryNames`, backed by `party_deals_in_categories` and the shared `Category` master/`CategoryService`). Since `Party` is not consumed by `Sales/Purchase` (see §1), adding or changing categories here has no effect on `Customer/Supplier` or on `Sales/Purchase` forms — it only affects `PartyMaster.tsx`'s "Categories & Addresses" tab and whatever (if anything) elsewhere in the app reads `Category`/`categoryApi` (`Products/Items`, per `frontend/src/types/product.ts`'s `Category` import in `PartyMaster.tsx`). `Supplier` and `Customer` have no equivalent "deals-in" concept at all.
- **Modules affected by Customer/Supplier field changes:** `Sales` forms (`SalesBillForm.tsx`, `SalesOrderForm.tsx`, `KacchiSaleForm.tsx`, `CustomerQuickAddModal.tsx`) for `Customer`; `Purchase` forms (`PurchaseBillForm.tsx`, `PurchaseOrderForm.tsx`, `KacchiPurchaseForm.tsx`, `SupplierQuickAddModal.tsx`) for `Supplier`; GST tax-mode suggestion (`utils/gst.ts`) for both `state` fields; Accounting/ledger reporting (`PartyLedgerService`, `CustomerLedgerEntry` / `SupplierLedgerEntry`) for any field that ledger reports display; and `SupplierPurchaseSummary` / `purchase-history` display for `Supplier`-side purchase fields.

Product / Item Master & Inventory Module

1. Overview

This module covers the **Item Master** (Product + Category) and **core Inventory** (per-store stock, stock history, manual adjustment, CSV import/export). Stock **transfer between stores** is a separate module (`StockTransferService` / `StockTransferController`) and is only referenced here as a dependency (section 13).

SKU / barcode generation. A Product's `sku` is mandatory, always stored **trimmed + uppercased** (`ProductService#normalizeSku`), and unique. It can be typed manually or auto-generated via `SkuGeneratorService#generateNext()`, which produces `ITEM-000001`-style values from a single global counter row (`item_sku_sequence`, entity `ItemSkuSequence`) locked with `PESSIMISTIC_WRITE` inside its own `REQUIRES_NEW` transaction, so two concurrent "Generate SKU" clicks never collide. A generated number is never reused even if unused (gaps are fine, duplicates are not).

A Product's `barcode` is optional, stored **trimmed only** (never uppercased — preserves leading zeros / Code128 case), and unique. It can be scanned/typed manually or auto-generated via `BarcodeGeneratorService#generateNext()`, which produces `INT-000001`-style values from its own counter row (`item_barcode_sequence`, entity `ItemBarcodeSequence`), using the identical locking pattern. `BarcodeUtil.detectType()` auto-classifies a barcode's shape server-side (`EAN13` / `EAN8` / `UPC` / `INTERNAL` / `OTHER`) — the user never picks a type — and only a 13-digit barcode is checked against the standard EAN-13 checksum algorithm.

Inventory is the single source of truth for stock, per (product, store). There is no stock quantity stored on `Product` itself (the legacy `products.stock_quantity` column is kept in the DB for historical-data safety but is no longer mapped by the entity and is relaxed to nullable at startup). One `Inventory` row exists per (`product_id`, `store_id`) pair (unique constraint `uk_inventory_product_store`); a product's stock at a given store is exactly that row's `current_stock`, and a "global" figure (Products list, CSV export) is always a `SUM()` across a product's rows, never stored directly. `InventoryService.applyMovement(...)` is the single choke point every stock-mutating flow in the system must go through — see section 13 for every caller.

CSV import/export. Products and Inventory each support CSV export (filtered, matching the current list-page filters) and — Products only — CSV import (create-or-update by SKU match, with opening stock for new rows applied via a real `STOCK_IN` movement through `InventoryService.applyMovement`, never a direct DB write). Export values are passed through `CsvUtil.escape()`, which neutralizes spreadsheet formula injection (a leading `=`, `+`, `-`, `@`, tab or CR gets a literal-text `'` prefix) before RFC4180 quoting.

2. Frontend

Files and Components

File	Purpose
<code>frontend/src/pages/Products.tsx</code>	Product list: search/category/status/stock filters, sort, pagination, activate/deactivate, CSV export/import dialog, opens <code>ProductViewModal</code> and <code>CategoryManagerDialog</code> .
<code>frontend/src/pages/ProductForm.tsx</code>	Create/Edit product form (Basic Info, Pricing, Inventory thresholds, Item Master Details cards); SKU/barcode Generate buttons; edit-lock when <code>hasTransactions</code> is true; surfaces backend <code>fieldErrors</code> .

File	Purpose
<code>frontend/src/components/ProductViewModal.tsx</code>	Read-only product detail modal (basic info, pricing, current stock/status).
<code>frontend/src/components/ProductQuickAddModal.tsx</code>	Minimal inline "Add Product" modal (name, SKU + generate, category + quick-add, unit, prices) used from other modules' item pickers.
<code>frontend/src/components/CategoryQuickAddModal.tsx</code>	Minimal inline "Add Category" modal (name, description).
<code>frontend/src/components/CategoryManagerDialog.tsx</code>	Full category CRUD dialog: add (with itemType/applicableProperty), inline edit, activate/deactivate.
<code>frontend/src/pages/Inventory.tsx</code>	Inventory list: summary cards (total/low/out-of-stock/overstock/reorder), search/category/store/stock-status filters, CSV export, "Adjust Stock" action, opens view/history/adjust modals.
<code>frontend/src/components/InventoryViewModal.tsx</code>	Read-only inventory row detail (stock levels, status, timestamps).
<code>frontend/src/components/StockAdjustmentDialog.tsx</code>	Two-step (form → confirm) manual stock adjustment: STOCK_IN / STOCK_OUT / ADJUSTMENT, live "new stock" preview, low-stock warning, negative-stock client-side guard.
<code>frontend/src/components/StockHistoryModal.tsx</code>	Paginated stock-movement history for a product, filterable by movement type, reference type, date range.
<code>frontend/src/api/productApi.ts</code>	Axios wrapper for <code>/api/products*</code> .
<code>frontend/src/api/categoryApi.ts</code>	Axios wrapper for <code>/api/categories*</code> .
<code>frontend/src/api/inventoryApi.ts</code>	Axios wrapper for <code>/api/inventory*</code> .
<code>frontend/src/types/product.ts</code>	<code>Product</code> , <code>ProductPayload</code> , <code>Category</code> , <code>CategoryPayload</code> , <code>ProductStatus</code> , <code>BarcodeType</code> , <code>TaxTreatment</code> , <code>CategoryStatus</code> .
<code>frontend/src/types/inventory.ts</code>	<code>Inventory</code> , <code>StockHistory</code> , <code>StockAdjustmentPayload</code> , <code>InventorySummary</code> , <code>StockStatus</code> , <code>StockMovementType</code> , <code>ManualAdjustmentType</code> , <code>ReferenceType</code> .
<code>frontend/src/types/importResult.ts</code>	<code>ImportResult</code> (totalRows/created/updated/skipped/errors).
<code>frontend/src/utils/stockStatus.ts</code>	Client-side stock-status classifier used on the Products page (<code>getStockStatus</code> , based on <code>stockQuantity</code> / <code>minStockLevel</code> only — does not know about <code>maxStockLevel</code> / <code>overstock</code> , unlike the backend's <code>InventoryResponse.fromEntity</code>).
<code>frontend/src/utils/apiError.ts</code>	<code>parseApiError()</code> — turns an axios error into a message + per-field <code>fieldErrors</code> map.

Frontend Flow

Product create/edit. `ProductForm.tsx` loads categories/item groups/HSN codes/units in parallel, and (edit mode) the product itself via `productApi.getById`, populating `hasTransactions` from the response. The SKU and Barcode fields each have a "Generate" button (`Sparkles` icon) calling `productApi.generateSku()` / `generateBarcode()`; both the input **and** the generate button are disabled when `isEdit && hasTransactions` — the edit-protection lock (backend: `ProductService.updateProduct`'s `skuChanging` / `barcodeChanging` + `hasTransactionHistory` guard). On submit, `validate()` runs client-side (required/non-negative checks mirroring the backend `@NotBlank` / `@DecimalMin` / `@Min`), then POST/PUT; a 400 response's `fieldErrors` populate both a per-field message (`fieldErrors.sku` etc., shown via `invalid` / red text under each Input) and a top-level `error` Alert, per `parseApiError`.

Barcode entry/scan. Manual typing or the Generate button in `ProductForm`; separately, `productApi.getByBarcode(barcode)` (used elsewhere, e.g. POS/Sales/Purchase scanning) hits `GET /api/products/barcode/{barcode}` which 404s if no product matches and 400s if the product is `INACTIVE`.

Inventory list with store filter. `Inventory.tsx` shows a Store Select only when `myStores.length > 1` (single-store users never see it); `storeFilter` flows into both `inventoryApi.list()` and `inventoryApi.getSummary()`. Sorting supports `currentStock/updatedAt`. Rows show a computed `StockStatus` badge from the backend response (`IN_STOCK` / `LOW_STOCK` / `OUT_OF_STOCK` / `OVERSTOCK`).

Stock adjustment dialog flow. `StockAdjustmentDialog` is opened either pre-selected (from an Inventory row's "Adjust Stock" action, `preselected` prop) or blank (from the page-level "Adjust Stock" button, which loads up to 200 inventory rows into a picker). It is a two-step wizard: **form** step computes a live `newStock` preview (`STOCK_IN` → +qty, `STOCK_OUT` → -qty, `ADJUSTMENT` → set-to-qty) and blocks submission client-side if it would go negative or if `selected.storeId` is missing; **confirm** step shows `current` → `new` stock and a low-stock warning (`newStock/currentStock <= 0.2`) before calling `inventoryApi.adjust()`.

Stock history view. `StockHistoryModal` loads `inventoryApi.getHistory({ productId, ... })` (the `productId`-level endpoint, not store-scoped) with movement-type, reference-type and date-range filters, paginated.

CSV import/export flow. Products page: Export button calls `productApi.exportCsv()` with current search/category/status filters and downloads the blob; Import opens a dialog with a hidden file input, calls `productApi.importCsv(file)` (multipart), and renders the `ImportResult` (created/updated/skipped counts + a scrollable list of per-row error strings). Inventory page: Export only (no import), same filtered-CSV pattern via `inventoryApi.exportCsv()`.

3. API Calls

Function	Method	URL	Request shape	Response shape
<code>productApi.list</code>	GET	<code>/api/products</code>	query: <code>search</code> , <code>categoryId</code> , <code>status</code> , <code>page</code> , <code>size</code> , <code>sortBy</code> , <code>sortDir</code>	<code>PagedResponse<Product></code>
<code>productApi.getById</code>	GET	<code>/api/products/{id}</code>	—	<code>Product</code>
<code>productApi.getByBarcode</code>	GET	<code>/api/products/barcode/{barcode}</code>	—	<code>Product</code> (404/400 on miss/inactive)
<code>productApi.generateSku</code>	POST	<code>/api/products/generate-sku</code>	—	<code>{ sku: string }</code>
<code>productApi.generateBarcode</code>	POST	<code>/api/products/generate-barcode</code>	—	<code>{ barcode: string }</code>
<code>productApi.create</code>	POST	<code>/api/products</code>	<code>ProductPayload</code> (→ <code>ProductCreateRequest</code>)	<code>Product</code> (201)

Function	Method	URL	Request shape	Response shape
productApi.update	PUT	/api/products/{id}	ProductPayload (→ ProductUpdateRequest)	Product
productApi.activate	PATCH	/api/products/{id}/activate	—	Product
productApi.deactivate	PATCH	/api/products/{id}/deactivate	—	Product
productApi.exportCsv	GET	/api/products/export	query: search, categoryId, status	Blob (text/csv)
productApi.importCsv	POST	/api/products/import	multipart/form-data , field file	ImportResult
categoryApi.list	GET	/api/categories	—	Category[]
categoryApi.getById	GET	/api/categories/{id}	—	Category
categoryApi.create	POST	/api/categories	CategoryPayload (→ CategoryCreateRequest)	Category (201)
categoryApi.update	PUT	/api/categories/{id}	CategoryPayload (→ CategoryUpdateRequest)	Category
categoryApi.activate	PATCH	/api/categories/{id}/activate	—	Category
categoryApi.deactivate	PATCH	/api/categories/{id}/deactivate	—	Category
inventoryApi.list	GET	/api/inventory	query: search, categoryId, storeId, stockStatus, page, size, sortBy, sortDir	PagedResponse<Inventory>
inventoryApi.getById	GET	/api/inventory/{id}	—	Inventory
inventoryApi.getSummary	GET	/api/inventory/summary	query: storeId?	InventorySummary
inventoryApi.adjust	POST	/api/inventory/adjust	StockAdjustmentPayload (→ StockAdjustmentRequest)	Inventory

Function	Method	URL	Request shape	Response shape
<code>inventoryApi.getHistoryForInventory</code>	GET	<code>/api/inventory/{id}/history</code>	query: <code>page, size</code>	<code>PagedResponse<StockHistory></code>
<code>inventoryApi.getHistory</code>	GET	<code>/api/inventory/history</code>	query: <code>productId, storeId, movementType, referenceType, fromDate, toDate, page, size</code>	<code>PagedResponse<StockHistory></code>
<code>inventoryApi.exportCsv</code>	GET	<code>/api/inventory/export</code>	query: <code>search, categoryId, storeId, stockStatus</code>	<code>Blob (text/csv)</code>

4. Backend

Controllers

- **ProductController** (`/api/products`): GET (list, no explicit `@PreAuthorize`), GET `/api/products/{id}`, GET `/api/products/{id}/barcode/{barcode}`, POST `/api/products/generate-sku` (`PERM_ITEM_CREATE`), POST `/api/products/generate-barcode` (`PERM_ITEM_CREATE`), POST (`PERM_ITEM_CREATE`), PUT `/api/products/{id}` (`PERM_ITEM_EDIT`), PATCH `/api/products/{id}/activate` (`PERM_ITEM_EDIT`), PATCH `/api/products/{id}/deactivate` (`PERM_ITEM_EDIT`), GET `/api/products/export` (`PERM_ITEM_EDIT`), POST `/api/products/import` (`PERM_ITEM_CREATE`).
- **CategoryController** (`/api/categories`): GET, GET `/api/categories/{id}` (no `@PreAuthorize`); POST, PUT `/api/categories/{id}`, PATCH `/api/categories/{id}/activate`, PATCH `/api/categories/{id}/deactivate` all require `PERM_MASTER_MANAGE`.
- **InventoryController** (`/api/inventory`): GET, GET `/api/inventory/summary`, GET `/api/inventory/history`, GET `/api/inventory/{id}`, GET `/api/inventory/{id}/history` (no explicit `@PreAuthorize`, but every list/summary/history endpoint resolves/validates `storeId` through `StoreAccessService.resolveViewableStoreId / assertStoreAccess`); POST `/api/inventory/adjust` (`PERM_INVENTORY_ADJUST`, plus explicit `storeAccessService.assertStoreAccess`); GET `/api/inventory/export` (`PERM_INVENTORY_ADJUST`).

All GET endpoints above with no method-level `@PreAuthorize` still require an authenticated session — `SecurityConfig` sets `.anyRequest().authenticated()` with only `/api/auth/**` and `/actuator/health` as `permitAll()`.

DTOs (fields + validation)

- **ProductCreateRequest / ProductUpdateRequest** (identical field sets): `name @NotBlank`; `sku @NotBlank @Size(max=50) @Pattern(^[A-Za-z0-9\-\_]+)$`; `barcode @Size(max=50) @Pattern(^[A-Za-z0-9\-\_]*$)` (optional, `*` allows empty); `categoryId @NotNull`; `brand, unit` (plain strings); `purchasePrice @NotNull @DecimalMin(0)`; `sellingPrice @NotNull @DecimalMin(0)`; `tax @DecimalMin(0)`; `taxTreatment` (defaults to `TAXABLE` in the service if null); `minStockLevel / reorderLevel / reorderQuantity / maxStockLevel @Min(0)`; `mrp / wholesalePrice / freeValue / tolerancePercent @DecimalMin(0)`; plus free-text `description, manualCode, itemGroupId, hsnId, purchaseUnitId, saleUnitId, itemType, taxNature, taxBasedOn, partyName, partyProductName, applicableProperty` (all unvalidated/optional).
- **ProductResponse**: full product projection plus `stockQuantity` (current stock — see section 12 for aggregate vs. per-store), `hasTransactions` (drives frontend SKU/barcode lock), `category/item-group/HSN/unit` names denormalized alongside their ids.
- **CategoryCreateRequest / CategoryUpdateRequest**: `name @NotBlank`; `description, itemType, applicableProperty` unvalidated.
- **CategoryResponse**: `id, name, description, itemType, applicableProperty, status, timestamps`.

- **StockAdjustmentRequest**: productId @NotNull; storeId @NotNull; movementType @NotNull (any StockMovementType, but InventoryService.adjustStock further restricts it to STOCK_IN / STOCK_OUT / ADJUSTMENT — a BadRequestException if not); quantity @NotNull @Min(1); reason @NotBlank; notes optional.
- **InventoryResponse**: id, product id/name/sku/category, store id/name/code, unit, currentStock, minStockLevel (defaults 0), maxStockLevel (from Product, nullable), reorderLevel / reorderQuantity, computed stockStatus (OUT_OF_STOCK if ≤ 0 , else LOW_STOCK if $\leq$ minStockLevel, else OVERSTOCK if maxStockLevel set and exceeded, else IN_STOCK), timestamps.
- **StockHistoryResponse**: id, product id/name/sku, store id/name/code, movementType, quantity, previousStock, newStock, reason, referenceType, referenceId, notes, createdAt, createdBy.
- **InventorySummaryResponse**: totalProducts, totalStockUnits, lowStockCount, outOfStockCount, overstockCount, reorderCandidateCount.
- **ImportResultResponse**: totalRows, created, updated, skipped, errors (List<String>).
- **GenerateSkuResponse** / **GenerateBarcodeResponse**: single-field wrappers (sku / barcode).

Services

ProductService

- normalizeSku(String sku) — private; sku.trim().toUpperCase(); the canonical form used everywhere (create/update/import/lookup), so " abc-001 " and "ABC-001" are recognized as the same SKU.
- validateBarcodeOrThrow(String barcode) — private; null-safe no-op for a null barcode; else checks BarcodeUtil.isValidCharacters (400 if not) and, only when BarcodeUtil.detectType(barcode) == EAN13, BarcodeUtil.isValidEan13Checksum (400 "checksum failed" if not). Called from createProduct, updateProduct, and per-row in importCsv — deliberately at the service layer (not just DTO @Pattern) because CSV import builds a Product directly and never goes through the create/update DTOs.
- createProduct(ProductCreateRequest) — normalizes SKU, normalizes+detects barcode type; checks name/SKU/barcode uniqueness (existsByNameIgnoreCase, existsBySkuIgnoreCase, existsByBarcodeIgnoreCase) each throwing BadRequestException; resolves Category (categoryService.findCategoryOrThrow, mandatory) and optional ItemGroup / Hsn / Unit refs; builds+saves Product; **then calls inventoryService.createInventoryForProduct(saved)** to create the baseline (0-stock) Inventory row; logs AuditAction.CREATE on module "ITEM_MASTER"; returns ProductResponse.fromEntity(saved, 0, request.getMaxStockLevel()).
- updateProduct(Long id, ProductUpdateRequest) — the **edit-protection rule**: reads hasTransactionHistory = stockHistoryRepository.existsByProductId(id); computes skuChanging / barcodeChanging by comparing normalized old vs. new values; if skuChanging && hasTransactionHistory → BadRequestException ("SKU cannot be changed because this item already has recorded stock, sales, or purchase history..."); same guard for barcodeChanging && hasTransactionHistory. Name/SKU/barcode uniqueness re-checked with *AndIdNot variants. On success, audit-logs either an SKU-changed entry, a barcode-changed entry, or (if neither changed) a generic "updated" entry — via three separate auditService.log(...) calls guarded by if (skuChanging) / if (barcodeChanging) / if (!skuChanging && !barcodeChanging).
- setStatus(Long id, ProductStatus status) — activate/deactivate; audit-logs UPDATE with a lower-cased status description.
- generateSku() / generateBarcode() — thin passthroughs to SkuGeneratorService.generateNext() / BarcodeGeneratorService.generateNext().
- findByBarcode(String barcode) — normalizes then productRepository.findByBarcodeIgnoreCase(...).orElseThrow(ProductNotFoundException); if the matched product is INACTIVE, throws BadRequestException ("is inactive and cannot be used in new transactions") — used by POS/Sales/Purchase scanning.
- backfillMissingSkus() — startup safety net; iterates all products, generates+saves an SKU for any with none, audit-logs each backfill. Idempotent.
- **CSV export**: exportCsv(search, categoryId, status) — reuses productRepository.search(...) (the

Sort -based, `LEFT JOIN FETCH category` variant) plus `inventoryService.getCurrentStockBulk(productIds)` (aggregate stock), builds a CSV via `CsvUtil.row(...)` with header `EXPORT_HEADER` (name, sku, barcode, category, brand, unit, purchasePrice, sellingPrice, tax, minStockLevel, reorderLevel, reorderQuantity, maxStockLevel, mrp, wholesalePrice, currentStock, status, description).

- **CSV import:** `importCsv(MultipartFile file)` — parses the file with `CsvUtil.parseLine`; requires header columns name, sku, category, purchasePrice, sellingPrice (`IMPORT_REQUIRED_COLUMNS`, else 400); per row: validates required fields present, resolves `Category` by name (row skipped with an error if not found), parses numeric fields (negative purchase/selling price rejected), validates barcode; if a product with the normalized SKU exists → **update** path (name/category/brand/unit/prices/tax/minStockLevel/reorder fields/mrp/wholesalePrice/description/barcode updated, **current stock is never touched**); else → **create** path (new `Product` built and saved, `inventoryService.createInventoryForProduct(saved)` called for the baseline row, and if `openingStock > 0`, `inventoryService.applyMovement(saved.getId(), openingStock, StockMovementType.STOCK_IN, ReferenceType.MANUAL, null, "Opening stock via CSV import")` — i.e. opening stock is applied through the normal movement choke point, never a direct write). Per-row exceptions are caught and appended to `errors` without aborting the whole import. Returns `ImportResultResponse` with totals.

InventoryService

- `getCurrentStock(Long productId, Long storeId)` — **store-specific**;
`inventoryRepository.findByProductIdAndStoreId(...).map(getCurrentStock).orElse(0)` . "The only figure a store-scoped transaction may ever act on."
- `getCurrentStock(Long productId)` — **cross-store aggregate**;
`inventoryRepository.sumCurrentStockForProduct(productId)` . For global Item Master displays only, never transaction validation.
- `getCurrentStockBulk(List<Long> productIds)` — bulk aggregate equivalent (Products list / CSV export).
- `getCurrentStockBulk(List<Long> productIds, Long storeId)` — bulk **store-scoped** equivalent (store-filtered inventory/report views).
- `createInventoryForProduct(Product product)` — called from `ProductService.createProduct / importCsv`; resolves `storeService.getOrCreateDefaultStore()` and, if no (product, defaultStore) row exists yet, saves a 0-stock `Inventory` row. A real store-aware movement later creates whichever actual (product, store) row it needs via `getOrCreateInventory` .
- `getOrCreateInventory(Product, Store)` — private; `findByProductIdAndStoreId(...).orElseGet(save new 0-stock row)` .
- `adjustStock(StockAdjustmentRequest request)` — the manual-adjustment entry point: rejects any `movementType` not in {`STOCK_IN`, `STOCK_OUT`, `ADJUSTMENT`} (400); resolves `Product` , `resolveActiveStore(storeId)` (400 if store `INACTIVE`), `getOrCreateInventory`; computes `newStock` per type (`STOCK_IN` → +qty, `STOCK_OUT` → -qty, `ADJUSTMENT` → set exactly to qty); rejects negative result (400 with current/requested detail); saves `Inventory` , writes a `StockHistory` row (`referenceType` = `MANUAL`), and audit-logs `UPDATE` on module "INVENTORY" with old/new stock and the store's `storeId` .
- `applyMovement(Long productId, Long storeId, int delta, StockMovementType, ReferenceType, Long referenceId, String reason)` — **primary store-aware overload**, the single choke point for every system-triggered movement (spec: "the only one a store-attributed transaction should call"); resolves `Product` , `resolveActiveStore(storeId)` , `getOrCreateInventory`; `newStock` = `previousStock` + `delta`; 400 if negative; saves `Inventory` , writes `StockHistory` (`quantity` = `Math.abs(delta)`).
- `applyMovement(Long productId, int delta, StockMovementType, ReferenceType, Long referenceId, String reason)` — **transitional overload** without a `storeId`; delegates to the primary overload using `storeService.getOrCreateDefaultStore().getId()` . Used by call sites not yet migrated to pass their own resolved store (see section 13).
- `getSummary(Long storeId)` — `storeId == null` aggregates across every store (`ALL_STORES` admin view); else scoped counts via `InventoryRepository` .
- `getReorderCandidates(Long storeId)` — products at/below `reorderLevel` , `ACTIVE` only, ordered by stock ascending; used by the Alert Center.

- `exportCsv(search, categoryId, storeId, stockStatus)` — CSV of the filtered Inventory list (header: store, product, sku, category, unit, currentStock, minStockLevel, maxStockLevel, reorderLevel, reorderQuantity, stockStatus, lastUpdated).
- Startup-only maintenance methods: `relaxLegacyStockQuantityColumn()` (ALTERS `products.stock_quantity` to NULL DEFAULT 0, best-effort/idempotent), `relaxLegacyInventoryProductUniqueConstraint()` (drops any leftover single-column unique index on `inventory.product_id` other than `uk_inventory_product_store`, via `INFORMATION_SCHEMA`), `backfillInventoryForExistingProducts()` (seeds an `Inventory` row per pre-existing product from the legacy `stock_quantity` column, and backfills a null `store` on any pre-Multi-Store row, onto the Default Store).
- `resolveActiveStore(Long storeId)` — private; `storeRepository.findById(...).orElseThrow(MasterNotFoundException)`, 400 if `StoreStatus.INACTIVE`.
- `currentUsername()` — private; resolves the audit `createdBy` string from `SecurityContextHolder`, "System" if none.

CategoryService — straightforward CRUD: `getAllCategories()` (sorted by name), `getCategoryById`, `createCategory` (name-uniqueness check), `updateCategory` (name-uniqueness check excluding self), `setStatus` (activate/deactivate), `findCategoryOrThrow`. No audit logging calls in this service (unlike `ProductService/InventoryService`).

SkuGeneratorService — `generateNext(): @Transactional(propagation = REQUIRES_NEW); itemSkuSequenceRepository.ensureRowExists(1L)` (native upsert-if-absent, avoids constraint-violation rollback under a race), then `lockForUpdate(1L)` (`PESSIMISTIC_WRITE`); increments `lastNumber`, formats "ITEM-" + %06d, loops while `productRepository.existsBySkuIgnoreCase(candidate)` (defensive, in case a manually-entered SKU already matches the pattern); saves the sequence row; returns the candidate.

BarcodeGeneratorService — `generateNext(): identical pattern against item_barcode_sequence / ItemBarcodeSequence`, prefix "INT-", loop-guard against `productRepository.existsByBarcodeIgnoreCase(candidate)`.

Repository (exact method names)

- **ProductRepository**: `existsByNameIgnoreCase`, `existsBySkuIgnoreCase`, `existsBySkuIgnoreCaseAndIdNot`, `existsByBarcodeIgnoreCase`, `existsByBarcodeIgnoreCaseAndIdNot`, `findBySkuIgnoreCase`, `findByBarcodeIgnoreCase`, `search(search, categoryId, status, Pageable)`, `search(search, categoryId, status, Sort)` (export-only, `LEFT JOIN FETCH p.category`).
- **CategoryRepository**: `existsByNameIgnoreCase`, `existsByNameIgnoreCaseAndIdNot`, `findByNameIgnoreCase`.
- **ItemSkuSequenceRepository**: `lockForUpdate(id)` (`@Lock(PESSIMISTIC_WRITE)`), `ensureRowExists(id)` (native `INSERT ... ON DUPLICATE KEY UPDATE`).
- **ItemBarcodeSequenceRepository**: `lockForUpdate(id)`, `ensureRowExists(id)` — identical shape.
- **InventoryRepository** (store-aware methods are the primary/authoritative ones): `findByProductIdAndStoreId(productId, storeId)`, `findByProductId(productId)` (every store's row — cross-store aggregation only), `findByStoreId(storeId)`, `sumCurrentStockForProduct(productId)`, `sumCurrentStockGroupedByProduct(productIds)`, `findCurrentStockByProductIdsAndStoreId(productIds, storeId)`, `search(search, categoryId, storeId, stockStatus, Pageable)`, `search(..., Sort)` (export-only, `JOIN FETCH`), `sumCurrentStock(storeId)`, `countLowStock(storeId)`, `countOutOfStock(storeId)`, `countOverstock(storeId)`, `findReorderCandidates(storeId)`, `countReorderCandidates(storeId)`.
- **StockHistoryRepository**: `findByProductIdOrderByCreatedAtDesc(productId, Pageable)`, `existsByProductId(productId)` (drives the edit-protection rule), `search(productId, storeId, movementType, referenceType, fromDateTime, toDateTime, Pageable)`.

5. Database

Tables

- **products** — id, name (unique), sku (unique), barcode (unique), barcode_type, category_id (FK), brand, unit, purchase_price, selling_price, tax, tax_treatment, min_stock_level, reorder_level, reorder_quantity, max_stock_level, mrp, wholesale_price, status, description, manual_code, item_group_id (FK), hsn_id (FK), purchase_unit_id (FK), sale_unit_id (FK), tolerance_percent, item_type, tax_nature, tax_based_on, party_name, party_product_name, free_value, applicable_property, created_at, updated_at. Legacy `stock_quantity` column still physically present but unmapped (nullable, defaulted to 0 at startup).
- **categories** — id, name (unique), description, item_type, applicable_property, status, created_at, updated_at.
- **inventory** — id, product_id (FK, not null), store_id (FK, nullable at schema level only), current_stock, created_at, updated_at. Unique constraint `uk_inventory_product_store` on (product_id, store_id).
- **stock_history** — id, product_id (FK, not null), store_id (FK, nullable — pre-Multi-Store rows only), movement_type, quantity, previous_stock, new_stock, reason, reference_type, reference_id, notes, created_at, created_by.
- **item_sku_sequence** — id (always 1), last_number.
- **item_barcode_sequence** — id (always 1), last_number.

Relationships

- `Inventory.product` → `Product` (@ManyToOne, product_id, not null).
- `Inventory.store` → `Store` (@ManyToOne, store_id, nullable only for schema-migration safety).
- `StockHistory.product` → `Product` (@ManyToOne, not null).
- `StockHistory.store` → `Store` (@ManyToOne, nullable for pre-Multi-Store rows).
- `Product.category` → `Category` (@ManyToOne, category_id).
- `Product.itemGroup` → `ItemGroup`, `Product.hsn` → `Hsn`, `Product.purchaseUnit` / `Product.saleUnit` → `Unit` (all @ManyToOne, optional, other masters modules).

6. Validation

- **DTO-level (Bean Validation):** see section 4's DTO list — @NotBlank, @NotNull, @Size, @Pattern, @DecimalMin, @Min on `ProductCreateRequest` / `ProductUpdateRequest`, `CategoryCreateRequest` / `CategoryUpdateRequest`, `StockAdjustmentRequest`.
- **Service-level (beyond DTO annotations, since CSV import bypasses the DTOs entirely):** SKU/barcode/name uniqueness checks in `ProductService`; `validateBarcodeOrThrow` (character set + EAN-13 checksum); `StockHistoryRepository.existsByProductId` edit-protection guard on SKU/barcode change; `InventoryService.adjustStock` / `applyMovement` negative-stock rejection; `resolveActiveStore` inactive-store rejection; `CategoryService` name-uniqueness.
- **CSV import row-level:** required-column presence, required-field-per-row presence, category-name lookup, non-negative numeric parsing (purchase/selling price), barcode validation, duplicate-name/barcode detection distinct from the create/update DTO path.

7. Authentication / Authorization

Every endpoint in this module requires an authenticated session

(`SecurityConfig`'s `.anyRequest().authenticated()`); there is no `permitAll()` carve-out for `Product`/

Category/Inventory. Mutating/administrative endpoints additionally require a specific `@PreAuthorize("hasAuthority('PERM_...')")` (see section 4); plain read endpoints (list/detail/barcode-lookup) rely only on authentication, so any authenticated role can view products/categories/inventory. `InventoryController` additionally layers **store-access authorization** via `StoreAccessService.resolveViewableStoreId / assertStoreAccess`, checked against the authenticated `UserPrincipal`'s accessible stores — this is independent of the `PERM_*` permission checks.

8. Permissions

From `Permission.java / RolePermissions.java`:

- `ITEM_VIEW`, `ITEM_CREATE`, `ITEM_EDIT` — module "Master". `ITEM_CREATE / ITEM_EDIT` gate `ProductController`'s mutating/import/export endpoints. `ITEM_VIEW` is defined but **not** enforced via `@PreAuthorize` on any `Product/Category` GET endpoint in this codebase (those rely on `.anyRequest().authenticated()` only); it is granted to `ADMIN` (implicit)/ `ACCOUNTANT` / `SALES_USER` / `PURCHASE_USER` / `INVENTORY_USER` / `STAFF` roles per `RolePermissions`.
- `MASTER_MANAGE` — module "Master". Gates all mutating `CategoryController` endpoints.
- `INVENTORY_VIEW`, `INVENTORY_ADJUST` — module "Inventory". `INVENTORY_ADJUST` gates `POST /api/inventory/adjust` and `GET /api/inventory/export`. `INVENTORY_VIEW` is defined and assigned to `ACCOUNTANT / INVENTORY_USER` roles but, like `ITEM_VIEW`, is **not** enforced via `@PreAuthorize` on `InventoryController`'s GET endpoints in the current code — access there is controlled by authentication + `StoreAccessService`, not this permission.
- `STORE_TRANSFER_*` permissions exist but belong to the separate Stock Transfer module (see section 13).

Role assignment excerpt (`RolePermissions`): `INVENTORY_USER` gets `ITEM_VIEW`, `INVENTORY_VIEW`, `INVENTORY_ADJUST`, `STOCK_TRANSFER_VIEW`, `STOCK_TRANSFER_CREATE`, `STOCK_TRANSFER_DISPATCH`, `STOCK_TRANSFER_RECEIVE`, `MASTER_VIEW`, `STORE_VIEW`; `ACCOUNTANT` gets `ITEM_VIEW`, ..., `INVENTORY_VIEW` (view-only, no adjust); `SALES_USER / PURCHASE_USER / STAFF` get `ITEM_VIEW` but no `INVENTORY_*` permissions at all.

9. Transaction Handling

- `ProductService.createProduct`, `updateProduct`, `setStatus`, `backfillMissingSkus`, `importCsv` are all `@Transactional` (default propagation) — a failure anywhere (including the `inventoryService.createInventoryForProduct` call inside `createProduct`) rolls back the whole product write.
- `InventoryService.adjustStock`, `applyMovement` (both overloads), `createInventoryForProduct`, and the startup migration methods are all `@Transactional` (default propagation) — an `Inventory` update and its paired `StockHistory` insert commit or roll back together.
- `SkuGeneratorService.generateNext()` and `BarcodeGeneratorService.generateNext()` run in their **own REQUIRES_NEW transaction**, deliberately separate from the caller's transaction, so the counter row's pessimistic lock is acquired/released independently — mirrors `VoucherNumberService`'s pattern. This means a generated SKU/barcode number is "spent" even if the caller's own transaction later rolls back (by design — gaps are acceptable, duplicate numbers are not).
- `AuditService.log(...)` is `@Transactional` with **default propagation** (explicitly not `REQUIRES_NEW`) — an audit entry participates in and rolls back with the same transaction as the business action it describes, and any exception inside audit-writing itself is caught/logged rather than propagated, so a transient audit failure never blocks the business operation.
- CSV import (`ProductService.importCsv`) processes all rows inside one `@Transactional` method but catches per-row exceptions internally (adding them to `errors` and continuing) rather than letting one bad row abort the whole batch.

10. Error Handling

`GlobalExceptionHandler` (backend/src/main/java/com/storehub/exception/GlobalExceptionHandler.java) maps:

- `MethodArgumentNotValidException` (DTO `@Valid` failures) → 400 with `ApiError.fieldErrors` populated from each field's `FieldError` (field name → `defaultMessage`), message "Validation failed".
- `ProductNotFoundException` → 404 (has both an `id`-based and a `barcode`-based message constructor).
- `CategoryNotFoundException` → 404.
- `InventoryNotFoundException` → 404.
- `BadRequestException` (used throughout `ProductService` / `InventoryService` / `CategoryService` for uniqueness violations, edit-protection, invalid barcode, negative stock, inactive store/product) → 400, message is the exact string thrown by the service (always specific — SKU/barcode/name/value named explicitly per the project's debugging-rule convention).

On the frontend, `parseApiError()` (frontend/src/utils/apiError.ts) reads `err.response.data` as `ApiErrorResponse`; if `fieldErrors` is non-empty it builds a combined "Field: reason | Field: reason" message and returns the raw map for per-field `invalid / Form.Control.Feedback`-style rendering (`ProductForm.tsx`, `ProductQuickAddModal.tsx`, `CategoryQuickAddModal.tsx`, `StockAdjustmentDialog.tsx` all consume `fieldErrors`); otherwise it falls back to `apiError.message` or a caller-supplied fallback string. A missing HTTP response at all (network/CORS/backend-down) is distinguished with its own message.

11. Audit Flow

`AuditService.log(...)` call sites in this module (all via `com.storehub.entity.AuditAction`):

- `ProductService.createProduct` — `AuditAction.CREATE`, module "ITEM_MASTER", entityType "Product", description "Item '<name>' (SKU <sku>) created".
- `ProductService.updateProduct` — up to one of three `AuditAction.UPDATE` calls, module "ITEM_MASTER": (a) if SKU changed, description "SKU changed for item '<name>' with old/new SKU; (b) if barcode changed, "Barcode changed for item '<name>' with old/new barcode; (c) if neither changed, generic "Item '<name>' (SKU <sku>) updated".
- `ProductService.setStatus` — `AuditAction.UPDATE`, module "ITEM_MASTER", description "Item '<name>' (SKU <sku>) <active/inactive>".
- `ProductService.backfillMissingSkus` — `AuditAction.UPDATE`, module "ITEM_MASTER", description "SKU backfilled for legacy item '<name>' which had none", per product.
- `InventoryService.adjustStock` — `AuditAction.UPDATE`, module "INVENTORY", entityType "Product", old/new value = previous/new stock as strings, description "Stock adjustment (<type>) on '<name>' at store '<storeCode>': <reason>", **with the store's id passed** (the `storeId` overload of `log`, so the entry is attributed to a specific store — Multi-Store spec).

`CategoryService` has **no** `AuditService` calls — category create/update/status changes are not audited in this module.

12. Important Side Effects

- **Store-aware inventory isolation.** The same `Product` can have completely independent `current_stock` at every store it has an `Inventory` row for; nothing about the `Product` entity itself carries a quantity. A store-scoped operation (`adjustStock`, the primary `applyMovement` overload) only ever reads/writes the one (`product`, `store`) row it resolves — it is architecturally impossible for a store-scoped write to affect another store's stock.

- **Default Store fallback.** `createInventoryForProduct` (new product's baseline row) and the transitional `applyMovement(productId, delta, ...)` overload (used by call sites not yet passing their own `storeId` — see section 13) both attribute to `storeService.getOrCreateDefaultStore()`. This is the same "historical-data-safety anchor" used by the Multi-Store startup backfill (`backfillInventoryForExistingProducts`) for pre-existing data.
- **Low stock / reorder threshold logic — two distinct thresholds, not one.** `minStockLevel` (on `Product`) drives `LOW_STOCK` classification (`InventoryResponse.fromEntity: currentStock <= minStockLevel → LOW_STOCK`) and the `InventoryRepository.countLowStock/.search` queries. `reorderLevel` (also on `Product`, separately configurable) drives `InventoryRepository.findReorderCandidates / countReorderCandidates (currentStock <= reorderLevel AND status = ACTIVE)` — used by the Alert Center, distinct from the `LOW_STOCK` stock-status badge. `maxStockLevel` (on `Product`, moved off `Inventory` specifically so it isn't duplicated per-store) drives `OVERSTOCK` classification.
- **CSV injection protection.** `CsvUtil.escape()` prefixes any cell whose first character is `=`, `+`, `-`, `@`, tab, or CR with a literal `'` before RFC4180 quoting — guards against a product name/category planted by one user executing as a spreadsheet formula when another user opens the exported CSV. `BarcodeUtil`'s `SAFE_CHARACTERS` pattern (`^[A-Za-z0-9\-\_/\]+$`) also incidentally closes off a leading `=` at the barcode-validation layer.
- **CSV import never double-counts stock.** Re-importing an existing SKU updates all its master-data fields but explicitly never touches `currentStock`; opening stock only applies to newly-created rows, and even then only through `applyMovement` (a real, audited `STOCK_IN` movement), never a direct `Inventory` write.
- **Legacy schema accommodations.** `products.stock_quantity` remains in the DB (per migration policy, never dropped) but is unmapped by the entity; `InventoryStartupRunner` relaxes it to nullable and backfills `Inventory` rows from it on every startup (idempotent, best-effort, swallows failures).

13. Dependencies on Other Modules

`InventoryService.applyMovement(...)` callers (grepped across `backend/src/main/java/com/storehub/service`):

Caller	File:Line	Context
Purchase reversed (cancel)	<code>PurchaseService.java:437-438</code>	<code>restoreStock: applyMovement(productId, purchase.getStore().getId(), -qty, PURCHASE_CANCEL, ReferenceType.PURCHASE, purchase.getId(), reason)</code>
Purchase posted	<code>PurchaseService.java:445-446</code>	<code>addStock: applyMovement(productId, purchase.getStore().getId(), +qty, PURCHASE, ReferenceType.PURCHASE, purchase.getId(), reason)</code>
Sale cancelled	<code>SaleService.java:488-489</code>	<code>restoreStock: applyMovement(productId, sale.getStore().getId(), +qty, SALE_CANCEL, ReferenceType.SALE, sale.getId(), reason)</code>
Sale posted	<code>SaleService.java:496-497</code>	<code>deductStock: applyMovement(productId, sale.getStore().getId(), -qty, SALE, ReferenceType.SALE, sale.getId(), reason)</code>
Credit Note stock return	<code>CreditNoteService.java:217-218 (store known) / 220-221 (transitional overload, store unknown)</code>	<code>applyMovement(productId, [storeId,] +qty, SALES_RETURN, ReferenceType.CREDIT_NOTE, note.getId(), "Sales return: Credit Note ...")</code>

Caller	File:Line	Context
Credit Note cancel/ reverse	CreditNoteService.java:257-258 (store known) / 260-261 (transitional)	applyMovement(productId, [storeId,] -qty, ADJUSTMENT, ReferenceType.CREDIT_NOTE, note.getId(), reason)
Debit Note stock return	DebitNoteService.java:206-207 (store known) / 209-210 (transitional)	applyMovement(productId, [storeId,] -qty, PURCHASE_RETURN, ReferenceType.DEBIT_NOTE, note.getId(), "Purchase return: Debit Note ...")
Debit Note cancel/ reverse	DebitNoteService.java:245-246 (store known) / 248-249 (transitional)	applyMovement(productId, [storeId,] +qty, ADJUSTMENT, ReferenceType.DEBIT_NOTE, note.getId(), reason)
Stock Transfer dispatch	StockTransferService.java:148-150	applyMovement(productId, transfer.getFromStore().getId(), -qty, TRANSFER_OUT, ReferenceType.STOCK_TRANSFER, transfer.getId(), "...dispatched...")
Stock Transfer receive	StockTransferService.java:172-174	applyMovement(productId, transfer.getToStore().getId(), +qty, TRANSFER_IN, ReferenceType.STOCK_TRANSFER, transfer.getId(), "...received...")
CSV opening stock (this module)	ProductService.java:515-516	applyMovement(saved.getId(), openingStock, STOCK_IN, ReferenceType.MANUAL, null, "Opening stock via CSV import") (transitional overload)

So this module's `InventoryService` is a dependency **of** the Purchase, Sale, Credit Note, Debit Note, and Stock Transfer modules (all call into it), while this module itself only calls into the **Store** module: `InventoryService.applyMovement(productId, delta, ...)` (the transitional, no-storeId overload) and `createInventoryForProduct` both call `storeService.getOrCreateDefaultStore()` — the only outbound dependency of Product/Inventory on another module (`StoreService`, `StoreAccessService`). `InventoryController` also depends on `StoreAccessService` for view-access scoping (section 7).

Note on Stock Transfer: `StockTransferService/StockTransferController` (a separate module) is the only caller that moves stock **between** two stores for the same product (`TRANSFER_OUT` from source, `TRANSFER_IN` to destination) — it is documented separately but depends on this module's `InventoryService.applyMovement` exactly like every other caller.

14. Key Operation Flows

Create Product (with baseline Inventory row).

`ProductForm.tsx` (submit) → `productApi.create(payload)` → `POST /api/products` → `ProductController.createProduct (@PreAuthorize PERM_ITEM_CREATE, @Valid ProductCreateRequest)` → `ProductService.createProduct`: normalize SKU/barcode → uniqueness checks (name/SKU/barcode) → resolve Category (+ optional ItemGroup/Hsn/Unit) → build+save Product (`ProductRepository.save`) → **`InventoryService.createInventoryForProduct(saved)`** → `storeService.getOrCreateDefaultStore()` → `InventoryRepository.findByProductIdAndStoreId` (miss) → `InventoryRepository.save` (new 0-stock row, table `inventory`) → `AuditService.log(CREATE, "ITEM_MASTER", ...)` (table `audit_logs`, other module) → returns `ProductResponse` → 201 Created → frontend navigates to `/products` with a success toast.

Generate SKU.

`ProductForm.tsx` "Generate" button → `productApi.generateSku()` → `POST /api/products/generate-sku` (`PERM_ITEM_CREATE`) → `ProductController.generateSku` → `ProductService.generateSku()` → `SkuGeneratorService.generateNext()` (own `REQUIRES_NEW` tx) → `ItemSkuSequenceRepository.ensureRowExists(1L)` (native upsert) → `lockForUpdate(1L)` (row lock on `item_sku_sequence`) → increment `lastNumber`, format `ITEM-%06d`, loop while

productRepository.existsBySkuIgnoreCase(candidate) → save the sequence row → returns candidate string → GenerateSkuResponse → frontend sets the SKU input value (still editable/submittable, not yet persisted to a product).

Generate Internal Barcode.

Same shape as SKU generation: ProductForm.tsx → productApi.generateBarcode() → POST /api/products/generate-barcode (PERM_ITEM_CREATE) → ProductService.generateBarcode() → BarcodeGeneratorService.generateNext() (OWN REQUIRES_NEW tx, item_barcode_sequence row lock) → format INT-%06d, loop while productRepository.existsByBarcodeIgnoreCase(candidate) → GenerateBarcodeResponse → frontend sets the Barcode field.

Manual Stock Adjustment (STOCK_IN / STOCK_OUT / ADJUSTMENT).

StockAdjustmentDialog.tsx (form step: pick product/store, type, quantity, reason → client-side validate() → review step → Confirm) → inventoryApi.adjust(payload) → POST /api/inventory/adjust (PERM_INVENTORY_ADJUST , @Valid StockAdjustmentRequest) → InventoryController.adjustStock : storeAccessService.assertStoreAccess(user, request.getStoreId()) (403 if not accessible) → InventoryService.adjustStock(request) : reject non-manual movement types → resolve Product , resolveActiveStore (400 if inactive) → getOrCreateInventory → compute newStock per type → reject if negative (400, exact current/requested numbers in the message) → InventoryRepository.save (updates inventory.current_stock) → StockHistoryRepository.save (new row in stock_history, referenceType = MANUAL) → AuditService.log(UPDATE, "INVENTORY", ..., storeId) → returns InventoryResponse → frontend success toast, reloads list + summary.

CSV Import.

Products.tsx (file picker) → productApi.importCsv(file) → POST /api/products/import (PERM_ITEM_CREATE , multipart) → ProductController.importProducts → ProductService.importCsv(file) : read+parse lines (CsvUtil.parseLine) → validate header has required columns → per row: validate required fields, resolve Category, parse numerics, validate barcode → **existing SKU** → update fields on the existing Product (never touches stock) → **new SKU** → build+save Product , createInventoryForProduct (baseline row), if openingStock > 0 → InventoryService.applyMovement(id, openingStock, STOCK_IN, MANUAL, null, "Opening stock via CSV import") (writes inventory + stock_history) → accumulate created/updated/skipped/errors → ImportResultResponse → frontend renders the result summary + per-row error list, reloads the product list on any success.

CSV Export.

Products.tsx / Inventory.tsx Export button → productApi.exportCsv(filters) / inventoryApi.exportCsv(filters) → GET /api/products/export (PERM_ITEM_EDIT) or GET /api/inventory/export (PERM_INVENTORY_ADJUST) → ProductService.exportCsv / InventoryService.exportCsv : re-run the same filtered search(...) query used by the list page (unpaged, Sort/LEFT JOIN FETCH variant), plus bulk stock lookup for products → build CSV rows via CsvUtil.row/CsvUtil.escape (formula-injection-safe) → controller sets Content-Disposition: attachment; filename="products-<date>.csv" (or inventory-<date>.csv), text/csv; charset=UTF-8 → frontend downloadBlob(res.data, filename) triggers the browser download.

15. Manual Changes

- **Add a new Product field:** add the column to Product entity (backend/.../entity/Product.java), add it to ProductCreateRequest / ProductUpdateRequest with appropriate Bean Validation annotations, wire it through ProductService.createProduct / updateProduct (builder/setters) and ProductResponse.fromEntity, add it to the CSV EXPORT_HEADER/export row in ProductService.exportCsv and (if importable) to the CSV import row-parsing block in ProductService.importCsv, then add the corresponding field to frontend types/product.ts

(`Product / ProductPayload`) and the relevant form section in `ProductForm.tsx` (and `ProductViewModal.tsx` if it should be displayed).

- **Change SKU format:** `SkuGeneratorService` (prefix `"ITEM-"`, `%06d` padding in `format(long)`) — the `@Pattern` on `ProductCreateRequest / ProductUpdateRequest.sku` (`^[A-Za-z0-9\-\_]+`) must stay permissive enough for whatever new format is chosen, or be updated in lockstep. `normalizeSku` in `ProductService` controls case/whitespace normalization independent of format.
- **Change barcode format/detection:** `BarcodeUtil` (`backend/.../util/BarcodeUtil.java`) — `SAFE_CHARACTERS`, `detectType()`'s pattern order (INTERNAL prefix checked first), and `isValidEan13Checksum()`. `BarcodeGeneratorService`'s `PREFIX = "INT-"` must stay in sync with `BarcodeUtil.INTERNAL_PREFIX` so generated codes are still auto-detected as `INTERNAL`.
- **Change the edit-protection rule:** `ProductService.updateProduct` — the `hasTransactionHistory` check is `stockHistoryRepository.existsByProductId(id)`; the guard conditions are `if (skuChanging && hasTransactionHistory)` and `if (barcodeChanging && hasTransactionHistory)`. To protect additional fields (e.g. category) the same pattern (compute a `*Changing` boolean, guard on `hasTransactionHistory`) would need to be added here, and `ProductResponse.hasTransactions / ProductForm.tsx`'s `disabled={isEdit && hasTransactions}` logic extended to the new field's inputs.
- **Add a new StockMovementType:** add the enum constant to `backend/.../entity/StockMovementType.java`; if it should be selectable via manual adjustment, add it to `InventoryService.MANUAL_MOVEMENT_TYPES`; add it to the frontend `types/inventory.ts` `StockMovementType` union (and `ManualAdjustmentType` if manual) and to `StockHistoryModal.tsx`'s `MOVEMENT_TYPES` filter list / `movementVariant()` color mapping; decide which caller (Purchase/Sale/Credit Note/Debit Note/Stock Transfer/this module) should pass it to `applyMovement`.
- **Change low-stock threshold logic:** `InventoryResponse.fromEntity` (`stockStatus` computation: `LOW_STOCK` at `currentStock <= minStockLevel`, `OVERSTOCK` at `currentStock > maxStockLevel`) and the matching JPQL in `InventoryRepository.search / countLowStock / countOverstock / countOutOfStock` must be changed together (the JPQL literally repeats the same boundary logic as the Java code — no shared helper). Reorder-specific logic (distinct from low-stock) lives in `InventoryRepository.findReorderCandidates / countReorderCandidates` against `Product.reorderLevel`. The frontend's own classifier `utils/stockStatus.ts` (`getStockStatus`, `Products` page only, ignores `maxStockLevel / overstock`) would need a matching update if threshold semantics change, since it currently duplicates only the low/out-of-stock half of the backend logic.
- **Modules affected by any InventoryService.applyMovement signature or behavior change:** every caller listed in section 13 — `PurchaseService`, `SaleService`, `CreditNoteService`, `DebitNoteService`, `StockTransferService`, and this module's own `ProductService.importCsv`. **Every place stock is displayed** that would need re-checking after a stock-computation change: `Products.tsx` (`stockQuantity`, `aggregate`), `Inventory.tsx / InventoryViewModal.tsx / StockAdjustmentDialog.tsx` (`currentStock`, `per-store`), `ProductViewModal.tsx`, `StockHistoryModal.tsx` (previous/new stock columns), plus `ProductResponse.stockQuantity` and `InventoryResponse.currentStock / stockStatus` on the backend, and any Dashboard/Alert Center widgets that read `InventoryService.getSummary / getReorderCandidates` (outside this module's ground-truth file list — NOT FOUND IN CURRENT CODEBASE for this doc's read set, cross-check `AlertService / dashboard` controllers separately if changing summary semantics).

Purchase Module

1. Overview

The Purchase module covers the full buy-side transactional lifecycle in StoreHub:

Purchase Order (optional) → Purchase Bill (GST-applicable, `TransactionType.PURCHASE`) **OR Kacchi Purchase / Purchase Challan** (`TransactionType.PURCHASE_CHALLAN`, starts as `PurchaseStatus.DRAFT`, non-GST-reportable) → **Payment** (against one or more bills, or on-account) → **Debit Note** (purchase return / supplier debit / price or tax adjustment against a posted Purchase Bill).

All five documents share one `Purchase` entity (`backend/src/main/java/com/storehub/entity/Purchase.java`) and one `PurchaseService` (`backend/src/main/java/com/storehub/service/PurchaseService.java`) — a Purchase Bill and a Kacchi Purchase are the same table/entity, differentiated only by `transactionType` and, while in draft, `status`.

TransactionType (`backend/src/main/java/com/storehub/entity/TransactionType.java`)

```
public enum TransactionType { SALE, SALE_CHALLAN, PURCHASE, PURCHASE_CHALLAN }
```

- `PURCHASE` — a normal GST or Non-GST Purchase Bill. Always created already `COMPLETED` (posted).
- `PURCHASE_CHALLAN` — a Kacchi Purchase / Purchase Challan. GST is **still calculated in full** exactly like a normal purchase (see `gstReportingApplicable` below); it is only excluded from GST return reporting (GSTR-1/GSTR-3B). Only a `PURCHASE_CHALLAN` may be saved as `DRAFT` (`PurchaseService.createPurchase`: "Only a Kacchi Purchase / Purchase Challan can be saved as a draft").

`Purchase.gstReportingApplicable` is set at creation to `transactionType != PURCHASE_CHALLAN` (`Purchase.onCreate()` / `PurchaseService.createPurchase`) and is read everywhere GST reporting eligibility is decided (`PurchaseRepository.findEligibleForReconciliation`, `PurchaseItemRepository.hsnSummary` / `taxRateSummary`).

PurchaseStatus (`backend/src/main/java/com/storehub/entity/PurchaseStatus.java`)

```
public enum PurchaseStatus { DRAFT, PENDING, COMPLETED, CANCELLED }
```

- `DRAFT` — saved but not posted; **no stock, ledger, GST-log, or accounting effects yet**. Only reachable via a Kacchi Purchase / Purchase Challan (`saveAsDraft=true`).
- `PENDING` — defined in the enum and used as the JPA `@PrePersist` default (`Purchase.onCreate()`), but no code path in `PurchaseService` / `PurchaseController` ever sets a purchase to `PENDING` — every non-draft create goes straight to `COMPLETED`. Treat `PENDING` as effectively unused in the current codebase (`grep` confirms no assignment other than the default).
- `COMPLETED` — posted: stock added, supplier ledger CREDIT entry, GST log entry (if GST type), accounting journal posted, purchase-order received-quantity consumed, and (if `paidAmount > 0`) a system-generated `Payment`.
- `CANCELLED` — soft reversal; all of the above effects are reversed, `payableAmount` is zeroed, and the row is kept for history.

A `PurchaseOrder` is a separate entity/table (`purchase_orders`) with no stock/ledger/accounting effects at all — it is a pre-billing commitment tracked through `PurchaseOrderStatus` (`DRAFT`, `CONFIRMED`,

PARTIALLY_RECEIVED , COMPLETED , CANCELLED), and its lines carry a receivedQuantity that PurchaseService increments/decrements as bills are posted/reversed against it.

2. Frontend

Files and Components

File	Purpose
frontend/src/pages/purchases/PurchaseHub.tsx	Landing page for the module; shows PurchaseSummary stat cards and links to the 5 sub-sections (Orders, Bills, Kacchi, Payments, Debit Notes).
frontend/src/pages/purchases/PurchaseOrders.tsx	Purchase Order list with search/supplier/status/date filters, cancel dialog.
frontend/src/pages/purchases/PurchaseOrderForm.tsx	Create/edit Purchase Order (supplier, items, GST%, discount).
frontend/src/pages/purchases/PurchaseOrderDetail.tsx	View a PO; actions: Create Purchase Bill, Edit, Cancel.
frontend/src/pages/purchases/PurchaseBills.tsx	Purchase Bill (transactionType=PURCHASE) list; view/edit/payment/delete actions.
frontend/src/pages/purchases/PurchaseBillForm.tsx	Create/edit a Purchase Bill, from scratch or pre-filled ?orderId= from a PO; barcode scan add-item; GST/Non-GST toggle, tax-mode suggestion from supplier state.
frontend/src/pages/purchases/PurchaseBillDetail.tsx	Printable bill view; Edit / Payment / Return (Debit Note) / Delete actions.
frontend/src/pages/purchases/KacchiPurchases.tsx	Kacchi Purchase list (transactionType=PURCHASE_CHALLAN filter); Post action for DRAFT rows.
frontend/src/pages/purchases/KacchiPurchaseForm.tsx	Create/edit a Kacchi Purchase; "Save as Draft" vs "Post Now"/"Save & Post" buttons; always GST-type internally.
frontend/src/pages/purchases/KacchiPurchaseDetail.tsx	Printable challan view with "GST Calculated: YES · GST Reporting: NO" banner; Post/Edit/Return/Delete actions.
frontend/src/pages/purchases/Payments.tsx	Payment Entry list; delete action.
frontend/src/pages/purchases/PaymentForm.tsx	Record a payment: supplier → loads outstanding bills + outstanding credit expenses (paymentApi.getOutstanding) → optional per-row checkbox allocation (else server FIFO-allocates).
frontend/src/pages/purchases/PaymentDetail.tsx	Printable payment voucher; shows allocations; delete action.
frontend/src/pages/purchases/DebitNotes.tsx	Debit Note list with status filter.
frontend/src/pages/purchases/DebitNoteForm.tsx	Search/select a COMPLETED source purchase, then enter per-line return quantities; "Save as Draft" or "Save and Post".
frontend/src/pages/purchases/DebitNoteDetail.tsx	Note detail; Post/Cancel actions gated on user.role === 'ADMIN' 'STORE_MANAGER' client-side (canManage).
frontend/src/api/purchaseApi.ts	purchaseApi — list/getById/create/update/post/cancel/remove against / purchases .
frontend/src/api/purchaseOrderApi.ts	purchaseOrderApi — list/getById/create/update/setStatus/cancel against / purchase-orders .
frontend/src/api/paymentApi.ts	paymentApi — list/getById/getOutstanding/create/remove against / payments .

File	Purpose
frontend/src/api/purchaseSummaryApi.ts	purchaseSummaryApi.get() against /purchase-summary.
frontend/src/api/debitNoteApi.ts	debitNoteApi — list/getById/create/post/cancel against /debit-notes.
frontend/src/types/purchase.ts	Purchase, PurchaseItem, PurchaseCreatePayload, PurchaseUpdatePayload, PurchaseStatus, PurchaseTransactionType, PaymentStatus, GstType, TaxMode, PaymentMode.
frontend/src/types/purchaseOrder.ts	PurchaseOrder, PurchaseOrderItem, PurchaseOrderPayload, PurchaseOrderStatus.
frontend/src/types/payment.ts	Payment, PaymentAllocation, PaymentPayload, SupplierOutstanding, OutstandingPurchaseBill, OutstandingExpense, PurchaseSummary.
frontend/src/types/note.ts	DebitNote, DebitNoteItem, DebitNoteCreatePayload, NoteStatus, StockImpactType, DebitNoteType (also CreditNote* for the Sales-side mirror).

Frontend Flow

Purchase Order create/edit: PurchaseOrderForm collects supplier + item rows (product, qty, rate, discount, GST%); client computes row/grand totals only for display — the server recomputes and is authoritative. Submits purchaseOrderApi.create / .update → on success navigates to PurchaseOrderDetail. There is no separate "approve" step in the UI; PurchaseOrderDetail / PurchaseOrders expose a setStatus API (DRAFT / CONFIRMED only) but **no page currently calls purchaseOrderApi.setStatus** (grep finds no caller) — status otherwise advances automatically to PARTIALLY_RECEIVED / COMPLETED as bills are posted against it (PurchaseOrderService.recomputeStatus). Cancel is only allowed while nothing has been received.

Purchase Bill create from scratch or from PO: PurchaseBillForm either starts empty or, when navigated to with ?orderId=<id>, calls purchaseOrderApi.getById and pre-fills supplier + only the order's lines with remainingQuantity > 0, capping each row's editable quantity at that remainingQuantity and tagging it with purchaseOrderItemId. GST type/tax-mode toggle recomputes CGST/SGST/IGST client-side for display; server is authoritative. Editing an existing bill that already hasPayments locks the whole form (<fieldset disabled={locked}>) — the backend rejects such edits outright (see §6).

Kacchi Purchase create: KacchiPurchaseForm is a near-duplicate of PurchaseBillForm but always sends gstType: 'GST' and transactionType: 'PURCHASE_CHALLAN', and offers two submit buttons — "Save as Draft" (saveAsDraft: true) and "Post Now"/"Save & Post" (posts immediately, with a confirm dialog warning that posting adds stock/ledger/journal and "cannot be undone from here"). Editing only allowed while still DRAFT (page redirects otherwise). KacchiPurchaseDetail exposes a standalone **Post** button (purchaseApi.post(id) → POST /api/purchases/{id}/post) for a saved draft.

Payment entry with multi-bill allocation: PaymentForm picks a supplier, then loads paymentApi.getOutstanding(supplierId) which returns both outstanding Purchase Bills and outstanding credit Expenses for that supplier. The user may check zero, some, or all rows and edit each row's "Apply Amount" (defaults to the full payable). If no rows are checked, the payload's allocations array is empty and the backend allocates FIFO by date across bills+expenses combined. Client validates each checked row's amount ≤ its payable and the sum of checked amounts ≤ the payment amount.

Debit Note create from a Bill: DebitNoteForm either receives ?purchaseId= (from PurchaseBillDetail / KacchiPurchaseDetail "Return" button) or lets the user search COMPLETED purchases by number/supplier. Once a source purchase is selected, the form lists its items with an editable "Return Qty" per line

(capped client-side at `item.quantity`; the server further caps at `quantity - alreadyReturned`). Submits either "Save as Draft" (`post: false`) or "Save and Post" (`post: true`).

3. API Calls

Function	Method	URL	Request shape	Response shape
<code>purchaseOrderApi.list</code>	GET	<code>/purchase-orders</code>	query: <code>search</code> , <code>supplierId</code> , <code>status</code> , <code>fromDate</code> , <code>toDate</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<PurchaseOrder></code>
<code>purchaseOrderApi.getById</code>	GET	<code>/purchase-orders/{id}</code>	—	<code>PurchaseOrder</code>
<code>purchaseOrderApi.create</code>	POST	<code>/purchase-orders</code>	<code>PurchaseOrderPayload</code>	<code>PurchaseOrder</code> (201)
<code>purchaseOrderApi.update</code>	PUT	<code>/purchase-orders/{id}</code>	<code>PurchaseOrderPayload</code>	<code>PurchaseOrder</code>
<code>purchaseOrderApi.setStatus</code>	PATCH	<code>/purchase-orders/{id}/status?status=</code>	—	<code>PurchaseOrder</code>
<code>purchaseOrderApi.cancel</code>	PATCH	<code>/purchase-orders/{id}/cancel</code>	—	<code>PurchaseOrder</code>
<code>purchaseApi.list</code>	GET	<code>/purchases</code>	query: <code>search</code> , <code>paymentStatus</code> , <code>status</code> , <code>fromDate</code> , <code>toDate</code> , <code>transactionType</code> , <code>page</code> , <code>size</code> (<code>storeId</code> supported server-side, not sent by current frontend query object)	<code>PagedResponse<Purchase></code>
<code>purchaseApi.getById</code>	GET	<code>/purchases/{id}</code>	—	<code>Purchase</code>
<code>purchaseApi.create</code>	POST	<code>/purchases</code>	<code>PurchaseCreatePayload</code>	<code>Purchase</code> (201)
<code>purchaseApi.update</code>	PUT	<code>/purchases/{id}</code>	<code>PurchaseUpdatePayload</code>	<code>Purchase</code>
<code>purchaseApi.post</code>	POST	<code>/purchases/{id}/post</code>	—	<code>Purchase</code> (posts a DRAFT Kacchi challan)
<code>purchaseApi.cancel</code>	PATCH	<code>/purchases/{id}/cancel</code>	—	<code>Purchase</code> (soft reverse; not called by any current page — <code>PurchaseBills / KacchiPurchases</code> use <code>remove</code> instead)
<code>purchaseApi.remove</code>	DELETE	<code>/purchases/{id}</code>	—	204 (hard delete + full reversal)
<code>paymentApi.list</code>	GET	<code>/payments</code>	query: <code>search</code> , <code>supplierId</code> , <code>fromDate</code> , <code>toDate</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<Payment></code>
<code>paymentApi.getById</code>	GET	<code>/payments/{id}</code>	—	<code>Payment</code>

Function	Method	URL	Request shape	Response shape
paymentApi.getOutstanding	GET	/payments/ outstanding/ {supplierId}	—	SupplierOutstanding
paymentApi.create	POST	/payments	PaymentPayload	Payment (201)
paymentApi.remove	DELETE	/payments/ {id}	—	204
debitNoteApi.list	GET	/debit- notes	query: search, status, fromDate, toDate, page, size	PagedResponse<DebitNote>
debitNoteApi.getById	GET	/debit- notes/{id}	—	DebitNote
debitNoteApi.create	POST	/debit- notes	DebitNoteCreatePayload	DebitNote (201)
debitNoteApi.post	POST	/debit- notes/{id}/ post	—	DebitNote
debitNoteApi.cancel	PATCH	/debit- notes/{id}/ cancel	—	DebitNote
purchaseSummaryApi.get	GET	/purchase- summary	—	PurchaseSummary

4. Backend

Controllers

PurchaseController (/api/purchases)

- GET /api/purchases — PERM_PURCHASE_VIEW — search/filter/paginate.
- GET /api/purchases/{id} — PERM_PURCHASE_VIEW.
- POST /api/purchases — PERM_PURCHASE_CREATE — create (Bill or Kacchi, draft or posted).
- PUT /api/purchases/{id} — PERM_PURCHASE_EDIT — full update.
- POST /api/purchases/{id}/post — PERM_PURCHASE_POST — posts a DRAFT Kacchi Purchase.
- PATCH /api/purchases/{id}/cancel — PERM_PURCHASE_CANCEL — soft reversal (keeps the row).
- DELETE /api/purchases/{id} — PERM_PURCHASE_CANCEL — hard delete with full reversal.

PurchaseOrderController (/api/purchase-orders)

- GET /api/purchase-orders — PERM_PURCHASE_VIEW.
- GET /api/purchase-orders/{id} — PERM_PURCHASE_VIEW.
- POST /api/purchase-orders — PERM_PURCHASE_CREATE.
- PUT /api/purchase-orders/{id} — PERM_PURCHASE_EDIT.
- PATCH /api/purchase-orders/{id}/status?status= — PERM_PURCHASE_EDIT.
- PATCH /api/purchase-orders/{id}/cancel — PERM_PURCHASE_CANCEL.

PaymentController (/api/payments)

- GET /api/payments — PERM_PAYMENT_VIEW.
- GET /api/payments/{id} — PERM_PAYMENT_VIEW.
- GET /api/payments/outstanding/{supplierId} — PERM_PAYMENT_VIEW.
- POST /api/payments — PERM_PAYMENT_CREATE.

- DELETE /api/payments/{id} — PERM_PAYMENT_POST (note: delete is gated on the POST permission, not a dedicated cancel permission — there is no PAYMENT_CANCEL).

PurchaseSummaryController (/api/purchase-summary)

- GET /api/purchase-summary — **no @PreAuthorize** on the endpoint or class (relies on the global authenticated-request filter chain only). NOT FOUND: any permission check for this endpoint.

DebitNoteController (/api/debit-notes)

- GET /api/debit-notes — **no @PreAuthorize** (view is open to any authenticated user; unlike PurchaseController / PaymentController there is no PERM_DEBIT_NOTE_VIEW check here despite that permission existing).

- GET /api/debit-notes/{id} — **no @PreAuthorize**.

- POST /api/debit-notes — PERM_DEBIT_NOTE_CREATE.

- POST /api/debit-notes/{id}/post — PERM_DEBIT_NOTE_POST.

- PATCH /api/debit-notes/{id}/cancel — PERM_DEBIT_NOTE_CANCEL.

DTOs

- **PurchaseCreateRequest**: supplierId (@NotNull), storeId (optional), purchaseDate (@NotNull), gstType (@NotNull), taxMode (required only when GST — checked in service, not annotation), supplierPhone, supplierGstin, billingAddress, shippingAddress, paymentMode (@NotNull), paidAmount (@NotNull @DecimalMin("0")), notes, purchaseOrderId, items (@NotEmpty @Valid List<PurchaseItemRequest>), transactionType, saveAsDraft (boolean, Kacchi-only).
- **PurchaseUpdateRequest**: same core fields as create plus status (@NotNull PurchaseStatus); no storeId / purchaseOrderId / transactionType / saveAsDraft (store and transaction type are fixed at creation).
- **PurchaseItemRequest**: productId (@NotNull), quantity (@NotNull @Min(1)), purchasePrice (@NotNull @DecimalMin("0")), discount (@NotNull @DecimalMin("0")), tax (@NotNull @DecimalMin("0")) — client-computed, not authoritative), gstPercent (@DecimalMin("0")), purchaseOrderItemId (optional).
- **PurchaseResponse** / **PurchaseItemResponse**: full read model including derived subtotalAmount / totalDiscount / totalTax (fromEntity sums), hasPayments, gstReportingApplicable, transactionType, store fields.
- **PurchaseOrderRequest**: supplierId (@NotNull), storeId, supplierPhone/Gstin, addresses, orderDate (@NotNull), expectedDeliveryDate, remarks, items (@NotEmpty @Valid List<PurchaseOrderItemRequest>).
- **PurchaseOrderItemRequest**: productId (@NotNull), quantity (@NotNull @Min(1)), rate (@NotNull @DecimalMin("0")), discount (@NotNull @DecimalMin("0")), gstPercent (@NotNull @DecimalMin("0")).
- **PurchaseOrderResponse** / **PurchaseOrderItemResponse**: adds derived receivedAmount / remainingAmount (proportional to receivedQuantity), and per-item remainingQuantity.
- **PaymentRequest**: supplierId (@NotNull), storeId, paymentDate (@NotNull), amount (@NotNull @DecimalMin("0.01")), paymentMode (@NotNull), remarks, allocations (@Valid List<PaymentAllocationRequest>, optional/empty ⇒ server FIFO).
- **PaymentAllocationRequest**: exactly one of purchaseId / expenseId (enforced in service, not by annotation), amountApplied (@NotNull @DecimalMin("0.01")).
- **PaymentResponse** / **PaymentAllocationResponse**: includes resolved purchaseNumber / expenseNumber.
- **SupplierOutstandingResponse** / **OutstandingPurchaseBillResponse** / **OutstandingExpenseResponse**: read models for GET /payments/outstanding/{supplierId}.
- **PurchaseSummaryResponse**: totalPurchaseOrders, pendingOrders, totalPurchaseBills, totalPayables, todaysPurchases.

- **DebitNoteCreateRequest**: sourcePurchaseId (@NotNull), noteType (@NotNull DebitNoteType), noteDate (@NotNull), stockImpact (@NotNull StockImpactType), reason, remarks, items (@NotEmpty @Valid List<DebitNoteItemRequest>), post (boolean — create-then-post in one call).
- **DebitNoteItemRequest**: purchaseItemId (@NotNull), quantity (@NotNull @Min(1)) — rate/tax are **not** accepted from the client; the service derives them proportionally from the original PurchaseItem.
- **DebitNoteResponse** / **DebitNoteItemResponse**: full read model including createdBy / postedBy / postedAt / cancelledBy / cancelledAt audit trail fields.

Services

PurchaseService (backend/src/main/java/com/storehub/service/PurchaseService.java)

- getPurchases(...) — store-scoped search via storeAccessService.resolveViewableStoreId.
- getPurchaseById(id) — asserts caller has access to the purchase's store (storeAccessService.assertStoreAccess); flags hasPayments from PaymentAllocationRepository.findByPurchaseId.
- createPurchase(request) — validates GST/tax-mode consistency and GSTIN format (GstinValidator.isValid), resolves supplier/store (rejecting inactive ones), resolves storeId via storeAccessService.resolveEffectiveStoreId, builds the Purchase with status = DRAFT only if saveAsDraft && transactionType == PURCHASE_CHALLAN (else COMPLETED), computes line items + totals (applyItems), computes paid/payable/paymentStatus (applyPayment), saves, assigns the voucher number (voucherNumberService.next(PURCHASE_CHALLAN or PURCHASE, purchaseDate)), and — **only if not draft** — calls applyPostingEffects. Always writes an AuditAction.CREATE audit log.
- postPurchaseChallan(id) — the only way a DRAFT (Kacchi) purchase becomes COMPLETED; guards transactionType == PURCHASE_CHALLAN and status == DRAFT, flips status, then applyPostingEffects.
- applyPostingEffects(saved) (private) — the single place every "posting" side effect happens: addStock (inventory), ledgerService.recordPurchaseCredit, ledgerService.recordInputGstEntry, accountingService.postPurchaseJournal, consumeOrderQuantities(+1) (PO received-qty), gstTransactionSyncService.syncPurchase, and — if paidAmount > 0 — paymentService.createSystemPaymentForPurchase.
- updatePurchase(id, request) — blocks editing a CANCELLED purchase, blocks setting status to CANCELLED via this endpoint ("Use the delete action to cancel and reverse a purchase"), and **blocks editing entirely once any payment allocation exists** ("Delete the payment first..."). If the old row was COMPLETED, first fully reverses it (restoreStock + ledger/GST/journal/PO-qty reversal), then rebuilds items/totals from the new request and, if the new status isn't DRAFT, re-applies all posting effects (stock add, ledger, journal, PO-qty, GST sync, system payment).
- cancelPurchase(id) (soft) — rejects if already CANCELLED; guardNoManualPayments (rejects if any manual — non-system — payment allocation exists); calls reversePurchaseEffects, sets status = CANCELLED and payableAmount = 0; audit-logs CANCEL.
- deletePurchase(id) (hard) — same manual-payment guard, reversePurchaseEffects, then purchaseRepository.delete.
- reversePurchaseEffects(purchase, reason) (private) — if COMPLETED, restoreStock; if still DRAFT, returns immediately (nothing was ever posted, nothing to reverse); otherwise deletes every system-generated Payment tied to it (paymentService.deleteSystemPayment per allocation), reverses the supplier-ledger CREDIT, reverses the GST log entry, reverses the accounting journal, decrements PO received-qty, reverses the GST-return sync.
- applyItems(...) (private) — rejects duplicate products in one purchase, rejects inactive products (unless already on the purchase being edited), validates a PO-linked line doesn't exceed orderItem.getRemainingQuantity(), computes taxable/CGST/SGST/IGST per line via GstCalculationService.calculateLine, accumulates purchase-level totals.
- restoreStock / addStock (private) — call inventoryService.applyMovement(productId, storeId, ±qty, StockMovementType.PURCHASE_CANCEL|PURCHASE, ReferenceType.PURCHASE, purchaseId, reason).
- findPurchaseOrThrow(id) — throws PurchaseNotFoundException.

PurchaseOrderService (backend/src/main/java/com/storehub/service/PurchaseOrderService.java)

- search(...) / getById(id) — store-scoped, same storeAccessService pattern as Purchase.
- create(request) — validates supplier/store active, builds items (applyItems , rejects duplicate products), status always starts DRAFT , assigns voucher number (VoucherDocType.PURCHASE_ORDER).
- update(id, request) — blocks editing a CANCELLED or COMPLETED order; guards that any product with receivedQuantity > 0 stays present with quantity ≥ what was already received; rebuilds items, recomputeStatus .
- setStatus(id, status) — only DRAFT / CONFIRMED allowed as a manual target; rejects if the order already has any received quantity ("status is now managed automatically").
- cancel(id) — rejects if already cancelled or if any quantity has been received (i.e., a bill exists against it).
- applyReceivedQuantityDelta(purchaseOrderItemId, delta) — called by PurchaseService.consumeOrderQuantities on every posted-purchase create/update/reversal; clamps at [0, orderedQuantity], then recomputeStatus .
- recomputeStatus(order) (private) — COMPLETED once every line's remainingQuantity ≤ 0, else PARTIALLY_RECEIVED if anything has been received, else left as-is (never auto-downgrades a CANCELLED order).
- countPendingOrders() / countTotalOrders() — used by PurchaseSummaryService .
- **No AuditService calls anywhere in this service** — Purchase Order create/update/cancel is **not** audit-logged (unlike Purchase, Payment and Debit Note). NOT FOUND: any auditService.log(...) call in PurchaseOrderService .

PaymentService (backend/src/main/java/com/storehub/service/PaymentService.java) — handles both a Purchase Bill payment and a credit (party) Expense payment through the same PaymentAllocation mechanism.

- search(...) / getById(id) — store-scoped.
- getOutstandingForSupplier(supplierId) — combines purchaseRepository.findOutstandingBySupplier and expenseRepository.findOutstandingBySupplier into one SupplierOutstandingResponse .
- create(request) — resolves supplier/store, builds Payment (systemGenerated=false), assigns voucher number (VoucherDocType.PAYMENT), calls allocate(...) , saves, then ledgerService.recordPaymentDebit + ledgerService.recordCashEntryOut + accountingService.postPaymentJournal ; audit-logs PAYMENT .
- createSystemPaymentForPurchase(purchase) — invoked only from PurchaseService when a bill records an immediate paidAmount ; builds a systemGenerated=true Payment with a single allocation to that purchase for the full paidAmount , posts the same ledger/journal effects. No separate audit log (the parent Purchase's audit entry covers it).
- delete(id) — rejects deleting a **system-generated** payment directly ("Delete the purchase bill instead to reverse it."); otherwise reverseAndRemove + audit-log CANCEL .
- deleteSystemPayment(id) (package-private) — used only by PurchaseService when reversing a purchase; same reverseAndRemove , no separate audit entry.
- reverseAndRemove(payment) (private) — for each allocation, restores the linked Purchase 's or Expense 's paidAmount / payableAmount / paymentStatus ; reverses ledger debit, reverses cash-out entry, reverses the payment journal; deletes the Payment row (allocations cascade).
- allocate(payment, explicit, supplierId) (private) — if explicit allocations given: validates each references exactly one of purchase/expense, belongs to the same supplier, doesn't exceed that document's payableAmount , and the sum doesn't exceed the payment amount. If **no** explicit allocations: FIFO across the supplier's combined outstanding purchases + expenses sorted by date, applying min(remaining, payable) to each until the payment amount is exhausted; any leftover becomes an on-account credit (ledger DEBIT only, not tied to a document).
- findOrThrow(id) — throws PaymentNotFoundException .

PurchaseSummaryService (backend/src/main/java/com/storehub/service/PurchaseSummaryService.java)

- getSummary() — a single read-only aggregation: purchaseOrderRepository.count() ,

```
purchaseOrderService.countPendingOrders(), purchaseRepository.count(),  
ledgerService.getTotalPayables(), purchaseRepository.getTotalPurchasesForDate(today).
```

DebitNoteService (backend/src/main/java/com/storehub/service/DebitNoteService.java) — the Purchase-side mirror of **CreditNoteService** (Sales module).

- search(...) / getById(id) — store access resolved via the **source purchase's** store.
- create(request) — rejects a debit note against a CANCELLED or still-DRAFT (unposted) source purchase; for each requested item, validates the purchaseItemId belongs to the source purchase, quantity > 0, and quantity ≤ purchaseItem.quantity - alreadyReturned (debitNoteItemRepository.sumReturnedQuantity, excluding cancelled notes) — the exact "eligible" message names the product, requested qty, and remaining eligible qty. Each item's taxable/discount/CGST/SGST/IGST is **derived proportionally** from the source PurchaseItem (proportion = totalForLine * returnedQty / purchasedQty, HALF_UP to 2dp) — never re-entered by the client. Resolves financialYearId (financialYearService.resolveForDate). Saves, assigns voucher number (VoucherDocType.DEBIT_NOTE), audit-logs CREATE. If request.isPost(), immediately calls post(id).
- post(id) — only from DRAFT; rejects if the source purchase has since been cancelled. Posts a journal (DEBIT SUPPLIER_PAYABLE for totalAmount, CREDIT PURCHASE for taxableAmount, plus CREDIT lines for any positive CGST/SGST/IGST) via accountingService.postJournal(VoucherType.DEBIT_NOTE, ...). If stockImpact == STOCK_RETURN, calls inventoryService.applyMovement(..., -quantity, StockMovementType.PURCHASE_RETURN, ReferenceType.DEBIT_NOTE, ...) per item (store-aware overload when the source purchase has a store). Calls ledgerService.recordDebitNoteEntry. Flips status = POSTED, stamps postedBy/postedAt, **then** calls gstTransactionSyncService.syncDebitNote (comment: "Must run after the status flip: GstReportingEligibility checks status == POSTED"). Audit-logs POST.
- cancel(id) — idempotent no-op if already CANCELLED. If it was POSTED: reverses the journal (accountingService.reverseJournal), if STOCK_RETURN reverses stock with StockMovementType.ADJUSTMENT (positive delta, putting the returned stock back), reverses the ledger entry, reverses the GST sync. Sets status = CANCELLED, stamps cancelledBy/cancelledAt; audit-logs CANCEL.
- proportion(...) (private) — the shared line-splitting helper described above.
- findOrThrow(id) — throws NoteNotFoundException.

Repository (exact method names)

- **PurchaseRepository**: findBySupplierIdOrderByCreatedAtDesc, findOutstandingBySupplier, findAllOutstanding, findCompletedIds, findAllIds, getTotalPurchasesForDate, sumTotalAmountByDateRange, countByDateRange, sumAndCountByDateRangeAndStore, search, findEligibleForReconciliation.
- **PurchaseItemRepository**: hsnSummary, taxRateSummary.
- **PurchaseOrderRepository**: search, countByStatusIn.
- **PurchaseOrderItemRepository**: no custom methods (plain JpaRepository<PurchaseOrderItem, Long>).
- **PaymentRepository**: findAllIds, sumAmountForDate, search.
- **PaymentAllocationRepository**: findByPurchaseId, findByExpenseId.
- **SupplierLedgerEntryRepository**: findBySupplierIdOrderByEntryDateAscCreatedAtAsc, getOutstandingForSupplier, getTotalOutstanding, findByReferenceTypeAndReferenceId, sumBySupplier.
- **PurchaseGstEntryRepository**: findByPurchaseId.
- **DebitNoteRepository**: search, findPostedIds, findAllIds.
- **DebitNoteItemRepository**: sumReturnedQuantity.

5. Database

Tables

`purchases`, `purchase_items`, `purchase_orders`, `purchase_order_items`, `payments`, `payment_allocations`, `supplier_ledger_entries`, `purchase_gst_entries`, `debit_notes`, `debit_note_items`.

Relationships

- **Purchase ↔ PurchaseItem:** `purchases.id` ← `purchase_items.purchase_id` (`@OneToMany(mappedBy="purchase", cascade=ALL, orphanRemoval=true)`).
- **Purchase ↔ Supplier:** `purchases.supplier_id` → `suppliers.id` (`@ManyToOne, nullable=false`).
- **Purchase ↔ Store:** `purchases.store_id` → `stores.id` (`@ManyToOne, nullable` — fixed at creation, never changed on edit per the class comment).
- **Purchase ↔ PurchaseOrder:** `purchases.purchase_order_id` → `purchase_orders.id` (`@ManyToOne, nullable` — set only when the bill originates from a PO).
- **PurchaseItem ↔ PurchaseOrderItem:** `purchase_items.purchase_order_item_id` → `purchase_order_items.id` (`nullable` — links a billed line back to the order line it fulfilled, driving `receivedQuantity` bookkeeping).
- **PurchaseOrder ↔ PurchaseOrderItem:** `purchase_orders.id` ← `purchase_order_items.purchase_order_id` (`@OneToMany, cascade ALL, orphanRemoval`).
- **Payment ↔ PaymentAllocation ↔ Purchase:** `payments.id` ← `payment_allocations.payment_id`; `payment_allocations.purchase_id` → `purchases.id` (`nullable` — exactly one of `purchase_id` / `expense_id` set, enforced in `PaymentService`, not a DB constraint); `payment_allocations.expense_id` → `expenses.id` (`nullable`, the credit-Expense path).
- **DebitNote ↔ DebitNoteItem ↔ Purchase:** `debit_notes.source_purchase_id` → `purchases.id` (`@ManyToOne, not null`); `debit_notes.id` ← `debit_note_items.debit_note_id`; `debit_note_items.purchase_item_id` → `purchase_items.id` (`not null` — links each returned line back to its original bill line, used by `sumReturnedQuantity` for the eligible-quantity guard).
- **DebitNote ↔ Supplier:** `debit_notes.supplier_id` → `suppliers.id` (copied from the source purchase at creation).
- **SupplierLedgerEntry:** `supplier_ledger_entries.supplier_id` → `suppliers.id`; polymorphic `reference_type` + `reference_id` (no FK) pointing at whichever document (Purchase, Payment, DebitNote, Expense) posted the entry.
- **PurchaseGstEntry:** `purchase_gst_entries.purchase_id` — plain Long column, no FK/`@ManyToOne` (looked up via `findByPurchaseId`).

6. Validation

- **Bean validation** (`jakarta.validation` annotations on the DTOs) — see §4 DTOs for the exact constraints (`@NotNull`, `@NotEmpty`, `@Min(1)`, `@DecimalMin`). A violation throws `MethodArgumentNotValidException`, mapped to 400 Bad Request with `ApiError.fieldErrors` populated from each field's `getDefaultMessage()` (`GlobalExceptionHandler.handleValidation`).
- **Service-level business validation** (throws `BadRequestException`, mapped to 400 with `no fieldErrors` — only a top-level message):
- GST purchase without a `taxMode` ("Tax mode (Intra-State or Inter-State) is required for a GST purchase").
- Malformed `supplierGstin` (`GstinValidator.isValid` — 15-char GSTIN format).
- Inactive supplier / inactive store used for a new purchase, PO, or edit onto a different supplier.
- Draft save requested for a non-Kacchi `transactionType`.
- `saveAsDraft` / `challan`-only actions on the wrong `transactionType` or wrong current `status`.

- Duplicate product within one purchase or PO's item list.
- A PO-linked bill line's quantity exceeding `orderItem.getRemainingQuantity()`.
- Discount exceeding the line's taxable amount.
- `paidAmount` exceeding `totalAmount`.
- Editing a `CANCELLED` purchase, or setting status to `CANCELLED` via `PUT /purchases/{id}` (must use delete/cancel endpoints).
- Editing a purchase that already has any payment allocation.
- Cancelling/deleting a purchase that has a **manual** (non-system) payment against it — the payment must be deleted first.
- PO: editing a `CANCELLED / COMPLETED` order; reducing/removing a line already partially received; manually setting status once anything has been received; cancelling an order with any received quantity.
- Payment: allocation referencing neither/both purchase and expense; allocation amount exceeding the target's payable amount; total explicit allocations exceeding the payment amount; allocation's purchase/expense not belonging to the stated supplier.
- Payment delete: rejecting deletion of a system-generated payment directly.
- Debit Note: source purchase `CANCELLED` or still `DRAFT`; a requested `purchaseItemId` not belonging to the stated source purchase; return quantity ≤ 0 or exceeding `quantity - alreadyReturned`; posting anything but a `DRAFT` note; posting when the source purchase has since been cancelled.
- Per this repo's `CLAUDE.md` debugging rule (Login/Registration/User Management scope), the field-error-surfacing pattern (`isValid / Form.Control.Feedback` -equivalent via React `fieldErrors` state + `parseApiError`) is already followed in `PurchaseOrderForm.tsx`, `PurchaseBillForm.tsx`, `KacchiPurchaseForm.tsx` for the annotated DTO fields (e.g. `supplierId`, `orderDate`, `purchaseDate`, `paidAmount`) — business-rule `BadRequestException` messages (no `fieldErrors`) are shown only via the top-level `error / Alert` banner, since the backend does not attach a field key to them.

7. Authentication / Authorization

Every endpoint in this module sits behind the application's standard JWT bearer-token filter chain (`backend/src/main/java/com/storehub/config/SecurityConfig.java`); all `PurchaseController`, `PurchaseOrderController`, `PaymentController` and most `DebitNoteController` methods additionally require a specific granted authority via `@PreAuthorize("hasAuthority('PERM_...')")`. Two `DebitNoteController` read endpoints (`GET /api/debit-notes`, `GET /api/debit-notes/{id}`) and the whole `PurchaseSummaryController` carry **no** `@PreAuthorize` — see §4 Controllers. Authorities are granted per `Role` via the fixed map in `backend/src/main/java/com/storehub/service/RolePermissions.java` (see §8).

8. Permissions

From `backend/src/main/java/com/storehub/entity/Permission.java` and `RolePermissions.java`:

Permission	Purpose	Granted to
<code>PURCHASE_VIEW</code>	List/view purchases & purchase orders	ADMIN, STORE_MANAGER, ACCOUNTANT, PURCHASE_USER
<code>PURCHASE_CREATE</code>	Create a purchase / PO	ADMIN, STORE_MANAGER, PURCHASE_USER
<code>PURCHASE_EDIT</code>	Update a purchase / PO, PO status	ADMIN, STORE_MANAGER, PURCHASE_USER
<code>PURCHASE_POST</code>	Post a DRAFT Kacchi challan	ADMIN, STORE_MANAGER, PURCHASE_USER

Permission	Purpose	Granted to
PURCHASE_CANCEL	Cancel (soft) / delete (hard) a purchase or PO	ADMIN, STORE_MANAGER only (excluded from PURCHASE_USER's set)
PAYMENT_VIEW	List/view payments, outstanding	ADMIN, STORE_MANAGER, ACCOUNTANT, PURCHASE_USER
PAYMENT_CREATE	Record a payment	ADMIN, STORE_MANAGER, PURCHASE_USER
PAYMENT_POST	Delete/reverse a payment (also gates DELETE / api/payments/{id})	ADMIN, STORE_MANAGER, PURCHASE_USER
DEBIT_NOTE_VIEW	(defined, but not enforced by DebitNoteController 's GET endpoints — see §4/ §7)	ADMIN, STORE_MANAGER, ACCOUNTANT, PURCHASE_USER
DEBIT_NOTE_CREATE	Create a debit note	ADMIN, STORE_MANAGER only
DEBIT_NOTE_POST	Post a debit note	ADMIN, STORE_MANAGER only
DEBIT_NOTE_CANCEL	Cancel a debit note	ADMIN, STORE_MANAGER only

PURCHASE_USER therefore cannot cancel/delete a purchase or PO, nor create/post/cancel a debit note (matching DebitNoteDetail.tsx 's client-side `canManage = role === 'ADMIN' || 'STORE_MANAGER'` gate — the client and server checks agree for Debit Note actions). ACCOUNTANT has view-only access across Purchase/Payment/Debit-Note. There is no PAYMENT_CANCEL permission — deletion of a payment reuses PAYMENT_POST.

9. Transaction Handling

Every mutating method in PurchaseService, PurchaseOrderService, PaymentService, and DebitNoteService is @Transactional (class-level @Transactional is not used; each public mutator is annotated individually) — read paths in DebitNoteService use @Transactional(readOnly = true); read paths elsewhere run without an explicit annotation (plain reads under the default propagation of whatever calls them, typically none needed for a single findById).

Atomicity boundaries:

- **createPurchase**: item/total computation, save, voucher-number assignment, and (if not draft) the entire applyPostingEffects cascade (stock, ledger CREDIT, GST log, accounting journal, PO received-qty, GST-return sync, and an immediate system Payment with its own ledger/journal) all commit or roll back together in one transaction.
- **postPurchaseChallan**: status flip + applyPostingEffects — one transaction.
- **updatePurchase**: full reversal-then-reapply cycle (if the old row was posted) plus the new item/total rebuild — one transaction; a failure partway (e.g. a bad new item) rolls back the reversal too, leaving the original posted state intact.
- **cancelPurchase** / **deletePurchase**: the manual-payment guard, every system-payment deletion (paymentService.deleteSystemPayment, itself @Transactional but running inside the caller's transaction due to default REQUIRED propagation), ledger/GST/journal/PO-qty reversal, and the status flip or row delete — one transaction.
- **PaymentService.create** / **createSystemPaymentForPurchase**: allocation loop (which mutates every allocated Purchase / Expense row), ledger debit, cash-out entry, and accounting journal — one transaction.
- **PaymentService.delete** / **deleteSystemPayment**: reversing every allocation's target document, ledger/cash/journal reversal, and the Payment row delete — one transaction.
- **DebitNoteService.post**: journal posting, conditional stock reversal-movement, ledger entry, status flip, and GST sync — one transaction; note the ordering comment that GST sync must run after the status

flip.

- `DebitNoteService.cancel` : journal reversal, conditional stock re-add, ledger reversal, GST-sync reversal, and status flip — one transaction.
- `VoucherNumberService.next(...)` runs with `REQUIRES_NEW` propagation (per its own Javadoc, referenced from `PurchaseService / PurchaseOrderService / PaymentService / DebitNoteService`) so a voucher number, once issued, is never rolled back even if the enclosing purchase/payment/note transaction later fails — preventing gaps from being reused, at the cost of a possible gap in the sequence on failure.
- `AuditService.log(...)` deliberately runs in the same transaction as its caller (default propagation) — an audit entry for an action that rolls back rolls back with it.

10. Error Handling

All exceptions are centralized in `backend/src/main/java/com/storehub/exception/GlobalExceptionHandler.java`, returning a uniform `ApiError { timestamp, status, error, message, path, fieldErrors }`:

Exception	HTTP status	fieldErrors
<code>MethodArgumentNotValidException</code> (bean validation)	400	populated per invalid field
<code>PurchaseNotFoundException</code>	404	null
<code>PurchaseOrderNotFoundException</code>	404	null
<code>PaymentNotFoundException</code>	404	null
<code>NoteNotFoundException</code>	404	null
<code>BadRequestException</code> (all business-rule violations in this module)	400	null
<code>AccessDeniedException</code> (a <code>@PreAuthorize</code> failure)	403	null
any other <code>Exception</code>	500, generic "An unexpected error occurred" message (full exception logged server-side)	null

Frontend: `parseApiError(err, fallbackMessage)` (`frontend/src/utils/apiError.ts`, used throughout every Purchase page) reads `err.response.data.message` and `err.response.data.fieldErrors`, falling back to the caller-supplied message on a network/parse failure. Field-level errors are then wired to `invalid / fieldErrors.<field>` props on the relevant `Input / Select` in `PurchaseOrderForm`, `PurchaseBillForm`, `KacchiPurchaseForm`; all other pages surface only the top-level `message` via a `sonner` toast or an `Alert` banner.

11. Audit Flow

`AuditService.log(action, module, entityType, entityId, documentNumber, oldValue, newValue, description, storeId)` — `oldValue/newValue` are passed `null` everywhere in this module (no field-level diff is recorded, only a free-text `description`). Exact call sites:

- `PurchaseService.createPurchase` → `AuditAction.CREATE`, module `"PURCHASE"`, entityType `"Purchase"` — description names whether it was "saved as draft" or "created and posted".
- `PurchaseService.cancelPurchase` → `AuditAction.CANCEL`, `"PURCHASE" / "Purchase"` — "stock, ledger, accounting and GST effects reversed".
- `PaymentService.create` → `AuditAction.PAYMENT`, `"PURCHASE" / "Payment"`.

- `PaymentService.delete` → `AuditAction.CANCEL`, "PURCHASE" / "Payment" — "allocations and ledger/accounting effects reversed".
- `DebitNoteService.create` → `AuditAction.CREATE`, "PURCHASE" / "DebitNote".
- `DebitNoteService.post` → `AuditAction.POST`, "PURCHASE" / "DebitNote".
- `DebitNoteService.cancel` → `AuditAction.CANCEL`, "PURCHASE" / "DebitNote".

Not audit-logged (confirmed by absence of any `AuditService` reference in the file):

`PurchaseService.updatePurchase`, `PurchaseService.deletePurchase` (hard delete), `PurchaseService.postPurchaseChallan`, and the entire `PurchaseOrderService` (create/update/setStatus/cancel). `PaymentService.createSystemPaymentForPurchase` / `deleteSystemPayment` also do not write their own audit entries (the parent Purchase's CREATE/CANCEL entry is treated as covering them).

12. Important Side Effects

Triggered from `PurchaseService.applyPostingEffects` (a normal Purchase Bill create, or posting a Kacchi draft), and mirrored (reversed) in `reversePurchaseEffects` / the reversal half of `updatePurchase`:

- **Stock movement:** `inventoryService.applyMovement(productId, storeId, ±quantity, StockMovementType.PURCHASE | PURCHASE_CANCEL, ReferenceType.PURCHASE, purchaseId, reason)` — one call per line item.
- **Accounting journal posting:** `accountingService.postPurchaseJournal(purchase)` — debits `SystemAccountCode.PURCHASE` for the taxable amount, debits `INPUT_CGST` / `INPUT_SGST` / `INPUT_IGST` (if GST), credits `SUPPLIER_PAYABLE` for the full total (always books the full payable regardless of any immediate payment — the Payment's own journal clears it separately); reversed via `accountingService.reversePurchaseJournal` → `reverseJournal(VoucherType.PURCHASE, purchaseId, reason)`.
- **GST transaction sync:** `gstTransactionSyncService.syncPurchase(purchase) / .reversePurchase(purchase)` — feeds the GST reporting module; a Kacchi challan still calls this but the purchase's `gstReportingApplicable=false` flag is what actually excludes it from GSTR-1/GSTR-3B downstream.
- **Supplier ledger entry:** `ledgerService.recordPurchaseCredit(purchase)` (CREDIT, increases payable) / `ledgerService.reversePurchaseCredit(purchase, reason)`; plus `ledgerService.recordInputGstEntry(purchase) / .reverseInputGstEntry(purchase)` writing to `purchase_gst_entries`.
- **Voucher numbering:** `voucherNumberService.next(VoucherDocType.PURCHASE_CHALLAN | PURCHASE | PURCHASE_ORDER | PAYMENT | DEBIT_NOTE, date)` — assigned once, immediately after the initial save, in a separate `REQUIRES_NEW` transaction so the sequence never rolls back with the enclosing operation.
- **Purchase-Order received-quantity consumption:** `consumeOrderQuantities(items, ±1)` → `PurchaseOrderService.applyReceivedQuantityDelta` — only for lines carrying a `purchaseOrderItemId`; also drives `PurchaseOrderStatus` transitions to `PARTIALLY_RECEIVED` / `COMPLETED`.
- **Immediate payment:** if `paidAmount > 0` at posting time, `paymentService.createSystemPaymentForPurchase(purchase)` creates a `systemGenerated=true` Payment with its own ledger-debit, cash-out entry, and payment journal — this payment can only be reversed by reversing/deleting the purchase itself (`PaymentService.delete` explicitly refuses to delete a system-generated payment directly).
- **Debit Note posting** additionally: conditional stock reversal (`StockMovementType.PURCHASE_RETURN`, only if `stockImpact == STOCK_RETURN`), its own journal (`DEBIT SUPPLIER_PAYABLE` / `CREDIT PURCHASE` + tax lines), `ledgerService.recordDebitNoteEntry`, and `gstTransactionSyncService.syncDebitNote`.

13. Dependencies on Other Modules

- **Product / Inventory** — `ProductService.findProductOrThrow` (line validation, rejects inactive products), `InventoryService.applyMovement` (stock in/out on post/cancel/return, documented in the Inventory module).
- **Supplier (party)** — `SupplierService.findSupplierOrThrow`; every purchase, PO, payment, and debit note is scoped to exactly one `Supplier`; inactive suppliers are rejected for new documents.
- **Store** — `StoreService.findOrThrow`, `StoreAccessService.resolveEffectiveStoreId` (create-time, validated against the caller's own access) / `.resolveViewableStoreId` (list-time) / `.assertStoreAccess` (detail-view-time) — every Purchase/PO/Payment row is fixed to one store at creation (Multi-Store spec, referenced throughout the service Javadoc).
- **Accounting** — `AccountingService.postPurchaseJournal` / `reversePurchaseJournal`, `.postPaymentJournal` / `reversePaymentJournal`, `.postJournal` / `.reverseJournal` (used directly by `DebitNoteService` for the DEBIT_NOTE voucher type); `VoucherNumberService.next(...)` for every document number in this module (documented fully in the Accounting Core module).
- **GST** — `GstCalculationService.calculateLine` (per-line CGST/SGST/IGST split by `TaxMode`), `GstTransactionSyncService.syncPurchase` / `reversePurchase` / `syncDebitNote` / `reverseDebitNote`, `GstinValidator.isValid`.
- **Financial Year** — `FinancialYearService.resolveForDate(noteDate)` used by `DebitNoteService.create` to stamp `financialYearId`; `VoucherNumberService` (Accounting Core module) performs period/sequence validation as part of voucher-number issuance.
- **Expense** (Accounting/Expenses module) — `PaymentService` treats a credit Expense as an alternate allocation target alongside a Purchase Bill, reusing the same `PaymentAllocation` entity and FIFO logic (`ExpenseRepository.findOutstandingBySupplier`).

14. Key Operation Flows

Create Purchase Order

`PurchaseOrderForm` (submit) → `purchaseOrderApi.create(payload)` → `POST /api/purchase-orders` → `PurchaseOrderController.create` (`PERM_PURCHASE_CREATE`) → `PurchaseOrderRequest` (validated) → `PurchaseOrderService.create` → resolves supplier/store (rejects inactive), builds `PurchaseOrder(status=DRAFT)`, `applyItems` computes per-line taxable/GST/total and rejects duplicate products, `purchaseOrderRepository.save` → `voucherNumberService.next(PURCHASE_ORDER, orderDate)` → save again → `purchase_orders/purchase_order_items` rows → `PurchaseOrderResponse.fromEntity` → 201 → UI navigates to `PurchaseOrderDetail`.

Create Purchase Bill (direct)

`PurchaseBillForm` (submit, no `orderId`) → `purchaseApi.create(payload)` (`gstType`, `taxMode`, `items`, `paidAmount`) → `POST /api/purchases` → `PurchaseController.createPurchase` (`PERM_PURCHASE_CREATE`) → `PurchaseCreateRequest` (validated) → `PurchaseService.createPurchase` → GST/tax-mode + GSTIN checks → resolve supplier/store (active) → `purchaseOrder = null` → `type = PURCHASE`, `draft = false` → build `Purchase(status=COMPLETED, gstReportingApplicable=true)` → `applyItems` (per-line `GstCalculationService.calculateLine`, duplicate-product/discount guards) → `applyPayment` (`paid/payable/paymentStatus`) → save → `voucherNumberService.next(PURCHASE, purchaseDate)` → save → `applyPostingEffects`: `addStock` (`InventoryService.applyMovement` × `items`), `ledgerService.recordPurchaseCredit` + `recordInputGstEntry`, `accountingService.postPurchaseJournal`, `consumeOrderQuantities` (no-op, no PO link), `gstTransactionSyncService.syncPurchase`, and if `paidAmount>0` `paymentService.createSystemPaymentForPurchase` (its own ledger/journal) → `auditService.log(CREATE)` → `PurchaseResponse.fromEntity(saved, hasPayments)` → 201 → UI navigates to `PurchaseBillDetail`.

Create Purchase Bill from Purchase Order

PurchaseOrderDetail / PurchaseOrders "Create Bill" → navigate /purchases/bills/new?orderId=<id> → PurchaseBillForm loads the order (purchaseOrderApi.getById), pre-fills supplier and only lines with remainingQuantity>0 (capped, tagged purchaseOrderItemId) → submit builds PurchaseCreatePayload with purchaseOrderId and each item's purchaseOrderItemId → POST /api/purchases → PurchaseService.createPurchase resolves purchaseOrder = purchaseOrderService.findOrThrow(id) and stores it on the Purchase; applyItems additionally validates each PO-linked line's quantity ≤ orderItem.getRemainingQuantity() → same posting cascade as above, but consumeOrderQuantities(items, +1) now calls purchaseOrderService.applyReceivedQuantityDelta per line → PurchaseOrder's receivedQuantity increments and recomputeStatus may flip it to PARTIALLY_RECEIVED / COMPLETED → response includes purchaseOrderId / purchaseOrderNumber .

Create Kacchi Purchase

KacchiPurchaseForm → "Save as Draft" or "Post Now" → buildPayload(saveAsDraft) forces gstType: 'GST' , transactionType: 'PURCHASE_CHALLAN' → purchaseApi.create → POST /api/purchases → PurchaseService.createPurchase → draft = request.isSaveAsDraft() && type==PURCHASE_CHALLAN → if draft: Purchase(status=DRAFT, gstReportingApplicable=false) saved with a PURCHASE_CHALLAN voucher number, **applyPostingEffects is skipped entirely** — no stock/ledger/GST-log/journal/payment yet → 201, UI shows it in KacchiPurchases as DRAFT. Later, "Post" (KacchiPurchaseDetail / KacchiPurchases) → purchaseApi.post(id) → POST /api/purchases/{id}/post → PurchaseController.postPurchaseChallan (PERM_PURCHASE_POST) → PurchaseService.postPurchaseChallan guards transactionType==PURCHASE_CHALLAN && status==DRAFT , flips to COMPLETED , runs the full applyPostingEffects cascade (stock, ledger, GST log, journal, GST sync, system payment) — identical effects to a directly-posted bill, just deferred. If the form instead chose "Post Now"/"Save & Post" (or editing a draft with "Save & Post"), saveAsDraft=false skips the draft state and posts immediately in the same create call.

Cancel a Purchase (and reversal)

PurchaseBills / PurchaseBillDetail / KacchiPurchases / KacchiPurchaseDetail "Delete" (the UI uses hard delete, purchaseApi.remove, not the soft cancel endpoint — no page currently calls purchaseApi.cancel) → confirm dialog → DELETE /api/purchases/{id} → PurchaseController.deletePurchase (PERM_PURCHASE_CANCEL) → PurchaseService.deletePurchase → findPurchaseOrThrow → guardNoManualPayments (rejects if any **manual** payment allocation exists — "Delete those payments first") → reversePurchaseEffects :

- If status==COMPLETED: restoreStock reverses every line (StockMovementType.PURCHASE_CANCEL , negative delta) — **Inventory reversal**.
- If status==DRAFT (never posted): returns immediately — nothing to reverse.
- Otherwise (posted, not draft): for every PaymentAllocation on this purchase, paymentService.deleteSystemPayment — reverses that system payment's ledger/cash/journal and deletes it (**system payments are the only ones allowed to exist here, per the manual-payment guard**); ledgerService.reversePurchaseCredit + reverseInputGstEntry (**Supplier ledger reversal**); accountingService.reversePurchaseJournal — **Accounting reversal**; consumeOrderQuantities(items, -1) decrements any linked PO's received quantity (PurchaseOrderStatus may revert from COMPLETED / PARTIALLY_RECEIVED); gstTransactionSyncService.reversePurchase — **GST resync**.
- purchaseRepository.delete(purchase) removes the row (cascades to purchase_items).

The soft PATCH /api/purchases/{id}/cancel (PurchaseService.cancelPurchase) performs the identical reversePurchaseEffects cascade but **keeps** the row, setting status=CANCELLED and payableAmount=0, and writes an AuditAction.CANCEL log (the hard-delete path writes **no** audit entry — see §11).

Record a Payment against one or more bills

PaymentForm → select supplier → paymentApi.getOutstanding(supplierId) → GET /api/payments/outstanding/{supplierId} → PaymentController.getOutstanding (PERM_PAYMENT_VIEW) → PaymentService.getOutstandingForSupplier → purchaseRepository.findOutstandingBySupplier +

expenseRepository.findOutstandingBySupplier → SupplierOutstandingResponse renders as checkable rows
→ user optionally checks bills/expenses and edits amounts → submit → paymentApi.create(payload) →
POST /api/payments → PaymentController.create (PERM_PAYMENT_CREATE) → PaymentRequest (validated) →
PaymentService.create → resolve supplier/store → build Payment(systemGenerated=false) → save →
voucherNumberService.next(PAYMENT, paymentDate) → allocate(payment, request.getAllocations(),
supplierId):
- if explicit allocations given: validate each (supplier match, amount ≤ payable, sum ≤ payment
amount), applyAllocation / applyExpenseAllocation per row (creates a PaymentAllocation, updates the
target's paidAmount / payableAmount / paymentStatus);
- else: FIFO across combined outstanding purchases+expenses sorted by date, applying min(remaining,
payable) until exhausted; leftover becomes an unallocated on-account amount.
→ save → ledgerService.recordPaymentDebit + recordCashEntryOut →
accountingService.postPaymentJournal → auditService.log(PAYMENT) → PaymentResponse.fromEntity → 201
→ UI navigates to PaymentDetail, which lists each PaymentAllocation under "Applied Against".

Create a Debit Note against a Bill

PurchaseBillDetail / KacchiPurchaseDetail "Return" → navigate /purchases/debit-notes/new?
purchaseId=<id> → DebitNoteForm loads the purchase (purchaseApi.getById), lists its items with an
editable return-quantity per line → submit ("Save as Draft" or "Save and Post") →
debitNoteApi.create({ sourcePurchaseId, noteType, noteDate, stockImpact, reason, remarks, items,
post }) → POST /api/debit-notes → DebitNoteController.create (PERM_DEBIT_NOTE_CREATE) →
DebitNoteCreateRequest (validated) → DebitNoteService.create → purchaseRepository.findById or
PurchaseNotFoundException → rejects if source is CANCELLED or still DRAFT → builds
DebitNote(status=DRAFT) copying gstType / gstReportingApplicable from the source purchase, resolves
financialYearId → for each item: validates the purchaseItemId belongs to this purchase and the return
quantity ≤ purchaseItem.quantity - alreadyReturned (debitNoteItemRepository.sumReturnedQuantity),
derives taxable/discount/CGST/SGST/IGST proportionally, appends a DebitNoteItem → totals summed →
save → voucherNumberService.next(DEBIT_NOTE, noteDate) → save → auditService.log(CREATE) → if
request.isPost(), immediately calls post(saved.getId()):
- post guards status==DRAFT and source purchase not since cancelled → posts a journal (DEBIT
SUPPLIER_PAYABLE / CREDIT PURCHASE + tax credit lines) via
accountingService.postJournal(VoucherType.DEBIT_NOTE, ...) → if stockImpact==STOCK_RETURN, reverses
stock per item (StockMovementType.PURCHASE_RETURN, negative delta) →
ledgerService.recordDebitNoteEntry → status=POSTED, stamps postedBy / postedAt →
gstTransactionSyncService.syncDebitNote (after the status flip) → auditService.log(POST).
→ DebitNoteResponse.fromEntity → 201 → UI navigates to DebitNoteDetail, which offers a standalone
Post button (for a saved draft) and a Cancel button (debitNoteApi.cancel → DebitNoteService.cancel,
reversing journal/stock/ledger/GST-sync if it had been posted).

15. Manual Changes

- **Add a new field to Purchase:** add the column to backend/src/main/java/com/storehub/entity/
Purchase.java, then thread it through PurchaseCreateRequest / PurchaseUpdateRequest (with
validation annotations), PurchaseService.createPurchase / .updatePurchase (builder/setter), and
PurchaseResponse.fromEntity. If it participates in GST or totals, also update
PurchaseService.applyItems. Mirror the corresponding frontend edits in frontend/src/types/
purchase.ts, PurchaseBillForm.tsx / KacchiPurchaseForm.tsx (and PurchaseOrderForm.tsx if the field
also belongs on the order), and purchaseApi.ts if new query params are needed.
- **Add a new field to PurchaseOrder:** same pattern in backend/src/main/java/com/storehub/entity/
PurchaseOrder.java, PurchaseOrderRequest, PurchaseOrderService.create / .update,
PurchaseOrderResponse.fromEntity, frontend/src/types/purchaseOrder.ts, PurchaseOrderForm.tsx.

- **Change the cancel/reversal logic:** the single source of truth is `PurchaseService.reversePurchaseEffects` (private method) — both `cancelPurchase` (soft) and `deletePurchase` (hard) call it, so a fix there fixes both paths. `guardNoManualPayments` is the gate that decides whether a reversal is even allowed; changing what counts as "blocking" payment lives there. The Debit Note equivalent is the reversal half of `DebitNoteService.cancel`; the Payment equivalent is `PaymentService.reverseAndRemove`.
- **Add a new `PurchaseStatus`:** edit `backend/src/main/java/com/storehub/entity/PurchaseStatus.java`, then update every `switch/if` that branches on status — `PurchaseService` (`applyPostingEffects` gating, `updatePurchase`'s old/new-status branches, `cancelPurchase/reversePurchaseEffects`'s `COMPLETED/DRAFT` checks), `PurchaseRepository.search`'s implicit status filter, and the frontend `PurchaseStatus` union type (`frontend/src/types/purchase.ts`) plus every page's `statusVariant()/status-Select` options (`PurchaseBills.tsx`, `KacchiPurchases.tsx`, `KacchiPurchaseDetail.tsx`).
- **Change payment allocation logic:** entirely inside `PaymentService.allocate` (private) — the explicit-allocation branch and the FIFO fallback branch are both there, along with `applyAllocation/applyExpenseAllocation` for the actual balance mutation. The reversal counterpart (undoing an allocation) is `PaymentService.reverseAndRemove`. Any change to what counts as "outstanding" starts in `PurchaseRepository.findOutstandingBySupplier/ExpenseRepository.findOutstandingBySupplier`.
- **Other modules affected by a Purchase-lifecycle change:**
- **Inventory** — any change to when/how much stock moves must be mirrored in both `PurchaseService.addStock/restoreStock` (bill) and `DebitNoteService.post/cancel`'s `inventoryService.applyMovement` calls (return), keeping `StockMovementType` (`PURCHASE`, `PURCHASE_CANCEL`, `PURCHASE_RETURN`, `ADJUSTMENT`) consistent with `ReferenceType` so inventory history stays traceable back to the right document.
- **Accounting** — journal shape changes belong in `AccountingService.postPurchaseJournal/postPaymentJournal` (Purchase/Payment) or `DebitNoteService.post`'s inline `JournalLine` list (Debit Note); a reversal always goes through `accountingService.reverseJournal/reversePurchaseJournal/reversePaymentJournal`, which look up the original journal by (`voucherType`, `voucherId`) — never hand-roll a reversing entry elsewhere.
- **GST** — a resync is required (`gstTransactionSyncService.syncPurchase/reversePurchase/syncDebitNote/reverseDebitNote`) any time taxable amounts, GST split, or `gstReportingApplicable` eligibility changes, and must stay ordered after any status flip that `GstReportingEligibility` reads.
- **Supplier ledger** — `ledgerService.recordPurchaseCredit/recordInputGstEntry` (bill), `recordPaymentDebit/recordCashEntryOut` (payment), `recordDebitNoteEntry` (note) and their `reverse*` counterparts must stay paired 1:1 with the accounting-journal change, since `SupplierLedgerEntryRepository.getOutstandingForSupplier` is what the Payment module's FIFO allocation and the Payables report both read from.

Sales Module

1. Overview

The Sales module covers the full customer-facing revenue lifecycle in StoreHub: **Sales Order** (optional pre-sale commitment) → **Sales Bill** (`SaleController / SaleService` , covering GST Sale, Kacchi Sale/Sale Challan, and POS/Counter Sale — all the same `Sale` entity and `SaleService.createSale`) → **Receipt Entry** (payment collection against outstanding bills) → **Credit Note** (sales return / adjustment against a posted bill).

TransactionType (`com.storehub.entity.TransactionType`) distinguishes document kind at creation and is fixed thereafter:

- `SALE` — a normal GST or Non-GST Sales Bill. `gstReportingApplicable = true` .
- `SALE_CHALLAN` — a Kacchi Sale / Sale Challan. GST is still calculated **in full** (CGST/SGST/IGST populated exactly like a normal sale) but `gstReportingApplicable = false` , so it is excluded from GSTR-1/GSTR-3B reporting. It is the only `TransactionType` allowed to be saved as `SaleStatus.DRAFT` .
- `PURCHASE` / `PURCHASE_CHALLAN` belong to the Purchase module (same enum, out of scope here).

SaleStatus (`com.storehub.entity.SaleStatus`):

- `DRAFT` — Kacchi Sale only, saved but not posted: no stock/ledger/GST-log/accounting effects exist yet.
- `PENDING` — the `@PrePersist` default if `status` is left null (not reachable through the documented create flow, since `createSale` always sets `DRAFT` or `COMPLETED` explicitly).
- `COMPLETED` — posted: all side effects (stock deduction, customer ledger debit, GST log, accounting journal) have been applied.
- `CANCELLED` — soft-reversed: record kept for history, all posting side effects reversed.

POS / Counter Sale (`Pos.tsx`) is not a separate backend endpoint — it calls the exact same `POST /api/sales` (`SaleService.createSale`) as a normal Sales Bill, with `transactionType: 'SALE'` . It is protected from duplicate submission by a client-generated **idempotency key** (`clientRequestId`), detailed in section 4 (Services) and section 12.

Sales Orders (`SalesOrderController / SalesOrderService`) never touch stock or accounts — only a Sales Bill created against an order's remaining quantity does (`SalesOrderItem.billedQuantity / remainingQuantity` , tracked via `SalesOrderService.applyBilledQuantityDelta`).

2. Frontend

Files and Components

File	Role
<code>frontend/src/pages/sales/SalesHub.tsx</code>	Landing page for the module: summary stat cards (<code>salesSummaryApi.get()</code>) and navigation cards to POS, Sales Orders, Sales Bills, Kacchi Sales, Receipts, Credit Notes.
<code>frontend/src/pages/sales/Pos.tsx</code>	POS / Counter Sale screen: barcode-scanner-friendly product search, cart, customer lookup, payment, and idempotent <code>saleApi.create</code> submission with a printable receipt view.
<code>frontend/src/pages/sales/SalesOrders.tsx</code>	Sales Order list: search/filter (customer, status, date range), cancel action.
<code>frontend/src/pages/sales/SalesOrderForm.tsx</code>	Create/edit Sales Order: customer, dates, item rows with quantity/rate/discount/GST%, "already billed" quantity floor enforced client-side.

File	Role
frontend/src/pages/sales/SalesOrderDetail.tsx	Sales Order detail: status, totals, items with billed/remaining columns, actions to Create Bill / Edit / Cancel.
frontend/src/pages/sales/SalesBills.tsx	Sales Bill (GST/Non-GST) list: search/filter, row actions (View/Print/Edit/Receipt/Delete).
frontend/src/pages/sales/SalesBillForm.tsx	Create/edit Sales Bill from scratch or (?orderId=) pre-filled from a Sales Order's remaining items; barcode scan-to-add; GST/Non-GST + tax-mode toggle; place-of-supply tax-mode suggestion on customer select; locked (fieldset disabled) once the bill has receipts.
frontend/src/pages/sales/SalesBillDetail.tsx	Printable invoice view; actions for Edit, Receipt, Return (Credit Note), Delete (guarded by hasPermission('SALES_CANCEL')).
frontend/src/pages/sales/KacchiSales.tsx	Kacchi Sale / Sale Challan list, filtered server-side by transactionType: 'SALE_CHALLAN' ; shows GST Reporting column (always NO); Post action for DRAFT rows.
frontend/src/pages/sales/KacchiSaleForm.tsx	Create/edit challan: always GST (gstType: 'GST' fixed), "Save as Draft" vs "Post Now"/"Save & Post" buttons; a confirm dialog before posting.
frontend/src/pages/sales/KacchiSaleDetail.tsx	Printable challan view with explicit "GST Calculated: YES · GST Reporting: NO" banner; Post/Edit (DRAFT only)/Return/Delete actions.
frontend/src/pages/sales/Receipts.tsx	Receipt list: search/filter by customer/date, Delete action.
frontend/src/pages/sales/ReceiptForm.tsx	Record a receipt: customer select loads receiptApi.getOutstanding , an optional per-bill checkbox/amount allocation table (falls back to FIFO auto-allocation server-side if none checked).
frontend/src/pages/sales/ReceiptDetail.tsx	Printable receipt view with "Applied Against" allocation table (or "on-account credit" if none); Delete action.
frontend/src/pages/sales/CreditNotes.tsx	Credit Note list: search/filter by status.
frontend/src/pages/sales/CreditNoteForm.tsx	Create a Credit Note: search/select a COMPLETED source sale (or pre-loaded via ?saleId=), note type/date/stock-impact/reason, per-line return-quantity inputs, "Save as Draft" vs "Save and Post".
frontend/src/pages/sales/CreditNoteDetail.tsx	Credit Note detail/print view; Post (DRAFT) / Cancel actions gated to ADMIN / STORE_MANAGER client-side.
frontend/src/api/saleApi.ts	Axios wrapper for /api/sales/** (list, getByld, create, update, post, cancel, remove).
frontend/src/api/salesOrderApi.ts	Axios wrapper for /api/sales-orders/** (search, getByld, create, update, setStatus, cancel).
frontend/src/api/receiptApi.ts	Axios wrapper for /api/receipts/** (search, getByld, getOutstanding, create, remove).
frontend/src/api/salesSummaryApi.ts	Axios wrapper for GET /api/sales-summary .
frontend/src/api/creditNoteApi.ts	Axios wrapper for /api/credit-notes/** (list, getByld, create, post, cancel).
frontend/src/types/sale.ts	Sale , SaleItem , SaleCreatePayload / SaleUpdatePayload , SaleStatus , SaleTransactionType , GstType , TaxMode , PaymentMode , Customer .
frontend/src/types/salesOrder.ts	SalesOrder , SalesOrderItem , SalesOrderPayload , SalesOrderStatus .
frontend/src/types/receipt.ts	Receipt , ReceiptAllocation , ReceiptPayload , OutstandingBill , CustomerOutstanding , SalesSummary .

File	Role
frontend/src/types/note.ts	CreditNote, CreditNoteItem, CreditNoteCreatePayload, NoteStatus, StockImpactType, CreditNoteType (also carries the unrelated Debit Note types for the Purchase module).

Frontend Flow

Sales Order create/edit — `SalesOrderForm.tsx` builds a `SalesOrderPayload` (customer, dates, item rows with `productId/quantity/rate/discount/gstPercent`) and calls `salesOrderApi.create/update`. On edit, an item that already has `billedQuantity > 0` cannot be removed or reduced below that quantity (client-side `validate()`, mirrored server-side). New orders always start `DRAFT`; `SalesOrderDetail.tsx` offers Edit while `DRAFT / CONFIRMED / PARTIALLY_BILLED`, Cancel while `DRAFT / CONFIRMED`, and "Create Sales Bill" whenever not `COMPLETED / CANCELLED`.

Sales Bill — from scratch — `SalesBillForm.tsx` with no `orderId/id`: pick GST/Non-GST + tax mode, customer (best-effort tax-mode suggestion via `suggestTaxMode(sellerState, customer.state)`), item rows (with a barcode-scan-to-add box), payment mode + paid amount. `handleSubmit` builds a `SaleCreatePayload` (`transactionType` omitted → defaults to `SALE` server-side) and calls `saleApi.create`.

Sales Bill — from a Sales Order — navigated to via `/sales/bills/new?orderId={id}`; `loadFromOrder()` fetches the order and pre-fills customer/address fields and one row per item with `remainingQuantity > 0`, carrying `salesOrderItemId` and a `maxQuantity` cap that the quantity input and `validate()` both enforce. The submitted payload includes `salesOrderId` and each row's `salesOrderItemId`.

Sales Bill — edit — loads the existing `Sale`; if `sale.hasReceipts` is true the whole form is locked (`<fieldset disabled>`) because `SaleService.updateSale` rejects edits once a receipt has been recorded.

Kacchi Sale create — `KacchiSaleForm.tsx` is a near-identical form to `SalesBillForm` but `gstType` is fixed to `GST` (a banner states "GST Calculated: YES · GST Reporting: NO") and every submit sets `transactionType: 'SALE_CHALLAN'`. Two submit paths: "Save as Draft" (`saveAsDraft: true`) and "Post Now"/"Save & Post" (goes through a confirm dialog, then either `saleApi.create` with `saveAsDraft: false` or, on edit, `saleApi.update` followed by `saleApi.post(id)`).

POS flow (including idempotency) — `Pos.tsx`:

1. On mount, `clientRequestIdRef` is seeded once via `newClientRequestId()` (`pos-${Date.now()}-${Math.random()}`).
2. Product search is debounced (200ms); pressing Enter in the search box first tries an exact `productApi.getByBarcode` lookup (scanner-friendly), falling back to a name/SKU search.
3. Cart lines compute taxable/GST/total client-side as a **preview only** — the comment in the code is explicit that `SaleService` computes the authoritative amounts.
4. `handlePost()` builds a `SaleCreatePayload` with `clientRequestId: clientRequestIdRef.current` (the same ref value on every retry within this cart) and `transactionType: 'SALE'`, then calls `saleApi.create`.
5. On success the screen switches to a printable receipt view; "New Sale" (`clearCart`) generates a **fresh** `clientRequestId` for the next transaction.
6. On failure, the UI explicitly tells the cashier: "Not charged. Safe to retry — the same sale won't be created twice." — because retrying resends the same `clientRequestId`, and the backend returns the original `Sale` instead of creating a duplicate (see section 4/12).

Receipt entry with multi-bill allocation — `ReceiptForm.tsx`: selecting a customer calls `receiptApi.getOutstanding(customerId)`, populating one checkbox row per outstanding bill (pre-filled with its full `dueAmount`). The cashier may check zero, one, or several bills and adjust each row's amount (capped at that bill's due amount); if none are checked, `allocations: []` is sent and the backend auto-

allocates FIFO. Client validation additionally blocks the sum of checked allocations from exceeding the entered receipt `amount`.

Credit Note create from a Bill — `CreditNoteForm.tsx`: either search+select a `COMPLETED` sale with a customer, or arrive pre-loaded via `/sales/credit-notes/new?saleId={id}` (wired from both `SalesBillDetail` and `KacchiSaleDetail`'s "Return" button). Once a sale is selected, one return-quantity input per `SaleItem` is shown (capped at `item.quantity`); `handleSubmit(post: boolean)` collects only lines with `qty > 0` into `CreditNoteItemPayload[]` and calls `creditNoteApi.create` with `post: false` (Save as Draft) or `post: true` (Save and Post, which the backend immediately runs through `post()` after creating).

3. API Calls

Function	HTTP Method	URL	Request shape	Response shape
<code>salesOrderApi.list</code>	GET	<code>/sales-orders</code>	query: <code>search?</code> , <code>customerId?</code> , <code>status?</code> , <code>fromDate?</code> , <code>toDate?</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<SalesOrder></code>
<code>salesOrderApi.getById</code>	GET	<code>/sales-orders/{id}</code>	—	<code>SalesOrder</code>
<code>salesOrderApi.create</code>	POST	<code>/sales-orders</code>	<code>SalesOrderPayload</code>	<code>SalesOrder</code>
<code>salesOrderApi.update</code>	PUT	<code>/sales-orders/{id}</code>	<code>SalesOrderPayload</code>	<code>SalesOrder</code>
<code>salesOrderApi.setStatus</code>	PATCH	<code>/sales-orders/{id}/status?</code> <code>status=</code>	—	<code>SalesOrder</code>
<code>salesOrderApi.cancel</code>	PATCH	<code>/sales-orders/{id}/cancel</code>	—	<code>SalesOrder</code>
<code>saleApi.list</code>	GET	<code>/sales</code>	query: <code>search?</code> , <code>paymentStatus?</code> , <code>status?</code> , <code>fromDate?</code> , <code>toDate?</code> , <code>transactionType?</code> , <code>page</code> , <code>size</code> (<code>storeId</code> supported server-side, not sent by these pages)	<code>PagedResponse<Sale></code>
<code>saleApi.getById</code>	GET	<code>/sales/{id}</code>	—	<code>Sale</code>
<code>saleApi.create</code>	POST	<code>/sales</code>	<code>SaleCreatePayload</code> (used for Sales Bill, Kacchi Sale, and POS — <code>transactionType</code> / <code>saveAsDraft</code> / <code>clientRequestId</code> vary by caller)	<code>Sale</code>
<code>saleApi.update</code>	PUT	<code>/sales/{id}</code>	<code>SaleUpdatePayload</code> (<code>SaleCreatePayload</code> + <code>status</code>)	<code>Sale</code>

Function	HTTP Method	URL	Request shape	Response shape
saleApi.post	POST	/sales/{id}/post	— (Kacchi Sale only)	Sale
saleApi.cancel	PATCH	/sales/{id}/cancel	—	Sale (defined in saleApi.ts ; no page in this module's read set calls it — SalesBills /detail pages call remove instead)
saleApi.remove	DELETE	/sales/{id}	—	204 No Content
receiptApi.list	GET	/receipts	query: search?, customerId?, fromDate?, toDate?, page, size	PagedResponse<Receipt>
receiptApi.getById	GET	/receipts/{id}	—	Receipt
receiptApi.getOutstanding	GET	/receipts/outstanding/{customerId}	—	CustomerOutstanding
receiptApi.create	POST	/receipts	ReceiptPayload	Receipt
receiptApi.remove	DELETE	/receipts/{id}	—	204 No Content
salesSummaryApi.get	GET	/sales-summary	—	SalesSummary
creditNoteApi.list	GET	/credit-notes	query: search?, status?, fromDate?, toDate?, page, size	PagedResponse<CreditNote>
creditNoteApi.getById	GET	/credit-notes/{id}	—	CreditNote
creditNoteApi.create	POST	/credit-notes	CreditNoteCreatePayload	CreditNote
creditNoteApi.post	POST	/credit-notes/{id}/post	—	CreditNote
creditNoteApi.cancel	PATCH	/credit-notes/{id}/cancel	—	CreditNote

4. Backend

Controllers

SaleController (/api/sales):

| Endpoint | Method | @PreAuthorize |

|---|---|---|

| GET /api/sales | getSales | none (any authenticated user; query filters: search, paymentStatus, status, fromDate, toDate, transactionType, storeId, page, size) |

| GET /api/sales/{id} | getSaleById | none |

| POST /api/sales | createSale | PERM_SALES_CREATE |

| PUT /api/sales/{id} | updateSale | PERM_SALES_EDIT |

```
| POST /api/sales/{id}/post | postSaleChallan | PERM_SALES_POST |
| PATCH /api/sales/{id}/cancel | cancelSale | PERM_SALES_CANCEL |
| DELETE /api/sales/{id} | deleteSale | PERM_SALES_CANCEL |
```

SalesOrderController (/api/sales-orders):

```
| Endpoint | Method | @PreAuthorize |
|---|---|---|
| GET /api/sales-orders | search | none |
| GET /api/sales-orders/{id} | getById | none |
| POST /api/sales-orders | create | PERM_SALES_CREATE |
| PUT /api/sales-orders/{id} | update | PERM_SALES_EDIT |
| PATCH /api/sales-orders/{id}/status | setStatus | PERM_SALES_EDIT |
| PATCH /api/sales-orders/{id}/cancel | cancel | PERM_SALES_CANCEL |
```

ReceiptController (/api/receipts):

```
| Endpoint | Method | @PreAuthorize |
|---|---|---|
| GET /api/receipts | search | none |
| GET /api/receipts/{id} | getById | none |
| GET /api/receipts/outstanding/{customerId} | getOutstanding | none |
| POST /api/receipts | create | PERM_RECEIPT_CREATE |
| DELETE /api/receipts/{id} | delete | PERM_RECEIPT_POST |
```

SalesSummaryController (/api/sales-summary):

```
| Endpoint | Method | @PreAuthorize |
|---|---|---|
| GET /api/sales-summary | getSummary | none |
```

CreditNoteController (/api/credit-notes):

```
| Endpoint | Method | @PreAuthorize |
|---|---|---|
| GET /api/credit-notes | search | none |
| GET /api/credit-notes/{id} | getById | none |
| POST /api/credit-notes | create | PERM_CREDIT_NOTE_CREATE |
| POST /api/credit-notes/{id}/post | post | PERM_CREDIT_NOTE_POST |
| PATCH /api/credit-notes/{id}/cancel | cancel | PERM_CREDIT_NOTE_CANCEL |
```

All GET endpoints in this module have no explicit `@PreAuthorize` — access is still gated by the global JWT filter (must be authenticated) and by row-level store scoping inside the service layer (`StoreAccessService.assertStoreAccess / resolveViewableStoreId`), not by a permission annotation.

DTOs

SaleCreateRequest — `clientId` (optional idempotency key, no annotation — validated by uniqueness at the DB/service layer, not `@NotNull`), `customerId` (optional), `storeId` (optional), `saleDate` `@NotNull`, `gstType` `@NotNull`, `taxMode` (required only when `gstType == GST`, enforced in service), `customerPhone`, `customerGstin`, `billingAddress`, `shippingAddress`, `paymentMode` `@NotNull`, `paidAmount` `@NotNull @DecimalMin("0")`, `notes`, `salesOrderId` (optional), `items` `@NotEmpty @Valid List<SaleItemRequest>`, `transactionType` (optional, defaults to `SALE`), `saveAsDraft` (boolean, Kacchi Sale only).

SaleItemRequest — `productId` `@NotNull`, `quantity` `@NotNull @Min(1)`, `sellingPrice` `@NotNull @DecimalMin("0")`, `discount` `@NotNull @DecimalMin("0")`, `tax` `@NotNull @DecimalMin("0")` (client-computed preview, not authoritative), `gstPercent` `@DecimalMin("0")`, `salesOrderItemId` (optional).

SaleUpdateRequest — same shape as `SaleCreateRequest` minus `clientRequestId` / `salesOrderId` / `transactionType` / `saveAsDraft`, plus `status @NotNull (SaleStatus)`.

SalesOrderRequest — `customerId`, `storeId` (both optional), `customerPhone`, `customerGstin`, `billingAddress`, `shippingAddress`, `orderDate @NotNull`, `expectedDeliveryDate`, `remarks`, `items @NotEmpty @Valid List<SalesOrderItemRequest>`.

SalesOrderItemRequest — `productId @NotNull`, `quantity @NotNull @Min(1)`, `rate @NotNull @DecimalMin("0")`, `discount @NotNull @DecimalMin("0")`, `gstPercent @NotNull @DecimalMin("0")`.

ReceiptRequest — `customerId @NotNull`, `storeId (optional)`, `receiptDate @NotNull`, `amount @NotNull @DecimalMin("0.01")`, `paymentMode @NotNull`, `remarks`, `allocations @Valid List<ReceiptAllocationRequest>` (optional; empty $\Rightarrow$ FIFO auto-allocation).

ReceiptAllocationRequest — `saleId @NotNull`, `amountApplied @NotNull @DecimalMin("0.01")`.

CreditNoteCreateRequest — `sourceSaleId @NotNull`, `noteType @NotNull (CreditNoteType)`, `noteDate @NotNull`, `stockImpact @NotNull (StockImpactType)`, `reason`, `remarks`, `items @NotEmpty @Valid List<CreditNoteItemRequest>`, `post (boolean; if true, post() runs immediately after create)`.

CreditNoteItemRequest — deliberately carries only `saleItemId @NotNull` and `quantity @NotNull @Min(1)`; rate/discount/GST are always derived server-side from the source `SaleItem` (per the code comment: "backend validation is the source of truth" — the client can never claim a different price/tax on a return than what was actually billed).

Response DTOs (`SaleResponse`, `SaleItemResponse`, `SalesOrderResponse`, `SalesOrderItemResponse`, `ReceiptResponse`, `ReceiptAllocationResponse`, `CreditNoteResponse`, `CreditNoteItemResponse`, `SalesSummaryResponse`, `CustomerOutstandingResponse`, `OutstandingBillResponse`) are plain `@Builder` read models built by static `fromEntity(...)` mappers; none carry validation annotations.

Services

SaleService

- `getSales(...)` — resolves the viewable store via `storeAccessService.resolveViewableStoreId`, delegates to `SaleRepository.search`, maps to `SaleResponse`.
- `getSaleById(id)` — loads the sale, calls `storeAccessService.assertStoreAccess` (a sale never returned to a caller without access to its store), sets `hasReceipts` from `ReceiptAllocationRepository.findBySaleId`.
- `createSale(request)` — **the central Sales Bill/Kacchi Sale/POS entry point:**
 1. **Idempotency check:** if `clientRequestId` is present and non-blank, looks it up via `SaleRepository.findByClientRequestId`; if found, returns the existing `Sale` unchanged instead of creating a new one.
 2. Validates `taxMode` is present when `gstType == GST`, and that `customerGstin` (if given) passes `GstinValidator.isValid`.
 3. Resolves `customer` (optional), `salesOrder` (optional), and `store` via `storeAccessService.resolveEffectiveStoreId + storeService.findOrThrow`; rejects an `INACTIVE` store.
 4. Determines `TransactionType` (default `SALE`) and rejects `saveAsDraft` for anything but `SALE_CHALLAN`.
 5. Builds the `Sale` (`status = DRAFT` if `saveAsDraft`, else `COMPLETED`; `gstReportingApplicable = type != SALE_CHALLAN`), calls `applyItems(...)` (validates no duplicate product, inactive-product guard, stock availability, sales-order-remaining-quantity guard, calls `GstCalculationService.calculateLine` per line) and `applyPayment(...)` (`paid  $\leq$  total`, derives `PaymentStatus`).
 6. Saves the sale; **catches `DataIntegrityViolationException`** from the unique `client_request_id` DB constraint — if a concurrent duplicate request already won the race, re-looks-up by `clientRequestId` and returns that result instead of a 500 (see section 12 for the full mechanism).
 7. Generates the invoice number via `voucherNumberService.next(SALE_CHALLAN or SALE, saleDate)`.

8. If not a draft, calls `applyPostingEffects(saved)`.
9. `auditService.log(CREATE, "SALES", "Sale", ...)`.
 - `postSaleChallan(id)` — Kacchi Sale only: rejects if not `SALE_CHALLAN`/not `DRAFT`; re-checks stock availability per line (stock may have moved since the draft was saved); flips to `COMPLETED`; calls `applyPostingEffects`.
 - `applyPostingEffects(saved)` (private) — the single posting pipeline used by both `createSale` (non-draft) and `postSaleChallan`: `deductStock` → `ledgerService.recordSaleDebit` → `ledgerService.recordGstEntry` → `accountingService.postSaleJournal` → `consumeOrderQuantities(+1)` → `gstTransactionSyncService.syncSale` → if `paidAmount > 0`: `receiptService.createSystemReceiptForSale` (customer sale) or `ledgerService.recordCashEntryForSale` (walk-in/no customer).
 - `updateSale(id, request)` — blocked if the sale is `CANCELLED`, if the new status is `CANCELLED` (must use `delete/cancel` instead), if the sale already has any receipt allocation, or if `gstType == GST` with no `taxMode`. If the old status was posted (`!= DRAFT`), fully reverses the old posting effects first (`restoreStock` if it was `COMPLETED`, `ledgerService.reverseSaleDebit / reverseGstEntry`, `accountingService.reverseSaleJournal`, `reverseCashEntryForSale` if walk-in, `consumeOrderQuantities(-1)`, `gstTransactionSyncService.reverseSale`), then rebuilds items/payment from the new request and re-applies posting effects if the new status isn't `DRAFT`.
 - `cancelSale(id)` — soft reversal: rejects if already `CANCELLED`; `guardNoManualReceipts` (rejects if any manual — non-system-generated — receipt is allocated against it); calls `reverseSaleEffects`; sets `status = CANCELLED`, `dueAmount = 0`; `auditService.log(CANCEL, ...)`.
 - `deleteSale(id)` — hard delete: same `guardNoManualReceipts` guard, `reverseSaleEffects`, then `saleRepository.delete`.
 - `reverseSaleEffects(sale, reason)` (private) — if `COMPLETED`, `restoreStock`; if `DRAFT`, returns immediately (nothing was ever posted); otherwise deletes every system-generated receipt allocated against it (`receiptService.deleteSystemReceipt`), reverses the walk-in cash entry if there were no allocations and no customer, then `reverseSaleDebit / reverseGstEntry / reverseSaleJournal / consumeOrderQuantities(-1) / gstTransactionSyncService.reverseSale`.
 - `applyItems(...)` (private) — per-line validation (no duplicate product; inactive product blocked unless it was already on the sale being edited; stock availability via `inventoryService.getCurrentStock`; sales-order-remaining-quantity check) and GST computation via `gstCalculationService.calculateLine`; accumulates sale-level `taxableAmount / cgstAmount / sgstAmount / igstAmount / totalAmount`.
 - `restoreStock / deductStock` (private) — call `inventoryService.applyMovement(productId, storeId, ±quantity, SALE_CANCEL|SALE, ReferenceType.SALE, saleId, reason)` per line.
 - `findSaleOrThrow(id)` — throws `SaleNotFoundException`.
 - `countTodaysSalesCount()` — `saleRepository.count()` (used by a dashboard, not this module's pages).

SalesOrderService

- `search(...)` — store-scoped list via `SalesOrderRepository.search`.
- `getById(id)` — store-access-asserted single read.
- `create(request)` — resolves customer/store (rejects `INACTIVE` store), builds items via `applyItems` (duplicate-product guard, no stock check — orders don't touch stock), status always starts `DRAFT`, generates `orderNumber` via `voucherNumberService.next(SALES_ORDER, orderDate)`.
- `update(id, request)` — blocked if `CANCELLED` or `COMPLETED`; enforces that any item already partially/fully billed (`billedQuantity > 0`) stays present at $\geq$ its billed quantity; rebuilds items, then `recomputeStatus`.
- `setStatus(id, status)` — manual transition restricted to `DRAFT / CONFIRMED` only; blocked once any item has billed quantity (status then becomes automatic).
- `cancel(id)` — blocked if already `CANCELLED` or if any item has been billed.
- `applyBilledQuantityDelta(salesOrderItemId, delta)` — called by `SaleService.consumeOrderQuantities` on post (+quantity) and on reversal (−quantity); clamps at 0 and rejects exceeding the ordered quantity; calls `recomputeStatus` after.
- `recomputeStatus(order)` (private) — `COMPLETED` if every item's `remainingQuantity <= 0`, else `PARTIALLY_BILLED` if any item has `billedQuantity > 0`, else left unchanged (`DRAFT / CONFIRMED`).

- `findOrThrow(id)` — throws `SalesOrderNotFoundException`.
- `countPendingOrders()` / `countTotalOrders()` — used by `SalesSummaryService`.

ReceiptService

- `search(...)` / `getById(id)` (store-access-asserted) / `getOutstandingForCustomer(customerId)` — reads; the latter sums `SaleRepository.findOutstandingByCustomer`'s `dueAmount`s.
- `create(request)` — builds a manual (`systemGenerated = false`) `Receipt`, generates `receiptNumber` via `voucherNumberService.next(RECEIPT, receiptDate)`, calls `allocate(...)`, then `ledgerService.recordReceiptCredit` + `ledgerService.recordCashEntry` + `accountingService.postReceiptJournal`; `auditService.log(RECEIPT, ...)`.
- `createSystemReceiptForSale(sale)` — called only from `SaleService.applyPostingEffects` / `updateSale` when a Sales Bill records an immediate payment against a customer: builds a `systemGenerated = true` `Receipt` allocated 1:1 against that sale for `paidAmount`, same ledger/accounting posting as a manual receipt.
- `delete(id)` — rejects deleting a system-generated receipt directly (instructs the caller to delete the sales bill instead); otherwise `reverseAndRemove`; `auditService.log(CANCEL, ...)`.
- `deleteSystemReceipt(id)` (package-private) — used only by `SaleService` when reversing a sale that has its own auto-generated receipt; same `reverseAndRemove`.
- `reverseAndRemove(receipt)` (private) — for every allocation, restores the sale's `paidAmount` / `dueAmount` / `paymentStatus`; reverses `ledgerService.reverseReceiptCredit` / `reverseCashEntry` / `accountingService.reverseReceiptJournal`; hard-deletes the `Receipt` row (cascades to its `ReceiptAllocation`s).
- `allocate(receipt, explicit, customerId)` (private) — if explicit allocations were given, validates each sale belongs to the customer and each amount doesn't exceed that sale's `dueAmount`, and that the sum doesn't exceed the receipt amount; otherwise walks `SaleRepository.findOutstandingByCustomer` oldest-first (FIFO) applying `min(remaining, dueAmount)` to each until the receipt amount is exhausted — any leftover becomes an unlinked on-account credit (still reduces the customer's net outstanding via the ledger CREDIT entry).
- `findOrThrow(id)` — throws `ReceiptNotFoundException`.

SalesSummaryService.getSummary() — aggregates `salesOrderRepository.count()`, `salesOrderService.countPendingOrders()`, `saleRepository.count()`, `ledgerService.getTotalOutstanding()`, `saleRepository.getTotalSalesForDate(today)`.

CreditNoteService (doc comment: DRAFT has zero side effects; POSTED reuses the same centralized engines as every other document — `AccountingService.postJournal`, `InventoryService.applyMovement`, `LedgerService`, `GstTransactionSyncService` — never a parallel implementation; CANCELLED reverses everything POSTED applied and is idempotent).

- `search(...)` / `getById(id)` (store-access-asserted via the source sale's store) — reads.
- `create(request)` — rejects if the source sale is `CANCELLED`, `DRAFT` (not yet posted), or has no customer (walk-in). Per item: looks up the `SaleItem`, validates it belongs to the source sale, and validates the requested return quantity against `saleItem.quantity` - `CreditNoteItemRepository.sumReturnedQuantity(saleItemId)` (already-returned/pending across every non-cancelled note, so two concurrent drafts can't double-claim the same units). Line amounts (taxable/discount/CGST/SGST/IGST) are computed by proportion of the original sale line (`proportion = totalForLine * returnedQty / soldQty`, `HALF_UP` to 2dp) — never re-derived from a client-supplied rate. `gstReportingApplicable` is copied verbatim from the source sale. Generates `voucherNumber` via `voucherNumberService.next(CREDIT_NOTE, noteDate)`; `auditService.log(CREATE, ...)`; if `request.isPost()`, immediately calls `post(saved.getId())`.
- `post(id)` — rejects if not `DRAFT`, or if the source sale has since been `CANCELLED`. Builds a journal (debit SALES for taxable amount, debit OUTPUT_CGST/SGST/IGST for any positive tax lines, credit CUSTOMER_RECEIVABLE for the total) and posts it via

accountingService.postJournal(VoucherType.CREDIT_NOTE, ...). If stockImpact == STOCK_RETURN, calls inventoryService.applyMovement(..., SALES_RETURN, ReferenceType.CREDIT_NOTE, ...) per item (store-aware overload when the source sale has a store). Calls ledgerService.recordCreditNoteEntry. Sets status = POSTED, postedBy/postedAt, then — **after** the status flip, because GstReportingEligibility checks status == POSTED — calls gstTransactionSyncService.syncCreditNote. auditService.log(POST, ...).

- cancel(id) — idempotent no-op if already CANCELLED. If POSTED, reverses the journal (accountingService.reverseJournal), reverses the stock movement if STOCK_RETURN (opposite-sign applyMovement with StockMovementType.ADJUSTMENT), ledgerService.reverseCreditNoteEntry, gstTransactionSyncService.reverseCreditNote. Sets status = CANCELLED, cancelledBy/cancelledAt. auditService.log(CANCEL, ...).
- findOrThrow(id) — throws NoteNotFoundException.

POS idempotency guard mechanism (exact): Sale.clientRequestId is a @Column(unique = true, length = 100) field, optional. Pos.tsx generates one token per cart (newClientRequestId()) before the first submit and resends the same token on every retry of that same cart (double-click, network timeout, page refresh before the response arrives). SaleService.createSale (1) looks the token up first via SaleRepository.findByClientRequestId and returns the existing Sale verbatim if found — no new row, no re-posting; (2) if two requests race past that check simultaneously, the DB-level unique constraint on client_request_id throws DataIntegrityViolationException on the losing insert (the Sale.id is IDENTITY-generated, so the insert — and the constraint check — happens synchronously in this call, not deferred to commit); the service catches that exception and re-queries by clientRequestId, returning the winning request's Sale instead of surfacing a 500. Either path is safe to retry from the client with no risk of a duplicate sale.

Repository

SaleRepository: findByClientRequestId, findOutstandingByCustomer, findAllOutstanding, findCompletedIds, findAllIds, getTotalSalesForDate, sumByPaymentModeForDate, sumCreditSalesForDate, sumTotalAmountByDateRange, countByDateRange, sumAndCountByDateRangeAndStore, search, findEligibleForReconciliation.

SaleItemRepository: hsnSummary, taxRateSummary (both GST-report-only, scoped to COMPLETED + gstReportingApplicable = true).

SalesOrderRepository: search, countByStatusIn.

SalesOrderItemRepository: plain JpaRepository<SalesOrderItem, Long> (no custom methods).

ReceiptRepository: findAllIds, sumAmountForDate, search.

ReceiptAllocationRepository: findBySaleId.

CustomerLedgerEntryRepository: findByCustomerIdOrderByEntryDateAscCreatedAtAsc, getOutstandingForCustomer, getTotalOutstanding, findByReferenceTypeAndReferenceId, sumByCustomer.

GstEntryRepository: findBySaleId.

CreditNoteRepository: search, findPostedIds, findAllIds.

CreditNoteItemRepository: sumReturnedQuantity (sums quantity already returned against one SaleItem across every non-CANCELLED note — DRAFT notes count too, so this is the concurrency guard against double-returning).

5. Database

Tables

- `sales` — one row per Sales Bill / Kacchi Sale / POS sale (all the same entity).
- `sale_items` — line items of `sales`.
- `sales_orders` — one row per Sales Order.
- `sales_order_items` — line items of `sales_orders`.
- `receipts` — one row per receipt (manual or system-generated).
- `receipt_allocations` — join table linking a `receipts` row to one or more `sales` rows with the amount applied.
- `customer_ledger_entries` — immutable DEBIT/CREDIT log of customer receivable movements.
- `gst_entries` — output GST log, one set per GST-type sale (used for GST reporting, not directly rendered by this module's pages).
- `credit_notes` — one row per Credit Note.
- `credit_note_items` — line items of `credit_notes`.

Relationships

- `Sale` ↔ `SaleItem`: one-to-many, `mappedBy = "sale"`, `cascade = ALL`, `orphanRemoval = true` (`sale.addItem/clearItems`).
- `Sale` ↔ `Customer`: many-to-one, nullable (walk-in sale has no customer).
- `Sale` ↔ `Store`: many-to-one, fixed at creation, never changed on edit.
- `Sale` ↔ `SalesOrder`: many-to-one, nullable — set when the bill originates from an order.
- `SaleItem` ↔ `SalesOrderItem`: many-to-one, nullable — set per line when that line bills against a specific order line (drives `SalesOrderItem.billedQuantity` tracking).
- `SalesOrder` ↔ `SalesOrderItem`: one-to-many, `cascade = ALL`, `orphanRemoval = true`.
- `Receipt` ↔ `ReceiptAllocation`: one-to-many, `cascade = ALL`, `orphanRemoval = true` (`receipt.addAllocation`).
- `ReceiptAllocation` ↔ `Sale`: many-to-one (`ReceiptAllocation.sale`) — this is the join that ties a receipt's payment to one or more specific bills; `ReceiptAllocationRepository.findBySaleId` is how `SaleService` determines `hasReceipts/blocks` edits/blocks cancel.
- `Receipt` ↔ `Customer`: many-to-one, `nullable = false`.
- `Receipt` ↔ `Store`: many-to-one, fixed at creation.
- `CreditNote` ↔ `CreditNoteItem`: one-to-many, `cascade = ALL`, `orphanRemoval = true` (`creditNote.addItem`).
- `CreditNote` ↔ `Sale (sourceSale)`: many-to-one, `nullable = false` — every Credit Note is always linked to the sale it adjusts.
- `CreditNote` ↔ `Customer`: many-to-one, `nullable = false` (copied from the source sale's customer at creation).
- `CreditNoteItem` ↔ `SaleItem`: many-to-one, `nullable = false` — the specific line being returned/adjusted; `CreditNoteItemRepository.sumReturnedQuantity` sums against this FK.
- `CustomerLedgerEntry` ↔ `Customer`: many-to-one, `nullable = false`; `referenceType/referenceId` polymorphically point back at the originating `Sale/Receipt/CreditNote` (not a JPA FK).
- `GstEntry` ↔ `Sale`: `saleId` is a plain `Long` column (not a JPA relationship) — `GstEntryRepository.findBySaleId` looks it up manually.

6. Validation

Backend (authoritative — see `CLAUDE.md` 's field-level-errors rule):

- Bean Validation on every request DTO (`@NotNull`, `@NotEmpty`, `@Min`, `@DecimalMin`) surfaces as 400 with

`ApiError.fieldErrors` keyed by DTO field name (`GlobalExceptionHandler.handleValidationExceptions` , triggered by `MethodArgumentNotValidException`).

- Service-level `BadRequestException` (plain 400, message only, no `fieldErrors` map) enforces business rules that span fields/entities: GST sale requires `taxMode`; GSTIN format (`GstinValidator.isValid`); inactive store/product blocks; per-line stock availability (`quantity > currentStock`); duplicate product in one sale/order; discount exceeding the line amount; sales-order-remaining-quantity guard; paid amount exceeding total; edit blocked once a receipt exists; cancel/delete blocked while a manual receipt is allocated; `saveAsDraft` only allowed for `SALE_CHALLAN`; Kacchi Sale post only from `DRAFT`; Credit Note return quantity exceeding `quantity - alreadyReturned`; Credit Note against a `CANCELLED` / `DRAFT` /no-customer sale; Credit Note post only from `DRAFT`; source sale cancelled since the note was created.
- Frontend mirrors the same rules pre-submit (`validate()` functions in each form) purely as a fast-fail UX layer — every form also renders `fieldErrors[field]` inline (via `parseApiError`) next to the offending input where the backend returns them, per the project's field-error-surfacing convention.

7. Authentication / Authorization

Every endpoint requires an authenticated request (standard JWT bearer auth, enforced globally — not specific to this module; see `01-authentication-authorization.md`). Within that, write/action endpoints are further gated by `@PreAuthorize("hasAuthority('PERM_...')")` per section 4; list/detail (GET) endpoints have no `@PreAuthorize` and rely on row-level store scoping instead:

`StoreAccessService.resolveViewableStoreId` / `resolveEffectiveStoreId` restrict which store's records a non-"all-stores" user can see or create against, and `StoreAccessService.assertStoreAccess` throws when a single-record `getById` targets a store the caller cannot access.

8. Permissions

From `Permission.java` / `RolePermissions.java`:

- `SALES_VIEW`, `SALES_CREATE`, `SALES_EDIT`, `SALES_POST`, `SALES_CANCEL`
- `RECEIPT_VIEW`, `RECEIPT_CREATE`, `RECEIPT_POST`
- `CREDIT_NOTE_VIEW`, `CREDIT_NOTE_CREATE`, `CREDIT_NOTE_POST`, `CREDIT_NOTE_CANCEL`
- `POS_ACCESS`

Role grants (`RolePermissions.build()`):

- `ADMIN` — every permission.
- `STORE_MANAGER` — every permission except the security-admin tier (`USER_*`, `ROLE_*`, `PERMISSION_VIEW`), `FY_MANAGE`, `AUDIT_VIEW`, `GST_CONFIG`, and store-management (`STORE_CREATE/EDIT/ASSIGN/ACCESS_ALL`) — i.e. it has full Sales module access.
- `ACCOUNTANT` — `SALES_VIEW`, `RECEIPT_VIEW`, `CREDIT_NOTE_VIEW` only (read-only on this module).
- `SALES_USER` — `SALES_VIEW/CREATE/EDIT/POST`, `RECEIPT_VIEW/CREATE/POST`, `CREDIT_NOTE_VIEW`, `POS_ACCESS` (no `SALES_CANCEL` / `CREDIT_NOTE_CREATE` / `CREDIT_NOTE_POST` / `CREDIT_NOTE_CANCEL` — cannot cancel a sale, delete a receipt, or manage credit notes beyond viewing).
- `PURCHASE_USER` / `INVENTORY_USER` — no Sales module permissions.
- `STAFF` — `POS_ACCESS`, `SALES_VIEW`, `SALES_CREATE`, `RECEIPT_VIEW`, `RECEIPT_CREATE` only (the exact set POS needs — create a sale, create its receipt — per the code comment noting these were already reachable pre-authorization and are now explicit rather than implicit).

9. Transaction Handling

Every mutating `SaleService` / `SalesOrderService` / `ReceiptService` / `CreditNoteService` method is `@Transactional` (class-level default propagation `REQUIRED`), so each request's full side-effect chain —

item validation, stock movement, ledger entries, GST log, accounting journal, order-quantity consumption, GST-sync, and (for a paid sale) the system receipt — commits or rolls back atomically as one unit. Read methods on `CreditNoteService` are explicitly `@Transactional(readOnly = true)`.

`VoucherNumberService.next(...)` runs in its **own** `@Transactional(propagation = Propagation.REQUIRES_NEW)` transaction with a `SELECT ... FOR UPDATE` row lock on the sequence, so voucher-number allocation commits independently of (and is never rolled back by) the outer sale/order/receipt/note transaction — it is claimed even if the calling transaction later fails for an unrelated reason, matching the documented "never re-issue a number" guarantee.

On **post** (`applyPostingEffects` / `CreditNoteService.post`): stock deduction/return, customer ledger debit/credit, GST log entry, accounting journal posting, sales-order billed-quantity consumption, and GST-transaction sync all happen inside the same transaction as the status flip — a failure partway through (e.g. `accountingService.postSaleJournal` throwing because the financial year is closed) rolls back the stock movement and ledger entry that ran before it, leaving the `Sale/CreditNote` at its pre-post status.

On **cancel/delete** (`reverseSaleEffects` / `CreditNoteService.cancel`): stock restoration, deletion of any system-generated receipts (which itself reverses ledger/accounting for that receipt), ledger/GST/accounting reversal, order-quantity release, and GST-sync reversal are likewise one atomic unit with the status flip to `CANCELLED` (or the hard delete).

10. Error Handling

`GlobalExceptionHandler` (shared across all modules):

- `MethodArgumentNotValidException` → 400 with `ApiError.fieldErrors` (per-DTO-field messages from `@NotNull`/`@Min`/`@DecimalMin` etc.).
- `SaleNotFoundException`, `SalesOrderNotFoundException`, `ReceiptNotFoundException`, `NoteNotFoundException` → 404 with `ex.getMessage()` (e.g. "Sale not found with id: 42"), `fieldErrors: null`.
- `BadRequestException` → 400 with `ex.getMessage()` only (all the business-rule messages listed in section 6).
- `AccessDeniedException` (thrown by `StoreAccessService.assertStoreAccess` or Spring Security's `@PreAuthorize` rejection) → 403, generic message "You do not have permission to access this resource".
- Any other unhandled `Exception` → 500, generic "An unexpected error occurred" (logged server-side with full stack trace).

Frontend: every list/form/detail page wraps its API calls in try/catch and surfaces the error via `parseApiError(err, fallbackMessage)`, which reads `ApiError.message` and `ApiError.fieldErrors` from the response — the message is shown as a page-level `Alert/toast`, and `fieldErrors[field]` (when present) drives per-input `invalid/error-text` styling in the forms (`SalesOrderForm`, `SalesBillForm`, `KacchiSaleForm`).

11. Audit Flow

`AuditService.log(...)` call sites in this module (all inside `@Transactional` service methods, so an audit row commits together with the business change it describes):

- `SaleService.createSale` — `AuditAction.CREATE`, module "SALES", entity "Sale", on every create (draft or posted).
- `SaleService.cancelSale` — `AuditAction.CANCEL`, entity "Sale".
- `ReceiptService.create` — `AuditAction.RECEIPT`, entity "Receipt".
- `ReceiptService.delete` — `AuditAction.CANCEL`, entity "Receipt".
- `CreditNoteService.create` — `AuditAction.CREATE`, entity "CreditNote".
- `CreditNoteService.post` — `AuditAction.POST`, entity "CreditNote".
- `CreditNoteService.cancel` — `AuditAction.CANCEL`, entity "CreditNote".

Not audited in the read files: `SaleService.updateSale`, `SaleService.deleteSale`, `SaleService.postSaleChallan`, and all of `SalesOrderService` — NOT FOUND IN CURRENT CODEBASE (no `auditService.log` call in those methods).

12. Important Side Effects

- **Stock movement on sale post:** `SaleService.deductStock` calls `inventoryService.applyMovement(productId, storeId, -quantity, StockMovementType.SALE, ReferenceType.SALE, saleId, reason)` per line. An **insufficient-stock guard** runs twice: once per line inside `applyItems` at create/update time (`itemRequest.getQuantity() > availableStock` → `BadRequestException`), and again in `postSaleChallan` immediately before posting a draft challan (stock may have moved since the draft was saved).
- **Stock reversal on cancel/delete:** `restoreStock` calls the same `applyMovement` with a positive delta and `StockMovementType.SALE_CANCEL`.
- **Credit Note stock impact:** only when `stockImpact == StockImpactType.STOCK_RETURN` — `CreditNoteService.post` calls `applyMovement(..., +quantity, StockMovementType.SALES_RETURN, ReferenceType.CREDIT_NOTE, ...)`; `cancel` reverses with `-quantity, StockMovementType.ADJUSTMENT`. `FINANCIAL_ADJUSTMENT` notes never touch stock.
- **Accounting journal posting:** `AccountingService.postSaleJournal(sale)` on sale post, `reverseSaleJournal` on reversal; `postReceiptJournal / reverseReceiptJournal` on receipt create/delete; `CreditNoteService` builds its own journal lines (`SALES` debit, `OUTPUT_CGST/SGST/IGST` debits, `CUSTOMER_RECEIVABLE` credit) and calls the generic `accountingService.postJournal(VoucherType.CREDIT_NOTE, ...) / reverseJournal`.
- **GST transaction sync:** `GstTransactionSyncService.syncSale / reverseSale` on sale post/reversal; `syncCreditNote / reverseCreditNote` on note post/cancel (note: `syncCreditNote` is called **after** the status flip to `POSTED` specifically because GST-reporting eligibility is computed from `status`).
- **Customer ledger entry creation:** `LedgerService.recordSaleDebit / reverseSaleDebit` on sale post/reversal; `recordGstEntry / reverseGstEntry` (writes/reverses `gst_entries`, GST-type sales only); `recordReceiptCredit / reverseReceiptCredit` + `recordCashEntry / reverseCashEntry` on receipt create/delete; `recordCashEntryForSale / reverseCashEntryForSale` specifically for a walk-in (no-customer) sale paid in full; `recordCreditNoteEntry / reverseCreditNoteEntry` on note post/cancel.
- **Voucher numbering:** `VoucherNumberService.next(docType, date)` — format `PREFIX/FY-CODE/000001`; called for `SALE / SALE_CHALLAN` (sale), `SALES_ORDER` (order), `RECEIPT` (receipt, both manual and system-generated), `CREDIT_NOTE` (note). Runs in its own `REQUIRES_NEW` transaction with a locking `SELECT ... FOR UPDATE` so numbers are never reused even if the outer transaction rolls back.
- **POS idempotency key mechanism:** see section 4 for the exact mechanism (`Sale.clientRequestId`, unique DB constraint, `findByClientRequestId` pre-check plus `DataIntegrityViolationException` catch-and-requery on a race). This is the specific mechanism that makes POS (and any other caller passing a `clientRequestId`) safe to retry.
- **Immediate system receipt on a paid sale:** any sale posted with `paidAmount > 0` and a `customer` set automatically gets a `systemGenerated = true` `Receipt` (`ReceiptService.createSystemReceiptForSale`), fully allocated against that sale — this is why `SalesBillDetail / SalesBills` treat `hasReceipts` as a lock on further item/amount edits (the sale must be reversed, not edited, once a receipt of any kind exists).

13. Dependencies on Other Modules

- **Product/Inventory** — `ProductService.findProductOrThrow` /inactive-product checks; `InventoryService.getCurrentStock / applyMovement` for stock deduction on sale, restoration on cancel, and return on Credit Note.

- **Customer (party)** — `CustomerService.findCustomerOrThrow`; every `Sale / SalesOrder / Receipt / CreditNote` optionally or mandatorily links to a `Customer`.
- **Store** — `StoreService.findOrThrow`, `StoreAccessService ( resolveEffectiveStoreId / resolveViewableStoreId / assertStoreAccess )` — every sale/order/receipt is fixed to one store at creation and scoped for viewing per Multi-Store spec section 14/22/24.
- **Accounting** — `AccountingService.postSaleJournal / reverseSaleJournal`, `postReceiptJournal / reverseReceiptJournal`, `postJournal / reverseJournal (Credit Note)`; `VoucherNumberService.next` for all voucher numbering in this module.
- **GST** — `GstCalculationService.calculateLine` (per-line CGST/SGST/IGST split, called from `SaleService.applyItems`); `GstTransactionSyncService.syncSale / reverseSale / syncCreditNote / reverseCreditNote` (feeds GSTR-1/GSTR-3B reporting, respecting `gstReportingApplicable`).
- **Financial Year** — `FinancialYearService.resolveForDate` (stamps `CreditNote.financialYearId` at creation); `AccountingService.postJournal / postSaleJournal` internally enforce the financial-year-open rule (not re-documented here — see the Accounting module doc).

14. Key Operation Flows

Create Sales Order: `SalesOrderForm.tsx` submit → `salesOrderApi.create(payload)` → `POST /api/sales-orders` → `SalesOrderController.create ( PERM_SALES_CREATE )` → `SalesOrderRequest` (validated) → `SalesOrderService.create` → resolves customer/store, `applyItems` (duplicate-product guard, computes taxable/GST/total per line), status `DRAFT`, `voucherNumberService.next(SALES_ORDER, orderDate)` → `SalesOrderRepository.save (×2, for the generated order number)` → `sales_orders / sales_order_items` rows → `SalesOrderResponse.fromEntity` → UI navigates to the new order's detail page.

Create Sales Bill (direct): `SalesBillForm.tsx` (no `orderId`) submit → `saleApi.create(payload)` (`transactionType` omitted ⇒ `SALE`) → `POST /api/sales` → `SaleController.createSale ( PERM_SALES_CREATE )` → `SaleCreateRequest` (validated) → `SaleService.createSale` → no `clientRequestId` short-circuit (or first attempt) → GST/GSTIN checks → resolve customer/store → `applyItems` (stock check, GST calc) + `applyPayment` → save (invoice number via `voucherNumberService.next(SALE, saleDate)`) → `applyPostingEffects` (deduct stock, customer ledger debit, GST log, accounting journal, `gstTransactionSyncService.syncSale`, system receipt if paid) → `auditService.log(CREATE)` → `sales / sale_items` (+ `stock_movements / journal / gst_entries / customer_ledger_entries / possibly receipts` rows from the dependent services) → `SaleResponse` → UI navigates to the invoice detail/print view.

Create Sales Bill from Sales Order: `SalesOrderDetail / SalesOrders` "Create Bill" → `SalesBillForm.tsx?orderId={id}` → `loadFromOrder()` fetches the order, pre-fills rows (`salesOrderItemId`, capped at `remainingQuantity`) → submit builds `SaleCreatePayload` with `salesOrderId` + per-line `salesOrderItemId` → same `POST /api/sales` → `SaleService.createSale` → `applyItems` additionally validates each line's quantity against `SalesOrderItem.remainingQuantity` → on posting, `consumeOrderQuantities(items, +1)` calls `SalesOrderService.applyBilledQuantityDelta` per line, which increments `billedQuantity` and calls `recomputeStatus` (order becomes `PARTIALLY_BILLED` or `COMPLETED`) → `sales_order_items.billed_quantity` updated, `sales_orders.status` updated.

Create Kacchi Sale: `KacchiSaleForm.tsx` submit ("Save as Draft" or "Post Now") → `saleApi.create(payload)` with `transactionType: 'SALE_CHALLAN'`, `saveAsDraft: true|false` → `POST /api/sales` → `SaleService.createSale` → `gstReportingApplicable = false` set on the entity; if `saveAsDraft`, status `DRAFT` and `applyPostingEffects` is **skipped** (no stock/ledger/GST-log/accounting rows yet) — only the `sales / sale_items` rows exist. If posting immediately, same `applyPostingEffects` pipeline as a normal sale runs (GST is still fully calculated and logged in `gst_entries`, but excluded from reporting via the flag). A later "Post" on a saved draft → `saleApi.post(id)` → `POST /api/sales/{id}/post` → `SaleService.postSaleChallan` → re-checks stock, flips to `COMPLETED`, runs `applyPostingEffects`.

POS checkout: Pos.tsx scan/search → addToCart → handlePost() → saleApi.create({ clientRequestId, transactionType: 'SALE', ... }) → POST /api/sales → SaleController.createSale (PERM_SALES_CREATE , granted to STAFF/SALES_USER via POS_ACCESS + SALES_CREATE) → SaleService.createSale — clientRequestId lookup first (idempotency, section 4/12); on a fresh token, proceeds through the normal create → validate → applyItems / applyPayment → save (unique-constraint race handled) → voucherNumberService.next(SALE, ...) → applyPostingEffects (status is always COMPLETED , POS never drafts) → SaleResponse → Pos.tsx switches to the printable receipt view; retry on network failure resends the same clientRequestId and gets the original Sale back, never a duplicate.

Cancel a Sale: SalesBills / SalesBillDetail / KacchiSales / KacchiSaleDetail "Delete" → confirm dialog → saleApi.remove(id) → DELETE /api/sales/{id} → SaleController.deleteSale (PERM_SALES_CANCEL) → SaleService.deleteSale → guardNoManualReceipts (rejects if a manual receipt is allocated — must delete that receipt first) → reverseSaleEffects: if COMPLETED , restoreStock (inventoryService.applyMovement(+qty, SALE_CANCEL)); if DRAFT , nothing to reverse; otherwise deletes any system-generated receipt (receiptService.deleteSystemReceipt , which itself reverses that receipt's ledger/accounting entries), reverses the walk-in cash entry if applicable, ledgerService.reverseSaleDebit / reverseGstEntry , accountingService.reverseSaleJournal , consumeOrderQuantities(items, -1) (releases any Sales Order billed quantity back), gstTransactionSyncService.reverseSale → saleRepository.delete (hard delete; PATCH ../cancel exists as a soft reversal alternative — same reversal logic via cancelSale , but keeps the Sale row with status = CANCELLED) → UI toasts and navigates back to the list. Net reversal: stock restored, customer ledger DEBIT offset by a reversing entry, GST log offset, accounting journal reversed, and (if linked) the Sales Order's billed quantity released.

Record a Receipt against one or more bills: ReceiptForm.tsx → customer select loads receiptApi.getOutstanding(customerId) → cashier optionally checks specific bills/amounts → submit → receiptApi.create(payload) → POST /api/receipts → ReceiptController.create (PERM_RECEIPT_CREATE) → ReceiptRequest (validated) → ReceiptService.create → builds Receipt (systemGenerated = false), voucherNumberService.next(RECEIPT, ...) → allocate(...) — explicit allocations (validated against customer ownership + due amount) or FIFO across findOutstandingByCustomer if none given, any leftover becomes an unlinked on-account credit → each allocation updates the target Sale's paidAmount / dueAmount / paymentStatus and is saved → ledgerService.recordReceiptCredit + recordCashEntry → accountingService.postReceiptJournal → auditService.log(RECEIPT) → receipts / receipt_allocations rows + updated sales rows + customer_ledger_entries + journal → ReceiptResponse → UI navigates to the receipt detail/print view.

Create a Credit Note against a Bill: SalesBillDetail / KacchiSaleDetail "Return" → CreditNoteForm.tsx?saleId={id} (or manual search) → set note type/date/stock-impact/reason, enter return quantities per line → handleSubmit(post) → creditNoteApi.create({ ..., post }) → POST /api/credit-notes → CreditNoteController.create (PERM_CREDIT_NOTE_CREATE) → CreditNoteCreateRequest (validated) → CreditNoteService.create — rejects a CANCELLED / DRAFT /no-customer source sale; per item validates against SaleItem ownership and sumReturnedQuantity (already-returned-or-pending guard); computes proportional taxable/CGST/SGST/IGST from the original sale line; copies gstReportingApplicable verbatim from the source sale; voucherNumberService.next(CREDIT_NOTE, ...); auditService.log(CREATE) → if post: true , immediately calls post(id) — builds and posts the accounting journal, applies the stock movement if STOCK_RETURN , ledgerService.recordCreditNoteEntry , flips to POSTED , then gstTransactionSyncService.syncCreditNote , auditService.log(POST) → credit_notes / credit_note_items rows (+ stock movement/journal/ledger/GST-sync rows if posted) → CreditNoteResponse → UI navigates to the note's detail/print view, where a separate "Post" action is also available for a note saved as DRAFT .

15. Manual Changes

- **Add a new field to Sale / SalesOrder** : add the `@Column` to the entity (`backend/src/main/java/com/storehub/entity/Sale.java` or `SalesOrder.java`), add it to the relevant request DTO(s) (`SaleCreateRequest / SaleUpdateRequest / SalesOrderRequest`) with validation annotations, wire it into the `Sale.builder() / applyItems/setter` calls in `SaleService / SalesOrderService` , add it to the response DTO's `fromEntity(...)` mapper, then add the field to the corresponding frontend type (`types/sale.ts / types/salesOrder.ts`) and the relevant form component(s) (`SalesBillForm.tsx` , `KacchiSaleForm.tsx` , `Pos.tsx` , `SalesOrderForm.tsx`) and payload builders. Remember a DB migration (this codebase does not use Flyway/Liquibase in the read files — confirm the schema-generation strategy before assuming Hibernate DDL auto-update is relied on in production).
- **Change the cancel/reversal logic**: `SaleService.reverseSaleEffects` (private method, used by both `cancelSale` and `deleteSale`) is the single place stock/ledger/GST/accounting/order-quantity reversal is orchestrated for a Sale — change it there, not in `cancelSale / deleteSale` individually, to keep soft-cancel and hard-delete behavior identical. For a Credit Note, the equivalent logic lives inline in `CreditNoteService.cancel` .
- **Add a new SaleStatus** : edit `backend/src/main/java/com/storehub/entity/SaleStatus.java` , then update every `switch/if`-based status check across `SaleService` (`createSale` , `updateSale` , `cancelSale` , `deleteSale` , `reverseSaleEffects` , `postSaleChallan`) and the frontend `statusVariant()` helpers/status filters in `SalesBills.tsx` , `KacchiSales.tsx` , `KacchiSaleDetail.tsx` (`types/sale.ts` 's `SaleStatus` union too).
- **Change receipt allocation logic**: `ReceiptService.allocate` (private) is the single place both explicit-allocation validation and FIFO auto-allocation live — change the FIFO ordering by changing `SaleRepository.findOutstandingByCustomer` 's `ORDER BY` , or change the auto-allocation strategy (e.g. largest-bill-first) entirely inside `allocate` . The reversal counterpart is `ReceiptService.reverseAndRemove` , which must stay symmetric with whatever `allocate` does.
- **Change the POS idempotency mechanism**: the DB constraint is `Sale.clientRequestId` (`@Column(unique = true, length = 100)`); the check-then-catch logic is in `SaleService.createSale` (the `findByClientRequestId` pre-check plus the `DataIntegrityViolationException` catch block). The frontend token generator is `newClientRequestId()` in `Pos.tsx` — any other POS-like caller must generate and persist its own token across retries the same way (one token per logical submission attempt, regenerated only after a confirmed success or an intentional new transaction).
- **Modules affected by a Sale/Credit-Note change**: Inventory (stock reversal via `InventoryService.applyMovement`), Accounting (journal reversal via `AccountingService.reverseSaleJournal / reverseJournal` , and the financial-year-open check inside posting), GST (resync via `GstTransactionSyncService.reverseSale / reverseCreditNote` , and the `gstReportingApplicable` flag that gates GSTR-1/GSTR-3B), Customer ledger (`LedgerService.reverseSaleDebit / reverseGstEntry / reverseCreditNoteEntry / reverseCashEntryForSale`) — any change to the posting pipeline should be mirrored exactly (same lines, opposite sign/direction) in the corresponding reversal path, since `reverseSaleEffects / CreditNoteService.cancel` are expected to fully undo `applyPostingEffects / CreditNoteService.post` .

Accounting Core Engine Module

1. Overview

StoreHub implements double-entry bookkeeping around three pillars:

- **Chart of Accounts** — `Account` rows (the GL/ledger accounts) optionally grouped under `AccountGroup` nodes (`Assets` -> `Current Assets` -> `Cash`, etc.), each `Account` tagged with an `AccountType` (`ASSET`, `LIABILITY`, `INCOME`, `EXPENSE`). A fixed subset of accounts (`Cash`, `Bank`, `Customer Receivable`, `Sales`, `Output CGST/SGST/IGST`, `Supplier Payable`, `Purchase`, `Input CGST/SGST/IGST`, `Expenses`, `Discount Allowed/Received`, `Round Off`, `Other Income`, `Inventory`) is enumerated in `SystemAccountCode` and seeded idempotently on startup by `AccountService.ensureSystemAccountsExist()` (a `@PostConstruct`). Business logic must never hard-code an account id — it resolves a system account via `AccountService.getSystemAccount(SystemAccountCode)` .
- **Journal posting** — one posted voucher = one `JournalHeader` row plus 2+ `JournalDetail` (line) rows, each line carrying exactly one of `debitAmount` / `creditAmount` (never both, never neither) referencing one `Account` . A journal is never physically deleted once POSTED; reversing it creates a second `JournalHeader` (linked back via `reversalOfJournalId`) with every line's debit/credit swapped, and flips the original's `status` to `REVERSED` .
- **Single point of entry** — every other transactional module (`Sale`, `Purchase`, `Receipt`, `Payment`, `Expense`, `CashTransaction`, `CreditNote`, `DebitNote`) posts through `AccountingService` (`postSaleJournal`, `postPurchaseJournal`, `postReceiptJournal`, `postPaymentJournal`, or the generic `postJournal` / `postJournalByAccountId`) rather than ever constructing `JournalHeader` / `JournalDetail` rows itself. `AccountingService` is a parallel financial ledger alongside the pre-existing operational sub-ledgers (`CustomerLedgerEntry`, `SupplierLedgerEntry`, `CashLedgerEntry`, `GstEntry`, `PurchaseGstEntry`) maintained by `LedgerService` — it does not replace them; both are updated side by side by each calling service, consuming the same already-calculated amounts (GST, discounts, taxable value are never recomputed by the accounting engine).
- **FY-aware voucher numbering** — `VoucherNumberService.next(VoucherDocType, LocalDate)` is documented as "the ONLY place a voucher number is ever generated." Every document-creating service (`Sale`, `Purchase`, `SalesOrder`, `PurchaseOrder`, `Receipt`, `Payment`, `CreditNote`, `DebitNote`, `Expense`, `CashTransaction`, `StockTransfer`) calls it instead of formatting its own `"%s-%06d"` string, producing numbers of the form `PREFIX/FY-CODE/000001` .

2. Frontend

Files and Components

File	Component	Purpose
<code>frontend/src/pages/accounting/AccountingHub.tsx</code>	<code>AccountingHub</code>	Landing page for the Accounting module: stat cards (Cash/Bank/Receivable/Payable balances, today's sales/purchases) plus a grid of links into every accounting sub-page/report (Accounts, Journals, Trial Balance, Expenses, Cash Management, Day Book, Account Ledger, P&L, Balance Sheet, Health Check, etc.).
<code>frontend/src/pages/accounting/AccountMaster.tsx</code>	<code>AccountMaster</code>	Chart of Accounts list (search by code/name, filter by <code>AccountType</code> , paginated) with an Add/Edit modal restricted to <code>user.role === 'ADMIN'</code> . System accounts have <code>accountCode</code> / <code>accountType</code> disabled for editing.

File	Component	Purpose
frontend/src/pages/ accounting/Journals.tsx	Journals	Journal Register: paginated list filterable by <code>voucherType</code> , <code>status</code> (<code>DRAFT</code> / <code>POSTED</code> / <code>REVERSED</code>), and free-text search; a "New Journal Entry" button (ADMIN only) links to <code>JournalEntryForm</code> ; a View action opens a modal with the full line breakdown.
frontend/src/pages/ accounting/ JournalEntryForm.tsx	JournalEntryForm	Manual Journal Entry creation form: date, narration, dynamic list of debit/credit lines (min 2), live-computed totals and a "Balanced"/"Not Balanced" indicator; client-side pre-check mirrors the backend's balance rule before submit.
frontend/src/pages/ accounting/ TrialBalance.tsx	TrialBalance	As-of-date Trial Balance: per-account debit/credit table plus total debit, total credit, difference, and a Balanced/Out of Balance badge.
frontend/src/api/ accountingApi.ts	<code>accountApi</code> , <code>journalApi</code> , <code>accountingReportApi</code>	Axios wrappers for <code>/api/accounts</code> , <code>/api/accounting/journals</code> , and <code>/api/accounting/reports/*</code> .
frontend/src/types/ accounting.ts	—	TypeScript types mirroring the backend DTOs (<code>Account</code> , <code>AccountGroup</code> , <code>JournalHeader</code> , <code>JournalLine</code> , <code>TrialBalanceResponse</code> , and the Phase-4 report response shapes).

Frontend Flow

Create/Edit Account (`AccountMaster.tsx`): `openAdd()` / `openEdit(account)` populate the `values` form state (a snapshot is stored in `initialSnapshot` for dirty-checking) → `handleSubmit` calls `accountApi.create(values)` OR `accountApi.update(id, values)` → on success, toast + `load()` refetches the page; on error, `parseApiError` splits the response into a top-level `error` message and a `fieldErrors` map, rendered per-field under each `Input` via `invalid={!fieldErrors.x}` and a `<p>` feedback line (the pattern this codebase standardizes on for surfacing backend `fieldErrors`).

Manual Journal Entry create (`JournalEntryForm.tsx`): user picks a `journalDate`, optional `narration`, and adds/removes line rows (`Account` select + debit/credit inputs, mutually exclusive per row in the UI). `totalDebit` / `totalCredit` are derived every render; a client-side guard blocks submit if fewer than 2 non-empty lines or `totalDebit !== totalCredit`, with the same "Journal is not balanced: total debit (...) does not equal total credit (...)" message format the backend also throws. `handleSubmit` calls `journalApi.create({journalDate, narration, lines})` → on success, navigates to `/accounting/journals`.

Journal list/detail view (`Journals.tsx`): `load()` calls `journalApi.list(query)` on mount and whenever `page` / `voucherType` / `status` change; `openView(journal)` calls `journalApi.getById(journal.id)` to fetch the full line breakdown (the list response omits lines) and renders it in a `Dialog` with per-line account/ narration/debit/credit and header-level total debit/credit.

Trial Balance view (`TrialBalance.tsx`): `load(asOfDate)` calls `accountingReportApi.trialBalance(asOfDate)` on mount and whenever the `asOfDate` date input changes; renders `report.rows` (account code/name/type/debit/credit) plus footer totals and a `balanced` badge, all computed server-side from `JournalDetailRepository.sumDebitCreditByAccount`.

3. API Calls

Function	HTTP Method	URL	Request Shape	Response Shape
<code>accountApi.list(query)</code>	GET	<code>/api/accounts</code>	query params <code>search?</code> , <code>accountType?</code> , <code>active?</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<Account></code>
<code>accountApi.groups()</code>	GET	<code>/api/accounts/groups</code>	—	<code>AccountGroupResponse[]</code>
<code>accountApi.getById(id)</code>	GET	<code>/api/accounts/{id}</code>	—	<code>AccountResponse</code>
<code>accountApi.create(payload)</code>	POST	<code>/api/accounts</code>	<code>AccountRequest</code>	<code>AccountResponse (201)</code>
<code>accountApi.update(id, payload)</code>	PUT	<code>/api/accounts/{id}</code>	<code>AccountRequest</code>	<code>AccountResponse</code>
<code>journalApi.list(query)</code>	GET	<code>/api/accounting/journals</code>	query params <code>voucherType?</code> , <code>status?</code> , <code>fromDate?</code> , <code>toDate?</code> , <code>search?</code> , <code>page</code> , <code>size</code>	<code>PagedResponse<Journal></code>
<code>journalApi.getById(id)</code>	GET	<code>/api/accounting/journals/{id}</code>	—	<code>JournalHeaderResponse</code>
<code>journalApi.create(payload)</code>	POST	<code>/api/accounting/journals</code>	<code>JournalCreateRequest (journalDate, narration?, lines: JournalLineRequest[])</code>	<code>JournalHeaderResponse</code>
<code>accountingReportApi.trialBalance(asOfDate?)</code>	GET	<code>/api/accounting/reports/trial-balance</code>	query param <code>asOfDate?</code>	<code>TrialBalanceResponse</code>
<code>accountingReportApi.dashboard()</code>	GET	<code>/api/accounting/reports/dashboard</code>	—	<code>AccountingDashboardSummary</code>
<code>accountingReportApi.dayBook/accountLedger/cashBook/bankBook/partyLedger/receivable/payable/outstandingCustomers/outstandingSuppliers/profitLoss/balanceSheet/accountSummary/expenseSummary/incomeSummary/healthCheck</code>	GET	<code>/api/accounting/reports/*</code>	various date/id query params	corresponding report DTO

Note: `accountingReportApi` (Trial Balance's data source and the wider report suite listed on `AccountingHub`) is served by `backend/src/main/java/com/storehub/controller/AccountingReportController.java`, a controller adjacent to but not in this task's backend ground-truth list; it is included here only because `TrialBalance.tsx` is a ground-truth frontend file. Its full endpoint-by-endpoint documentation (permissions, DTOs) is **NOT FOUND IN CURRENT CODEBASE's ground-truth scope for this doc** — see the Accounting Reports module doc if one exists separately.

4. Backend

Controllers

AccountController (@RequestMapping("/api/accounts"))

Method	Endpoint	@PreAuthorize	Description
search	GET /api/accounts	none (any authenticated user)	Paginated/filterable Chart of Accounts list.
listGroups	GET /api/accounts/groups	none	All active AccountGroup s.
getById	GET /api/accounts/{id}	none	Single account.
create	POST /api/accounts	hasRole('ADMIN')	Creates a new (non-system) account.
update	PUT /api/accounts/{id}	hasRole('ADMIN')	Updates an account.

AccountingController (@RequestMapping("/api/accounting/journals"))

Method	Endpoint	@PreAuthorize	Description
search	GET /api/accounting/journals	none	Journal Register search (voucherType , status , fromDate , toDate , search , page , size).
getById	GET /api/accounting/journals/{id}	none	Full journal with lines.
create	POST /api/accounting/journals	hasRole('ADMIN')	Posts a manual JOURNAL -type voucher.

Note on authorization granularity: `Permission.ACCOUNT_VIEW` , `JOURNAL_VIEW` , `JOURNAL_CREATE` , `JOURNAL_POST` are defined in `Permission.java` and granted to `Role.ACCOUNTANT` in `RolePermissions.java` , but neither `AccountController` nor `AccountingController` actually checks `hasAuthority('...')` for these permissions — both controllers gate their write endpoints with the coarser `hasRole('ADMIN')` only, and their read endpoints have no `@PreAuthorize` at all (any authenticated user can read). This means the fine-grained `JOURNAL_*` / `ACCOUNT_VIEW` permissions exist in the permission model but are **not enforced at the controller layer for these two controllers** — an `ACCOUNTANT` role (which lacks `ADMIN`) cannot actually `POST /api/accounts` or `/api/accounting/journals` despite holding `JOURNAL_CREATE` .

DTOs

DTO	Fields	Validation
AccountRequest	accountCode , accountName , accountType , accountGroupId , openingBalance , openingBalanceType , active	@NotBlank accountCode , @NotBlank accountName , @NotNull accountType , @NotNull openingBalance , @NotNull openingBalanceType
AccountResponse	id , accountCode , accountName , accountType , accountGroupId , accountGroupName , openingBalance , openingBalanceType , systemAccount , active , createdBy , createdAt , modifiedBy , updatedAt	— (response only)
AccountGroupResponse	id , name , accountType , parentGroupId , parentGroupName , active	—

DTO	Fields	Validation
JournalLineRequest	accountId, debitAmount (default 0), creditAmount (default 0), narration, partyType, partyId	@NotNull accountId
JournalCreateRequest	journalDate, narration, lines: List<JournalLineRequest>	@NotNull journalDate, @NotEmpty @Valid lines (cascades JournalLineRequest's own @NotNull accountId)
JournalDetailResponse	id, accountId, accountCode, accountName, debitAmount, creditAmount, narration, partyType, partyId, referenceType, referenceId	—
JournalHeaderResponse	id, journalNumber, journalDate, voucherType, voucherId, voucherNumber, narration, status, reversalOfJournalId, postedBy, postedAt, createdBy, createdAt, totalDebit, totalCredit, lines: List<JournalDetailResponse>	— (totalDebit/totalCredit are computed by summing lines in fromEntity)

Not a DTO but the two carriers used internally by `AccountingService` for lines before a `JournalDetail` exists:

- `JournalLine (record, com.storehub.service): (SystemAccountCode account, BigDecimal debitAmount, BigDecimal creditAmount, String narration, AccountingPartyType partyType, Long partyId)` with static factories `debit(...)/credit(...)` — used by the Sale/Purchase/Receipt/Payment posting methods, which reference accounts by `SystemAccountCode`.
- `ManualJournalLine (record, com.storehub.service): (Long accountId, BigDecimal debitAmount, BigDecimal creditAmount, String narration, AccountingPartyType partyType, Long partyId)` — used by `postManualJournal/postJournalByAccountId`, which reference accounts by real id (e.g. an Expense Category's configured account).

Services

`AccountingService` — every public method

- **`postSaleJournal(Sale sale) @Transactional`** — Books the Sale's accounting entries. If the sale has a named `Customer`: debits `CUSTOMER_RECEIVABLE` for the full `totalAmount` (tagged `AccountingPartyType.CUSTOMER` + customer id) and credits `SALES` for `taxableAmount` plus, if `GstType.GST`, `OUTPUT_CGST/OUTPUT_SGST/OUTPUT_IGST` for any positive tax amounts — regardless of how much was paid immediately (a later Receipt clears the receivable via its own journal). If there is no customer (a walk-in sale) and `paidAmount > 0`: debits Cash or Bank (via `resolveCashOrBank`) for `paidAmount` and credits the same Sales/GST lines. If no customer and nothing paid, the method returns without posting anything. Calls `postJournal(VoucherType.SALE, sale.getId(), sale.getInvoiceNumber(), sale.getSaleDate(), "Sale " + invoiceNumber, lines, storeId)`.
- **`reverseSaleJournal(Sale sale, String reason) @Transactional`** — delegates to `reverseJournal(VoucherType.SALE, sale.getId(), reason)`.
- **`postPurchaseJournal(Purchase purchase) @Transactional`** — Debits `PURCHASE` for `taxableAmount`, plus `INPUT_CGST/INPUT_SGST/INPUT_IGST` if `GstType.GST`, and always credits `SUPPLIER_PAYABLE` for the full `totalAmount` (tagged `SUPPLIER` + supplier id), regardless of how much is paid immediately.
- **`reversePurchaseJournal(Purchase purchase, String reason) @Transactional`** — delegates to `reverseJournal(VoucherType.PURCHASE, ...)`.

- **postReceiptJournal(Receipt receipt)** @Transactional — Debits Cash/Bank (via `resolveCashOrBank(receipt.getPaymentMode())`) for `receipt.getAmount()`, credits `CUSTOMER_RECEIVABLE` for the same amount (tagged `CUSTOMER` + customer id).
- **reverseReceiptJournal(Receipt receipt, String reason)** @Transactional — delegates to `reverseJournal(VoucherType.RECEIPT, ...)`.
- **postPaymentJournal(Payment payment)** @Transactional — Debits `SUPPLIER_PAYABLE` (tagged `SUPPLIER` + supplier id) for `payment.getAmount()`, credits Cash/Bank for the same amount.
- **reversePaymentJournal(Payment payment, String reason)** @Transactional — delegates to `reverseJournal(VoucherType.PAYMENT, ...)`.
- **getJournalById(Long id): JournalHeaderResponse** @Transactional(readOnly = true) — throws `JournalNotFoundException` if absent.
- **searchJournals(VoucherType, LocalDate from, LocalDate to, String search, int page, int size): PagedResponse<JournalHeaderResponse>** @Transactional(readOnly = true) — overload without status, delegates to the 7-arg version with `status = null`.
- **searchJournals(VoucherType, JournalStatus, LocalDate, LocalDate, String, int, int): PagedResponse<JournalHeaderResponse>** — the Journal Register query, sorted `journalDate DESC, id DESC`.
- **resolveCashOrBank(PaymentMode mode): SystemAccountCode** — maps `PaymentMode.CASH` → `SystemAccountCode.CASH`, anything else → `SystemAccountCode.BANK`. Documented as "the one place that maps a payment mode to a system account," reused by `ExpenseService` and `CashTransactionService`.
- **postJournal(VoucherType, Long voucherId, String voucherNumber, LocalDate journalDate, String narration, List<JournalLine> lines): JournalHeader** — 6-arg overload, delegates to the 7-arg version with `storeId = null`.
- **postJournal(VoucherType, Long voucherId, String voucherNumber, LocalDate journalDate, String narration, List<JournalLine> lines, Long storeId): JournalHeader** @Transactional — validates the lines (`validateJournal`), resolves each `SystemAccountCode` to a live, active `Account` (`resolveActiveAccount`, throwing `BadRequestException` if the account is inactive), then calls `buildAndSaveJournal`.
- **postManualJournal(LocalDate journalDate, String narration, List<ManualJournalLine> lines): JournalHeader** @Transactional — throws `BadRequestException("A journal must have at least one line")` if `lines` is null/empty; resolves each line's `accountId` to a live active `Account` (`resolveActiveAccountById`), validates balance (`validateResolvedLines`), then `buildAndSaveJournal(VoucherType.JOURNAL, null, null, journalDate, narration, resolved, null)` — no source `voucherId`, so duplicate-posting protection does not apply to manual journals.
- **postManualJournal(JournalCreateRequest request): JournalHeaderResponse** @Transactional — REST-facing overload; maps `request.getLines()` to `ManualJournalLine`s and wraps the saved entity in `JournalHeaderResponse.fromEntity`.
- **postJournalByAccountId(VoucherType, Long voucherId, String voucherNumber, LocalDate journalDate, String narration, List<ManualJournalLine> lines): JournalHeader** — 6-arg overload, delegates with `storeId = null`.
- **postJournalByAccountId(VoucherType, Long voucherId, String voucherNumber, LocalDate journalDate, String narration, List<ManualJournalLine> lines, Long storeId): JournalHeader** @Transactional — like `postJournal`, but lines reference accounts by real id rather than `SystemAccountCode` (used where the posting account is configurable per source record, e.g. an `ExpenseCategory`'s linked account); duplicate-posting protection and voucher-id-based reversal both still apply since a real `voucherId` / `voucherType` is supplied.
- **buildAndSaveJournal(VoucherType, Long voucherId, String voucherNumber, LocalDate journalDate, String narration, List<ResolvedLine> lines, Long storeId): JournalHeader** (private) @Transactional — the actual save path; see §9/§14.

- **reverseJournal(VoucherType voucherType, Long voucherId, String reason): void** @Transactional — see §14; no-op if `voucherId == null` or no active POSTED original exists (never posted, or already reversed).
- **validateJournal(List<JournalLine> lines): void** (private) — throws `BadRequestException` if `lines` is null/empty, else delegates amount validation.
- **validateResolvedLines(List<ResolvedLine> lines): void** (private) — same, for the id-based line path.
- **validateAmounts(List<BigDecimal[]> debitCreditPairs): void** (private) — the shared debit/credit rule engine; see §6.
- **resolveActiveAccount(SystemAccountCode code): Account** (private) — `accountService.getSystemAccount(code)`, throws `BadRequestException` if `!account.isActive()`.
- **resolveActiveAccountById(Long accountId): Account** (private) — `accountService.findOrThrow(accountId)`, same inactive-account guard.

`SystemAccountCode.values()`, `VoucherType` values, `AccountingPartyType` are documented in §1/§5. `AccountingService` depends on `AccountService`, `JournalHeaderRepository`, `FinancialYearService`, `StoreRepository`.

VoucherNumberService — FY-aware numbering algorithm

- **next(VoucherDocType docType, LocalDate transactionDate): String** — the only public method, annotated `@Transactional(propagation = Propagation.REQUIRES_NEW)`:
 1. `financialYearService.resolveForDate(transactionDate)` resolves the `FinancialYear` the date falls in (does **not** require it to be OPEN — a closed FY's historical documents must remain viewable/re-printable). Throws `BadRequestException` if no FY covers the date.
 2. `voucherSequenceRepository.ensureRowExists(docType.name(), fy.getId())` — a native `INSERT ... ON DUPLICATE KEY UPDATE id = id` upsert-if-absent, chosen specifically so a race never throws a constraint violation that would mark the JPA transaction rollback-only.
 3. `voucherSequenceRepository.lockForUpdate(docType, fy.getId())` — `SELECT ... FOR UPDATE (@Lock(LockModeType.PESSIMISTIC_WRITE))`, row-locking the `(docType, financialYearId)` counter for the duration of this `REQUIRES_NEW` transaction, serializing concurrent increments so two callers can never receive the same number.
 4. `seq.setLastNumber(seq.getLastNumber() + 1)`, saved.
 5. Returns `docType.getPrefix() + "/" + fy.getCode() + "/" + String.format("%06d", seq.getLastNumber())`, e.g. `SALE/26-27/000001`.
- **Concurrency design:** runs in its own `REQUIRES_NEW` transaction so the counter lock is acquired and released independently of the caller's (often much larger) transaction — a long-running Sale-posting transaction never holds the numbering lock, and a numbering failure never poisons the caller's transaction.
- **Draft numbering policy:** a number is allocated at document creation time regardless of draft/posted status; a draft later cancelled or never posted still permanently consumes its number (never reused).
- **VoucherDocType values and prefixes:** `SALE("SALE")`, `SALE_CHALLAN("SC")`, `PURCHASE("PUR")`, `PURCHASE_CHALLAN("PC")`, `SALES_ORDER("SO")`, `PURCHASE_ORDER("PO")`, `RECEIPT("REC")`, `PAYMENT("PAY")`, `CREDIT_NOTE("CN")`, `DEBIT_NOTE("DN")`, `EXPENSE("EXP")`, `CASH_TRANSACTION("CT")`, `STOCK_TRANSFER("ST")`. It is deliberately separate from `VoucherType` (the accounting journal classification) — e.g. a Kacchi Sale gets its own `SC/26-27/000001` numbering series while still posting its accounting journal under the single `VoucherType.SALE` bucket.

LedgerService — every public method

Every posting is an immutable row; a reversal is a new offsetting row, never a delete/update of the original.

Customer receivable ledger / output GST / cash movement (Sale, Receipt side):

- `recordSaleDebit(Sale sale): void @Transactional` — no-op if `sale.getCustomer() == null`; else saves a `CustomerLedgerEntry(DEBIT, sale.getTotalAmount(), referenceType=SALE)`.
- `reverseSaleDebit(Sale sale, String reason): void @Transactional` — mirrors with CREDIT.
- `recordGstEntry(Sale sale): void @Transactional` — no-op unless `GstType.GST`; saves `GstEntry(taxableAmount, cgstAmount, sgstAmount, igstAmount)`.
- `reverseGstEntry(Sale sale): void @Transactional` — same amounts negated.
- `recordReceiptCredit(Receipt receipt): void @Transactional` — `CustomerLedgerEntry(CREDIT, receipt.getAmount(), referenceType=RECEIPT)`.
- `reverseReceiptCredit(Receipt receipt, String reason): void @Transactional` — mirrors with DEBIT.
- `recordCashEntry(Receipt receipt): void @Transactional` — `CashLedgerEntry(paymentMode, amount, referenceType=RECEIPT)`.
- `reverseCashEntry(Receipt receipt, String reason): void @Transactional` — same, amount negated.
- `recordCashEntryForSale(Sale sale): void @Transactional` — no-op unless `paidAmount > 0`; used for a walk-in sale paid immediately (money moved, no receivable to track).
- `reverseCashEntryForSale(Sale sale, String reason): void @Transactional` — mirrors, negated.
- `getOutstandingForCustomer(Long customerId): BigDecimal` — delegates to repository.
- `getTotalOutstanding(): BigDecimal` — delegates to repository.

Supplier payable ledger / input GST / cash outflow (Purchase, Payment side):

- `recordPurchaseCredit(Purchase purchase): void @Transactional` — `SupplierLedgerEntry(CREDIT, totalAmount, referenceType=PURCHASE)`.
- `reversePurchaseCredit(Purchase purchase, String reason): void @Transactional` — mirrors with DEBIT.
- `recordInputGstEntry(Purchase purchase): void @Transactional` — no-op unless GST; saves `PurchaseGstEntry`.
- `reverseInputGstEntry(Purchase purchase): void @Transactional` — negated.
- `recordPaymentDebit(Payment payment): void @Transactional` — `SupplierLedgerEntry(DEBIT, amount, referenceType=PAYMENT)`.
- `reversePaymentDebit(Payment payment, String reason): void @Transactional` — mirrors with CREDIT.
- `recordCashEntryOut(Payment payment): void @Transactional` — `CashLedgerEntry(amount negated, referenceType=PAYMENT)`.
- `reverseCashEntryOut(Payment payment, String reason): void @Transactional` — amount positive (reversing the negation).
- `getOutstandingForSupplier(Long supplierId): BigDecimal`, `getTotalPayables(): BigDecimal` — delegate to repository.

Expense (Account-module) and Credit/Debit Note:

- `recordExpenseCredit(Expense expense): void @Transactional` — "A credit (party) Expense increases what is owed to the supplier — same polarity as `recordPurchaseCredit`"; saves `SupplierLedgerEntry(CREDIT, expense.getTotalAmount(), referenceType=EXPENSE)`.
- `reverseExpenseCredit(Expense expense, String reason): void @Transactional` — mirrors with DEBIT.
- `recordCreditNoteEntry(CreditNote note): void @Transactional` — "always REDUCES what the customer owes — the opposite polarity of `recordSaleDebit`"; `CustomerLedgerEntry(CREDIT, note.getTotalAmount(), referenceType=CREDIT_NOTE)`.
- `reverseCreditNoteEntry(CreditNote note, String reason): void @Transactional` — mirrors with DEBIT.
- `recordDebitNoteEntry(DebitNote note): void @Transactional` — "always REDUCES what is owed to the supplier — the opposite polarity of `recordPurchaseCredit`"; `SupplierLedgerEntry(DEBIT,`

`note.getTotalAmount(), referenceType=DEBIT_NOTE)`.

- `reverseDebitNoteEntry(DebitNote note, String reason): void @Transactional` — mirrors with `CREDIT`.

AccountService — CRUD for Account/AccountGroup

- `ensureSystemAccountsExist(): void @PostConstruct @Transactional` — for each `AccountType`, finds/creates a top-level `AccountGroup` (Assets/Liabilities/Income/Expenses); for each `SystemAccountCode`, finds-or-creates the corresponding `Account` (idempotent by `accountCode`), with `systemAccount=true`, `active=true`, `openingBalance=ZERO`.
- `getSystemAccount(SystemAccountCode code): Account` — resolves via an in-memory `ConcurrentHashMap<SystemAccountCode, Long>` cache (`code` → `account id`) to avoid a query per journal line; falls back to `accountRepository.findByAccountCode`, throws `AccountNotFoundException(code.getCode())` if missing.
- `search(String search, AccountType accountType, Boolean active, int page, int size): PagedResponse<AccountResponse>` — sorted `accountCode` ASC.
- `findById(Long id): AccountResponse` — throws `AccountNotFoundException` if absent.
- `listGroups(): List<AccountGroupResponse>` — all active groups, sorted by name.
- `create(AccountRequest request): AccountResponse @Transactional` — throws `BadRequestException` if `accountCode` already exists (case-insensitive); new accounts are always `systemAccount=false`.
- `update(Long id, AccountRequest request): AccountResponse @Transactional` — if the code changes: throws `BadRequestException("The code of a system account cannot be changed")` for a system account, or a duplicate-code `BadRequestException` otherwise; `accountType` is only updated for non-system accounts (a system account's type stays fixed).
- `findOrThrow(Long id): Account` — throws `AccountNotFoundException(id)`.

Repository

AccountRepository: `findByAccountCode(String)`, `existsByAccountCodeIgnoreCase(String)`, `findByActiveTrueOrderByAccountNameAsc()`, `search(search, accountType, active, Pageable)` (JPQL, case-insensitive `LIKE` on code/name).

AccountGroupRepository: `findByNameIgnoreCase(String)`, `findByActiveTrueOrderByNameAsc()`.

JournalHeaderRepository:

`existsByVoucherTypeAndVoucherIdAndStatusAndReversalOfJournalIsNull(VoucherType, Long, JournalStatus)`, `findByVoucherTypeAndVoucherIdAndStatusAndReversalOfJournalIsNull(...)`, `search(voucherType, fromDate, toDate, search, Pageable)`, `search(voucherType, status, fromDate, toDate, search, Pageable)`, `findDuplicateActivePostings()`, `findActivePostedVoucherIds(VoucherType)`, `findDistinctVoucherIds(VoucherType)`, `findPostedDatesWithoutFinancialYear()` (the last four back the Accounting Health Check report).

JournalDetailRepository: `findByJournalIdOrderById(Long)`, `findLedgerLines(accountId, from, to)`, `sumDebitCreditByAccount(asOfDate)` (Trial Balance), `sumDebitCreditBefore(accountId, beforeDate)`, `sumDebitCreditForAccount(accountId)`, `dayBookRows(from, to) / dayBookRows(voucherType, from, to)`, `sumDebitCreditByAccountRange(from, to)`, `accountVoucherRows(accountId, from, to)`, `sumByPartyBefore(partyType, beforeDate)`, `sumByPartyAndReferenceTypeRange(partyType, from, to)`, `findLedgerLinesByParty(partyType, partyId, from, to)`, `sumByParty(partyType)`, `findUnbalancedJournals()`.

VoucherSequenceRepository: `lockForUpdate(VoucherDocType, Long financialYearId)` (`@Lock(PESSIMISTIC_WRITE)`), `ensureRowExists(String docType, Long financialYearId)` (native upsert).

5. Database

Tables

- **accounts** — columns: `id`, `account_code` (unique, not null), `account_name`, `account_type` (VARCHAR(20)), `account_group_id` (FK), `opening_balance` (precision 12, scale 2), `opening_balance_type` (VARCHAR(10)), `system_account` (boolean), `active` (boolean), `created_by`, `created_at`, `modified_by`, `updated_at`.
- **account_groups** — `id`, `name`, `account_type`, `parent_group_id` (self-FK), `active`, `created_at`, `updated_at`.
- **journal_headers** — `id`, `journal_number` (unique), `journal_date` (indexed), `voucher_type` (VARCHAR(20), indexed with `voucher_id`), `voucher_id`, `store_id` (FK), `voucher_number`, `narration` (TEXT), `status` (VARCHAR(10), indexed), `reversal_of_journal_id` (self-FK), `posted_by`, `posted_at`, `created_by`, `created_at`.
- **journal_details** — `id`, `journal_id` (FK, not null), `account_id` (FK, not null, indexed), `debit_amount` (precision 12, scale 2, default 0), `credit_amount` (precision 12, scale 2, default 0), `narration`, `party_type` (VARCHAR(10), indexed with `party_id`), `party_id`, `reference_type` (VARCHAR(20)), `reference_id`.
- **voucher_sequences** — `id`, `doc_type` (VARCHAR(20)), `financial_year_id`, `last_number`; unique constraint `uk_voucher_sequence` on (`doc_type`, `financial_year_id`).

Relationships

- `JournalHeader 1—* JournalDetail`: `@OneToMany(mappedBy = "journal", cascade = ALL, orphanRemoval = true)` on `JournalHeader.lines`; `JournalDetail.journal` is the owning `@ManyToOne`. `JournalHeader.addLine(line)` sets both sides.
- `JournalDetail *—1 Account`: `@ManyToOne` on `JournalDetail.account`.
- `Account *—1 AccountGroup`: `@ManyToOne` on `Account.accountGroup` (nullable).
- `AccountGroup *—1 AccountGroup`: self-referential `parentGroup` for the group hierarchy.
- `JournalHeader *—1 Store`: `@ManyToOne` on `JournalHeader.store`, nullable — present when the source voucher has a meaningful store (Sale/Purchase/Receipt/Payment/etc.), null for a manual `JOURNAL` entry or a pre-multi-store legacy voucher.
- `JournalHeader *—1 JournalHeader (self)`: `reversalOfJournal` links a reversal journal back to the original it offsets.
- `JournalDetail.partyType (AccountingPartyType) + partyId` is a loose (non-FK) tag pointing at a `Customer` or `Supplier` id, letting a control account (e.g. Customer Receivable) be drilled down per party without a hard foreign key.

6. Validation

Debit=credit balance check and related invariants, from `AccountingService.validateAmounts` (exact code):

```

private void validateAmounts(List<BigDecimal[]> debitCreditPairs) {
    BigDecimal totalDebit = BigDecimal.ZERO;
    BigDecimal totalCredit = BigDecimal.ZERO;
    boolean hasDebit = false;
    boolean hasCredit = false;

    for (BigDecimal[] pair : debitCreditPairs) {
        BigDecimal debit = pair[0];
        BigDecimal credit = pair[1];

        if (debit.signum() < 0 || credit.signum() < 0) {
            throw new BadRequestException("A journal line amount cannot be negative");
        }
        if (debit.signum() > 0 && credit.signum() > 0) {
            throw new BadRequestException("A journal line cannot have both a debit and a credit amount");
        }
        if (debit.signum() == 0 && credit.signum() == 0) {
            throw new BadRequestException("A journal line must have either a debit or a credit amount");
        }

        hasDebit = hasDebit || debit.signum() > 0;
        hasCredit = hasCredit || credit.signum() > 0;
        totalDebit = totalDebit.add(debit);
        totalCredit = totalCredit.add(credit);
    }

    if (!hasDebit || !hasCredit) {
        throw new BadRequestException("A journal must have at least one debit line and one credit line");
    }
    if (totalDebit.compareTo(totalCredit) != 0) {
        throw new BadRequestException("Journal is not balanced: total debit (" + totalDebit + ") does not equal total credit (" + totalCredit + ")");
    }
}

```

Other invariants enforced by `AccountingService` :

- **At least one line:** `postManualJournal` throws `BadRequestException("A journal must have at least one line")` if `lines == null || lines.isEmpty()`; `validateJournal/validateResolvedLines` throw the same for the `postJournal/postJournalByAccountId` paths.
- **FY-open check:** `buildAndSaveJournal` calls `financialYearService.resolveOpenForPosting(journalDate)` before saving; this throws `BadRequestException` if no FY covers the date, or `BadRequestException("Financial year " + fy.getName() + " is closed. This transaction (" + date + ") cannot be posted.")` if the FY is `CLOSED`. `reverseJournal` performs the same check against `LocalDate.now()` (the reversal's own posting date).
- **No double-posting:** `buildAndSaveJournal` throws `BadRequestException("An accounting journal has already been posted for this " + voucherType.name().toLowerCase())` if `journalHeaderRepository.existsByVoucherTypeAndVoucherIdAndStatusAndReversalOfJournalIsNull(voucherType, voucherId, POSTED)` is already true (checked only when `voucherId != null`, so manual journals are exempt).
- **Active-account check:** `resolveActiveAccount/resolveActiveAccountById` throw `BadRequestException("Account '" + account.getAccountName() + "' is inactive and cannot be posted to")` if the resolved `Account.active` is false.

- **Account uniqueness (AccountService):** create/update throw `BadRequestException("An account with code '...' already exists")` on a duplicate `accountCode` (case-insensitive), and `BadRequestException("The code of a system account cannot be changed")` if an attempt is made to rename a system account's code.

7. Authentication / Authorization

All `/api/accounts/**` and `/api/accounting/**` endpoints require an authenticated request — `SecurityConfig.filterChain` permits only `/api/auth/**` and `/actuator/health` without authentication (`.anyRequest().authenticated()`), enforced via a stateless JWT filter chain (`JwtAuthenticationFilter` added before `UsernamePasswordAuthenticationFilter`), with `@EnableMethodSecurity` enabling the controllers' `@PreAuthorize` annotations. There is no accounting-specific authentication mechanism; it relies entirely on the application-wide JWT/`UserPrincipal` setup.

8. Permissions

Defined in `Permission.java` under the "Account" module:

- `ACCOUNT_VIEW`
- `JOURNAL_VIEW`, `JOURNAL_CREATE`, `JOURNAL_POST`
- `EXPENSE_VIEW`, `EXPENSE_CREATE`, `EXPENSE_POST`, `EXPENSE_CANCEL` (Expense module, posts through this engine but is documented elsewhere)
- `REPORT_VIEW`, `REPORT_EXPORT`
- `CASH_MANAGE`
- `FY_MANAGE` (kept deliberately separate from `ACCOUNT_VIEW` per the code comment — "financial year close requires the highest authorization")

Per `RolePermissions.java`: `Role.ADMIN` holds every permission; `Role.STORE_MANAGER` holds all of ADMIN's operational permissions except `FY_MANAGE` (and the security/GST-config/store-admin tiers) — it does hold `JOURNAL_*/ACCOUNT_VIEW`; `Role.ACCOUNTANT` explicitly holds `ACCOUNT_VIEW`, `JOURNAL_VIEW`, `JOURNAL_CREATE`, `JOURNAL_POST` plus the related Sales/Purchase view, Expense, Report, Cash, and GST permissions. As noted in §4, these permissions are defined and assigned but **not actually checked** by `AccountController/AccountingController`, whose write endpoints instead gate on `hasRole('ADMIN')` directly — so in practice only ADMIN can create/edit an Account or post a manual Journal Entry through these two controllers, regardless of `JOURNAL_CREATE/JOURNAL_POST` grants elsewhere.

9. Transaction Handling

Posting must be atomic (all lines saved together, or none), and this is enforced entirely through Spring's declarative `@Transactional`:

- Every `post*Journal/reverse*Journal` public method on `AccountingService` (`postSaleJournal`, `postPurchaseJournal`, `postReceiptJournal`, `postPaymentJournal`, their `reverse*` counterparts, `postJournal(..., storeId)`, `postManualJournal` (both overloads), `postJournalByAccountId(..., storeId)`, `reverseJournal`) is annotated `@Transactional`. Because these are default-propagation (`REQUIRED`), they join the calling module's own transaction (e.g. `SaleService.createSale`'s transaction) when called from within one — so a Sale row, its `CustomerLedgerEntry/GstEntry` (via `LedgerService`), and its `JournalHeader + JournalDetail` rows (via `AccountingService`) all commit or roll back together as a single unit of work.
- `buildAndSaveJournal` (private, called from within an already-`@Transactional` public method) performs, in order: the FY-open check, the duplicate-posting check, builds and saves the

`JournalHeader`, saves each `JournalDetail` line via `header.addLine(...)` (cascaded by `CascadeType.ALL` on `JournalHeader.lines`), then re-saves the header twice more to stamp `journalNumber = "JRN-%06d".formatted(id)` (needs the generated id first) and, if still null, `voucherNumber = journalNumber`. All of this happens inside one transaction — a failure partway (e.g. the duplicate-posting check firing) throws before any row is persisted, and Spring rolls back the whole enclosing transaction (the `RuntimeException` subclass `BadRequestException` triggers rollback by default).

- `reverseJournal` similarly does its lookup, FY check, builds the reversal `JournalHeader` + lines, saves it, then flips the original's `status` to `REVERSED` and saves it — all within the same `@Transactional` boundary, so the reversal journal and the original's status change are atomic together.
- **`VoucherNumberService.next(...)` is the one deliberate exception:** it is `@Transactional(propagation = Propagation.REQUIRES_NEW)`, meaning it always runs in its own, independent transaction rather than joining the caller's. This is intentional (see §4) — it decouples the pessimistic row lock on `voucher_sequences` from the (potentially much longer) posting transaction, so the lock is held only as long as the increment itself takes, and a numbering failure does not roll back whatever transaction called it (though in practice a numbering failure typically happens before the rest of the document is built, e.g. in `SaleService.createSale` before `accountingService.postSaleJournal` is even reached).
- Read paths (`getJournalById`, `searchJournals`) are `@Transactional(readOnly = true)`.

10. Error Handling

`GlobalExceptionHandler` (`@RestControllerAdvice`) maps exceptions to a uniform `ApiError` body (`timestamp`, `status`, `error`, `message`, `path`, `fieldErrors`):

- `AccountNotFoundException` → 404, message `"Account not found with id: {id}"` or `"Account not found with code: {code}"`.
- `JournalNotFoundException` → 404, message `"Journal not found with id: {id}"`.
- `BadRequestException` (every validation failure in §6, plus `AccountService`'s duplicate-code/system-account-code-change errors) → 400, `ex.getMessage()` as message, `fieldErrors = null`.
- `MethodArgumentNotValidException` (bean-validation failures on `AccountRequest` / `JournalCreateRequest` / `JournalLineRequest`, e.g. a missing `accountCode` or `accountId`) → 400, message = `"Validation failed"`, `fieldErrors` populated per-field from the binding result — per the project-wide rule, the frontend must surface these via `isInvalid`/feedback text rather than only the top-level message (see `AccountMaster.tsx`'s `fieldErrors` handling in §2).
- `AccessDeniedException` (a non-ADMIN hitting a `hasRole('ADMIN')` endpoint) → 403, generic message.
- Any other `Exception` → 500, generic `"An unexpected error occurred"`, logged server-side.

11. Audit Flow

`AccountingService`, `AccountService`, `LedgerService`, and `VoucherNumberService` do **not** call `AuditService` anywhere — grepped for `auditService.` across all four files with zero matches. Audit logging for financial-year state changes (open/close/mark-current), which gates whether a posting is even allowed, is done in `FinancialYearService` (`auditService.log(AuditAction.FY_CLOSE/FY_OPEN/CREATE/UPDATE, "ADMIN", "FinancialYear", ...)`), not in the accounting engine itself. There is **NOT FOUND IN CURRENT CODEBASE** any direct audit-trail entry written when a Journal is posted or reversed, or when an Account is created/updated — the only record of who posted/reversed a journal is the `postedBy` / `createdBy` string columns on `JournalHeader` itself (populated from `SecurityUtil.currentUsername()` / the authenticated `UserPrincipal`'s name).

12. Important Side Effects

- Posting a Sale/Purchase/Receipt/Payment/Expense/CreditNote/DebitNote/CashTransaction always writes to **two** parallel ledgers: the `LedgerService` operational sub-ledger (`CustomerLedgerEntry` / `SupplierLedgerEntry` / `CashLedgerEntry` / `GstEntry` / `PurchaseGstEntry`) and the `AccountingService` double-entry journal (`JournalHeader` + `JournalDetail`). These are maintained independently by each calling service (see §13) — nothing in `AccountingService` itself updates the sub-ledgers, and nothing in `LedgerService` posts a journal.
- `AccountService.ensureSystemAccountsExist()` runs on every application startup and is idempotent (find-or-create), so it has no effect after the first run except guaranteeing the fixed set of `SystemAccountCode` accounts and their top-level groups exist.
- `AccountService.getSystemAccount` caches `SystemAccountCode` → `Account.id` in an unbounded, never-invalidated `ConcurrentHashMap` for the process lifetime — updating a system account's other fields (name, group, balance) is reflected immediately since the cache only stores the id, but the cache is never cleared, which is fine because system-account ids never change once created.
- A walk-in Sale (no customer) with `paidAmount == 0` posts **no** journal at all (`postSaleJournal` returns early) — matching the pre-existing gap already present in `LedgerService.recordSaleDebit`'s own no-op for the same case.
- `postJournal` / `postJournalByAccountId` resolve accounts and validate balance before touching the database, but the FY-open check and duplicate-posting check happen inside `buildAndSaveJournal`, after line resolution — a caller could resolve valid accounts and then still fail on a closed FY or a duplicate voucher.
- `buildAndSaveJournal` saves the `JournalHeader` row up to three times (once to get a generated `id`, once to stamp `journalNumber`, and a third time if `voucherNumber` was still null) — each save happens within the same transaction, so this is not visible to other transactions until commit, but it means `journalNumber` generation depends on the auto-increment `id`, not on `VoucherNumberService`.

13. Dependencies on Other Modules

Modules that call `AccountingService`

Calling module (service)	Method(s) called
<code>SaleService</code>	<code>postSaleJournal(saved)</code> ; <code>reverseSaleJournal(sale, "Sale revised: " + invoiceNumber)</code> ; <code>reverseSaleJournal(sale, reason)</code>
<code>PurchaseService</code>	<code>postPurchaseJournal(saved)</code> ; <code>reversePurchaseJournal(purchase, "Purchase revised: " + purchaseNumber)</code> ; <code>reversePurchaseJournal(purchase, reason)</code>
<code>ReceiptService</code>	<code>postReceiptJournal(saved)</code> (×2 call sites); <code>reverseReceiptJournal(receipt, reason)</code>
<code>PaymentService</code>	<code>postPaymentJournal(saved)</code> (×2 call sites); <code>reversePaymentJournal(payment, reason)</code>
<code>ExpenseService</code>	<code>accountingService.resolveCashOrBank(expense.getPaymentMode())</code> ; <code>postJournalByAccountId(VoucherType.EXPENSE, expense.getId(), expense.getExpenseNumber(), ...)</code> ; <code>reverseJournal(VoucherType.EXPENSE, expense.getId(), reason)</code>
<code>CashTransactionService</code>	<code>resolveCashOrBank(txn.getPaymentMode())</code> ; <code>postJournal(VoucherType.CASH_TRANSACTION, txn.getId(), txn.getTransactionNumber(), ...)</code> ; <code>reverseJournal(VoucherType.CASH_TRANSACTION, txn.getId(), reason)</code>

Calling module (service)	Method(s) called
CreditNoteService	postJournal(VoucherType.CREDIT_NOTE, note.getId(), note.getVoucherNumber(), ...); reverseJournal(VoucherType.CREDIT_NOTE, note.getId(), reason)
DebitNoteService	postJournal(VoucherType.DEBIT_NOTE, note.getId(), note.getVoucherNumber(), ...); reverseJournal(VoucherType.DEBIT_NOTE, note.getId(), reason)

8 distinct caller modules call into `AccountingService`.

Modules that call `VoucherNumberService.next(...)`

`SalesOrderService` (SALES_ORDER), `ReceiptService` (RECEIPT, ×2), `PurchaseService` (PURCHASE), `CashTransactionService` (CASH_TRANSACTION), `CreditNoteService` (CREDIT_NOTE), `StockTransferService` (STOCK_TRANSFER), `SaleService` (SALE), `ExpenseService` (EXPENSE), `PurchaseOrderService` (PURCHASE_ORDER), `PaymentService` (PAYMENT, ×2), `DebitNoteService` (DEBIT_NOTE) — **11 distinct caller modules**.

Modules that call `LedgerService`

`SaleService` (`recordSaleDebit`, `recordGstEntry`, `recordCashEntryForSale`, and their `reverse*` counterparts), `PurchaseService` (`recordPurchaseCredit`, `recordInputGstEntry`, `reverses`), `ReceiptService` (`recordReceiptCredit`, `recordCashEntry`, `reverses`), `PaymentService` (`recordPaymentDebit`, `recordCashEntryOut`, `reverses`), `ExpenseService` (`recordExpenseCredit`, `reverseExpenseCredit`), `CreditNoteService` (`recordCreditNoteEntry`, `reverseCreditNoteEntry`), `DebitNoteService` (`recordDebitNoteEntry`, `reverseDebitNoteEntry`), `SalesSummaryService` (read-only: `getTotalOutstanding()`), `PurchaseSummaryService` (read-only: `getTotalPayables()`) — **9 distinct caller modules**.

What this module depends on

- **FinancialYearService** — `AccountingService.buildAndSaveJournal / reverseJournal` call `resolveOpenForPosting(date)`; `VoucherNumberService.next` calls `resolveForDate(date)`. This is the only cross-module inbound dependency of the core posting/numbering engine.
- **StoreRepository / Store** — `AccountingService.buildAndSaveJournal` calls `storeRepository.getReferenceById(storeId)` to stamp `JournalHeader.store` when a `storeId` is supplied by the caller.
- **AccountService** — used internally by `AccountingService` (`resolveActiveAccount`, `resolveActiveAccountById`) to resolve accounts; part of this same module.

Per the task's framing, this module is depended upon by nearly every transactional module in the codebase; it in turn depends on almost nothing outside itself except Financial Year (for FY-open validation/FY code resolution) and Store (for stamping `storeId` on journals).

14. Key Operation Flows

Create Manual Journal Entry (post)

1. **UI:** `JournalEntryForm.tsx` — user fills `journalDate`, `narration`, and 2+ line rows; client computes `totalDebit / totalCredit` and blocks submit unless balanced and both totals are present.
2. **Function:** `handleSubmit` builds `payloadLines: JournalLinePayload[]` (filtering out empty rows) and calls `journalApi.create({ journalDate, narration, lines: payloadLines })`.
3. **API:** `POST /api/accounting/journals` with body `JournalCreateRequest`.

4. **Controller:** `AccountingController.create(@Valid @RequestBody JournalCreateRequest request)` — `@PreAuthorize("hasRole('ADMIN')")`; bean validation runs first (`@NotNull journalDate`, `@NotEmpty @Valid lines`, each line's `@NotNull accountId`) — a failure here short-circuits into `GlobalExceptionHandler.handleValidation` before the controller body executes.
5. **DTO → Service:** controller calls `accountingService.postManualJournal(request)`.
6. **Service:** `AccountingService.postManualJournal(JournalCreateRequest)` maps each `JournalLineRequest` to a `ManualJournalLine`, then calls the 3-arg `postManualJournal(journalDate, narration, lines)`, which: (a) throws if `lines` is empty; (b) resolves each `accountId` via `accountService.findOrThrow` + active-account check (`resolveActiveAccountById`); (c) `validateResolvedLines` runs the debit=credit balance rule (§6); (d) `buildAndSaveJournal(VoucherType.JOURNAL, null, null, journalDate, narration, resolved, null)` — no `voucherId`, so no duplicate-posting check applies; the FY-open check still runs.
7. **Repository:** `buildAndSaveJournal` builds a `JournalHeader` (`status=POSTED`, `postedBy/createdBy` = current username, `postedAt=now()`), adds one `JournalDetail` per line via `header.addLine(...)`, and calls `journalHeaderRepository.save(header)` (cascades the lines via `CascadeType.ALL`); saves again to stamp `journalNumber = "JRN-%06d"`; saves a third time if `voucherNumber` is still null (set to `journalNumber`).
8. **Database:** one `INSERT` into `journal_headers`, `N INSERT`s into `journal_details` (via Hibernate's cascade), plus the follow-up `UPDATE`s to `journal_headers` for the number stamps — all inside the single `@Transactional` boundary.
9. **Response:** `postManualJournal(request)` wraps the saved `JournalHeader` in `JournalHeaderResponse.fromEntity(saved)`; controller returns `201 Created` with that body.
10. **UI:** `journalApi.create` resolves; `JournalEntryForm` toasts "Journal posted successfully" and navigates to `/accounting/journals`, where `Journals.tsx` will show the new row on next load.

Reverse a Journal

There is no dedicated "reverse" UI/endpoint in the ground-truth frontend/controller files read for this module (`AccountingController` exposes only `search/getById/create`) — reversal is invoked programmatically by the owning transactional module when its source document is cancelled/revised, not by a direct user action against the accounting engine. Trace (using Sale cancellation as the example, `SaleService.cancelSale/equivalent` calling `accountingService.reverseSaleJournal(sale, reason)`):

1. **Caller service** (e.g. `SaleService`) reaches a point where a previously-posted Sale must be undone (cancel, or revise-then-repost) and calls `accountingService.reverseSaleJournal(sale, reason)` (or the equivalent `reverseJournal(VoucherType.X, id, reason)` for Purchase/Receipt/Payment/Expense/CreditNote/DebitNote/CashTransaction), inside its own `@Transactional` method.
2. **AccountingService.reverseSaleJournal** delegates to `reverseJournal(VoucherType.SALE, sale.getId(), reason)`.
3. **reverseJournal**: if `voucherId == null`, returns immediately (no-op). Otherwise looks up the currently-active original via `journalHeaderRepository.findByVoucherTypeAndVoucherIdAndStatusAndReversalOfJournalIsNull(voucherType, voucherId, POSTED)`; if absent (never posted, or already reversed), returns without error (a deliberate no-op, not an exception).
4. If found: `financialYearService.resolveOpenForPosting(LocalDate.now())` re-validates the current date's FY is open (the reversal itself is dated today, independent of the original journal's date).
5. Builds a new `JournalHeader` (`journalDate = LocalDate.now()`, same `voucherType/voucherId/voucherNumber` as the original, `narration = reason`, `status = POSTED`, `reversalOfJournal = original`, same `store`), and for every line of the original adds a mirrored `JournalDetail` with debit and credit **swapped** (`debitAmount = originalLine.getCreditAmount()`, `creditAmount = originalLine.getDebitAmount()`) — an exact offsetting entry.

6. Saves the reversal `JournalHeader` (cascading its lines), stamps its `journalNumber`.
7. Sets `original.setStatus(JournalStatus.REVERSED)` and saves the original — all within the same `@Transactional` boundary as steps 5-6, so the new reversal journal and the original's status flip commit together.
8. **Database:** net effect — the original journal's `status` column flips to `REVERSED` (its rows are never deleted), and a brand-new `journal_headers / journal_details` row set exists representing the offsetting entry, permanently preserving both for audit.
9. **UI:** nothing in the ground-truth frontend triggers this directly, but `Journals.tsx`'s list/detail view will show both the original (now `REVERSED`, styled `muted`) and the new reversal journal (still `voucherType / voucherNumber` of the source, `POSTED`) once the caller's own cancel/revise flow completes and the user reloads the Journal Register.

Cross-module example: Sale posting (SaleService → AccountingService)

(`SaleService` is documented in its own module doc; this traces only the call chain into the accounting engine, per `backend/src/main/java/com/storehub/service/SaleService.java` lines ~216-218.)

1. `SaleService.createSale(...)` (or the complete/revise variants), after validating and saving the `Sale` entity (`saved`), calls, in order: `ledgerService.recordSaleDebit(saved)`, `ledgerService.recordGstEntry(saved)`, (conditionally) `ledgerService.recordCashEntryForSale(saved)`, then `accountingService.postSaleJournal(saved)` — all within `SaleService`'s own `@Transactional` method, so `Sale + CustomerLedgerEntry / GstEntry / CashLedgerEntry + JournalHeader / JournalDetail` all commit atomically.
2. `AccountingService.postSaleJournal(Sale sale)`: builds a `List<JournalLine>` per the branching in §4 (customer present → debit `CUSTOMER_RECEIVABLE` for the full total + credit `SALES / GST` lines; no customer but paid → debit `Cash/Bank` + credit `SALES / GST` lines; no customer and unpaid → return, no journal). Calls `postJournal(VoucherType.SALE, sale.getId(), sale.getInvoiceNumber(), sale.getSaleDate(), "Sale " + sale.getInvoiceNumber(), lines, storeIdOf(sale.getStore()))`.
3. `postJournal(..., storeId)`: `validateJournal(lines)` runs the debit=credit balance check (§6); each `JournalLine.account()` (a `SystemAccountCode`, e.g. `SALES`, `CUSTOMER_RECEIVABLE`, `OUTPUT_CGST`) is resolved to a live `Account` via `resolveActiveAccount` → `accountService.getSystemAccount(code)` (cached lookup by `accountCode` in the `accounts` table).
4. `buildAndSaveJournal(VoucherType.SALE, sale.getId(), sale.getInvoiceNumber(), sale.getSaleDate(), narration, resolved, storeId)`: FY-open check on `sale.getSaleDate()`; duplicate-posting check on `(SALE, sale.getId())`; builds and saves the `JournalHeader` (`voucherType=SALE`, `voucherId=sale.getId()`, `voucherNumber=sale.getInvoiceNumber()`, `store` set if `storeId` given), adds one `JournalDetail` per resolved line (each tagged `referenceType=SALE`, `referenceId=sale.getId()`, and, for the receivable line, `partyType=CUSTOMER / partyId=customer.getId()`), saves, stamps `journalNumber`.
5. **Database:** one new `journal_headers` row (`voucher_type='SALE'`, `voucher_id=<sale.id>`) and its `journal_details` rows (e.g. a debit line on the `Customer Receivable` accounts row, credit lines on the `Sales` and `Output-GST` accounts rows) are persisted alongside the `sales` row and the `customer_ledger_entries / gst_entries` rows written by `LedgerService`, all in the one transaction `SaleService` opened.
6. **Response/UI:** `SaleService` returns its own `SaleResponse` to the Sale-creation UI (not documented here); separately, this posting becomes visible in the Accounting module's `Journals.tsx` (voucher type `SALE`, linked by `voucherNumber` = the sale's invoice number) and in `TrialBalance.tsx` / other reports once the transaction commits.

15. Manual Changes

- **Add a new Account:** use the Chart of Accounts UI (`AccountMaster.tsx` → `POST /api/accounts`) or insert via `AccountService.create` . A brand-new business account does **not** need a code change anywhere else. Only add a new `SystemAccountCode` constant (in `backend/src/main/java/com/storehub/entity/SystemAccountCode.java`) when a new fixed account the application posts to by code is needed — doing so requires: (1) adding the enum constant with its code/default name/ `AccountType/normal balance`, (2) `AccountService.ensureSystemAccountsExist()` will auto-seed it on next startup (no manual seed script needed), (3) updating whichever `AccountingService` posting method(s) should reference it. Affected: any new `AccountingService.post*Journal` method that references the new code, and — since seeding is idempotent and additive — no other module needs to change just because a code was added.
- **Add a new VoucherType:** edit `backend/src/main/java/com/storehub/entity/VoucherType.java` (currently `SALE`, `PURCHASE`, `RECEIPT`, `PAYMENT`, `JOURNAL`, `CREDIT_NOTE`, `DEBIT_NOTE`, `EXPENSE`, `CASH_TRANSACTION`). This is the accounting-journal classification stamped on `JournalHeader.voucherType / JournalDetail.referenceType` . Affected by this change: (a) the new source-transaction service must call `accountingService.postJournal(...)` / `postJournalByAccountId(...)` with the new `VoucherType` ; (b) `journalHeaderRepository` 's duplicate-posting guard (`existsByVoucherTypeAndVoucherIdAndStatusAndReversalOfJournalIsNull`) automatically covers it since it's parameterized by `VoucherType` ; (c) frontend `types/accounting.ts` 's `VoucherType` union and `Journals.tsx` 's `VOUCHER_TYPES` filter array should be extended to keep the UI filter in sync (note: as of this read, the frontend union is already missing `EXPENSE` and `CASH_TRANSACTION` , which the backend enum already has — see discrepancy below).
- **Add a new VoucherDocType** (a new independently-numbered document series): edit `backend/src/main/java/com/storehub/entity/VoucherDocType.java` , adding the enum constant with its prefix string (e.g. `MY_DOC("MD")`). No other code change is required for numbering itself — `VoucherNumberService.next(docType, date)` and `VoucherSequenceRepository.ensureRowExists/lockForUpdate` are fully generic over `VoucherDocType` ; a new counter row is created lazily on first use per `(docType, financialYearId)` . The new document-creating service must call `voucherNumberService.next(VoucherDocType.MY_DOC, date)` when assigning its number.
- **Change the voucher numbering format:** edit the single `return` statement in `VoucherNumberService.next(...)` (`docType.getPrefix() + "/" + fy.getCode() + "/" + String.format("%06d", seq.getLastNumber())`). Because every voucher-numbered module funnels through this one method, a format change here affects **every** transactional module's document numbers simultaneously (`Sale`, `Purchase`, `SalesOrder`, `PurchaseOrder`, `Receipt`, `Payment`, `CreditNote`, `DebitNote`, `Expense`, `CashTransaction`, `StockTransfer`) — there is no per-module override. Changing the digit-padding (`%06d`) or separator changes all of them at once; a module-specific prefix change instead belongs in that `VoucherDocType` constant's own `prefix` string.
- **Change the debit=credit validation:** edit `AccountingService.validateAmounts(...)` (§6). This one private method is the sole enforcement point for every posting path (`postJournal` , `postManualJournal` , `postJournalByAccountId`) — loosening or tightening it (e.g. allowing a zero-total journal, or requiring line-level party tagging) affects every caller module listed in §13 uniformly, since none of them perform their own balance check before calling in (the frontend's `JournalEntryForm.tsx` client-side check is a UX convenience only and is not authoritative — the backend re-validates independently).
- **Change AccountingService's public method signatures** (e.g. `postSaleJournal(Sale sale)` , `postJournal(...)` , `postJournalByAccountId(...)`): because these are called directly, by name, from `SaleService` , `PurchaseService` , `ReceiptService` , `PaymentService` , `ExpenseService` ,

`CashTransactionService`, `CreditNoteService`, and `DebitNoteService` (\$13), any signature change (new required parameter, renamed method, changed return type) requires updating every one of those 8 call sites. The generic `postJournal` / `postJournalByAccountId` / `reverseJournal` overloads are the lowest-risk to extend (add a new optional overload rather than changing an existing signature) since `CashTransactionService`, `CreditNoteService`, `DebitNoteService`, and `ExpenseService` already depend on their exact current shapes.

- **Frontend/backend `VoucherType` discrepancy found while documenting:** `frontend/src/types/accounting.ts`'s `VoucherType` union (`'SALE'` | `'PURCHASE'` | `'RECEIPT'` | `'PAYMENT'` | `'JOURNAL'` | `'CREDIT_NOTE'` | `'DEBIT_NOTE'`) and `Journals.tsx`'s `VOUCHER_TYPES` filter array both omit `EXPENSE` and `CASH_TRANSACTION`, which exist in the backend `VoucherType` enum and are actively posted to by `ExpenseService` / `CashTransactionService`. This does not break posting or viewing an individual `Expense` / `CashTransaction` journal (the `JournalHeader` object still carries whatever `voucherType` string the backend returns and TypeScript's structural typing does not reject it at runtime), but the `Journals` page's voucher-type filter dropdown has no option to filter specifically for `EXPENSE` or `CASH_TRANSACTION` journals.

Accounting Reports Module

1. Overview

The Accounting Reports module (spec sections 9–23 and 32, per the class-level Javadoc comments in the source) is a **pure read-only reporting/aggregation layer**. It owns **no database tables and no entities of its own** — every figure it returns is computed live, on each request, from data owned by other modules: the posted accounting journal (`JournalHeader / JournalDetail` , owned by the Accounting Core module), `Sale / Purchase` (Sales/Purchase modules), and the `Account` chart of accounts. Nothing in this module ever writes to those tables; every service method is `@Transactional(readOnly = true)` .

The controller's own class Javadoc states the design principle explicitly: "Every endpoint reads the posted accounting journal (or, for Outstanding/Ageing, the `Sale.dueAmount / Purchase.payableAmount` fields `ReceiptService / PaymentService` keep in exact sync with `ReceiptAllocation / PaymentAllocation`) — never a second, independently calculated source of truth." In other words, this module never re-derives figures `Sale/Purchase/Accounting Core` already maintain; it only reshapes and aggregates them for display.

There are 13 reports in total: Day Book, Account Ledger, Trial Balance, Cash Book, Bank Book, Party Ledger, Receivable, Payable, Outstanding (Customers/Suppliers, with ageing), Profit & Loss, Balance Sheet, Account Summary, Expense Summary, Income Summary, and Accounting Health Check — all served from one controller (`AccountingReportController`) backed by 9 focused services. (An Accounting Dashboard summary and Expense/Income Summary are also served from this controller/these services but are not part of the 12 named frontend report pages covered here in detail — see the note at the end of Section 2.)

2. Frontend

Files and Components

File	Purpose
<code>frontend/src/pages/accounting/reports/DayBook.tsx</code>	Every POSTED voucher for a date range/voucher-type filter; clicking a row opens a dialog with the full journal (via <code>journalApi.getById</code>).
<code>frontend/src/pages/accounting/reports/AccountLedger.tsx</code>	Running-balance statement for one selected <code>Account</code> , date-ranged.
<code>frontend/src/pages/accounting/TrialBalance.tsx</code>	Account-wise debit/credit balances as of a date, with a Balanced/Out-of-Balance badge. (Lives under <code>pages/accounting/</code> , not <code>pages/accounting/reports/</code> , but is the same report served by <code>AccountingReportService.trialBalance</code> .)
<code>frontend/src/pages/accounting/reports/CashBook.tsx</code>	Receipts/payments through the system <code>CASH</code> account, date-ranged.
<code>frontend/src/pages/accounting/reports/BankBook.tsx</code>	Receipts/payments through the system <code>BANK</code> account (or another bank <code>Account</code> once one exists), date-ranged.
<code>frontend/src/pages/accounting/reports/PartyLedger.tsx</code>	Running-balance statement for one Customer or Supplier; reads/writes <code>partyType / partyId</code> to the URL query string via <code>useSearchParams</code> .

File	Purpose
<code>frontend/src/pages/accounting/reports/Receivable.tsx</code>	Per-customer opening/credit-sales/receipts/notes/closing table; each row links to that customer's Party Ledger.
<code>frontend/src/pages/accounting/reports/Payable.tsx</code>	Per-supplier opening/purchases/payments/notes/closing table; each row links to that supplier's Party Ledger.
<code>frontend/src/pages/accounting/reports/Outstanding.tsx</code>	Tabbed Customers/Suppliers bill-wise outstanding with ageing-bucket summary cards; loads both reports in parallel via <code>Promise.all</code> .
<code>frontend/src/pages/accounting/reports/ProfitLoss.tsx</code>	Income vs. Expense account lines for a date range, with a Net Profit/Loss card.
<code>frontend/src/pages/accounting/reports/BalanceSheet.tsx</code>	Assets vs. Liabilities+Equity as of a date, with a Balanced/Out-of-Balance badge.
<code>frontend/src/pages/accounting/reports/AccountSummary.tsx</code>	Opening/debit/credit/closing per account, filterable by <code>AccountType</code> .
<code>frontend/src/pages/accounting/reports/HealthCheck.tsx</code>	Overall PASS/WARNING/ERROR status plus one card per check, each with a "Re-run" button.
<code>frontend/src/pages/accounting/reports/reportFormat.ts</code>	Shared helpers used by 11 of the 12 pages above: <code>money()</code> (₹, en-IN, 2 decimals), <code>todayIso()</code> , <code>firstDayOfMonthIso()</code> , <code>firstDayOfFinancialYearIso()</code> (defined but not currently called by any of these 12 pages).

Note on `TrialBalance.tsx`: it does **not** import from `reportFormat.ts` — it defines its own local `money / todayIso` functions (lines 16–17) that are functionally equivalent (same en-IN 2-decimal formatting) but duplicated rather than shared. This is the one inconsistency across the 12 pages.

Out of scope siblings (present in `pages/accounting/reports/` but not part of this module's 12 named reports per the task's ground-truth list): `ExpenseIncomeSummaryView.tsx`, `ExpenseSummary.tsx`, `IncomeSummary.tsx` (wrap the `expense-summary / income-summary` endpoints also served by `AccountSummaryService`), `ExpenseAnalysis.tsx`, and `StoreComparison.tsx` (a Multi-Store report, documented in the Store module doc).

Frontend Flow

All 12 pages follow the same generic shape:

- Filter state** — `fromDate / toDate` (defaulted via `firstDayOfMonthIso() / todayIso()`) for period reports, or a single `asOfDate` (defaulted via `todayIso()`) for point-in-time reports (Trial Balance, Balance Sheet, Outstanding). Day Book adds an optional `voucherType` `<Select>`; Account Ledger and Party Ledger add an account/party `<Select>` (Party Ledger also supports a `partyType` toggle between Customer/Supplier); Account Summary adds an optional `accountType` `<Select>`. Health Check has no filters at all — it is a re-runnable check, not a range query.
- Load** — an `async load()` function calls the matching `accountingReportApi.*` function inside a `try/catch`, showing a `toast.error(parseApiError(err, '...').message)` on failure and toggling a `loading` boolean in a `finally` block. Every page calls `load()` once on mount via `useEffect(() => { load(); }, [])`; Account Ledger and Party Ledger additionally reload whenever the selected account/party id changes. There is also an explicit "Apply" button (or, for Trial Balance, `onChange` fires the load directly) so users can re-run without changing the mount-time defaults.
- Render** — a `PageHeader` with the filter controls in its `actions` slot, a row of summary `Card`s (opening/

closing balances, totals, or ageing-bucket cards), then a `Table` of rows (or, for Profit & Loss/Balance Sheet, two side-by-side tables) with a `TableSkeleton` while loading and an `EmptyState` when the row list is empty. Day Book additionally opens a `Dialog` with full journal-line detail on row click.

Store-filter usage (Multi-Store): NOT FOUND IN CURRENT CODEBASE. None of the 12 report pages, `accountingReportApi` functions, `AccountingReportController` endpoints, or the underlying repository queries (`JournalDetailRepository`, `JournalHeaderRepository`, `SaleRepository.findAllOutstanding`, `PurchaseRepository.findAllOutstanding`) take or filter by a `storeId/store` parameter, even though `Sale`, `Purchase`, and `JournalHeader` entities do carry a `store` field elsewhere in the codebase. Every report in this module aggregates across **all stores** — the only store-scoped accounting/sales report is the separate `StoreComparison.tsx` page (Store module, out of scope here).

3. API Calls

All defined in `frontend/src/api/accountingApi.ts` under `accountingReportApi`, wrapping the shared `api` (axios) client.

Function	HTTP Method	URL	Request shape (query params)	Response shape
<code>accountingReportApi.dashboard</code>	GET	<code>/accounting/reports/dashboard</code>	—	<code>AccountingDashboardSummary</code>
<code>accountingReportApi.dayBook</code>	GET	<code>/accounting/reports/day-book</code>	<code>fromDate?</code> , <code>toDate?</code> , <code>voucherType?</code>	<code>DayBookResponse</code>
<code>accountingReportApi.accountLedger</code>	GET	<code>/accounting/reports/account-ledger/{accountId}</code>	<code>fromDate?</code> , <code>toDate?</code>	<code>AccountLedgerResponse</code>
<code>accountingReportApi.trialBalance</code>	GET	<code>/accounting/reports/trial-balance</code>	<code>asOfDate?</code>	<code>TrialBalanceResponse</code>
<code>accountingReportApi.cashBook</code>	GET	<code>/accounting/reports/cash-book</code>	<code>fromDate?</code> , <code>toDate?</code>	<code>CashBankBookResponse</code>
<code>accountingReportApi.bankBook</code>	GET	<code>/accounting/reports/bank-book</code>	<code>fromDate?</code> , <code>toDate?</code> , <code>accountId?</code>	<code>CashBankBookResponse</code>
<code>accountingReportApi.partyLedger</code>	GET	<code>/accounting/reports/party-ledger</code>	<code>partyType</code> , <code>partyId</code> , <code>fromDate?</code> , <code>toDate?</code>	<code>PartyLedgerResponse</code>
<code>accountingReportApi.receivable</code>	GET	<code>/accounting/reports/receivable</code>	<code>fromDate?</code> , <code>toDate?</code>	<code>ReceivablePayableResponse</code>
<code>accountingReportApi.payable</code>	GET	<code>/accounting/reports/payable</code>	<code>fromDate?</code> , <code>toDate?</code>	<code>ReceivablePayableResponse</code>

Function	HTTP Method	URL	Request shape (query params)	Response shape
<code>accountingReportApi.outstandingCustomers</code>	GET	<code>/accounting/reports/outstanding/customers</code>	<code>asOfDate?</code>	<code>OutstandingBillReportResponse</code>
<code>accountingReportApi.outstandingSuppliers</code>	GET	<code>/accounting/reports/outstanding/suppliers</code>	<code>asOfDate?</code>	<code>OutstandingBillReportResponse</code>
<code>accountingReportApi.profitLoss</code>	GET	<code>/accounting/reports/profit-loss</code>	<code>fromDate?, toDate?</code>	<code>ProfitLossResponse</code>
<code>accountingReportApi.balanceSheet</code>	GET	<code>/accounting/reports/balance-sheet</code>	<code>asOfDate?</code>	<code>BalanceSheetResponse</code>
<code>accountingReportApi.accountSummary</code>	GET	<code>/accounting/reports/account-summary</code>	<code>accountType?, fromDate?, toDate?</code>	<code>AccountSummaryResponse</code>
<code>accountingReportApi.expenseSummary</code>	GET	<code>/accounting/reports/expense-summary</code>	<code>fromDate?, toDate?</code>	<code>ExpenseIncomeSummaryResponse</code>
<code>accountingReportApi.incomeSummary</code>	GET	<code>/accounting/reports/income-summary</code>	<code>fromDate?, toDate?</code>	<code>ExpenseIncomeSummaryResponse</code>
<code>accountingReportApi.healthCheck</code>	GET	<code>/accounting/reports/health-check</code>	—	<code>AccountingHealthCheckResponse</code>

All dates are ISO `yyyy-MM-dd` strings (Java-side bound with `@DateTimeFormat(iso = DateTimeFormat.ISO.DATE)`). The base path prefix `/api` is added by the shared axios client (`frontend/src/api/axios.ts`) — the controller's `@RequestMapping` is `/api/accounting/reports`.

4. Backend

Controllers

Single controller: `AccountingReportController` (`@RequestMapping("/api/accounting/reports")`), 17 endpoints, all `@GetMapping`, delegating to 8 injected services (`AccountingReportService` + 7 focused services). No endpoint has a request body; every parameter is a `@RequestParam` (or `@PathVariable` `Long` `accountId` for `account-ledger`), all optional except `party-ledger`'s `partyType`/`partyId`.

Endpoint	Delegates to
GET <code>/dashboard</code>	<code>AccountingReportService.getDashboardSummary()</code>
GET <code>/day-book</code>	<code>AccountingReportService.dayBook(voucherType, fromDate, toDate)</code>

Endpoint	Delegates to
GET /account-ledger/{accountId}	AccountingReportService.accountLedger(accountId, fromDate, toDate)
GET /trial-balance	AccountingReportService.trialBalance(asOfDate)
GET /cash-book	CashBankBookService.cashBook(fromDate, toDate)
GET /bank-book	CashBankBookService.bankBook(accountId, fromDate, toDate)
GET /party-ledger	PartyLedgerService.partyLedger(partyType, partyId, fromDate, toDate)
GET /receivable	ReceivablePayableService.receivable(fromDate, toDate)
GET /payable	ReceivablePayableService.payable(fromDate, toDate)
GET /outstanding/customers	OutstandingBillService.customerOutstanding(asOfDate)
GET /outstanding/suppliers	OutstandingBillService.supplierOutstanding(asOfDate)
GET /profit-loss	ProfitLossService.profitAndLoss(fromDate, toDate)
GET /balance-sheet	BalanceSheetService.balanceSheet(asOfDate)
GET /account-summary	AccountSummaryService.accountSummary(accountType, fromDate, toDate)
GET /expense-summary	AccountSummaryService.expenseSummary(fromDate, toDate)
GET /income-summary	AccountSummaryService.incomeSummary(fromDate, toDate)
GET /health-check	AccountingHealthCheckService.runHealthCheck()

DTOs

All under `backend/src/main/java/com/storehub/dto/`, plain Lombok `@Getter @Builder @AllArgsConstructor` classes (no `@Setter`, no persistence annotations — confirming they are pure response shapes, not entities).

Response DTO	Fields	Row DTO	Row fields
DayBookResponse	rows: List<DayBookRow>, totalDebit, totalCredit	DayBookRow	journalDate, journalId, journalNumber, voucherType, voucherNumber, narration, debit, credit, status
AccountLedgerResponse	accountId, accountCode, accountName, openingBalance, openingBalanceType, rows: List<AccountLedgerRow>, closingBalance, closingBalanceType	AccountLedgerRow	journalDate, voucherType, voucherNumber, particulars, debit, credit, balance, balanceType
TrialBalanceResponse	asOfDate, rows: List<TrialBalanceRow>, totalDebit, totalCredit, difference, balanced	TrialBalanceRow	accountId, accountCode, accountName, accountType, debit, credit

Response DTO	Fields	Row DTO	Row fields
CashBankBookResponse	accountId, accountCode, accountName, fromDate, toDate, openingBalance, rows: List<CashBankBookRow>, totalReceipts, totalPayments, closingBalance	CashBankBookRow	voucherDate, voucherType, voucherNumber, particulars, receipt, payment, runningBalance
PartyLedgerResponse	partyType, partyId, partyName, fromDate, toDate, openingBalance, openingBalanceType, rows: List<PartyLedgerRow>, closingBalance, closingBalanceType	PartyLedgerRow	voucherDate, voucherType, voucherNumber, particulars, debit, credit, balance, balanceType
ReceivablePayableResponse	partyType, fromDate, toDate, rows: List<ReceivablePayableRow>, totalOpening, totalTransactions, totalPayments, totalOutstanding	ReceivablePayableRow	partyId, partyName, openingBalance, transactionAmount, paymentAmount, creditNoteAmount, debitNoteAmount, closingOutstanding
OutstandingBillReportResponse	partyType, asOfDate, ageingBasis, rows: List<OutstandingBillDetailRow>, ageingSummary: List<AgeingBucketSummary>, totalInvoiceAmount, totalReceivedOrPaid, totalOutstanding	OutstandingBillDetailRow	partyId, partyName, billId, invoiceNumber, invoiceDate, invoiceAmount, receivedOrPaidAmount, outstanding, daysOutstanding, ageingBucket; AgeingBucketSummary, bucket, count, amount
ProfitLossResponse	fromDate, toDate, incomeLines: List<PnLAccountLine>, expenseLines: List<PnLAccountLine>, totalIncome, totalExpense, netProfitOrLoss	PnLAccountLine	accountId, accountCode, accountName, amount
BalanceSheetResponse	asOfDate, assetLines: List<BalanceSheetLine>, liabilityLines: List<BalanceSheetLine>, equityLines: List<BalanceSheetLine>, totalAssets, totalLiabilities, totalEquity, currentYearProfit, currentFinancialYear, difference, balanced	BalanceSheetLine	accountId (nullable), accountCode (nullable), accountName, amount
AccountSummaryResponse	fromDate, toDate, rows: List<AccountSummaryRow>, totalOpening, totalDebit, totalCredit, totalClosing	AccountSummaryRow	accountId, accountCode, accountName, accountType, openingBalance, debit, credit, closingBalance

Response DTO	Fields	Row DTO	Row fields
ExpenseIncomeSummaryResponse	fromDate, toDate, accounts: List<ExpenseIncomeAccountGroup>, totalAmount	ExpenseIncomeAccountGroup → ExpenseIncomeVoucherRow	Group: accountId, accountCode, accountName, total, vouchers . Voucher ro voucherDate, journalId, journalNumber, voucherType, voucherNumber, narration, amount
AccountingHealthCheckResponse	generatedAt, overallStatus: HealthCheckStatus, findings: List<HealthCheckFinding>	HealthCheckFinding	checkName, status: HealthCheckStatus, message, details: List<String>
AccountingDashboardResponse	cashBalance, bankBalance, receivableBalance, payableBalance	—	—

HealthCheckStatus (backend/src/main/java/com/storehub/entity/HealthCheckStatus.java) is a 3-value enum: PASS, WARNING, ERROR .

Services

All 9 services are @Service @RequiredArgsConstructor , and every public method is @Transactional(readOnly = true) .

1. AccountingReportService — Day Book, Account Ledger, Trial Balance, Dashboard

- **Day Book** (dayBook(voucherType, fromDate, toDate)): calls JournalDetailRepository.dayBookRows(voucherType, fromDate, toDate) , which returns one row per JournalDetail line (columns: journal date, id, number, voucher type, voucher number, narration, debit, credit, status). Sums totalDebit / totalCredit by simple accumulation across all returned rows.
- **Account Ledger** (accountLedger(accountId, fromDate, toDate)): loads the Account (AccountService.findOrCreate), computes a signed opening balance — +openingBalance if openingBalanceType == DEBIT, else -openingBalance — then, if fromDate is given, adds all activity strictly before it via JournalDetailRepository.sumDebitCreditBefore(accountId, fromDate) (openingSigned += priorDebit - priorCredit). It then iterates JournalDetailRepository.findLedgerLines(accountId, fromDate, toDate) in order, maintaining a running signed balance (running = running + debitAmount - creditAmount per line) and emitting each row's balance as running.abs() with balanceType = DEBIT if running.signum() >= 0 else CREDIT .
- **Trial Balance** (trialBalance(asOfDate)): for every active Account (AccountRepository.findByActiveTrueOrderByAccountNameAsc() , then re-sorted by accountCode), computes the same "opening balance + all activity up to asOfDate " signed net (activity from JournalDetailRepository.sumDebitCreditByAccount(asOfDate) , a map keyed by account id).
Formula: netSigned = openingSigned + activityDebit - activityCredit ; then debit = netSigned > 0 ? netSigned : 0 and credit = netSigned < 0 ? -netSigned : 0 (i.e. every account's net balance is placed on exactly one side). totalDebit / totalCredit are the column sums; difference = totalDebit - totalCredit ; balanced = (totalDebit == totalCredit) (by BigDecimal.compareTo, not equals , so scale differences don't cause a false "unbalanced").

- **Dashboard** (`getDashboardSummary()`): returns `currentBalance()` (opening + all-time journal activity, same signed formula as above with no date cutoff) for the `CASH`, `BANK`, and `CUSTOMER_RECEIVABLE` system accounts, plus `SUPPLIER_PAYABLE`'s balance negated (so it reads as a positive "amount owed" figure).

2. `CashBankBookService` — Cash Book, Bank Book

Both reuse one private method, `bookForAccount(accountId, fromDate, toDate)`, which is **exactly the same opening-balance-plus-running-balance algorithm as**

`AccountingReportService.accountLedger`, reshaped into receipt/payment columns instead of debit/credit: a debit into the account is a `receipt`, a credit out of it is a `payment` (the class Javadoc states this explicitly). `cashBook()` resolves the account via

`AccountService.getSystemAccount(SystemAccountCode.CASH)`; `bankBook(accountId, ...)` uses the given `accountId` if supplied, else `SystemAccountCode.BANK`. `totalReceipts / totalPayments` are the column sums (not net); `closingBalance` is the final signed running balance (not `.abs()`, unlike Account Ledger — Cash/Bank running balance is always debit-side so no sign-flip display is needed).

3. `PartyLedgerService` — Party Ledger

A "second VIEW over the existing journal" (class Javadoc) — reads `JournalDetail` lines tagged with a `partyType / partyId` (the `CUSTOMER_RECEIVABLE / SUPPLIER_PAYABLE` control-account lines), via `JournalDetailRepository.sumByPartyBefore` (opening) and `findLedgerLinesByParty` (period rows). Same running-balance algorithm as Account Ledger, but starting opening balance is **unsigned zero** unless `fromDate` is given (no account-level `openingBalance` field exists for a party, only journal activity). Explicitly documented as not replacing the operational `CustomerLedgerEntry / SupplierLedgerEntry` sub-ledger tables from the Sales/Purchase modules — those "remain untouched and keep reconciling against this" (checked by the Health Check's "Party Ledger Match").

4. `ReceivablePayableService` — Receivable, Payable

Both call one shared `build(partyType, transactionRef, paymentRef, noteRef, creditIncreases, fromDate, toDate)` with polarity flipped: Receivable is `build(CUSTOMER, SALE, RECEIPT, CREDIT_NOTE, creditIncreases=false, ...)`, Payable is `build(SUPPLIER, PURCHASE, PAYMENT, DEBIT_NOTE, creditIncreases=true, ...)`.

Formula per party: `closingOutstanding = openingBalance + transactionAmount - paymentAmount - noteAmount`, where:

- `openingBalance` = every posted/reversed control-account line strictly before `fromDate` (via `sumByPartyBefore`), net-signed by polarity (`creditIncreases ? credit-debit : debit-credit`).
- `transactionAmount` (credit sales / purchases) and `paymentAmount` (receipts / payments) and `noteAmount` (credit/debit notes) are each read from `JournalDetailRepository.sumByPartyAndReferenceTypeRange`, one bucket per `VoucherType`, and **both sides of each bucket are netted, not just the increasing side** — a reversed Sale/Purchase posts its offsetting entry as the opposite side of the same `referenceType` bucket, so reading only the increasing side would double-count a cancelled voucher forever (explicit in the source comment).
- A Credit Note (Receivable) or Debit Note (Payable) always reduces outstanding — same polarity as a payment — so it nets the same way `paymentAmount` does; `creditNoteAmount / debitNoteAmount` in the row are just the note amount routed into whichever of the two output fields matches the report (the other stays zero, since a Sales Credit Note only ever affects Receivable and a Purchase Debit Note only ever affects Payable).

Rows are sorted by `partyName` (case-insensitive); totals are simple column sums across all parties with any opening balance or activity.

5. OutstandingBillService — Outstanding (Customers/Suppliers) with Ageing

Built directly from `Sale.dueAmount / Purchase.payableAmount` — fields `ReceiptService / PaymentService` keep in exact sync with `ReceiptAllocation / PaymentAllocation` — rather than re-deriving outstanding from the journal. `customerOutstanding(asOfDate)` reads `SaleRepository.findAllOutstanding()` (every `Sale` with a non-null customer, `dueAmount > 0`, `status = COMPLETED`); `supplierOutstanding(asOfDate)` reads `PurchaseRepository.findAllOutstanding()` (every `Purchase` with `payableAmount > 0`, `status = COMPLETED`).

Ageing formula: `daysOutstanding = ChronoUnit.DAYS.between(invoiceDate, asOfDate)` (defaulting `asOfDate` to today if not given); **ageing basis is always INVOICE_DATE**, stated explicitly in the `OutstandingBillReportResponse` Javadoc because no due-date/credit-period/credit-terms field exists anywhere on `Sale/Purchase/Receipt/Payment/allocation` entities in this codebase (confirmed absent by inspection). **Bucket boundaries** (`bucketFor`): 0–30 Days (≤ 30), 31–60 Days (≤ 60), 61–90 Days (≤ 90), 91–180 Days (≤ 180), 180+ Days (> 180). `ageingSummary` is a per-bucket count+amount rollup (`AgeingBucketSummary`), built with all 5 buckets pre-seeded at zero so empty buckets still appear. `totalInvoiceAmount / totalReceivedOrPaid / totalOutstanding` are simple sums over all rows.

6. ProfitLossService — Profit & Loss

Built **only** from accounts classified `AccountType.INCOME / AccountType.EXPENSE` in the Chart of Accounts — never from `Sale/Purchase` tables directly, and never special-cased by account name. For every active account with any activity in `[fromDate, toDate]` (via `JournalDetailRepository.sumDebitCreditByAccountRange()`): an `INCOME` account's line amount is `credit - debit`; an `EXPENSE` account's line amount is `debit - credit`. `netProfitOrLoss = totalIncome - totalExpense`. A `netProfit(fromDate, toDate)` convenience method (just the net figure) is used by Balance Sheet.

Documented simplification (class Javadoc): `SystemAccountCode.PURCHASE` is itself classified `EXPENSE` and is debited for the taxable amount on every posted `Purchase`, so every purchase is expensed immediately in the period posted — there is a Chart-of-Accounts `INVENTORY` account, but nothing in the codebase ever posts to it (confirmed by inspection), so there is **no perpetual-inventory/COGS-matching treatment**. This is called out as "a genuine simplification of the existing... accounting design, not something invented here — P&L simply reports what the ledger actually contains."

7. BalanceSheetService — Balance Sheet

Reuses the exact same per-account "opening balance + all activity up to a date" math as `AccountingReportService.trialBalance` (via `sumDebitCreditByAccount`), so `Assets/Liabilities` here are guaranteed consistent with the Trial Balance, but bucketed by `AccountType` instead of listed flat:

- `ASSET`: net-signed balance (`netSigned`) added directly to `assetLines / totalAssets` when non-zero.
- `LIABILITY`: `amount = netSigned.negate()` (liabilities are naturally credit-balance) added to `liabilityLines / totalLiabilities`.
- `INCOME / EXPENSE`: **not** shown as lines — instead accumulated into `cumulativeIncome / cumulativeExpense` (`INCOME: cumulativeIncome += netSigned.negate()`; `EXPENSE: cumulativeExpense += netSigned`) across **all-time**, not just the period.

Equity: there is no `EQUITY AccountType` in this app (confirmed — Chart of Accounts only classifies `ASSET/LIABILITY/INCOME/EXPENSE`). `equityLines` is therefore always exactly one synthetic line, "Retained Earnings (Accumulated Profit/Loss)", computed as `retainedEarnings = cumulativeIncome - cumulativeExpense` — a report-level figure, never a real `Account` row and never a posted journal entry. `totalEquity = retainedEarnings`.

Balance check formula: `difference = totalAssets - (totalLiabilities + totalEquity); balanced = (difference == 0) (BigDecimal.compareTo)`. This is the literal "Assets = Liabilities + Equity" identity.

`currentYearProfit` is a separate, informational-only figure:

`ProfitLossService.netProfit(FinancialYearUtil.startOf(asOfDate), asOfDate)` — the financial-year-to-date slice of the same P&L, distinct from the all-time `totalEquity` figure the balance check uses.

`currentFinancialYear` is a display label from `FinancialYearUtil.label(asOfDate)`.

8. AccountSummaryService — Account Summary, Expense Summary, Income Summary

- **Account Summary** (`accountSummary(accountType, fromDate, toDate)`): for every active account (optionally filtered by `accountType`), computes `openingBalance` (`opening + activity` strictly before `fromDate`, same formula as elsewhere), `debit/credit` (activity within `[fromDate, toDate]` via `sumDebitCreditByAccountRange`), and `closingBalance = openingBalance + debit - credit`. Accounts with zero opening, debit, and credit are skipped. Rows sorted by `accountCode`; four running totals (`totalOpening/Debit/Credit/Closing`) are simple sums.
- **Expense Summary / Income Summary** (`expenseSummary / incomeSummary`, both delegate to `voucherGroupedSummary(accountType, ...)`): for every active account of the given type with any activity, lists individual vouchers via `JournalDetailRepository.accountVoucherRows(accountId, fromDate, toDate)`. Per-voucher `amount = debitIsAmount ? (debit - credit) : (credit - debit)`, where `debitIsAmount = (accountType == EXPENSE)` — i.e. Expense accounts read their debit side as the "amount", Income accounts read their credit side. Zero-amount vouchers are dropped. Grouped into `ExpenseIncomeAccountGroup` per account (with an `accountTotal`), groups sorted by `accountCode`, and an overall `grandTotal` summed across all groups.

9. AccountingHealthCheckService — Accounting Health Check

Runs 7 independent checks in a fixed sequence, each returning a `HealthCheckFinding` with an explicit status — "every check reports PASS/WARNING/ERROR explicitly rather than hiding a mismatch" (class Javadoc). `overallStatus` is the worst of the 7 (`ERROR` beats `WARNING` beats `PASS`).

#	Check	Status if failing	Logic
1	Journal Balance	ERROR	<code>JournalHeaderRepository ... JournalDetailRepository.findUnbalancedJournals()</code> — any journal whose own lines don't sum <code>debit == credit</code> . Documented as "should be impossible" given posting-time validation.
2	Duplicate Posting	ERROR	<code>JournalHeaderRepository.findDuplicateActivePostings()</code> — any (<code>voucherType, voucherId</code>) pair with more than one currently-active posted journal. "Should be impossible given the duplicate-post guard."
3	Source Posting	WARNING	For every <code>COMPLETED / POSTED</code> Sale, Purchase, Receipt, and Payment (via each module's repository <code>findCompletedIds() / findAllIds()</code>), checks it has a matching active journal via <code>JournalHeaderRepository.findActivePostedVoucherIds(voucherType)</code> . Flags any source transaction with no journal.
4	Orphan Journal	WARNING	For <code>SALE/PURCHASE/RECEIPT/PAYMENT/CREDIT_NOTE/DEBIT_NOTE</code> , checks every journal's <code>voucherId</code> (via <code>JournalHeaderRepository.findDistinctVoucherIds(voucherType)</code>) still points at a row that exists in the owning module's repository. Documented as reachable legitimately — <code>SaleService.deleteSale / PurchaseService.deletePurchase</code> reverse the journal (kept for audit) and then hard-delete the source row, so a hard "delete" (not "cancel") of an old bill always surfaces here; "that is expected, not a bug."
5	Party Ledger Match	WARNING	Compares, per customer/supplier, the accounting journal's control-account net balance (<code>JournalDetailRepository.sumByParty(partyType)</code>) against the operational sub-ledger balance (<code>CustomerLedgerEntryRepository.sumByCustomer() / SupplierLedgerEntryRepository.sumBySupplier()</code> , from the Sales/Purchase modules' own parallel ledger tables). Any mismatch (<code>compareTo != 0</code>) is flagged.

#	Check	Status if failing	Logic
6	Financial Year Coverage	WARNING	<code>JournalHeaderRepository.findPostedDatesWithoutFinancialYear()</code> — any POSTED journal dated outside every defined <code>FinancialYear</code> . "Should not occur going forward" since posting always resolves a FY first; a non-empty result means pre-FY-feature data, or a later-deleted FY.
7	Note Posting	WARNING	Mirrors check 3 for <code>CreditNoteRepository.findPostedIds()</code> / <code>DebitNoteRepository.findPostedIds()</code> against <code>JournalHeaderRepository.findActivePostedVoucherIds(CREDIT_NOTE/DEBIT_NOTE)</code> .

Each `pass()` / `warning()` / `error()` builder sets `checkName`, `status`, a human-readable `message`, and a `details: List<String>` (empty on PASS, one line per offending record otherwise).

Repository

This module has no repositories of its own. Every service injects and calls repositories owned by other modules:

Repository (owning module)	Used by	Key methods called
<code>JournalDetailRepository</code> (Accounting Core)	<code>AccountingReportService</code> , <code>CashBankBookService</code> , <code>PartyLedgerService</code> , <code>ReceivablePayableService</code> , <code>ProfitLossService</code> , <code>BalanceSheetService</code> , <code>AccountSummaryService</code> , <code>AccountingHealthCheckService</code>	<code>dayBookRows</code> , <code>findLedgerLines</code> , <code>findLedgerLinesByParty</code> , <code>sumDebitCreditBefore</code> , <code>sumDebitCreditByAccount</code> , <code>sumDebitCreditByAccountRange</code> , <code>sumDebitCreditForAccount</code> , <code>sumByPartyBefore</code> , <code>sumByPartyAndReferenceTypeRange</code> , <code>sumByParty</code> , <code>accountVoucherRows</code> , <code>findUnbalancedJournals</code>
<code>JournalHeaderRepository</code> (Accounting Core)	<code>AccountingHealthCheckService</code>	<code>findDuplicateActivePostings</code> , <code>findActivePostedVoucherIds</code> , <code>findDistinctVoucherIds</code> , <code>findPostedDatesWithoutFinancialYear</code>
<code>AccountRepository</code> (Accounting Core)	<code>AccountingReportService</code> , <code>BalanceSheetService</code> , <code>AccountSummaryService</code> , <code>ProfitLossService</code>	<code>findByActiveTrueOrderByAccountNameAsc</code>
<code>SaleRepository</code> (Sales module)	<code>OutstandingBillService</code> , <code>AccountingHealthCheckService</code>	<code>findAllOutstanding</code> , <code>findCompletedIds</code> , <code>findAllIds</code>
<code>PurchaseRepository</code> (Purchase module)	<code>OutstandingBillService</code> , <code>AccountingHealthCheckService</code>	<code>findAllOutstanding</code> , <code>findCompletedIds</code> , <code>findAllIds</code>
<code>ReceiptRepository</code> (Sales/ Receipt module)	<code>AccountingHealthCheckService</code>	<code>findAllIds</code>
<code>PaymentRepository</code> (Purchase/ Payment module)	<code>AccountingHealthCheckService</code>	<code>findAllIds</code>
<code>CustomerLedgerEntryRepository</code> (Customer module)	<code>AccountingHealthCheckService</code>	<code>sumByCustomer</code>
<code>SupplierLedgerEntryRepository</code> (Supplier module)	<code>AccountingHealthCheckService</code>	<code>sumBySupplier</code>
<code>CreditNoteRepository</code> (Sales module)	<code>AccountingHealthCheckService</code>	<code>findAllIds</code> , <code>findPostedIds</code>

Repository (owning module)	Used by	Key methods called
DebitNoteRepository (Purchase module)	AccountingHealthCheckService	findAllIds , findPostedIds
CustomerRepository (Customer module)	PartyLedgerService , ReceivablePayableService	findById , findAllById
SupplierRepository (Supplier module)	PartyLedgerService , ReceivablePayableService	findById , findAllById

AccountService and AccountingReportService itself (self-injected into AccountingReportController) are the only same-package service dependencies; AccountService.findOrThrow/getSystemAccount are used to resolve Account rows and system accounts (CASH, BANK, CUSTOMER_RECEIVABLE, SUPPLIER_PAYABLE).

5. Database

Tables

This module owns no database tables. Every report reads from tables owned by other modules:

Table (via entity)	Read by
journal_header (JournalHeader)	Day Book, Account Ledger, Trial Balance, Cash Book, Bank Book, Party Ledger, Receivable, Payable, Profit & Loss, Balance Sheet, Account Summary, Expense Summary, Income Summary, Health Check
journal_detail (JournalDetail)	Same set as above (every report except Outstanding reads JournalDetail rows/aggregates)
account (Account)	Account Ledger, Trial Balance, Cash Book, Bank Book, Profit & Loss, Balance Sheet, Account Summary, Expense Summary, Income Summary
sale (Sale)	Outstanding (customers), Health Check (Source/Orphan Posting)
purchase (Purchase)	Outstanding (suppliers), Health Check (Source/Orphan Posting)
receipt (Receipt)	Health Check (Source/Orphan Posting)
payment (Payment)	Health Check (Source/Orphan Posting)
credit_note (CreditNote)	Health Check (Note/Orphan Posting)
debit_note (DebitNote)	Health Check (Note/Orphan Posting)
customer_ledger_entry (CustomerLedgerEntry)	Health Check (Party Ledger Match)
supplier_ledger_entry (SupplierLedgerEntry)	Health Check (Party Ledger Match)
customer (Customer)	Party Ledger, Receivable, Payable (name resolution)
supplier (Supplier)	Party Ledger, Payable, Receivable (name resolution)
financial_year (FinancialYear, via FinancialYearUtil/ findPostedDatesWithoutFinancialYear)	Balance Sheet (current FY label/window), Health Check (FY coverage)

Relationships

N/A — this module defines no foreign keys or JPA relationships of its own. Its only "relationship" is a **read dependency graph**: every report service reads live from JournalHeader / JournalDetail (which

itself carries FKs to `Account`, `Customer/Supplier` via `partyType/partyId`, and back to its source voucher via `voucherType/voucherId`), plus, for Outstanding, directly from `Sale/Purchase`. None of these reads are cached or materialized — every request re-aggregates from the current table state, so this module has zero data of its own to go stale.

6. Validation

There are no `@Valid`/DTO-annotated request bodies in this module (every endpoint is a `GET` with `@RequestParam/@PathVariable`), so there is no `MethodArgumentNotValidException/fieldErrors` path for these endpoints specifically (that mechanism, per `GlobalExceptionHandler`, only fires for `@Valid` request-body DTOs elsewhere in the codebase). Date parameters are bound with `@DateTimeFormat(iso = DateTimeFormat.ISO.DATE)`; a malformed date string fails Spring's own parameter-conversion and surfaces as a generic `400/500` via `GlobalExceptionHandler`'s catch-all `Exception.class` handler (not a field-level error). All date-range/`accountId/partyId` parameters are optional or defaulted server-side (e.g. `asOfDate` defaults to `LocalDate.now()` in `BalanceSheetService/OutstandingBillService`) — **there is no explicit "fromDate must be before toDate" or similar cross-field validation anywhere in this module** (NOT FOUND IN CURRENT CODEBASE). `party-ledger`'s required `partyType/partyId` missing a matching `Customer/Supplier` row throws `BadRequestException` (`PartyLedgerService.resolvePartyName`), mapped by `GlobalExceptionHandler.handleBadRequest` to a plain `400` with the exception message (no `fieldErrors` map).

7. Authentication / Authorization

Every endpoint requires an authenticated request: `SecurityConfig.filterChain` permits only `/api/auth/**` and `/actuator/health`, and applies `.anyRequest().authenticated()` to everything else, including `/api/accounting/reports/**`. Requests are authenticated via `JwtAuthenticationFilter` (JWT bearer token → `CustomUserDetailsService`), the same mechanism used app-wide.

No endpoint-level authorization: unlike most other controllers in the codebase (e.g. `ExpenseController`, which annotates every method with `@PreAuthorize("hasAuthority('PERM_EXPENSE_VIEW')")` etc.), `AccountingReportController` has **zero** `@PreAuthorize` annotations — confirmed by reading the full controller source. Authorization for these 17 endpoints is therefore reduced to "any authenticated user of any role" at the backend. The only gate that exists is on the **frontend**: every one of these routes in `App.tsx` is nested inside `<Route element={<ManagerRoute extraRoles={['ACCOUNTANT']} />}>`, and `ManagerRoute` (`frontend/src/components/ManagerRoute.tsx`) redirects to `/unauthorized` unless `user.role` is one of `ADMIN`, `STORE_MANAGER`, or `ACCOUNTANT`. This is a client-side-only check; a `SALES_USER/PURCHASE_USER/INVENTORY_USER/STAFF` with a valid JWT could call any of these 17 endpoints directly (e.g. via curl) and receive a `200` with full accounting data.

8. Permissions

`Permission.REPORT_VIEW` and `Permission.REPORT_EXPORT` (both grouped under category "Account" in `entity/Permission.java`) exist and are assigned via `RolePermissions.java`:

- **ADMIN:** holds every `Permission` (`EnumSet.allOf(Permission.class)`), including `REPORT_VIEW/REPORT_EXPORT`.
- **STORE_MANAGER:** holds every permission ADMIN does except the `USER_*/ROLE_*/PERMISSION_VIEW` security-admin tier and `FY_MANAGE` — so it retains `REPORT_VIEW/REPORT_EXPORT`.
- **ACCOUNTANT:** explicitly granted `Permission.REPORT_VIEW`, `Permission.REPORT_EXPORT` (line 73) alongside its other accounting permissions.

- SALES_USER / PURCHASE_USER / INVENTORY_USER / STAFF: not granted `REPORT_VIEW / REPORT_EXPORT` (not present in their `EnumSet.of(...)` lists).

However, as noted in Section 7, **these permissions are never actually checked against `AccountingReportController`** — no `@PreAuthorize("hasAuthority('PERM_REPORT_VIEW')")` (or similar) exists on any of its 17 methods, so `REPORT_VIEW / REPORT_EXPORT` are defined in the permission model but not wired to backend enforcement for this module. There is no dedicated "export" action (CSV/PDF/etc.) implemented anywhere in this module's controller or services — `REPORT_EXPORT` has no corresponding endpoint (NOT FOUND IN CURRENT CODEBASE).

9. Transaction Handling

Every public method across all 9 services is annotated `@Transactional(readOnly = true)`. This is purely a performance/consistency optimization for the read (single-connection snapshot across multiple repository calls within one report build, e.g. Trial Balance's per-account loop) — there is no write transaction anywhere in this module, consistent with it owning no tables.

10. Error Handling

Handled entirely by the shared `GlobalExceptionHandler (@RestControllerAdvice)`, no module-specific exception handlers:

- `BadRequestException` (thrown by `PartyLedgerService.resolvePartyName` for an unknown customer/supplier id, or an unsupported `partyType`) → 400 Bad Request, `ApiError.message` set to the exception message, no `fieldErrors`.
- Any other unhandled exception (e.g. a malformed date query param failing Spring's binder) → falls through to the catch-all `@ExceptionHandler(Exception.class)` → 500 Internal Server Error, generic message "An unexpected error occurred".
- On the frontend, every one of the 12 pages wraps its `load()` call in `try { ... } catch (err) { toast.error(parseApiError(err, '<Report Name>-specific fallback message').message); }` — a toast notification, not an inline per-field error (there are no form fields to attach errors to; these are all read-only GET reports).

11. Audit Flow

NOT FOUND IN CURRENT CODEBASE. None of the 9 services or the controller write to any audit-log table, call an audit-logging service, or reference `AuditLogController` / an audit entity. This is expected for a pure read-only reporting module — there is nothing here to audit (no create/update/delete actions), and the module's only self-check mechanism is the Accounting Health Check (Section 4.9), which is a data-integrity check, not an audit trail of user actions.

12. Important Side Effects

None — every service method is `@Transactional(readOnly = true)` and every controller method is a `@GetMapping`; no service in this module calls `.save() / .delete() / .update()` on any repository, confirmed by inspection of all 9 service files. The only "write-adjacent" behavior is `PartyLedgerService`'s frontend companion writing `partyType / partyId` into the browser's URL query string (`useSearchParams`) for deep-linking, which is a client-side navigation convenience, not a data side effect.

CSV-injection / export protections: NOT FOUND IN CURRENT CODEBASE. There is no CSV, Excel, or PDF export endpoint implemented anywhere in this module (see Section 8 — `REPORT_EXPORT` has no wired action), so there is no export-formatting code to review for CSV-injection sanitization.

13. Dependencies on Other Modules

Report	Modules/data it depends on
Day Book	Accounting Core (<code>JournalHeader</code> / <code>JournalDetail</code>)
Account Ledger	Accounting Core (<code>JournalHeader</code> / <code>JournalDetail</code> , <code>Account</code>)
Trial Balance	Accounting Core (<code>JournalHeader</code> / <code>JournalDetail</code> , <code>Account</code>)
Cash Book	Accounting Core (<code>JournalHeader</code> / <code>JournalDetail</code> , <code>Account</code> — system <code>CASH</code> account)
Bank Book	Accounting Core (<code>JournalHeader</code> / <code>JournalDetail</code> , <code>Account</code> — system <code>BANK</code> account or any manually added bank <code>Account</code>)
Party Ledger	Accounting Core (<code>JournalHeader</code> / <code>JournalDetail</code>), Customer module, Supplier module
Receivable	Accounting Core (journal control-account activity: <code>SALE/RECEIPT/CREDIT_NOTE</code> reference types), Customer module
Payable	Accounting Core (journal control-account activity: <code>PURCHASE/PAYMENT/DEBIT_NOTE</code> reference types), Supplier module
Outstanding (Customers/Suppliers)	Sales module (<code>Sale.dueAmount</code>), Purchase module (<code>Purchase.payableAmount</code>) — indirectly, Receipt/Payment modules via <code>ReceiptAllocation</code> / <code>PaymentAllocation</code> (which keep those fields in sync, but are not queried directly here)
Profit & Loss	Accounting Core (<code>JournalHeader</code> / <code>JournalDetail</code> , <code>Account</code> filtered to <code>INCOME / EXPENSE</code>)
Balance Sheet	Accounting Core (<code>JournalHeader</code> / <code>JournalDetail</code> , <code>Account</code> filtered to <code>ASSET / LIABILITY</code>), Profit & Loss (for <code>currentYearProfit</code>), Financial Year module (<code>FinancialYearUtil</code>)
Account Summary	Accounting Core (<code>JournalHeader</code> / <code>JournalDetail</code> , <code>Account</code>)
Expense Summary / Income Summary	Accounting Core (<code>JournalHeader</code> / <code>JournalDetail</code> , <code>Account</code> filtered to <code>EXPENSE / INCOME</code>)
Accounting Health Check	Accounting Core (<code>JournalHeader</code> / <code>JournalDetail</code>), Sales module (<code>Sale</code> , <code>CreditNote</code>), Purchase module (<code>Purchase</code> , <code>DebitNote</code>), Receipt module (<code>Receipt</code>), Payment module (<code>Payment</code>), Customer module (<code>CustomerLedgerEntry</code>), Supplier module (<code>SupplierLedgerEntry</code>), Financial Year module

No report in this module depends on Store/Multi-Store data (confirmed — see Section 2's "Store-filter usage" note) or on Inventory/Product data.

14. Key Operation Flows

Flow A — Trial Balance

- UI:** `frontend/src/pages/accounting/TrialBalance.tsx` mounts, defaults `asOfDate = todayIso()` , calls `load(asOfDate)` in a `useEffect` . Changing the date `<Input>` calls `handleDateChange(value)` which both sets state and immediately calls `load(value)` .
- Function:** `load()` calls `accountingReportApi.trialBalance(date)` .
- API:** `GET /api/accounting/reports/trial-balance?asOfDate=<date>` (`frontend/src/api/accountingApi.ts`).
- Controller:** `AccountingReportController.trialBalance(@RequestParam LocalDate asOfDate) → accountingReportService.trialBalance(asOfDate)` .

5. **DTO in:** none (query param only).
6. **Service:** `AccountingReportService.trialBalance(asOfDate)` —
 - a. `journalDetailRepository.sumDebitCreditByAccount(asOfDate)` → `Map<accountId, [debit, credit]>` of all activity up to `asOfDate`.
 - b. `accountRepository.findByActiveTrueOrderByAccountNameAsc()`, re-sorted by `accountCode`.
 - c. For each account: `openingSigned` from `Account.openingBalance/openingBalanceType`; `netSigned = openingSigned + activityDebit - activityCredit`; split into `debit/credit` columns by sign.
 - d. Accumulate `totalDebit/totalCredit`; compute difference and balanced.
7. **Repository (multiple):** `JournalDetailRepository` (journal activity aggregate query), `AccountRepository` (active accounts list) — both owned by the Accounting Core module, not this one.
8. **Database:** `journal_detail` (joined to `journal_header` inside the JPQL aggregate, filtered to `POSTED/REVERSED` status and `journalDate <= asOfDate`), `account`.
9. **Response:** `TrialBalanceResponse { asOfDate, rows: TrialBalanceRow[], totalDebit, totalCredit, difference, balanced }` serialized as JSON.
10. **UI:** report state updates; the table renders one row per account (`accountCode`, `accountName`, `accountType`, `debit`, `credit`), and a footer shows `Total Debit`, `Total Credit`, `Difference`, and a `Balanced/Out of Balance` `Badge` keyed off `report.balanced`.

Flow B — Accounting Health Check

1. **UI:** `frontend/src/pages/accounting/reports/HealthCheck.tsx` mounts, calls `load()` in a `useEffect` (no filters — this report always runs against current data); a "Re-run" button re-triggers the same `load()`.
2. **Function:** `load()` calls `accountingReportApi.healthCheck()`.
3. **API:** `GET /api/accounting/reports/health-check` (no params).
4. **Controller:** `AccountingReportController.healthCheck()` → `accountingHealthCheckService.runHealthCheck()`.
5. **DTO in:** none.
6. **Service:** `AccountingHealthCheckService.runHealthCheck()` runs 7 private check methods in sequence (`checkJournalBalance`, `checkDuplicatePosting`, `checkSourcePosting`, `checkOrphanJournals`, `checkLedgerMismatch`, `checkFinancialYearCoverage`, `checkNotePosting` — see Section 4.9 for each one's exact logic), each producing a `HealthCheckFinding`. `overallStatus` is derived as the worst status among all 7 (`ERROR > WARNING > PASS`).
7. **Repository (multiple):** `JournalHeaderRepository`, `JournalDetailRepository` (Accounting Core); `SaleRepository`, `CreditNoteRepository` (Sales module); `PurchaseRepository`, `DebitNoteRepository` (Purchase module); `ReceiptRepository` (Receipt module); `PaymentRepository` (Payment module); `CustomerLedgerEntryRepository` (Customer module); `SupplierLedgerEntryRepository` (Supplier module) — 10 repositories across 7 modules, none owned by this one.
8. **Database:** `journal_header`, `journal_detail`, `sale`, `purchase`, `receipt`, `payment`, `credit_note`, `debit_note`, `customer_ledger_entry`, `supplier_ledger_entry`, plus `financial_year` (via `findPostedDatesWithoutFinancialYear`).
9. **Response:** `AccountingHealthCheckResponse { generatedAt, overallStatus, findings: HealthCheckFinding[] }` serialized as JSON.
10. **UI:** an overall-status `Card` (icon + `Badge` colored by `PASS/WARNING/ERROR`) followed by one `Card` per finding, each showing its `checkName`, `status` `Badge`, `message`, and — if non-empty — a scrollable `details` list of the specific offending records (e.g. `"Journal JRN-000123 (id=45): debit=500.00 credit=450.00"`).

15. Manual Changes

To add a new report, touch every one of these files:

1. **DTO(s)** — one or more new classes in `backend/src/main/java/com/storehub/dto/` (a `<Name>Response` and, if tabular, a `<Name>Row`), following the existing `@Getter @Builder @AllArgsConstructor` pattern (see Section 4's DTO table for the template).
2. **Service method** — add a method to the most relevant of the 9 existing services (e.g. a new journal-based statement belongs in `AccountingReportService` or a new focused service if it doesn't fit an existing one's theme), annotated `@Transactional(readOnly = true)`, reusing an existing repository method where possible rather than writing a new custom query (per this module's own "never a second, independently calculated source of truth" principle).
3. **Controller endpoint** — a new `@GetMapping` in `AccountingReportController`, injecting the owning service if it's a new one, with `@RequestParam / @DateTimeFormat` for any filters.
4. **Frontend API export** — a new function on `accountingReportApi` in `frontend/src/api/accountingApi.ts`, and the matching request/response TypeScript interfaces in `frontend/src/types/accounting.ts`.
5. **Frontend page** — a new `.tsx` file in `frontend/src/pages/accounting/reports/`, following the established pattern (Section 2's "Frontend Flow"): filter state → `load()` with try/catch/toast → summary cards → table, importing `money / date` helpers from `./reportFormat`.
6. **Route** — a new `<Route path="/accounting/reports/<slug>" element=<NewReport /> />` inside the existing `<Route element={<ManagerRoute extraRoles={['ACCOUNTANT']} />}>` block in `frontend/src/App.tsx`, plus the matching `import`.
7. **Nav entry** — a new report card entry in `frontend/src/pages/accounting/AccountingHub.tsx` (the array of `{ title, description, icon, path }` cards rendered on the Accounting hub page).

To change an existing report's formula/logic: edit only the relevant private/public method inside its owning service (Section 4's per-service breakdown gives the exact formula for each) — the DTO shape and controller signature typically stay unchanged unless the change adds a new field. Because every formula here is a live re-derivation (nothing cached), a formula fix takes effect immediately on next load with no migration or backfill needed.

Fragility — this module depends on other modules' schemas without owning them. Since none of these 9 services own their data, a schema/behavior change in another module can silently break a report here without that module's own tests catching it:

- Changing `JournalDetail`'s `debitAmount / creditAmount / partyType / partyId / referenceType` columns, or `JournalHeader`'s `status / voucherType / voucherId`, breaks **8 of the 9 services** (every one except `OutstandingBillService`) — this is the highest-fragility dependency in the module.
- Changing `Account`'s `accountType`, `openingBalance / openingBalanceType`, or `active` flag changes Trial Balance, Balance Sheet, Profit & Loss, and Account/Expense/Income Summary results, since all of them branch or filter on those exact fields.
- Changing `Sale.dueAmount / Purchase.payableAmount` semantics (e.g. how `ReceiptAllocation / PaymentAllocation` keep them in sync) directly changes Outstanding's figures, since this report deliberately reuses those fields instead of re-deriving them from receipts/payments itself.
- Renaming/removing a `SystemAccountCode` (`CASH`, `BANK`, `CUSTOMER_RECEIVABLE`, `SUPPLIER_PAYABLE`, `PURCHASE`, `INVENTORY`) breaks whichever service calls `AccountService.getSystemAccount(...)` for it (Cash/Bank Book, Dashboard) or whichever service's Javadoc assumes its accounting treatment (`ProfitLossService`'s Purchase-as-expense assumption).
- Adding a due-date/credit-terms field to `Sale / Purchase` would **not** automatically change Outstanding's ageing basis — `OutstandingBillService.AGEING_BASIS` is a hardcoded string and `bucketFor()` always computes from `invoiceDate`; that would need an explicit code change to pick it up.
- Any new `VoucherType` or `AccountingPartyType` enum value needs to be added to the relevant `switch / EnumSet` in this module's services (e.g. `ReceivablePayableService`'s reference-type buckets,

AccountingHealthCheckService's per-voucher-type checks) or it will silently be excluded from these reports rather than erroring.

GST Module

1. Overview

The GST module has three layers:

- 1. **GST calculation** — `GstCalculationService.calculateLine(...)` is the single central engine that every financial module (Sale, Purchase, Expense; Kacchi variants reuse Sale/Purchase directly) must call to compute a line's taxable value, GST amount, and CGST/SGST/IGST split. Before this service existed, `SaleService`, `PurchaseService`, and `ExpenseService` each carried an independently-maintained copy of the same arithmetic; the spec explicitly calls this out as a "one central engine" requirement. The caller is responsible for subtracting discount before calling (`taxableAmount = gross - discount`) and for zeroing the rate when the parent document is `NON_GST` or the item's tax treatment is `EXEMPT/NIL_RATED/ZERO_RATED` (there is also an item-taxability-aware overload that does this zeroing itself).
- 2. **GST transaction sync (`gst_transactions`)** — `GstTransactionSyncService` is the only place that writes to the `gst_transactions` table. It is a **normalized, synced mirror/audit log of already-computed GST data** on posted Sale/Purchase/CreditNote/DebitNote/Expense records — architecturally the same pattern as `StockHistory` mirroring posted stock movements. It never recalculates tax: every amount is copied verbatim from the source transaction, which remains the single source of truth. A cancelled/reversed source transaction flips its `GstTransaction` row's `status` to `REVERSED` in place (never deleted), so GST reports (which only read `ACTIVE` rows) stop counting it while the record stays auditable — mirroring the `JournalHeader` reversal convention.
- 3. **GST reporting** — `GstReportingService` reads only from `gst_transactions` (already scoped to `ACTIVE` + synced-at-post-time eligibility) or from `SaleItem/PurchaseItem` with the same explicit filters, to produce the full statutory report suite: GSTR-1 (B2B/B2C/HSN), Purchase GST Report (with ITC eligibility), GSTR-3B summary, Output GST report, Input GST report (Purchase + Expense), HSN Summary, Tax Rate Summary, GST Liability, and Reconciliation. GST tax calculation is deliberately kept separate from GST reporting eligibility: a Kacchi transaction still gets a fully correct CGST/SGST/IGST split from `GstCalculationService`, but is structurally excluded from ever having a `GstTransaction` row (see §4 `GstReportingEligibility`), so it can never appear in any report. All these reports are explicitly documented as return **preparation aids** built from StoreHub's own data — none of it is an actual GSTN filing integration.

Business GST config vs per-store GSTIN override: `BusinessGstConfig` is a business-wide singleton (always id 1) holding the seller's own GST registration (legal name, GSTIN, PAN, address, state, pincode) — the "seller state" anchor `GstCalculationService.suggestTaxMode` compares a party's state against. Multi-Store adds a per-store override: `Store.gstin` (and related fields), resolved through `StoreService.resolveGstContext`, which returns the store's own GSTIN/state when configured, and falls back to the business-wide config otherwise (see §4/§14).

2. Frontend

Files and Components

File	Purpose
<code>frontend/src/pages/gst/GstReportsHub.tsx</code>	Landing page linking to all 9 GST reports (static <code>SECTIONS</code> array of cards).

File	Purpose
<code>frontend/src/pages/gst/Gstr1Report.tsx</code>	GSTR-1 — tabs for B2B, B2C, HSN Summary, and a permanently-empty Credit/Debit Notes tab.
<code>frontend/src/pages/gst/PurchaseGstReport.tsx</code>	Paginated Purchase GST Report with ITC-eligibility badge per row.
<code>frontend/src/pages/gst/Gstr3bSummary.tsx</code>	GSTR-3B summary (Outward Supplies / ITC / Net Liability), by return period (month picker).
<code>frontend/src/pages/gst/OutputGstReport.tsx</code>	Paginated Output GST report (Sale side), by date range.
<code>frontend/src/pages/gst/InputGstReport.tsx</code>	Paginated Input GST report (Purchase + Expense), by date range, with ITC badge.
<code>frontend/src/pages/gst/HsnSummaryReport.tsx</code>	HSN Summary — tabs for Outward/Inward, by date range.
<code>frontend/src/pages/gst/TaxRateSummaryReport.tsx</code>	Tax Rate Summary — tabs for Outward/Inward, by date range.
<code>frontend/src/pages/gst/GstLiabilityReport.tsx</code>	Output/Input/Net liability table, by return period.
<code>frontend/src/pages/gst/GstReconciliation.tsx</code>	Reconciliation table (MATCHED/MISMATCHED/MISSING/ DUPLICATE counts + row list), by date range.
<code>frontend/src/pages/masters/ BusinessGstConfigMaster.tsx</code>	View/edit form for the singleton <code>BusinessGstConfig</code> (Masters section).
<code>frontend/src/api/gstReportApi.ts</code>	Typed API client for all 9 report endpoints.
<code>frontend/src/api/mastersApi.ts</code> (<code>businessGstConfigApi</code> , lines ~204-207)	GET / PUT client for Business GST Config.
<code>frontend/src/types/gstReport.ts</code>	TypeScript interfaces mirroring every report DTO.

Frontend Flow

Viewing a report: each report page holds its own `fromDate / toDate` (default: first day of current month → today, via `firstDayOfMonthIso() / todayIso()` in `./gstFormat`) or `returnPeriod` (default: `currentReturnPeriod()`, an HTML `<input type="month">`) as local React state, fetches on mount via `useEffect`, and re-fetches on an explicit "Apply" button click (not on every keystroke). Paginated reports (Purchase, Output, Input) also carry a `page` state reset to `0` on Apply and re-fetched via a `page`-only `useEffect`. Errors go through `parseApiError(...)` and a `toast.error(...)`; there is no page-level field-error surfacing since these are all read-only GET reports.

IMPORTANT finding: none of the 9 frontend report pages actually pass a `storeId` — `gstReportApi.ts`'s param objects never include it, even though every backend report endpoint accepts an optional `storeId` query parameter and every `GstReportingService` method takes a `storeId` argument (see §3/§4). `Store.resolveViewableStoreId` on the backend therefore always resolves to `null` (all-store aggregate) unless the current user only has access to exactly one store, or has a `currentStore` set — see §4 Services. There is no store-selector UI control anywhere under `frontend/src/pages/gst/`.

Editing Business GST Config: `BusinessGstConfigMaster.tsx` loads the config via `businessGstConfigApi.get()` (any authenticated user may view) and the active `State` list. Only `user?.role === 'ADMIN'` sees an enabled form and a Save button (all inputs are `disabled={!isAdmin}` otherwise, matching the backend's `@PreAuthorize("hasAuthority('PERM_GST_CONFIG')")` gate on PUT). On submit it does a client-side required check on `legalName` only, then calls `businessGstConfigApi.update(...)`; on a 400 it surfaces `parsed.fieldErrors` per-field (`invalid={!!fieldErrors.gstin}` + inline `<p>` message) and `parsed.message` in a top-level `Alert`, per the codebase's field-error surfacing convention.

3. API Calls

Function	Method	URL	Request shape	Response shape
gstReportApi.gstr1	GET	/api/gst-reports/gstr1	fromDate?, toDate?, returnPeriod? (no storeId sent by frontend; backend accepts storeId?)	Gstr1Response
gstReportApi.purchaseGstReport	GET	/api/gst-reports/purchase	fromDate?, toDate?, returnPeriod?, page=0, size=20 (backend also accepts storeId?)	PurchaseGstReportResponse
gstReportApi.gstr3b	GET	/api/gst-reports/gstr3b	returnPeriod (required; backend also accepts storeId?)	Gstr3bResponse
gstReportApi.outputGst	GET	/api/gst-reports/output	fromDate?, toDate?, page=0, size=20 (backend also accepts storeId?)	GstReportListResponse
gstReportApi.inputGst	GET	/api/gst-reports/input	fromDate?, toDate?, page=0, size=20 (backend also accepts storeId?)	GstReportListResponse
gstReportApi.hsnSummary	GET	/api/gst-reports/hsn	fromDate?, toDate? (backend also accepts storeId?)	HsnSummaryReportResponse
gstReportApi.taxRateSummary	GET	/api/gst-reports/tax-rate	fromDate?, toDate? (backend also accepts storeId?)	TaxRateSummaryReportResponse
gstReportApi.liability	GET	/api/gst-reports/liability	returnPeriod (required; backend also accepts storeId?)	GstLiabilityResponse
gstReportApi.reconciliation	GET	/api/gst-reports/reconciliation	fromDate?, toDate? (backend also accepts storeId?)	ReconciliationResponse
businessGstConfigApi.get	GET	/api/masters/business-gst-config	none	BusinessGstConfigResponse
businessGstConfigApi.update	PUT	/api/masters/business-gst-config	BusinessGstConfigRequest body	BusinessGstConfigResponse
(Store module, used for GST context)	GET	/api/masters/stores/{id}/gst-context	path id	StoreGstContextResponse

4. Backend

Controllers

GstReportController (@RequestMapping("/api/gst-reports")) — no @PreAuthorize on any method (protected only by the global anyRequest().authenticated() rule; see §7/§8):

Endpoint	Params	Returns
GET /gstr1	fromDate?, toDate?, returnPeriod?, storeId?	Gstr1Response
GET /purchase	fromDate?, toDate?, returnPeriod?, storeId?, page=0, size=20 (sorted voucherDate ASC)	PurchaseGstReportResponse
GET /gstr3b	returnPeriod (required), storeId?	Gstr3bResponse
GET /output	fromDate?, toDate?, storeId?, page=0, size=20 (sorted voucherDate ASC)	GstReportListResponse
GET /input	fromDate?, toDate?, storeId?, page=0, size=20 (sorted voucherDate ASC)	GstReportListResponse
GET /hsn	fromDate?, toDate?, storeId?	HsnSummaryReportResponse
GET /tax-rate	fromDate?, toDate?, storeId?	TaxRateSummaryReportResponse
GET /liability	returnPeriod (required), storeId?	GstLiabilityResponse
GET /reconciliation	fromDate?, toDate?, storeId?	ReconciliationResponse

BusinessGstConfigController (@RequestMapping("/api/masters/business-gst-config")) — class comment: "Any authenticated user may VIEW the business's GST configuration (Sales/Purchase forms need it for tax-mode suggestion); only ADMIN may edit it":

Endpoint	Auth	Returns
GET	any authenticated user	BusinessGstConfigResponse
PUT	@PreAuthorize("hasAuthority('PERM_GST_CONFIG')"), @Valid @RequestBody BusinessGstConfigRequest	BusinessGstConfigResponse

StoreController (in Store module, but the one relevant endpoint) — GET /api/masters/stores/{id}/gst-context, unguarded (matches other Store GETs), calls storeService.resolveGstContext(id) → StoreGstContextResponse.

DTOs

- **BusinessGstConfigRequest** — legalName (@NotBlank, @Size(max=200)), tradeName (@Size(max=200)), gstin (no annotation — validated manually in the service via GstinValidator.isValid), pan (@Size(max=10)), address, stateId, pincode (@Size(max=10)).
- **BusinessGstConfigResponse** — id, legalName, tradeName, gstin, pan, address, stateId, stateName, stateCode, pincode, configured (boolean, true iff legalName and gstin are both non-blank), createdAt, updatedAt.
- **GstTransactionRow** — one report row built directly from a GstTransaction: gstTransactionId, sourceTransactionType, sourceTransactionId, voucherNumber, voucherDate, partyType, partyId, partyName, partyGstin, placeOfSupplyStateCode, b2b, itcEligible (Boolean, meaningful only for PURCHASE rows; null for SALE), taxableAmount, cgstAmount, sgstAmount, igstAmount, totalTax, totalValue, returnPeriod.
- **GstSummaryTotals** — taxableAmount, cgstAmount, sgstAmount, igstAmount, totalTax, totalValue, transactionCount; has a static .zero() factory used when an aggregate query returns no rows.
- **Gstr1Response** — returnPeriod, fromDate, toDate, b2bTransactions[], b2cTransactions[], creditNotes[] (always empty — no Credit/Debit Note voucher type exists structurally in the GSTR-1 aggregation path), debitNotes[] (always empty), hsnSummary[], totals.
- **PurchaseGstReportResponse** — returnPeriod, fromDate, toDate, transactions (PagedResponse<GstTransactionRow>), totals, eligibleItcTotal.

- **Gstr3bResponse** — `returnPeriod`, `outwardSupplies`, `inputTaxCredit`, `netLiability`, `note` (a fixed disclaimer string that this is not a GSTN filing integration).
- **GstReportListResponse** — shared shape for Output/Input GST report pages: `fromDate`, `toDate`, `transactions` (`PagedResponse<GstTransactionRow>`), `totals`.
- **HsnSummaryReportResponse** — `fromDate`, `toDate`, `outward[]`, `inward[]` of `HsnSummaryRow{ hsnCode, description, unit, totalQuantity, taxableAmount, cgstAmount, sgstAmount, igstAmount, totalValue }`.
- **TaxRateSummaryReportResponse** — `fromDate`, `toDate`, `outward[]`, `inward[]` of `TaxRateSummaryRow{ gstPercent, taxableAmount, cgstAmount, sgstAmount, igstAmount, totalValue }`.
- **GstLiabilityResponse** — `returnPeriod`, `outputCgst/Sgst/Igst/Total`, `inputCgst/Sgst/Igst/Total`, `netCgst/Sgst/Igst/Total`.
- **ReconciliationResponse** — `fromDate`, `toDate`, `rows[]`, `matchedCount`, `mismatchedCount`, `missingCount`, `duplicateCount`.
- **ReconciliationRow** — `sourceTransactionType`, `sourceTransactionId`, `voucherNumber`, `voucherDate`, `status` ("MATCHED"|"MISMATCHED"|"MISSING", string not enum on the backend – frontend adds "DUPLICATE" as a 4th possible type value but the backend never actually emits a per-row DUPLICATE status; duplicates are only ever a top-level `duplicateCount`), `sourceTaxableAmount`, `reportedTaxableAmount` (nullable), `sourceTotalTax`, `reportedTotalTax` (nullable), `remarks`.
- **StoreGstContextResponse** — `storeId`, `gstIn`, `legalName`, `stateCode`, `source` ("STORE" | "BUSINESS").
- **PagedResponse<T>** — generic `content[]`, `page`, `size`, `totalElements`, `totalPages` (a `.fromPage(Page<T>)` factory).

Services

GstCalculationService — the exact split logic in `calculateLine(taxableAmount, gstPercent, taxMode, taxTreatment)`:

1. `effectivePercent` = `gstPercent` (nvl'd to ZERO if null) **unless** `taxTreatment` is non-null and not TAXABLE (i.e. EXEMPT/NIL_RATED/ZERO_RATED), in which case `effectivePercent` = ZERO. `taxTreatment == null` is treated as TAXABLE (legacy/unset items are not silently zeroed).
 2. `gstAmount` = `taxableAmount * effectivePercent / 100`, rounded HALF_UP to 2 decimal places (via `BigDecimal.divide(..., 2, RoundingMode.HALF_UP)`), or ZERO if `effectivePercent.signum() <= 0`.
 3. Split: if `gstAmount.signum() > 0` and `taxMode == TaxMode.INTER_STATE` → `igst` = `gstAmount`, `cgst` = `sgst` = 0. Otherwise (INTRA_STATE, the CGST+SGST path) → `cgst` = `gstAmount / 2` rounded HALF_UP to 2 decimals, `sgst` = `gstAmount - cgst` (so any odd-paisa remainder from the halving lands on `sgst`, not lost).
 4. Returns a `LineTaxResult{ taxableAmount, gstPercent (the *effective* rate used), gstAmount, cgstAmount, sgstAmount, igstAmount, totalAmount = taxableAmount + gstAmount }`.
- A 3-arg overload (`taxTreatment` omitted) delegates to the 4-arg one with `TaxTreatment.TAXABLE`.
 - `suggestTaxMode(sellerState, partyState)` returns `null` if either is null/blank (so callers keep their existing default rather than guessing), else `INTRA_STATE` if the trimmed, case-insensitive states match, else `INTER_STATE`. This is a **suggestion only** — it never overrides an explicitly chosen tax mode (the code comment calls out exports/SEZ as legitimate cases needing a human decision).
 - Called from: `SaleService` (line ~452), `PurchaseService` (line ~401), `ExpenseService` (line ~225) — each per line-item, in its own item-application loop.

GstReportingEligibility — the single source of truth for "does this transaction belong in GST return reporting" (kept deliberately separate from tax calculation):

- `Sale` — eligible iff `gstReportingApplicable == TRUE` **AND** `status == SaleStatus.COMPLETED`.
- `Purchase` — eligible iff `gstReportingApplicable == TRUE` **AND** `status == PurchaseStatus.COMPLETED`.
- `CreditNote` — eligible iff `gstReportingApplicable == TRUE` (copied verbatim from the source Sale at note creation, never re-derived) **AND** `status == NoteStatus.POSTED`.
- `DebitNote` — eligible iff `gstReportingApplicable == TRUE` **AND** `status == NoteStatus.POSTED`.

- Expense — eligible iff `taxMode != null` (GST was actually applied) **AND** `status == ExpenseStatus.POSTED`. (`itcEligible` on the expense controls whether it's counted as ITC in reports — see `GstTransactionSyncService.syncExpense` — it does not gate reporting inclusion itself.)
- Explicitly never inferred from `taxAmount > 0` or `transactionType == SALE` alone: a Kacchi transaction (`SALE_CHALLAN/PURCHASE_CHALLAN`) calculates GST in full via `GstCalculationService` but must never appear in reporting; a DRAFT transaction — Kacchi or not — has no accounting/stock effects yet and must never appear either.

GstTransactionSyncService — the ONLY writer to `gst_transactions`. Every sync/reverse pair:

Sync method	Reverse method	Guard (no-op if false)
<code>syncSale(Sale)</code>	<code>reverseSale(Sale)</code>	<code>GstReportingEligibility.isEligibleForGstReporting(s</code>
<code>syncPurchase(Purchase)</code>	<code>reversePurchase(Purchase)</code>	eligibility check on the purchase
<code>syncCreditNote(CreditNote)</code>	<code>reverseCreditNote(CreditNote)</code>	eligibility check on the note
<code>syncDebitNote(DebitNote)</code>	<code>reverseDebitNote(DebitNote)</code>	eligibility check on the note
<code>syncExpense(Expense)</code>	<code>reverseExpense(Expense)</code>	eligibility check on the expense

Sync-method mechanics (identical shape across all five):

1. `findOrCreate(voucherType, sourceId)` — looks up an existing `GstTransaction` by `(sourceTransactionType, sourceTransactionId)` (the row is unique on that pair), or builds a fresh one. This makes re-syncing idempotent — editing and re-posting a transaction refreshes its existing row rather than creating a duplicate.
2. Copies voucher number/date, party type/id/name, the party's GSTIN, `placeOfSupplyStateCode = GstinValidator.extractStateCode(gstin)`, and `b2b = GstinValidator.isValid(gstin)` — **except** `syncExpense`, where `b2b` is instead set directly to `expense.isItcEligible()` (the expense's own explicit ITC flag, not a GSTIN-validity proxy), so ITC-side reports only ever include what the user explicitly marked eligible.
3. Copies amounts verbatim via `applyAmounts(...): taxableAmount, cgstAmount, sgstAmount, igstAmount` (each null-coalesced to `ZERO`), `totalTax = cgst+sgst+igst`, `totalValue`.
4. Sets `returnPeriod = voucherDate.format("yyyy-MM")`, `status = ACTIVE`, `createdBy = current user's full name` (or `"System"` if no authenticated principal), then `save(txn)`.

Reverse-method mechanics (identical shape across all five): look up the existing row by `(sourceTransactionType, sourceTransactionId)`; if present, flip `status` to `REVERSED` and save; if absent (never-synced, e.g. Kacchi/ineligible), it's a no-op.

GstReportingService — 9 public report methods, all `@Transactional(readOnly = true)`, all first resolve `resolvedStoreId = storeAccessService.resolveViewableStoreId(currentUser, storeId)`:

1. `gstr1(fromDate, toDate, returnPeriod, storeId)` — pulls all `ACTIVE SALE` `GstTransaction`s via `findActiveForSummary`, splits into `b2bTransactions/b2cTransactions` by each row's `isB2b()`, adds HSN summary from `saleItemRepository.hsnSummary(...)`, and totals via

`gstTransactionRepository.aggregateTotals(SALE, ...)`. `creditNotes/debitNotes` are always empty lists (no such voucher type feeds this aggregation).

2. `taxRateSummary(fromDate, toDate, storeId)` — outward from `saleItemRepository.taxRateSummary(...)`, inward from `purchaseItemRepository.taxRateSummary(...)` — reads line-item data directly, not `gst_transactions`.
3. `hsnSummary(fromDate, toDate, storeId)` — same shape, `saleItemRepository.hsnSummary(...)` / `purchaseItemRepository.hsnSummary(...)`.
4. `purchaseGstReport(fromDate, toDate, returnPeriod, storeId, pageable)` — paginated `gstTransactionRepository.search(PURCHASE, ...)`, each row mapped with `itcEligible = txn.isB2b()`; totals via `aggregateTotals(PURCHASE, ...)`; `eligibleItcTotal` via `aggregateB2bTotals(PURCHASE, ...).getTotalTax()`.
5. `outputGstReport(fromDate, toDate, storeId, pageable)` — paginated `search(SALE, null, returnPeriod, ...)`, `itcEligible` left null on every row (not meaningful for sales); totals via `aggregateTotals(SALE, ...)`.
6. `inputGstReport(fromDate, toDate, storeId, pageable)` — paginated `searchByTypes([PURCHASE, EXPENSE], ...)` (the "generic ITC-side view" combining both voucher types, unlike `purchaseGstReport` which is Purchase-only); `itcEligible = txn.isB2b()`; totals via `aggregateTotalsByTypes([PURCHASE, EXPENSE], ...)`.
7. `gstr3bSummary(returnPeriod, storeId)` — outward = `aggregateTotals(SALE, returnPeriod, null, null, ...)`, `itc = aggregateB2bTotalsByTypes([PURCHASE, EXPENSE], returnPeriod, ...)`, `net = netOf(outward, itc)` (component-wise subtraction, `transactionCount` taken from outward's count).
8. `gstLiability(returnPeriod, storeId)` — same underlying outward/input aggregates as `gstr3bSummary`, laid out as explicit `outputCgst/Sgst/Igst/Total`, `inputCgst/Sgst/Igst/Total`, and `netCgst/Sgst/Igst/Total = output - input` per component.
9. `reconciliation(fromDate, toDate, storeId)` — for each of SALE and PURCHASE: pulls every source-side transaction eligible for reconciliation (`saleRepository.findEligibleForReconciliation` / `purchaseRepository.findEligibleForReconciliation` — raw `Object[]` rows of `id/voucherNumber/voucherDate/taxableAmount/cgst/sgst/igst`) and every currently-`ACTIVE` `GstTransaction` of that type, then for each source row: **MISSING** if no active `GstTransaction` row exists for that source id ("Eligible for GST reporting but no active `GstTransaction` row was found — re-sync needed"); **MATCHED** if `taxableAmount` and `totalTax` both compare equal (`BigDecimal.compareTo == 0`) to the reported row; **MISMATCHED** otherwise ("Amounts on the source transaction differ from the synced GST reporting row"). `duplicateCount` is computed separately via `countActiveBySource` (groups `ACTIVE` rows by `sourceTransactionId`, counts groups with `>1` row) — documented as a **defensive check only**, since the DB `uk_gst_transaction_source` unique constraint on (`source_transaction_type`, `source_transaction_id`) makes a true duplicate structurally impossible.

BusinessGstConfigService — singleton row always at id 1 (`findOrCreate()`):

- `get()` — safe to call before any ADMIN has configured anything; returns an empty/unconfigured row rather than 404, so callers like the suggest-tax-mode flow never need a special branch.
- `update(request)` — validates `gstIn` via `GstinValidator.isValid` if non-blank (throws `BadRequestException("GSTIN '<value>' is not a valid 15-character GSTIN")` otherwise), resolves `State` via `stateService.findOrThrow(stateId)` if `stateId` given, blank-to-null normalizes `tradeName/gstin/pan/address/pincode`, saves, and logs an audit entry (see §11).

StoreService.resolveGstContext(Long id) (the only method in scope from this file): finds the `Store`, and if `StringUtils.hasText(store.getGstin())`, returns a `StoreGstContextResponse` built from the store's own `gstIn/legalName` (fallback to `storeName`)/`state.code`, `source = "STORE"`. Otherwise calls `businessGstConfigService.get()` and returns a response built from the business config's `gstIn/legalName` / `stateCode`, `source = "BUSINESS"`.

Repository

GstTransactionRepository :

- `findBySourceTransactionTypeAndSourceTransactionId(type, id) → Optional<GstTransaction>` — the sync/reverse lookup key.
- `findAllBySourceTransactionTypeAndSourceTransactionId(type, id) → List<GstTransaction>`.
- `search(type, returnPeriod, fromDate, toDate, b2b, storeId, pageable) → Page<GstTransaction>` — ACTIVE only; **storeId-filtered** (`:storeId IS NULL OR g.store.id = :storeId`).
- `findActiveForSummary(type, returnPeriod, fromDate, toDate, storeId) → List<GstTransaction>` — same filter set as `search` minus `b2b/paging`; **storeId-filtered**.
- `aggregateTotals(type, returnPeriod, fromDate, toDate, storeId) → List<Object[]>` (single summed row); **storeId-filtered**.
- `aggregateB2bTotals(type, returnPeriod, fromDate, toDate, storeId) → same shape restricted to b2b = true`; **storeId-filtered**.
- `searchByTypes(types, returnPeriod, fromDate, toDate, storeId, pageable) → Page<GstTransaction>`, `sourceTransactionType IN :types`; **storeId-filtered**.
- `aggregateTotalsByTypes(types, returnPeriod, fromDate, toDate, storeId) → summed row across a type set`; **storeId-filtered**.
- `aggregateB2bTotalsByTypes(types, returnPeriod, fromDate, toDate, storeId) → same, b2b = true only`; **storeId-filtered**.
- `countActiveBySource(type) → List<Object[]>` of (`sourceTransactionId`, `count`) for duplicate detection — **not** storeId-filtered (reconciliation duplicate check is deliberately store-agnostic).
- `findBySourceTransactionTypeAndStatus(type, status) → List<GstTransaction>` — used by reconciliation to build the `activeById` lookup map.

BusinessGstConfigRepository — plain `JpaRepository<BusinessGstConfig, Long>`, no custom methods.

5. Database

Tables

- **gst_transactions** — backs `GstTransaction`. Unique constraint `uk_gst_transaction_source` on (`source_transaction_type`, `source_transaction_id`). Columns: `id`, `source_transaction_type`, `source_transaction_id`, `store_id` (FK, nullable – null for pre-migration data), `voucher_number`, `voucher_date`, `party_type`, `party_id`, `party_name`, `party_gstin`, `place_of_supply_state_code`, `b2b`, `taxable_amount`, `cgst_amount`, `sgst_amount`, `igst_amount`, `total_tax`, `total_value`, `return_period`, `status`, `created_by`, `created_at`, `updated_at`.
- **business_gst_config** — backs `BusinessGstConfig`. Singleton (`id` always 1, no auto-generation). Columns: `id`, `legal_name`, `trade_name`, `gstin`, `pan`, `address`, `state_id` (FK), `pincode`, `created_at`, `updated_at`.

Relationships

- `GstTransaction.sourceTransactionType` (enum `VoucherType`: `SALE/PURCHASE/CREDIT_NOTE/DEBIT_NOTE/EXPENSE`) + `sourceTransactionId` together identify the source `Sale/Purchase/CreditNote/DebitNote/Expense` row — a polymorphic reference by convention, not a JPA foreign key/join (no `@ManyToOne` to any of those entities).
- `GstTransaction.store` — `@ManyToOne` (lazy) to `Store`, nullable (Multi-Store; null for pre-migration data).
- `BusinessGstConfig.state` — `@ManyToOne` (lazy) to `State` (the state's own `code` is exposed as the business's GST state code — no duplicated state-code column).

- `Store.gstin/state` fields (Store module) are the per-store override consulted by `StoreService.resolveGstContext` before falling back to `BusinessGstConfig`.

6. Validation

GstinValidator (`backend/src/main/java/com/storehub/util/GstinValidator.java`) — the single reusable GSTIN validator for the whole app:

```
private static final Pattern GSTIN_PATTERN =
    Pattern.compile("[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$");
```

`isValid(gstin)` trims and uppercases before matching (null → false). A 15-character GSTIN is: 2-digit state code + 10-character PAN + 1-digit entity code + literal `Z` + 1 checksum character.

`extractStateCode(gstin)` returns the first 2 characters of a valid GSTIN (its embedded state code) or null if invalid — used to populate `GstTransaction.placeOfSupplyStateCode` and to classify a party as B2B/B2C.

`BusinessGstConfigService.update(...)` calls `GstinValidator.isValid(...)` on a non-blank `gstin` and throws `BadRequestException` with the exact message "GSTIN '<value>' is not a valid 15-character GSTIN" if it fails. `BusinessGstConfigRequest.gstin` itself carries no Bean Validation annotation — the format check happens in the service, not via `@Pattern`.

`BusinessGstConfigRequest` bean validation: `legalName @NotBlank + @Size(max=200)`; `tradeName @Size(max=200)`; `pan @Size(max=10)`; `pincode @Size(max=10)`.

7. Authentication / Authorization

- All `/api/gst-reports/**` endpoints and `GET /api/masters/business-gst-config` require only a valid authenticated session (global rule `anyRequest().authenticated()` in `SecurityConfig`, `/api/auth/**` and `/actuator/health` are the only `permitAll()` paths) — **no** `@PreAuthorize` gate is present on any `GstReportController` method or on the config `GET`.
- `PUT /api/masters/business-gst-config` requires `@PreAuthorize("hasAuthority('PERM_GST_CONFIG')")`.
- `GET /api/masters/stores/{id}/gst-context` requires only authentication (no `@PreAuthorize`).

8. Permissions

From `RolePermissions.java` / `Permission.java` (module tag "GST"): `GST_VIEW`, `GST_REPORT`, `GST_EXPORT`, `GST_CONFIG`.

- `ADMIN` holds all four (holds every `Permission`).
- `STORE_MANAGER` holds `GST_VIEW`/`GST_REPORT`/`GST_EXPORT` (inherited from the admin set) but **not** `GST_CONFIG` — explicitly excluded by name in `RolePermissions.build()` with the comment "GST config ... is system-level ... pre-existing controllers already restrict these to ADMIN only, so `STORE_MANAGER` never had them."
- `ACCOUNTANT` holds all four: `GST_VIEW`, `GST_REPORT`, `GST_EXPORT`, `GST_CONFIG`.
- `SALES_USER`, `PURCHASE_USER`, `INVENTORY_USER` — none of the four GST permissions appear in their permission sets (not read in full here beyond the grep hits; confirmed absent from the two role blocks read).

Important: none of `GST_VIEW`/`GST_REPORT`/`GST_EXPORT` is actually enforced by any `@PreAuthorize` check anywhere in the codebase (grepped for `PERM_GST` across the whole backend — the only hit is

PERM_GST_CONFIG on the config PUT). They exist in the permission model/role map but the report endpoints themselves are open to any authenticated user regardless of these three permissions.

9. Transaction Handling

- All 9 `GstReportingService` report methods are `@Transactional(readOnly = true)`.
- `BusinessGstConfigService.update(...)` is `@Transactional`.
- Every `GstTransactionSyncService` sync/reverse method is individually `@Transactional`. Because each is called synchronously from within the caller's own posting/reversal method (e.g. `SaleService.applyPostingEffects()`), it participates in — and is covered by — that caller's own transaction boundary (Spring's default `REQUIRED` propagation), so a GST sync failure rolls back the whole Sale/Purchase/Note/Expense posting operation, and vice versa. No explicit `@Transactional(propagation = ...)` override was found on any sync method.

10. Error Handling

- `MethodArgumentNotValidException` (bean-validation failures, e.g. `BusinessGstConfigRequest.legalName` blank) → `GlobalExceptionHandler.handleValidation` → HTTP 400 with `ApiError.fieldErrors` populated per DTO field name.
- `BadRequestException` (e.g. `BusinessGstConfigService.update`'s manual GSTIN-format check, or `StoreAccessService.resolveViewableStoreId`'s "Please select a store"/"You do not have access to any store") → `GlobalExceptionHandler.handleBadRequest` → HTTP 400, `message = ex.getMessage()`, `fieldErrors = null` (this path does **not** populate per-field errors — only the top-level message).
- `AccessDeniedException` (a `@PreAuthorize` failure, e.g. a non-ADMIN hitting `PUT /business-gst-config`) → HTTP 403, generic message "You do not have permission to access this resource".
- Any other unhandled exception → `GlobalExceptionHandler.handleGeneral` → HTTP 500, generic message, full exception logged server-side.
- `MasterNotFoundException` (e.g. `StoreService.findOrThrow` inside `resolveGstContext` for an unknown store id) — handled by the generic Master-not-found mapping in `GlobalExceptionHandler` (not itself part of the GST module's own code, standard 404 convention shared across Masters).

11. Audit Flow

- `BusinessGstConfigService.update(...)` is the **only** audit call site in this module:
`auditService.log(AuditAction.UPDATE, "GST_CONFIG", "BusinessGstConfig", saved.getId(), null, oldGstin, saved.getGstin(), "Business GST configuration updated")` — logs the module as "GST_CONFIG", entity type "BusinessGstConfig", entity id 1 (the singleton), `documentNumber = null`, `oldValue/newValue` = the GSTIN before/after the update, and a fixed description. No `storeId` is passed (this overload defaults it to `null` — a global/system-level action, consistent with the config being business-wide, not per-store).
- No audit call exists inside `GstTransactionSyncService` — sync/reverse of `gst_transactions` rows is not itself separately audited; it is a downstream side effect of the Sale/Purchase/Note/Expense posting action, which is audited by its own owning service.
- No audit call exists inside `GstReportingService` (read-only reports are not audited).

12. Important Side Effects

Every sync/reverse call is an automatic side effect of posting or cancelling a transaction in another module — none of it is user-visible or separately triggered. Complete list of call sites found via `grep "gstTransactionSyncService\" across backend/src/main/java/com/storehub/service/ :`

- **SaleService.java**: `syncSale` in `applyPostingEffects(Sale saved)` (line 220, called on initial post) and again after an edit-and-repost (line 303); `reverseSale` in the sale-cancel flow (line 270) and in `reverseSaleEffects(Sale sale, String reason)` (line 378).
- **PurchaseService.java**: `syncPurchase` in `applyPostingEffects(Purchase saved)` (line 187) and after edit-and-repost (line 265); `reversePurchase` on cancel (line 230) and in `reversePurchaseEffects(Purchase purchase, String reason)` (line 333).
- **CreditNoteService.java**: `syncCreditNote` on note post (line 233); `reverseCreditNote` on note cancel (line 266).
- **DebitNoteService.java**: `syncDebitNote` on note post (line 222); `reverseDebitNote` on note cancel (line 254).
- **ExpenseService.java**: `syncExpense` on expense post (line 302); `reverseExpense` on expense cancel (line 328).

Every call site is a `private void applyPostingEffects(...)` / `private void reverse*Effects(...)` -style internal method, always invoked from the owning service's own create/post/cancel/edit public methods — GST sync is never invoked directly by a controller.

13. Dependencies on Other Modules

- **Sale** — `SaleService` calls `GstCalculationService.calculateLine` per line item, and `GstTransactionSyncService.syncSale` / `reverseSale` on post/cancel/repost. Source of `taxableAmount`, `cgstAmount`, `sgstAmount`, `igstAmount`, `totalAmount`, `customerGstin`, `saleDate`, `invoiceNumber`, `gstReportingApplicable`, `status`.
- **Purchase** — `PurchaseService` calls `GstCalculationService.calculateLine` per line item, and `GstTransactionSyncService.syncPurchase` / `reversePurchase`. Source of the equivalent purchase-side fields plus `supplierGstin`.
- **Credit Note** — `CreditNoteService` calls `GstTransactionSyncService.syncCreditNote` / `reverseCreditNote`; `gstReportingApplicable` is copied verbatim from the source `Sale`.
- **Debit Note** — `DebitNoteService` calls `GstTransactionSyncService.syncDebitNote` / `reverseDebitNote`.
- **Expense** — `ExpenseService` calls `GstCalculationService.calculateLine` and `GstTransactionSyncService.syncExpense` / `reverseExpense`; supplies its own `itcEligible` flag (used as `GstTransaction.b2b` instead of a GSTIN-validity proxy).
- **Store (Multi-Store)** — `StoreService.resolveGstContext` reads `Store.gstin/state` as an override before falling back to `BusinessGstConfig`; `GstTransaction.store` and every `GstTransactionRepository` query/aggregate accept a `storeId` filter resolved via `StoreAccessService.resolveViewableStoreId`.
- **Party / Customer / Supplier** — `Sale.customer` / `Purchase.supplier` (and their GSTIN/state) are the source of the party name, GSTIN, and — via `GstinValidator.extractStateCode` — the place-of-supply state code used to determine B2B/B2C classification and, upstream at billing time, `GstCalculationService.suggestTaxMode`'s CGST+SGST vs IGST suggestion.
- **State** — both `BusinessGstConfig.state` and party addresses ultimately resolve to the shared `State` master for state-code comparison.

14. Key Operation Flows

A. Sale GST calculation → sync → GSTR-1 reporting (end-to-end):

1. User posts a Sale in the Sales module. `SaleService`'s line-item application loop calls `gstCalculationService.calculateLine(taxableAmount, gstPercent, taxMode, taxTreatment)` per `SaleItem` (line ~452), which returns the CGST/SGST/IGST split described in §4, and the totals are stored on the `Sale` entity (`taxableAmount`, `cgstAmount`, `sgstAmount`, `igstAmount`, `totalAmount`) together with `gstReportingApplicable` (set elsewhere in `Sale`'s own business rules, e.g. false for a Kacchi Sale Challan) and `status`.
2. `SaleService.applyPostingEffects(saved)` (line 220) calls `gstTransactionSyncService.syncSale(saved)`. `syncSale` checks `GstReportingEligibility.isEligibleForGstReporting(sale)` (`gstReportingApplicable == TRUE` && `status == COMPLETED`) — if false (Kacchi or not-yet-posted), it's a no-op and no `GstTransaction` row is ever created. If eligible, it finds-or-creates the row keyed on (`SALE`, `sale.getId()`), copies party/GSTIN/place-of-supply/amounts verbatim, sets `b2b = GstinValidator.isValid(customerGstin)`, `returnPeriod = saleDate.format("yyyy-MM")`, `status = ACTIVE`, and saves.
3. Later, a user opens GSTR-1 (`Gstr1Report.tsx` → `gstReportApi.gstr1(...)` → `GET /api/gst-reports/gstr1`). `GstReportingService.gstr1(...)` calls `gstTransactionRepository.findActiveForSummary(SALE, returnPeriod, fromDate, toDate, resolvedStoreId)`, splits the rows into `b2bTransactions` / `b2cTransactions` by each row's stored `b2b` flag, adds an HSN summary from `saleItemRepository.hsnSummary(...)`, and totals from `aggregateTotals(SALE, ...)` — all read straight from the `gst_transactions` mirror, never recomputed from the live `Sale` / `SaleItem` rows.
4. If the sale is later cancelled, `SaleService`'s cancel flow calls `gstTransactionSyncService.reverseSale(sale)`, which flips the existing row's `status` to `REVERSED` in place — it then silently drops out of every report above (`findActiveForSummary` / `search` / `aggregate*` all filter `status = ACTIVE`), while the row itself remains in the table for audit/reconciliation purposes.

B. resolveGstContext flow (store GSTIN override):

1. A billing form (Sale/Purchase UI, store-aware) requests `GET /api/masters/stores/{id}/gst-context`.
2. `StoreController.gstContext(id)` calls `storeService.resolveGstContext(id)`.
3. `StoreService.resolveGstContext` loads the `Store` via `findOrThrow`. If `StringUtils.hasText(store.getGstin())` (the store has its own GSTIN configured), it returns `StoreGstContextResponse{ gstin: store.gstin, legalName: store.legalName ?? store.storeName, stateCode: store.state?.code, source: "STORE" }`.
4. Otherwise it calls `businessGstConfigService.get()` and returns `StoreGstContextResponse{ gstin: business.gstin, legalName: business.legalName, stateCode: business.stateCode, source: "BUSINESS" }` — the business-wide singleton fallback.
5. The frontend uses `source` to label which anchor was used (store-level vs business-wide) when suggesting the seller line/tax mode for that store's transactions.

15. Manual Changes

- **To change the GST calculation/split logic** (rate application, HALF_UP rounding, CGST/SGST vs IGST split): edit `GstCalculationService.calculateLine` in `backend/src/main/java/com/storehub/service/GstCalculationService.java`. This is the single central engine — changing it affects every caller: `SaleService`, `PurchaseService`, `ExpenseService` (and transitively any Kacchi variant, since those reuse `Sale`/`Purchase` directly).
- **To change which transactions are GST-eligible for reporting** (e.g. adding a new eligible status, or a new source transaction type): edit `GstReportingEligibility` in `backend/src/main/java/com/storehub/service/GstReportingEligibility.java`. A new source type also requires a new `sync` / `reverse` method pair in `GstTransactionSyncService`, a new call site in that module's own `post` /

cancel service methods, and (if it should aggregate anywhere) inclusion in the relevant `GstTransactionRepository` type list (e.g. `INPUT_GST_TYPES` in `GstReportingService`).

- **To add a new GST report:** add a method to `GstReportingService` (reading only `ACTIVE` `GstTransaction` rows or `SaleItem`/`PurchaseItem` with equivalent explicit filters — never re-derive eligibility ad hoc), a new DTO in `backend/src/main/java/com/storehub/dto/`, a new endpoint in `GstReportController`, a matching function in `frontend/src/api/gstReportApi.ts`, a type in `frontend/src/types/gstReport.ts`, a new page under `frontend/src/pages/gst/`, and a card entry in `GstReportsHub.tsx`'s `SECTIONS` array.
- **To change the GSTIN validation regex:** edit `GSTIN_PATTERN` in `backend/src/main/java/com/storehub/util/GstinValidator.java`. This is the single reusable validator — changing it affects GSTIN format checks in `BusinessGstConfigService.update`, and B2B/B2C classification plus place-of-supply state-code extraction in every `GstTransactionSyncService.sync*` method (`isValid`/`extractStateCode` are both called there for `Sale`/`Purchase`/`CreditNote`/`DebitNote`/`Expense`).
- **Modules affected by any `GstCalculationService` change:** `SaleService`, `PurchaseService`, `ExpenseService`.
- **Modules affected by any `GstTransactionSyncService` change:** `SaleService`, `PurchaseService`, `CreditNoteService`, `DebitNoteService`, `ExpenseService` (every module listed in §12/§13 that calls a `sync/reverse` method).
- **Frontend note:** if per-store GST report filtering is ever required in the UI, the backend already supports it end-to-end (`storeId` query param on every `GstReportController` endpoint, `storeId`-filtered repository queries) — only the frontend `gstReportApi.ts` param objects and each report page's UI would need a store selector added (see §2 IMPORTANT finding).

Expense, Cash Management, Day Closing, Payment Method, Financial Year & Audit Trail Module

1. Overview

Expense — records operational business expenses (rent, electricity, transport, etc.). `DRAFT` has zero accounting effect; `POST` debits the Expense account (category's linked account, or the generic `SystemAccountCode.EXPENSES`) plus Input GST accounts when ITC-eligible, and credits either Cash/Bank or Supplier Payable (for a credit/party expense); `CANCEL` reverses the journal (idempotent). Backed by `backend/src/main/java/com/storehub/entity/Expense.java`, `backend/src/main/java/com/storehub/service/ExpenseService.java`.

Cash Management — ad-hoc Cash In / Cash Out entries for money that moves through Cash/Bank without a customer/supplier bill (petty cash, other income, owner's contribution). Posts through the same `AccountingService.postJournal` every other voucher uses; the existing Cash Book/Bank Book (`CashBankBookService`) reads straight off the journal — no separate balance is kept in `CashTransaction` itself. Backed by `backend/src/main/java/com/storehub/entity/CashTransaction.java`, `backend/src/main/java/com/storehub/service/CashTransactionService.java`.

Day Closing — one record per calendar date, computing expected cash (read live from `CashBankBookService.cashBook`, never independently calculated) versus the actual cash counted by the operator; requires an explicit reason when they differ. It is a record only — closing a day never adjusts accounting balances; a real shortage/overage needs an explicit `CashTransaction` (Cash Adjustment) if the business wants it reflected in the ledger. Backed by `backend/src/main/java/com/storehub/entity/DayClosing.java`, `backend/src/main/java/com/storehub/service/DayClosingService.java`.

Payment Method — a configurable, orderable master list of payment options a store accepts (Cash, Bank Transfer, UPI, Card, ...). Deliberately reference/configuration data only: it labels and orders the existing `PaymentMode` enum values, which is what every Sale/Purchase/Receipt/Payment/Expense actually posts against via `AccountingService.resolveCashOrBank`; adding a Payment Method row does not rewire that posting logic. Backed by `backend/src/main/java/com/storehub/entity/PaymentMethod.java`, `backend/src/main/java/com/storehub/service/PaymentMethodService.java`.

Financial Year — an Indian financial year (1-Apr to 31-Mar, e.g. code "26-27"). The single source of truth for "which FY does this date belong to, and is it OPEN". Every posting path resolves and gates on this before posting a journal. A `@PostConstruct` startup hook auto-seeds a `FinancialYear` row covering "today" if none exists. Backed by `backend/src/main/java/com/storehub/entity/FinancialYear.java`, `backend/src/main/java/com/storehub/service/FinancialYearService.java`.

Audit Trail — an append-only log of every important business action across the whole application (who did what, when, from where). `AuditService.log(...)` is the one place every audited action is recorded; it is a CENTRAL/COMMON service used by nearly every module (18 service files call it), not exclusive to this module — see section 11. Backed by `backend/src/main/java/com/storehub/entity/AuditLog.java`, `backend/src/main/java/com/storehub/service/AuditService.java`.

2. Frontend

Files and Components

Sub-area	File	Purpose
Expense	<code>frontend/src/pages/accounting/Expenses.tsx</code>	List/search page with filters (status, category, party, payment mode, GST, ITC, date range)
Expense	<code>frontend/src/pages/accounting/ExpenseForm.tsx</code>	Create/edit form (DRAFT only for edit); cash vs. credit-expense toggle, GST preview
Expense	<code>frontend/src/pages/accounting/ExpenseDetail.tsx</code>	Detail view with Post/Cancel/Edit/Print actions
Expense	<code>frontend/src/pages/masters/ExpenseCategoryMaster.tsx</code>	CRUD for Expense Category master, uses generic <code>MasterCrudPage</code>
Expense reports	<code>frontend/src/pages/accounting/reports/ExpenseAnalysis.tsx</code>	Category/payment-mode/party/GST-ITC totals from <code>expenseApi.summary</code> (<code>ExpenseRepository</code> GROUP BY aggregations)
Expense reports	<code>frontend/src/pages/accounting/reports/ExpenseSummary.tsx</code>	Thin wrapper around <code>ExpenseIncomeSummaryView</code> , fed by <code>accountingReportApi.expenseSummary</code> — a different, Chart-of-Accounts-account-grouped report (Accounting Core, not <code>ExpenseRepository</code>)
Expense reports	<code>frontend/src/pages/accounting/reports/IncomeSummary.tsx</code>	Same shared view, fed by <code>accountingReportApi.incomeSummary</code>
Expense reports	<code>frontend/src/pages/accounting/reports/ExpenseIncomeSummaryView.tsx</code>	Shared presentational component for both of the above (accordion of account groups + posted vouchers)
Cash Management	<code>frontend/src/pages/accounting/CashTransactions.tsx</code>	List/search page (type, status filters)
Cash Management	<code>frontend/src/pages/accounting/CashTransactionForm.tsx</code>	Create Cash In / Cash Out form
Cash Management	<code>frontend/src/pages/accounting/CashTransactionDetail.tsx</code>	Detail view with Post/Cancel actions
Day Closing	<code>frontend/src/pages/accounting/DayClosingPage.tsx</code>	Live summary (sales/purchases/cash-by-mode/receipts/payments/expenses/cash-in-out), expected-vs-actual reconciliation, close action, closing history table
Payment Method	<code>frontend/src/pages/accounting/PaymentMethods.tsx</code>	ADMIN-only CRUD list with activate/deactivate
Financial Year	<code>frontend/src/pages/admin/FinancialYears.tsx</code>	List, create, open/close, mark-current, per-year summary dialog
Audit Trail	<code>frontend/src/pages/admin/AuditTrail.tsx</code>	Read-only, filterable/searchable log table, ADMIN only

Frontend Flow

Expense: `Expenses.tsx` loads `expenseCategoryApi.listActive()` and `supplierApi.list()` for filter dropdowns, then `expenseApi.list(query)` on mount/filter-change; row click navigates to `ExpenseDetail`. `ExpenseForm.tsx` toggles "credit expense" (party/Supplier picked) vs. cash/bank (vendor free-text + payment mode); computes a live GST preview client-side (CGST/SGST split on Intra-State, IGST on Inter-State) purely for display — the server recomputes authoritatively. Submits via `expenseApi.create / update` with `post: boolean` to optionally post in the same call. `ExpenseDetail.tsx` exposes Post/Cancel/

Edit(DRAFT only)/Print gated by `canManage = role ADMIN|STORE_MANAGER` (note: this is a frontend-only convenience gate; the real authorization is the backend's `@PreAuthorize`).

Cash Management: `CashTransactions.tsx` lists/filters; `CashTransactionForm.tsx` is a simple type/mode/amount/reason form with Save-as-Draft/Save-and-Post; `CashTransactionDetail.tsx` mirrors the Expense detail pattern (Post/Cancel).

Day Closing: `DayClosingPage.tsx` calls `dayClosingApi.summary(date)` whenever date changes (always live-computed, whether or not already closed) and `dayClosingApi.history()` once. The UI computes `difference = actualCash - expectedCash` client-side to decide whether to show/require the Difference Reason textarea before enabling Close; the server independently re-validates the same rule.

Payment Method: `PaymentMethods.tsx` — ADMIN-only page; list + modal form (create/edit) + activate/deactivate toggle via dropdown menu.

Financial Year: `FinancialYears.tsx` — table with per-row actions (View Summary / Mark as Current / Close / Re-open), all ADMIN-gated in the UI; create dialog with optional name/code (server derives if blank).

Audit Trail: `AuditTrail.tsx` — filterable/paginated read-only table (search, action-type filter); no create/edit/delete UI exists, matching the append-only backend.

3. API Calls

Expense

Function	Method	URL	Request	Response
<code>expenseApi.list</code>	GET	<code>/expenses</code>	query params: search, status, category, categoryId, supplierId, paymentMode, gstApplicable, itcEligible, fromDate, toDate, page, size	<code>PagedResponse<Expense></code>
<code>expenseApi.getById</code>	GET	<code>/expenses/{id}</code>	—	<code>Expense</code>
<code>expenseApi.create</code>	POST	<code>/expenses</code>	<code>ExpenseCreatePayload</code>	<code>Expense</code>
<code>expenseApi.update</code>	PUT	<code>/expenses/{id}</code>	<code>ExpenseUpdatePayload</code>	<code>Expense</code>
<code>expenseApi.post</code>	POST	<code>/expenses/{id}/post</code>	—	<code>Expense</code>
<code>expenseApi.cancel</code>	PATCH	<code>/expenses/{id}/cancel</code>	—	<code>Expense</code>
<code>expenseApi.summary</code>	GET	<code>/expenses/reports/summary</code>	fromDate, toDate	<code>ExpenseSummaryReportResponse</code>
<code>expenseCategoryApi.list/create/update/activate/deactivate</code>	GET/ POST/ PUT/ PATCH	<code>/masters/expense-categories[...]</code>	<code>ExpenseCategoryRequest</code> (write)	<code>ExpenseCategoryResponse</code>

Cash Management

Function	Method	URL	Request	Response
cashTransactionApi.list	GET	/cash-transactions	search, transactionType, status, fromDate, toDate, page, size	PagedResponse<CashTransaction>
cashTransactionApi.getById	GET	/cash-transactions/{id}	—	CashTransaction
cashTransactionApi.create	POST	/cash-transactions	CashTransactionCreatePayload	CashTransaction
cashTransactionApi.post	POST	/cash-transactions/{id}/post	—	CashTransaction
cashTransactionApi.cancel	PATCH	/cash-transactions/{id}/cancel	—	CashTransaction

Day Closing

Function	Method	URL	Request	Response
dayClosingApi.summary	GET	/day-closing/summary	date	DayClosingSummary
dayClosingApi.history	GET	/day-closing/history	—	DayClosingSummary[]
dayClosingApi.close	POST	/day-closing/close	DayClosingCloseInput	DayClosingSummary

Payment Method

Function	Method	URL	Request	Response
paymentMethodApi.listAll	GET	/payment-methods	—	PaymentMethod[]
paymentMethodApi.listActive	GET	/payment-methods?activeOnly=true	—	PaymentMethod[]
paymentMethodApi.create	POST	/payment-methods	PaymentMethodPayload	PaymentMethod
paymentMethodApi.update	PUT	/payment-methods/{id}	PaymentMethodPayload	PaymentMethod
paymentMethodApi.activate	PATCH	/payment-methods/{id}/activate	—	PaymentMethod
paymentMethodApi.deactivate	PATCH	/payment-methods/{id}/deactivate	—	PaymentMethod

Financial Year

Function	Method	URL	Request	Response
financialYearApi.list	GET	/financial-years	—	FinancialYear[]
financialYearApi.getCurrent	GET	/financial-years/current	—	FinancialYear

Function	Method	URL	Request	Response
<code>financialYearApi.getById</code>	GET	<code>/financial-years/{id}</code>	—	<code>FinancialYear</code>
<code>financialYearApi.getSummary</code>	GET	<code>/financial-years/{id}/summary</code>	—	<code>FinancialYearSummary</code>
<code>financialYearApi.create</code>	POST	<code>/financial-years</code>	<code>FinancialYearCreatePayload</code>	<code>FinancialYear</code>
<code>financialYearApi.open</code>	PATCH	<code>/financial-years/{id}/open</code>	—	<code>FinancialYear</code>
<code>financialYearApi.close</code>	PATCH	<code>/financial-years/{id}/close</code>	—	<code>FinancialYear</code>
<code>financialYearApi.markCurrent</code>	PATCH	<code>/financial-years/{id}/mark-current</code>	—	<code>FinancialYear</code>

Audit Trail

Function	Method	URL	Request	Response
<code>auditLogApi.list</code>	GET	<code>/audit-logs</code>	<code>action, module, entityType, entityId, userId, search, fromDate, toDate, page, size</code>	<code>PagedResponse<AuditLog></code>

Note: the backend `AuditLogController` also accepts a `storeId` query param not currently sent by `auditLogApi.list` (frontend never filters by store on this page).

4. Backend

Controllers

ExpenseController (`/api/expenses`): GET (search, `PERM_EXPENSE_VIEW`) · GET `/reports/summary` (`PERM_EXPENSE_VIEW`) · GET `/_{id}` (`PERM_EXPENSE_VIEW`) · POST (create, `PERM_EXPENSE_CREATE`) · PUT `/_{id}` (update, `PERM_EXPENSE_CREATE`) · POST `/_{id}/post` (`PERM_EXPENSE_POST`) · PATCH `/_{id}/cancel` (`PERM_EXPENSE_CANCEL`).

ExpenseCategoryController (`/api/masters/expense-categories`): GET (list, no explicit `@PreAuthorize` — any authenticated user) · GET `/active` (no explicit auth) · GET `/_{id}` (no explicit auth) · POST (`PERM_MASTER_MANAGE`) · PUT `/_{id}` (`PERM_MASTER_MANAGE`) · PATCH `/_{id}/activate` (`PERM_MASTER_MANAGE`) · PATCH `/_{id}/deactivate` (`PERM_MASTER_MANAGE`).

CashTransactionController (/api/cash-transactions): class-level

@PreAuthorize("hasAuthority('PERM_CASH_MANAGE')") covers all its endpoints — GET, GET /{id}, POST, POST /{id}/post, PATCH /{id}/cancel.

DayClosingController (/api/day-closing): GET /summary (PERM_CASH_MANAGE) · GET /history (PERM_CASH_MANAGE) · POST /close (PERM_CASH_MANAGE).

PaymentMethodController (/api/payment-methods): GET (no explicit auth — any authenticated user, activeOnly query flag) · POST (hasRole('ADMIN')) · PUT /{id} (hasRole('ADMIN')) · PATCH /{id}/activate (hasRole('ADMIN')) · PATCH /{id}/deactivate (hasRole('ADMIN')).

FinancialYearController (/api/financial-years): GET (no explicit auth) · GET /current (no explicit auth) · GET /{id} (no explicit auth) · GET /{id}/summary (no explicit auth) · POST (PERM_FY_MANAGE) · PATCH /{id}/open (PERM_FY_MANAGE) · PATCH /{id}/close (PERM_FY_MANAGE) · PATCH /{id}/mark-current (PERM_FY_MANAGE). Comment in source: "Any authenticated user may VIEW financial years... only ADMIN may mutate them" (mutation is actually gated by PERM_FY_MANAGE, which only Role.ADMIN holds per RolePermissions).

AuditLogController (/api/audit-logs): class-level @PreAuthorize("hasAuthority('PERM_AUDIT_VIEW')"), read-only — GET (search) only. No POST/PUT/DELETE exists anywhere in the audit trail API surface; comment: "the audit trail is append-only and never editable/deletable via the API".

DTOs

Expense

- ExpenseCreateRequest — expenseDate (@NotNull), storeId (optional), categoryId (preferred) or category (legacy free text), supplierId (optional, makes it a credit expense), vendorName, paymentMode (@NotNull), taxMode, gstPercent (@DecimalMin("0")), itcEligible (boolean), grossAmount, discountAmount (@DecimalMin("0")), taxableAmount (used only if grossAmount absent), description, referenceNumber, remarks, post (boolean).
- ExpenseUpdateRequest — identical shape to create (no storeId / categoryId swap logic difference); only usable while the Expense is DRAFT.
- ExpenseResponse — full read model incl. categoryId, storeId/storeName/storeCode, supplierId/supplierGstin, tax breakdown (cgstAmount/sgstAmount/igstAmount), paidAmount/payableAmount/paymentStatus, status, audit fields (createdBy/postedBy/postedAt/cancelledBy/cancelledAt/createdAt/updatedAt).
- ExpenseCategoryRequest — code (@NotBlank), name (@NotBlank), linkedAccountId (optional), description.
- ExpenseCategoryResponse — incl. linkedAccountId/linkedAccountName, active + a mirrored status string ("ACTIVE"/"INACTIVE") for the generic MasterCrudPage component.
- ExpenseSummaryGroupRow — key, label, totalAmount, count (used for category/payment-mode/party groupings).
- ExpenseSummaryReportResponse — fromDate/toDate, totalAmount, totalCount, byCategory/byPaymentMode/byParty (List<ExpenseSummaryGroupRow>), gstApplicableAmount, nonGstAmount, itcEligibleTax, itcIneligibleTax.
- ExpenseIncomeSummaryResponse / ExpenseIncomeAccountGroup / ExpenseIncomeVoucherRow — used by the separate, Accounting-Core-driven Expense/Income Summary reports (account-grouped, not ExpenseRepository-grouped); populated by AccountingReportService, not ExpenseService.

Cash Management

- CashTransactionCreateRequest — transactionDate (@NotNull), storeId (optional), transactionType (@NotNull), paymentMode (@NotNull), amount (@NotNull @DecimalMin("0.01")), reason (@NotBlank), post (boolean).
- CashTransactionResponse — full read model incl. storeId/storeName/storeCode, status, audit fields.

- `CashBankBookResponse / CashBankBookRow` — belong to `CashBankBookService` (Phase 4), reused by `DayClosingService` to compute `expectedCash`; not owned by `CashTransactionService`.

Day Closing

- `DayClosingCloseRequest` — `closingDate` (@NotNull), `actualCash` (@NotNull), `differenceReason` (optional at DTO level; enforced conditionally in the service when `difference != 0`).

- `DayClosingResponse` — every summary figure (`totalSales/totalPurchases/cashSales/cardSales/upiSales/creditSales/totalReceipts/totalPayments/totalExpenses/cashIn/cashOut/expectedCash`) plus, once closed, `actualCash/difference/differenceReason/closedBy/closedAt` and `closed: boolean`.

Payment Method

- `PaymentMethodRequest` — `name` (@NotBlank), `type` (`PaymentMode`, @NotNull), `sortOrder` (optional).

- `PaymentMethodResponse` — `id/name/type/active/sortOrder/createdAt/updatedAt`.

Financial Year

- `FinancialYearRequest` — `name` (optional, auto-derived), `code` (optional, auto-derived), `startDate` (@NotNull), `endDate` (@NotNull).

- `FinancialYearResponse` — `id/name/code/startDate/endDate/status/current/createdAt/updatedAt`.

- `FinancialYearSummaryResponse` — `financialYear` (nested `FinancialYearResponse`), `totalSales/totalPurchases/saleCount/purchaseCount`.

Audit Trail

- `AuditLogResponse` — `id/userId/username/action/module/entityType/entityId/storeId/storeName/storeCode/documentNumber/oldValue/newValue/timestamp/ipAddress/userAgent/description`. No request DTO exists — writes only happen internally via `AuditService.log(...)`, never through a controller-exposed request body.

Services

ExpenseService — `create`: resolves the caller's effective store (`StoreAccessService.resolveEffectiveStoreId`), rejects an INACTIVE store, builds a DRAFT Expense via `applyRequest` (resolves category/supplier, computes `taxable = gross - discount` or uses raw `taxableAmount`, calls `GstCalculationService.calculateLine` for CGST/SGST/IGST split, requires `taxMode` when `GST > 0`), assigns the voucher number via `VoucherNumberService.next(EXPENSE, date)`, audit-logs CREATE, optionally posts immediately if `request.isPost()`. `update`: only allowed while `status == DRAFT`; re-runs `applyRequest`; audit-logs UPDATE. `post`: only from DRAFT; computes the Expense-account debit line (`expenseLineAmount = taxable-only if ITC-eligible, else taxable+tax, since a non-recoverable tax is expensed`); adds Input CGST/SGST/IGST debit lines when ITC-eligible; if `supplier != null` (credit expense) credits SUPPLIER_PAYABLE party-tagged to that Supplier and calls `LedgerService.recordExpenseCredit` (writes a `SupplierLedgerEntry`, NOT a `CashLedgerEntry`) and sets `paidAmount=0/payableAmount=total/paymentStatus=UNPAID`; else credits Cash/Bank (`AccountingService.resolveCashOrBank(paymentMode)`) and sets `paidAmount=total/payableAmount=0/paymentStatus=PAID`; posts via `AccountingService.postJournalByAccountId(VoucherType.EXPENSE, ...)`; syncs GST reporting via `GstTransactionSyncService.syncExpense`; audit-logs POST. `cancel`: idempotent no-op if already CANCELLED; refuses to cancel if any `PaymentAllocation` already exists against it (payments must be deleted first); if POSTED, reverses the journal (`AccountingService.reverseJournal`), reverses the supplier-ledger credit if applicable (`reverseExpenseCredit`), and reverses the GST sync row (`gstTransactionSyncService.reverseExpense`); resets `paidAmount/payableAmount` to 0, sets `status=CANCELLED`; audit-logs CANCEL. `resolveExpenseAccountId`: uses the category's `linkedAccount` if set and active, else falls back to `SystemAccountCode.EXPENSES`.

ExpenseCategoryService — plain CRUD with uniqueness checks on `code/name` (case-insensitive); `setActive` is the only "delete" path (never a hard delete — an already-referenced category must stay

visible on historical expenses); `findActiveOrThrow` additionally rejects an inactive category when used on a new/edited Expense.

CashTransactionService — `create`: same store-resolution/INACTIVE-store guard pattern as Expense; builds `DRAFT CashTransaction`, numbers it via `VoucherNumberService.next(CASH_TRANSACTION, date)`, audit-logs `CREATE`, optional immediate post. `post`: builds two journal lines depending on type — `CASH_IN` debits Cash/Bank and credits `OTHER_INCOME`; `CASH_OUT` debits `EXPENSES` and credits Cash/Bank; posts via `AccountingService.postJournal` (not `postJournalByAccountId` — this uses the simpler `JournalLine` helper, unlike Expense which needs party-tagging); audit-logs `POST`. `cancel`: idempotent; reverses the journal if `POSTED`; audit-logs `CANCEL`. No credit/party support exists for Cash Transactions (always cash/bank, never a payable).

DayClosingService — `computeSummary(date)` (read-only, safe to call repeatedly/preview): pulls `totalSales/byMode` from `SaleRepository`, `totalPurchases` from `PurchaseRepository`, `totalReceipts/totalPayments` from `ReceiptRepository/PaymentRepository`, `totalExpenses` from `ExpenseRepository.getTotalExpensesForDate`, `cashIn/cashOut` from `CashTransactionRepository.sumForDateAndType`, and critically `expectedCash = cashBankBookService.cashBook(date, date).getClosingBalance()` — the exact same Cash Book every other page reads, so it can never drift from the accounting journal. If a `DayClosing` row already exists for that date, wraps the computed figures with the persisted `actualCash/difference/differenceReason/closedBy/closedAt` via `DayClosingResponse.fromClosed`. `history()` re-runs `computeSummary` for every historical `DayClosing` row (not a cached snapshot). `close(request)`: rejects if `existsByClosingDate` already true (one close per date, ever); recomputes the summary fresh; computes `difference = actualCash - expectedCash`; **if difference != 0 and differenceReason is null/blank, throws `BadRequestException`** — this is the difference-reason-audit requirement; resolves and stamps `financialYearId` via `FinancialYearService.resolveForDate`; saves the `DayClosing`; audit-logs `DAY_CLOSE` with the difference amount/reason embedded in the description when non-zero. Closing never blocks a later backdated transaction itself — that gate is Financial Year open/closed status, per the class Javadoc.

PaymentMethodService — simple CRUD: `create/update` check name uniqueness (case-insensitive); `setActive` is the only deactivation path (never hard-deleted, so historical transactions still display the method they used).

FinancialYearService — `@PostConstruct ensureCurrentFinancialYearExists()`: idempotent startup hook — if no FY row covers `LocalDate.now()`, builds and saves one (via `buildFor`, using `FinancialYearUtil.startOf/endOf/shortCode/label` for the 1-Apr–31-Mar Indian convention) with `status=OPEN`, `current=true`. `resolveForDate(date)`: looks up the FY containing date; throws `BadRequestException` ("Ask an ADMIN to create it") if none exists — does NOT require the FY to be OPEN (used for numbering/reporting on historical/closed-year documents). `resolveOpenForPosting(date)`: calls `resolveForDate` then additionally throws if `status == CLOSED` — this is the one real posting gate, called from `AccountingService.buildAndSaveJournal` (and its reversal path) before every journal post, so FY enforcement lives in exactly one place. `create`: validates `endDate > startDate`, auto-derives `code/name` if blank, rejects a duplicate code and any overlapping date range (`countOverlapping`); audit-logs `CREATE`. `setStatus(id, status)`: refuses to `CLOSED` the currently-current FY ("Mark another year current first"); audit-logs `FY_CLOSE` or `FY_OPEN` with old/new status values. `markCurrent(id)`: refuses on a `CLOSED` target; unmarks whichever other FY was `current` (at most one `current=true` row ever, enforced here in application code, not a DB constraint); audit-logs `UPDATE`. `summary(id)`: aggregates `SaleRepository/PurchaseRepository` totals+counts over the FY's date range.

AuditService — the one place every audited action is recorded. Two overloads:

`log(AuditAction action, String module, String entityType, Long entityId, String documentNumber, String oldValue, String newValue, String description)` — delegates to the 9-arg overload with `storeId = null`.

`log(AuditAction action, String module, String entityType, Long entityId, String documentNumber, String oldValue, String newValue, String description, Long storeId)` — the real implementation: resolves the current user via `SecurityUtil.currentUserOrNull()` (username = "System" if null), builds and saves an `AuditLog` row with `userId/username, action, module, entityType, entityId, store` (via `storeRepository.getReferenceById(storeId)` when non-null), `documentNumber, oldValue, newValue, ipAddress` (from `X-Forwarded-For` header, falling back to `request.getRemoteAddr()`), `userAgent` (from the `User-Agent` header), `description`. Runs `@Transactional` with default propagation (deliberately NOT `REQUIRES_NEW` like `VoucherNumberService`) — an audit entry for an action that later rolls back rolls back with it. Wraps the whole body in try/catch and only `log.warn(...)`s on failure — a transient audit-write problem never blocks the business operation it describes.

Repository

Expense: `ExpenseRepository` — `search(...)` (dynamic JPQL over `search/status/category/categoryId/supplierId/paymentMode/gstApplicable/itcEligible/fromDate/toDate/storeId`), `findPostedIds`, `findAllIds`, `getTotalExpensesForDate`, `sumTotalAmountByDateRange`, `findOutstandingBySupplier` (POSTED credit expenses with `payableAmount > 0`), plus report aggregations `sumByCategory / sumByPaymentMode / sumByParty / gstSummary` (all server-side GROUP BY — no per-row loading). **ExpenseCategoryRepository** — `existsByCodeIgnoreCase(AndIdNot)`, `existsByNameIgnoreCase(AndIdNot)`, `findByActiveTrueOrderByNameAsc`, `search`.

Cash Management: `CashTransactionRepository` — `search(...)`, `sumForDateAndType(date, type)` (used by `DayClosingService`). **CashLedgerEntryRepository** — `findByReferenceTypeAndReferenceId`, `getBalanceForMode`; this ledger is a Receipt-side (customer money in) movement log per its Javadoc — it is NOT written to by `CashTransactionService` or `ExpenseService` (see section 5 relationships).

Day Closing: `DayClosingRepository` — `existsByClosingDate`, `findByClosingDate`, `findAllByOrderByClosingDateDesc`.

Payment Method: `PaymentMethodRepository` — `existsByNameIgnoreCase(AndIdNot)`, `findByActiveTrueOrderBySortOrderAscNameAsc`, `findAllByOrderBySortOrderAscNameAsc`.

Financial Year: `FinancialYearRepository` — `existsByCodeIgnoreCase`, `findByCodeIgnoreCase`, `findByCurrentTrue`, `findByDate(date)` (WHERE `:date BETWEEN startDate AND endDate`), `findAllByOrderByStartDateDesc`, `countOverlapping`.

Audit Trail: `AuditLogRepository` — `search(...)` only (dynamic JPQL over `action/module/entityType/entityId/userId/storeId/search/fromDate/toDate`). No update/delete method exists anywhere on this repository — enforced append-only at the code level.

5. Database

Tables

Sub-area	Table
Expense	<code>expenses</code> , <code>expense_categories</code>
Cash Management	<code>cash_transactions</code> , <code>cash_ledger_entries</code>
Day Closing	<code>day_closings</code>
Payment Method	<code>payment_methods</code>

Sub-area	Table
Financial Year	<code>financial_years</code>
Audit Trail	<code>audit_logs</code> (indexes: <code>idx_audit_logs_entity(entity_type, entity_id)</code> , <code>idx_audit_logs_timestamp(timestamp)</code> , <code>idx_audit_logs_user(user_id)</code>)

Relationships

- `expenses.expense_category_id` → `expense_categories.id` (`Expense.expenseCategory` , `ManyToOne` , nullable — null on legacy free-text-category rows). `expense_categories.linked_account_id` → `accounts.id` (optional Chart-of-Accounts mapping).
- `**expenses.supplier_id` → `suppliers.id` `**` (`Expense.supplier` , `ManyToOne` , nullable – set only for a credit expense; reuses the existing Supplier master, never a separate vendor entity). A credit expense's payable is tracked via a `SupplierLedgerEntry` row (written by `LedgerService.recordExpenseCredit / reverseExpenseCredit` , `referenceType=EXPENSE`), not by `CashLedgerEntry` .
- `**expenses.store_id` → `stores.id` `**` (Multi-Store – fixed at creation, `ManyToOne`).
- `cash_transactions.store_id` → `stores.id` (fixed at creation).
- `cash_ledger_entries` has NO foreign key to `cash_transactions` — it is a Receipt-side ledger (`referenceType / referenceId` polymorphic reference, one row per Receipt movement) per its class Javadoc; `CashTransactionService` never writes to it. The Cash Book/Bank Book instead reads straight off `journal_headers / journal_lines` (Accounting Core), which is what `CashTransactionService.post` and `ExpenseService.post` actually write to via `AccountingService` .
- `day_closings.financial_year_id` — a plain `Long` column (no JPA `@ManyToOne / FK`), set from `FinancialYearService.resolveForDate(closingDate).getId()` at close time; same denormalized-FK-by-id pattern used by `expenses.financial_year_id` and `cash_transactions.financial_year_id`.
- `**audit_logs.store_id` → `stores.id` `**` (`AuditLog.store` , `ManyToOne` , nullable – null for a global/system-level action). `**audit_logs.user_id` `**` is a plain `Long` , not a JPA relationship to `User` – `AuditLogResponse` carries a denormalized username` snapshot alongside it, so a row remains readable even if the user is later deleted/deactivated.
- `payment_methods` has no foreign keys — it is a standalone label/order master over the `PaymentMode` enum, not wired into any posting relationship.
- `financial_years` has no foreign keys; every other entity in this module (`expenses` , `cash_transactions` , `day_closings`) stores `financial_year_id` as a plain denormalized long, resolved on write and never re-tagged later.

6. Validation

Expense: `expenseDate` and `paymentMode` required (`@NotNull`); `gstPercent / discountAmount` must be `>= 0` (`@DecimalMin`); `category` is required — resolved from `categoryId` (must be an active `ExpenseCategory` , else `BadRequestException`) or the legacy `category` free-text field, and a blank result throws `BadRequestException("Category is required")` ; an inactive `Supplier` cannot be used (`BadRequestException`); an inactive `Store` cannot be used (`BadRequestException`); taxable amount (computed as `gross - discount` , or the raw `taxableAmount` if no `grossAmount` sent) must be `> 0` else `BadRequestException("Taxable amount must be greater than 0")` ; `taxMode` is required whenever the computed GST amount is `> 0` (`BadRequestException`); update is only permitted while `status == DRAFT` ; post is only permitted from `DRAFT` ; cancel is blocked while any `PaymentAllocation` exists against the expense.

Cash Management: `transactionDate`, `transactionType`, `paymentMode` required (`@NotNull`); `amount` required and `@DecimalMin("0.01")` (must be `> 0`); `reason` required (`@NotBlank`); an inactive Store cannot be used; post only from `DRAFT` ; cancel is idempotent (no error re-cancelling).

Day Closing: `closingDate` and `actualCash` required (`@NotNull` at DTO level); a date can only be closed once (`BadRequestException` if `existsByClosingDate`); **differenceReason is conditionally required** — enforced in `DayClosingService.close`, not by a DTO annotation: if `actualCash - expectedCash != 0` and `differenceReason` is null/blank, throws `BadRequestException` with the exact expected/actual/difference figures in the message.

Payment Method: `name` required (`@NotBlank`) and must be unique case-insensitively (create and update, excluding self on update); `type` (`PaymentMode`) required (`@NotNull`).

Financial Year: `startDate/endDate` required (`@NotNull`); service-level: `endDate` must be strictly after `startDate`; `code` must be unique case-insensitively; the new range must not overlap any existing FY (`countOverlapping`); closing the currently- current FY is blocked ("Mark another year current first"); marking a `CLOSED` FY as current is blocked ("Re-open it first").

Audit Trail: no write-side validation exists — this module only exposes a read/search endpoint; write path (`AuditService.log`) has no validation, only a try/catch that swallows failures.

7. Authentication / Authorization

All endpoints in this module require an authenticated request (standard Spring Security filter chain, JWT-based per `01-authentication-authorization.md`). Endpoint-level authorization is enforced via `@PreAuthorize("hasAuthority('PERM_...')")` (or, for Payment Method, `hasRole('ADMIN')`) — see section 8 for the exact permission-to-endpoint mapping. Several read endpoints (Expense Category list/active/getById, Payment Method list, Financial Year list/current/getById/summary) carry no explicit `@PreAuthorize`, meaning any authenticated user can read them (financial years in particular must be visible everywhere per the controller's own comment). Multi-Store access is additionally enforced in the service layer for Expense and Cash Transaction: `StoreAccessService.resolveEffectiveStoreId/resolveViewableStoreId/assertStoreAccess` scope creation and single-record reads to stores the caller has access to — `getById` throws if the caller lacks access to the record's store.

8. Permissions

Permission	Module (per <code>Permission.java</code>)	Used by
<code>EXPENSE_VIEW</code>	Account	<code>ExpenseController</code> GET/search/summary/getById
<code>EXPENSE_CREATE</code>	Account	<code>ExpenseController</code> POST/create, PUT/update
<code>EXPENSE_POST</code>	Account	<code>ExpenseController</code> POST /post
<code>EXPENSE_CANCEL</code>	Account	<code>ExpenseController</code> PATCH /cancel
<code>CASH_MANAGE</code>	Account	<code>CashTransactionController</code> (all endpoints, class-level) and <code>DayClosingController</code> (all endpoints) — comment: "Cash In/Out, Day Closing"
<code>FY_MANAGE</code>	Account	<code>FinancialYearController</code> create/open/close/mark-current — comment: "deliberately its own permission, never bundled with <code>ACCOUNT_VIEW</code> "
<code>AUDIT_VIEW</code>	Audit	<code>AuditLogController</code> (class-level, all endpoints)
<code>MASTER_MANAGE</code>	Master	<code>ExpenseCategoryController</code> create/update/activate/deactivate

Role coverage (from `RolePermissions.build()`): `ADMIN` holds every `Permission` (`EnumSet.allOf`). `STORE_MANAGER` holds everything `ADMIN` does **except** `FY_MANAGE` and `AUDIT_VIEW` (explicitly removed, alongside the `USER_/ROLE_/PERMISSION_VIEW` security tier) — so a Store Manager can manage Expenses/Cash/Day Closing but cannot manage Financial Years or view the Audit Trail. `ACCOUNTANT` explicitly holds `EXPENSE_VIEW/CREATE/POST/CANCEL` and `CASH_MANAGE` (listed among its granted set) but does NOT hold `FY_MANAGE` or `AUDIT_VIEW`. `SALES_USER`, `PURCHASE_USER`, `INVENTORY_USER`, and `STAFF` hold none of `EXPENSE_*`, `CASH_MANAGE`, `FY_MANAGE`, or `AUDIT_VIEW` — this module's write paths are Accountant/Store-Manager/Admin-tier only, and FY management plus the audit trail are Admin-only.

9. Transaction Handling

- `ExpenseService / CashTransactionService / DayClosingService / FinancialYearService / PaymentMethodService` methods are `@Transactional` (read methods `readOnly = true`), default propagation — the standard pattern across this codebase.
- Voucher numbering (`VoucherNumberService.next`, used by both Expense and Cash Transaction creation) runs in its own `@Transactional(propagation = REQUIRES_NEW)` transaction with a `SELECT ... FOR UPDATE` row lock on the sequence counter, so the numbering lock is independent of — and released before — the larger creation transaction, and a numbering failure never poisons the caller's transaction.
- Journal posting (`AccountingService.postJournalByAccountId / postJournal / reverseJournal`) runs inside the caller's own transaction (Expense/Cash Transaction posting and their DB state changes are atomic together — either both the journal and the entity status flip, or neither does).
- `AuditService.log(...)` deliberately runs in the SAME transaction as its caller (default propagation, NOT `REQUIRES_NEW`) — per its Javadoc, an audit entry for an action that later rolls back should roll back with it; recording "this happened" for something that didn't would be worse than not recording it. Its body is also wrapped in try/catch so a transient audit-write failure is logged (`log.warn`) but never thrown, never blocking or rolling back the business operation it describes.
- `FinancialYearService.resolveOpenForPosting(date)` is invoked from inside `AccountingService.buildAndSaveJournal` (and the reversal path) — i.e., the FY-open gate is checked in the SAME transaction as the journal write, so a closed-FY rejection prevents the posting atomically rather than as a separate pre-check that could race.
- `DayClosing.close` is a single-row insert guarded by `existsByClosingDate` inside the same transaction as the `dayClosingRepository.save` — no unique-constraint race window is explicitly discussed in the code beyond the DB's own `unique = true` constraint on `closing_date`.

10. Error Handling

`GlobalExceptionHandler` (`backend/src/main/java/com/storehub/exception/GlobalExceptionHandler.java`) centrally maps:

- `MethodArgumentNotValidException` (bean validation failures, e.g. `@NotNull / @NotBlank / @DecimalMin` on the DTOs above) → HTTP 400 with `ApiError.fieldErrors` populated per-field (`field → defaultMessage`), message = "Validation failed".
- `ExpenseNotFoundException` → HTTP 404 with the exception's own message ("Expense not found with id: {id}").
- `CashTransactionNotFoundException` → HTTP 404 ("Cash transaction not found with id: {id}").
- `FinancialYearNotFoundException` → HTTP 404 ("Financial year not found with id: {id}").
- `BadRequestException` (the generic business-rule violation type — used throughout this module for inactive store/supplier/category, invalid amounts, missing tax mode, already-closed day, overlapping FY range, closing the current FY, etc.) → HTTP 400 with the exception's own message as `ApiError.message` (no `fieldErrors` — this is a top-level, not per-field, error).

- `AuditService.log` never throws to its caller — any persistence failure inside it is caught and logged, so an audit-write problem can never surface as an HTTP error on the business operation.

The `ApiError` shape (`timestamp`, `status`, `error`, `message`, `path`, `fieldErrors`) is the standard shape used across the whole application; per project convention (see this repo's `CLAUDE.md`), a 400 with `fieldErrors` should always be checked field-by-field before assuming a generic cause.

11. Audit Flow

`AuditService` is a CENTRAL/COMMON logging mechanism, not exclusive to this module — a `grep` of `auditService.log` across `backend/src/main/java/com/storehub/service` found 18 distinct service files calling it (representative, not exhaustive): `ProductService`, `InventoryService`, `StockTransferService`, `StoreService`, `DebitNoteService`, `CreditNoteService`, `CashTransactionService`, `ExpenseService`, `PaymentService`, `ReceiptService`, `PurchaseService`, `SaleService`, `EmployeeService`, `UserService`, `AuthService`, `BusinessGstConfigService`, `DayClosingService`, `FinancialYearService`. Every module in the application funnels its "who did what" history through this one service and the single `audit_logs` table.

This module's OWN entities are audited as follows:

- **Expense:** `CREATE` on creation (description names the expense number + category), `UPDATE` when a DRAFT is edited, `POST` when posted ("accounting journal applied"), `CANCEL` when cancelled (reason echoes the expense number). Every call passes `module="EXPENSE"`, `entityType="Expense"`, and the expense's `storeId`.
- **Cash Transaction:** `CREATE`, `POST`, `CANCEL` — same pattern, `module="CASH"`, `entityType="CashTransaction"`, with `storeId`.
- **Day Closing:** `DAY_CLOSE` on close, with the difference amount and reason embedded directly in the free-text description when non-zero — `module="CASH"`, `entityType="DayClosing"`; notably `DayClosingService.close`'s audit call does NOT pass a `storeId` (uses the 8-arg overload), unlike `Expense/CashTransaction`.
- **Financial Year:** `CREATE` on creation, `FY_CLOSE` / `FY_OPEN` on status change (records `oldStatus` / `newStatus` in `oldValue` / `newValue`), `UPDATE` on mark-current — `module="ADMIN"`, `entityType="FinancialYear"`; none of these pass a `storeId` (financial years are global, not store-scoped).
- **Payment Method:** NOT FOUND IN CURRENT CODEBASE — `PaymentMethodService` never calls `auditService.log(...)`; create/update/activate/deactivate on Payment Method leave no audit trail entry.
- **Expense Category:** NOT FOUND IN CURRENT CODEBASE — `ExpenseCategoryService` never calls `auditService.log(...)` either.

12. Important Side Effects

- **Expense posting → Accounting Core:** `ExpenseService.post` calls `accountingService.postJournalByAccountId(VoucherType.EXPENSE, ...)` — verified directly in `ExpenseService.java`. It debits the resolved Expense account (category's `linkedAccount` if active, else `SystemAccountCode.EXPENSES`) for the ITC-adjusted amount, adds Input CGST/SGST/IGST debit lines when `itcEligible`, and credits either `SUPPLIER_PAYABLE` (party-tagged) for a credit expense or the resolved Cash/Bank system account otherwise.
- **Expense GST/ITC sync:** `ExpenseService.post` calls `gstTransactionSyncService.syncExpense(posted)` (only creates/refreshes a `GstTransaction` row if `GstReportingEligibility.isEligibleForGstReporting(expense)` — a non-GST expense, i.e. no `taxMode`, is never synced). Its `b2b` flag is set directly from `expense.isItcEligible()` (not a GSTIN-

validity proxy like Sale/Purchase use). `ExpenseService.cancel` calls

`gstTransactionSyncService.reverseExpense(expense)`, which flips the existing GST row to REVERSED in place (a no-op if the expense was never synced).

- **Cash Transaction posting → Accounting Core:** `CashTransactionService.post` calls `accountingService.postJournal(VoucherType.CASH_TRANSACTION, ...)` with simple `JournalLine.debit/credit` pairs (CASH_IN: debit Cash/Bank, credit OTHER_INCOME; CASH_OUT: debit EXPENSES, credit Cash/Bank) — no GST/ITC involvement for Cash Transactions at all.
- **Day Closing's difference-reason requirement:** `DayClosingService.close` throws `BadRequestException` if `actualCash != expectedCash` and no `differenceReason` was supplied — this is a hard server-side gate, not merely a frontend nicety (the frontend independently computes and enforces the same rule for UX, but the backend re-validates authoritatively).
- **Day Closing never touches accounting balances:** confirmed by the entity Javadoc and the service code — `close` only persists a `DayClosing` row; any real cash shortage/overage must be separately recorded as an explicit `CashTransaction` (Cash Adjustment) if the business wants it reflected in the ledger.
- **FinancialYear's startup auto-seed:** `FinancialYearService.ensureCurrentFinancialYearExists()` (`@PostConstruct`) runs once per application startup; it is idempotent — if a FY row already covers `LocalDate.now()`, it does nothing; otherwise it creates one with `status=OPEN`, `current=true` using `FinancialYearUtil` for the Apr-Mar boundaries. This guarantees "today" always has a resolvable FY on a fresh database, so the very first Sale/Purchase/Expense/Cash Transaction/Day Closing never fails purely for lack of a FY row.
- **FY-open gate is centralized, not per-service:** `AccountingService.buildAndSaveJournal` (and its reversal counterpart) call `financialYearService.resolveOpenForPosting(...)` before every journal write — this is the single choke point that blocks posting into a CLOSED financial year for every voucher type in the system, including Expense and Cash Transaction from this module.

13. Dependencies on Other Modules

- **Accounting Core** (`AccountingService`, `AccountService`, journal headers/lines) — Expense and Cash Transaction posting/reversal both go through `AccountingService.postJournal / postJournalByAccountId / reverseJournal`; `ExpenseService` also depends on `AccountService.getSystemAccount(SystemAccountCode...)` for the Expenses/Input-GST/Supplier-Payable system accounts, and `ExpenseCategoryService / ExpenseCategoryController` depend on `Account` for the optional linked-account mapping. `DayClosingService` depends on `CashBankBookService.cashBook` (also Accounting Core) for `expectedCash`.
- **GST** (`GstCalculationService`, `GstTransactionSyncService`, `GstReportingEligibility`, `GstinValidator`) — `ExpenseService.applyRequest` uses `GstCalculationService.calculateLine` for the CGST/SGST/IGST split; `ExpenseService.post / cancel sync/reverse` the expense's `GstTransaction` reporting row via `GstTransactionSyncService`.
- **Supplier** (Party module) — a credit Expense reuses the existing `Supplier` entity (`SupplierService.findSupplierOrThrow`) rather than a separate vendor master; its payable is tracked through `LedgerService's SupplierLedgerEntry` (`recordExpenseCredit / reverseExpenseCredit`), and later settlement of that payable happens entirely through the existing Payment module (`PaymentService`) — Expense never implements its own payment mechanism. `ExpenseRepository.findOutstandingBySupplier` mirrors `PurchaseRepository's` equivalent query.
- **Store (Multi-Store)** — `Expense.store`, `CashTransaction.store`, and `AuditLog.store` are all `@ManyToOne` to `Store`; `storeAccessService.resolveEffectiveStoreId / resolveViewableStoreId / assertStoreAccess` gate both creation and single-record access for Expense and Cash Transaction. `DayClosing` and `FinancialYear` are NOT store-scoped (no `store_id` column on either).

- **User** — `Expense.createdBy/postedBy/cancelledBy`, `CashTransaction.createdBy/postedBy/cancelledBy`, `DayClosing.closedBy`, and `AuditLog.userId/username` are all stamped from `SecurityUtil.currentUsername() / currentUserOrNull()`.
- **Day Closing → Sale/Purchase/Receipt/Payment/Expense/CashTransaction repositories** — `DayClosingService.computeSummary` directly queries `SaleRepository`, `PurchaseRepository`, `ReceiptRepository`, `PaymentRepository`, `ExpenseRepository`, and `CashTransactionRepository` for its daily aggregates — it is a read-only rollup across nearly every transactional module, not an independent ledger.
- **Financial Year → Sale/Purchase** — `FinancialYearService.summary(id)` aggregates `SaleRepository / PurchaseRepository` totals for the FY's date range; `VoucherNumberService` (used by `Expense/Cash Transaction` numbering) depends on `FinancialYearService.resolveForDate` for the FY code embedded in every voucher number.

14. Key Operation Flows

Create Expense (cash): `ExpenseForm.tsx` (credit-expense toggle OFF, vendor/paymentMode filled) → `expenseApi.create(payload)` → `POST /api/expenses` → `ExpenseController.create (@PreAuthorize PERM_EXPENSE_CREATE)` → `ExpenseCreateRequest` (validated: `expenseDate`, `paymentMode @NotNull`) → `ExpenseService.create`: resolves store, builds DRAFT `Expense` via `applyRequest` (resolves category, computes taxable/GST via `GstCalculationService`), saves, numbers via `VoucherNumberService.next(EXPENSE, date)`, audit-logs `CREATE` → if `post: true`, immediately calls `ExpenseService.post` (see below) → `expenses` table row(s) → `ExpenseResponse` → UI navigates to `ExpenseDetail`.

Create Expense (credit, against a Supplier): same form with the credit-expense toggle ON and a `supplierId` chosen → same create path, but `applyRequest` resolves and validates the Supplier (must be active) and sets `vendorName` from the Supplier's name → on `post`, `ExpenseService.post` detects `supplier != null` → credits `SUPPLIER PAYABLE` (party-tagged to that Supplier) instead of `Cash/Bank`, sets `paymentStatus=UNPAID / payableAmount=total`, and calls `LedgerService.recordExpenseCredit` → writes a `SupplierLedgerEntry` (`referenceType=EXPENSE`) → `GstTransactionSyncService.syncExpense` if GST-applicable → `ExpenseResponse` shows `supplierId/payableAmount`; later settlement happens via the existing `Payment` module, not this flow.

Cancel Expense: `ExpenseDetail.tsx` Cancel button → `expenseApi.cancel(id)` → `PATCH /api/expenses/{id}/cancel` → `ExpenseController.cancel (PERM_EXPENSE_CANCEL)` → `ExpenseService.cancel`: no-op if already `CANCELLED`; throws `BadRequestException` if any `PaymentAllocation` exists against it; if `POSTED`, reverses the journal (`AccountingService.reverseJournal`), reverses the supplier-ledger credit if it was a credit expense, reverses the GST sync row; resets `paidAmount/payableAmount` to 0, sets `status=CANCELLED`, stamps `cancelledBy/cancelledAt`; audit-logs `CANCEL` → `ExpenseResponse`.

Cash In: `CashTransactionForm.tsx` (`type = CASH_IN`) → `cashTransactionApi.create(payload)` → `POST /api/cash-transactions` → `CashTransactionController.create` (class-level `PERM_CASH_MANAGE`) → `CashTransactionCreateRequest` validated → `CashTransactionService.create`: resolves store, builds DRAFT, numbers via `VoucherNumberService.next(CASH_TRANSACTION, date)`, audit-logs `CREATE` → if `post: true`, `CashTransactionService.post`: debits resolved `Cash/Bank`, credits `OTHER_INCOME`, posts via `AccountingService.postJournal`, audit-logs `POST` → `cash_transactions` row → `CashTransactionResponse` → UI navigates to detail.

Cash Out: identical flow with `type = CASH_OUT`; `post` instead debits `SystemAccountCode.EXPENSES` and credits the resolved `Cash/Bank` account.

Perform Day Closing: `DayClosingPage.tsx` loads `dayClosingApi.summary(date)` (live, re-derivable any time) → operator enters `actualCash`; if it differs from `expectedCash` the UI requires a `differenceReason`

before enabling Close → `dayClosingApi.close(payload)` → `POST /api/day-closing/close` → `DayClosingController.close ( PERM_CASH_MANAGE )` → `DayClosingCloseRequest` → `DayClosingService.close`: rejects if the date was already closed; recomputes the summary fresh server-side; recomputes difference; throws if non-zero and no reason given; resolves `financialYearId` via `FinancialYearService.resolveForDate`; saves the `DayClosing` row; audit-logs `DAY_CLOSE` → `day_closings` table → `DayClosingResponse.fromClosed` → UI shows the closed state and appends to Closing History.

Create Financial Year: `FinancialYears.tsx` create dialog (ADMIN only in UI) →

`financialYearApi.create(payload)` → `POST /api/financial-years` → `FinancialYearController.create ( PERM_FY_MANAGE )` → `FinancialYearRequest ( startDate/endDate @NotNull )` → `FinancialYearService.create`: validates `endDate > startDate`, derives `code/name` if blank via `FinancialYearUtil`, rejects a duplicate code or an overlapping date range (`countOverlapping`), saves with `status=OPEN`, `current=false`, audit-logs `CREATE ( module="ADMIN" )` → `financial_years` row → `FinancialYearResponse`.

Close Financial Year: `FinancialYears.tsx` row action "Close Year" → `financialYearApi.close(id)` → `PATCH /api/financial-years/{id}/close` → `FinancialYearController.close ( PERM_FY_MANAGE )` → `FinancialYearService.setStatus(id, CLOSED)`: throws `BadRequestException` if the target FY is `isCurrent()` ("Mark another year current first"); flips `status` to `CLOSED`; audit-logs `FY_CLOSE ( oldValue/newValue = old/new status )` → `financial_years` row updated → `FinancialYearResponse` → from this point, `AccountingService.buildAndSaveJournal`'s call to `resolveOpenForPosting` rejects any new posting dated inside this FY, application-wide (Expense/Cash Transaction included).

15. Manual Changes

- **Add a new Expense field:** add the column + getter/setter to `backend/src/main/java/com/storehub/entity/Expense.java`; add the field to `ExpenseCreateRequest / ExpenseUpdateRequest` (with validation annotations as needed) and to `ExpenseResponse.fromEntity`; wire it through `ExpenseService.applyRequest`; if it affects the posted journal amount, update the line-building logic in `ExpenseService.post`. Affects: Accounting (if it changes what gets posted), GST (if it's tax-related — also touch `GstCalculationService / GstTransactionSyncService.syncExpense`), and the frontend `Expense` type (`frontend/src/types/expense.ts`), `ExpenseCreatePayload`, `ExpenseForm.tsx`, `ExpenseDetail.tsx`.
- **Change Day Closing's reconciliation logic:** the expected-cash source and the aggregation queries live in `DayClosingService.computeSummary` (each figure is a separate repository call — `SaleRepository`, `PurchaseRepository`, `ReceiptRepository`, `PaymentRepository`, `ExpenseRepository`, `CashTransactionRepository`, `CashBankBookService`); the difference-reason gate itself is the single `if (difference.signum() != 0 && ...)` check in `DayClosingService.close`. Never change `expectedCash` to be computed independently of `CashBankBookService.cashBook` — the class Javadoc explicitly states it must never drift from the accounting journal. Affects: Accounting Core (`CashBankBookService`) if the Cash Book's own logic changes; the frontend `DayClosingSummary` type and `DayClosingPage.tsx` if new fields are added to the response.
- **Change FY validation rules:** overlap/date-range/code-uniqueness checks live in `FinancialYearService.create`; the posting gate itself (`status == CLOSED` rejection) is in `resolveOpenForPosting`, called centrally from `AccountingService.buildAndSaveJournal / reversal` — changing that gate changes posting behavior for every voucher type in the whole application, not just this module. The startup auto-seed rule (`ensureCurrentFinancialYearExists`) and the Apr-Mar boundary calculation (`FinancialYearUtil.startOf/endOf/shortCode/label`) are separate concerns to touch only if the fiscal-year convention itself changes.
- **Add a new AuditAction type:** add the constant to `backend/src/main/java/com/storehub/entity/AuditAction.java`; add the matching string literal to the frontend `AuditAction` union type in `frontend/src/types/auditLog.ts` (note: the current frontend union is missing `DAY_CLOSE`, which the backend enum already has — this is an existing frontend/backend drift worth fixing alongside any

new addition); if the new action needs a distinct badge color, update `actionVariant` in `frontend/src/pages/admin/AuditTrail.tsx`.

- **Add a new `CashTransactionType`:** add the constant to `backend/src/main/java/com/storehub/entity/CashTransactionType.java`; extend the `if/else` in `CashTransactionService.post` that currently branches only on `CASH_IN/CASH_OUT` to build the correct journal lines for the new type; update the frontend `CashTransactionType` union in `frontend/src/types/cashTransaction.ts` and the type `<Select>` options in `CashTransactionForm.tsx/CashTransactions.tsx`. Affects: Accounting Core (new journal line combination for the new type).

Alerts, Global Search & Dashboard Module

1. Overview

This module covers three cross-cutting UI features that read data owned by other modules rather than owning data of their own:

- **Alert Center** (/alerts) — a read-only, computed-on-request list of business warnings (low/out-of-stock, overdue receivables/payables, financial year closing soon, accounting health-check failures). Backed by a single endpoint, `GET /api/alerts`.
- **Global Search** (the search box in the top layout bar) — an as-you-type, multi-entity search across Sales, Purchases, Products, Suppliers, Credit Notes, Debit Notes, Sales Orders, Purchase Orders, Receipts and Payments. Backed by a single endpoint, `GET /api/search`.
- **Dashboard** (/dashboard) — the landing page after login. It has **no backend module of its own**. It is entirely frontend-composed: it fans out to `productApi`, `customerApi`, `supplierApi`, `saleApi`, `purchaseApi`, `accountingReportApi.dashboard()`, `gstReportApi.liability()` and `alertApi.list()` — seven other modules' existing endpoints plus the Alert Center endpoint — and assembles the results client-side. This is verified directly from `Dashboard.tsx`'s imports and `Promise.allSettled` calls; there is no `DashboardController` or `DashboardService` anywhere in `backend/src/main/java/com/storehub`.

Both `AlertService` and `GlobalSearchService` are explicitly documented in their own Javadoc as deliberately holding **no new business logic and no persisted rows**: `AlertService` re-derives everything live each call from `InventoryRepository`, `OutstandingBillService`, `FinancialYearService` and `AccountingHealthCheckService`; `GlobalSearchService` re-uses each module's existing paginated `search(...)` repository query (the same one backing that module's list page) so results can never drift from the source page.

2. Frontend

Files and Components

File	Role
<code>frontend/src/pages/AlertsCenter.tsx</code>	Full-page Alert Center: loads all alerts, shows Critical/Warning/Total stat tiles, lists every alert, click-to-navigate via <code>path</code> . No dismiss/mark-read — alerts are not stored, so there is nothing to dismiss.
<code>frontend/src/pages/Dashboard.tsx</code>	Landing dashboard: stat cards, two 7-day bar charts (Sales/Purchases), an "Alerts" summary card (top 5), Recent Sales list, Low Stock Products list.
<code>frontend/src/components/layout/GlobalSearch.tsx</code>	Debounce (300ms) as-you-type search input rendered in the top bar; dropdown of grouped results; click-outside-to-close; Escape/clear button.
<code>frontend/src/api/alertApi.ts</code>	<code>alertApi.list()</code> → <code>GET /alerts</code> .
<code>frontend/src/api/globalSearchApi.ts</code>	<code>globalSearchApi.search(q)</code> → <code>GET /search?q=...</code> .
<code>frontend/src/types/alert.ts</code>	<code>AlertSeverity</code> , <code>AlertItem</code> (<code>severity</code> , <code>category</code> , <code>message</code> , <code>path?</code>).
<code>frontend/src/types/globalSearch.ts</code>	<code>GlobalSearchResultItem</code> , <code>GlobalSearchResponse</code> .

File	Role
frontend/src/utils/ stockStatus.ts	getStockStatus() — pure client-side helper Dashboard uses to compute its own "Low Stock" list from the already-fetched product list (independent of AlertService's server-side inventory-count logic).
frontend/src/components/ layout/page-title.ts	Maps /alerts route to page title "Alert Center".
frontend/src/components/ layout/Topbar.tsx	Notification bell / dropdown item that navigates to /alerts .

Frontend Flow

Alert Center (list):

1. On mount, AlertsCenter calls load() → alertApi.list() → GET /api/alerts .
2. Response AlertItem[] is split client-side into critical (severity === 'CRITICAL') and warning (severity === 'WARNING') for the two stat tiles; a third tile shows the total count.
3. Each row is clickable only if a.path is set; click calls navigate(a.path) .
4. A manual "Refresh" button re-runs load() . There is **no dismiss/acknowledge action** — the page's own description text states alerts are "not stored or dismissible."
5. Empty state ("No active alerts") is shown when the array is empty; loading state uses Skeleton placeholders.

Global Search (as-you-type):

1. User types into the Input in GlobalSearch.tsx ; query state updates on every keystroke and open is set true on focus.
2. A useEffect on query clears any pending debounce timer, and if the trimmed query is < 2 characters, clears results / loading and does nothing further (matches the backend's own q.length() < 2 short-circuit).
3. Otherwise loading is set true and a 300ms setTimeout fires globalSearchApi.search(trimmed) → GET /api/search?q=... .
4. On success, results is replaced with res.data.results ; on any error, results is silently reset to [] (no toast).
5. The dropdown renders each result's title , optional subtitle , and a category pill; clicking a row closes the dropdown, clears the query, and calls navigate(item.path) .
6. A click outside the container (mousedown listener on document) closes the dropdown without clearing the query.

Dashboard load:

1. On mount (and whenever canSeePurchases changes — derived from user.role === 'ADMIN' || 'STORE_MANAGER'), Dashboard fires a first Promise.allSettled batch of 6 calls: productApi.list({size: 200}) , customerApi.list() , supplierApi.list({size:1,status:'ACTIVE'}) , saleApi.list({fromDate:today,toDate:today,size:200}) (today's sales), saleApi.list({size:5}) (recent sales), saleApi.list({fromDate:from,toDate:today,size:200}) (7-day chart, from = daysAgoISO(6)).
2. If canSeePurchases , a second Promise.allSettled batch of 4 calls runs: purchaseApi.list({size:1}) (count), purchaseApi.list({fromDate,toDate,size:200}) (7-day chart), accountingReportApi.dashboard() , gstReportApi.liability(currentReturnPeriod()) . Non-manager/admin roles never issue these 4 calls (all four values stay null).
3. Independently, alertApi.list() is called (.catch(() => null)) so a failure never blocks the rest of the page).
4. Each settled promise is checked individually (status === 'fulfilled') and used to populate its own piece of state — a failure in one call does not block the others from rendering (this is the purpose of Promise.allSettled over Promise.all).
5. Cards rendered and their data source:

- **Total Products** — `totalProducts` from `productApi.list ( totalElements )`.
- **Today's Sales** — sum of today's non-cancelled sales' `totalAmount`, computed client-side from `saleApi.list({fromDate:today,toDate:today})`.
- **Total Purchases** — `purchaseApi.list({size:1}).totalElements`; shown as — and marked `restricted` (lock icon + tooltip "Only visible to Admin and Store Manager") for other roles.
- **Total Customers** — `customerApi.list().length`.
- **Low Stock Products** — computed client-side: `products.filter(p => getStockStatus(p) !== 'IN_STOCK')`, sorted ascending by `stockQuantity`. This is a **separate** client-side low-stock computation from `AlertService`'s server-side `countLowStock / countOutOfStock` — see §12 for the discrepancy this can cause.
- **Total Suppliers** — `supplierApi.list({status:'ACTIVE'}).totalElements`.
- **Cash + Bank Balance / Receivable / Payable** (manager/admin only) — `accountingReportApi.dashboard()` → `cashBalance + bankBalance, receivableBalance, payableBalance`.
- **GST Liability (this period)** — `gstReportApi.liability(currentReturnPeriod())` → `netTotal`.
- **Sales Overview / Purchase Overview bar charts** — client-aggregated daily totals (`buildDailyTotals`) over the last 7 days from the two chart `saleApi.list / purchaseApi.list` calls (non-cancelled rows only), rendered with `Recharts BarChart`.
- **Alerts card** — first 5 items of `alertApi.list()`'s response, with a "View all" link to `/alerts` and a critical-count badge.
- **Recent Sales** — last 5 sales from `saleApi.list({size:5})`, click-through to the sale's bill/challan detail page.
- **Low Stock Products list** — first 6 of the client-computed `lowStockProducts`, click-through to `/inventory`.

6. Financial stat cards and the Purchase Overview chart are conditionally rendered only when `canSeePurchases` is true (a **frontend-only** role gate — see §7/§8, there is no corresponding backend restriction on these underlying endpoints beyond their own module's permission checks).

3. API Calls

Alerts

Function	Method	URL	Request shape	Response shape
<code>alertApi.list()</code>	GET	<code>/api/alerts</code>	none	<code>AlertItem[]</code> — <code>{severity: 'CRITICAL' 'WARNING', category: string, message: string, path?: string}[]</code>

Global Search

Function	Method	URL	Request shape	Response shape
<code>globalSearchApi.search(q)</code>	GET	<code>/api/search?q={q}</code>	query param <code>q: string</code>	<code>GlobalSearchResponse</code> — <code>{query: string, results: {category, id, title, subtitle?, path}[]}</code>

Dashboard — every other module's endpoint it calls (`Dashboard.tsx` has no API file of its own; these are imported directly from other modules' API files):

Import (from <code>Dashboard.tsx</code>)	Function called	Method	URL
<code>productApi (@/api/productApi)</code>	<code>productApi.list({ size: 200 })</code>	GET	<code>/api/products</code>

Import (from Dashboard.tsx)	Function called	Method	URL
customerApi (@/api/customerApi)	customerApi.list()	GET	/api/customers
supplierApi (@/api/supplierApi)	supplierApi.list({ size: 1, status: 'ACTIVE' })	GET	/api/suppliers
saleApi (@/api/saleApi)	saleApi.list({ fromDate, toDate, size: 200 }) (today), saleApi.list({ size: 5 }) (recent), saleApi.list({ fromDate, toDate, size: 200 }) (7-day chart) — 3 separate calls	GET	/api/sales
purchaseApi (@/api/purchaseApi)	purchaseApi.list({ size: 1 }), purchaseApi.list({ fromDate, toDate, size: 200 }) — 2 separate calls, manager/admin only	GET	/api/purchases
accountingReportApi (@/api/accountingApi)	accountingReportApi.dashboard() — manager/admin only	GET	/api/accounting/reports/dashboard
gstReportApi (@/api/gstReportApi)	gstReportApi.liability(currentReturnPeriod()) — manager/admin only	GET	/api/gst-reports/liability?returnPeriod=...
alertApi (@/api/alertApi)	alertApi.list()	GET	/api/alerts

Confirmed: **Dashboard has no dedicated backend controller/service**. A repo-wide case-insensitive search for Dashboard under backend/src/main/java/com/storehub/controller and ../service finds nothing named DashboardController/DashboardService; the only backend hit for "Dashboard" is dto/AccountingDashboardResponse.java, which belongs to the Accounting module and is served by /api/accounting/reports/dashboard (accounting-owned, just reused by the frontend Dashboard page).

4. Backend

Controllers (every endpoint)

AlertController (backend/src/main/java/com/storehub/controller/AlertController.java)

- GET /api/alerts → alertService.getAlerts() → List<AlertItem>. No path/query params. No @PreAuthorize annotation on the controller or method.

GlobalSearchController (backend/src/main/java/com/storehub/controller/GlobalSearchController.java)

- GET /api/search?q={q} → globalSearchService.search(q) → GlobalSearchResponse. q defaults to "" (@RequestParam(defaultValue = "")). No @PreAuthorize annotation.

No dedicated Dashboard controller exists (see §3).

DTOs (fields)

- **AlertItem** (dto/AlertItem.java) — severity: String, category: String, message: String, path: String. Plain @Builder/@Getter/@AllArgsConstructor, NO @NoArgsConstructor, no validation annotations (it's an outbound-only DTO).
- **GlobalSearchResponse** (dto/GlobalSearchResponse.java) — query: String, results: List<GlobalSearchResultItem>.
- **GlobalSearchResultItem** (dto/GlobalSearchResultItem.java) — category: String, id: Long, title: String, subtitle: String, path: String. Javadoc: "One row in the Global Search dropdown — enough to render and to navigate straight to the source document."

- No Search*.java DTOs beyond the two above (glob of dto/Alert*.java, dto/GlobalSearch*.java, dto/Search*.java returns exactly AlertItem.java, GlobalSearchResponse.java, GlobalSearchResultItem.java).

Services

AlertService (backend/src/main/java/com/storehub/service/AlertService.java) — @Transactional(readOnly = true) . Exact trigger conditions, in the order they're evaluated in getAlerts() :

1. **Out of stock** — inventoryRepository.countOutOfStock(null) (aggregated across all stores, storeId = null). Repository query: Inventory i WHERE i.currentStock <= 0. If count > 0 → severity=CRITICAL, category=Inventory, message="{n} product(s) are OUT OF STOCK", path=/inventory?stockStatus=OUT_OF_STOCK.
2. **Low stock** — inventoryRepository.countLowStock(null). Query: Inventory i JOIN i.product p WHERE i.currentStock > 0 AND i.currentStock <= COALESCE(p.minStockLevel, 0). If count > 0 → severity=WARNING, category=Inventory, message="{n} product(s) are LOW STOCK", path=/inventory?stockStatus=LOW_STOCK.
3. **Reorder point reached** — inventoryRepository.countReorderCandidates(null). Query: Inventory i JOIN i.product p WHERE p.reorderLevel IS NOT NULL AND i.currentStock <= p.reorderLevel AND p.status = ACTIVE — explicitly a **distinct threshold from low-stock's minStockLevel** (code comment: "distinct from LOW_STOCK's minStockLevel threshold"). If count > 0 → severity=WARNING, category=Inventory, message="{n} product(s) have reached their reorder point", path=/inventory.
4. **Receivables overdue** — via addOutstandingAlert(alerts, outstandingBillService.customerOutstanding(LocalDate.now()), "Receivable", "/accounting/reports/receivable"). Sums every ageing bucket in the report **except** the "0-30 Days" bucket (bucket labels are {"0-30 Days", "31-60 Days", "61-90 Days", "91-180 Days", "180+ Days"} from OutstandingBillService) — i.e. **overdue = any bill older than 30 days**. If the summed overdue count > 0 → severity=WARNING, category=Receivable, message="{overdueCount} receivable bill(s) overdue (30+ days), totalling {overdueAmount}", path=/accounting/reports/receivable.
5. **Payables overdue** — same logic via outstandingBillService.supplierOutstanding(LocalDate.now()), category="Payable", path=/accounting/reports/payable.
6. **Financial year nearing close** — financialYearService.getCurrent(); if currentFy.getEndDate() is set, daysToClose = ChronoUnit.DAYS.between(today, endDate). If 0 <= daysToClose <= FY_NEARING_CLOSE_DAYS (constant = 30) → severity=WARNING, category="Financial Year", message="Financial year {name} closes in {daysToClose} day(s)", path=/admin/financial-years.
Caveat: FinancialYearService.getCurrent() throws BadRequestException("No current financial year is set") (via .orElseThrow(...)) rather than returning null when no FY is marked current — so the currentFy != null guard in AlertService is effectively dead code, and if no financial year is currently marked current in the database, GET /api/alerts (and therefore both the Alert Center page and the Dashboard's Alerts card) will fail with HTTP 400 instead of degrading gracefully. See §10/§12.
7. **Accounting health check findings** — iterates accountingHealthCheckService.runHealthCheck().getFindings(); any finding with status == WARNING or status == ERROR becomes an alert (ERROR → CRITICAL, WARNING → WARNING), category="Accounting Health Check", message="{checkName}: {message}", path=/accounting/reports/health-check. The underlying checks (in AccountingHealthCheckService) are: checkJournalBalance, checkDuplicatePosting, checkSourcePosting, checkOrphanJournals, checkLedgerMismatch, checkFinancialYearCoverage, checkNotePosting.

Alerts are **one per condition, not one per row** (e.g. one "12 products are LOW STOCK" alert, not 12 separate alerts) — stated explicitly in the class Javadoc to keep the list "meaningful rather than noisy."

GlobalSearchService (backend/src/main/java/com/storehub/service/GlobalSearchService.java) —
@Transactional(readOnly = true) , LIMIT_PER_CATEGORY = 5 .

- Query is trimmed; if `< 2` characters, returns an empty result list immediately
(`GlobalSearchResponse.builder().query(q).results(List.of())`).
- Store scoping (`resolveSearchStoreIdOrNull()`): if the caller has all-stores access
(`storeAccessService.hasAllStoresAccess(user)` , e.g. ADMIN) → `null` (no filter, searches every store). Otherwise, uses `user.getCurrentStore()` if set; if not set and the user has exactly one accessible store, uses that store's id; if not set and the user has multiple accessible stores, uses a sentinel id `-1L` (`NO_ACCESSIBLE_STORE_SENTINEL`) that no real store can match, so store-sensitive categories quietly return zero results rather than erroring the whole search.
- Queries each of the following repositories' own existing `search(...)` method, in this exact order, each capped to top 5 results (`PageRequest.of(0, 5)`), and appends results to one combined flat list (no cross-category ranking/sorting — order is purely the fixed category iteration order below, and within a category whatever order that module's own `search` query returns):
 1. `saleRepository.search(q, null, null, null, null, null, storeId, top)` → category "Sale", title = `invoiceNumber`, subtitle = customer name or "Walk-in", path = `/sales/kacchi/{id}` (if `SALE_CHALLAN`) or `/sales/bills/{id}`.
 2. `purchaseRepository.search(...)` → category "Purchase", title = `purchaseNumber`, subtitle = supplier name, path = `/purchases/kacchi/{id}` or `/purchases/bills/{id}`.
 3. `productRepository.search(q, null, null, top)` → category "Product", title = `name`, subtitle = "SKU: {sku}", path = `/products/{id}/edit`.
 4. `supplierRepository.search(q, null, top)` → category "Supplier", title = `name`, subtitle = mobile, path = `/suppliers/{id}/edit`.
 5. `creditNoteRepository.search(q, null, null, null, storeId, top)` → category "Credit Note", title = `voucherNumber`, subtitle = customer name, path = `/sales/credit-notes/{id}`.
 6. `debitNoteRepository.search(...)` → category "Debit Note", title = `voucherNumber`, subtitle = supplier name, path = `/purchases/debit-notes/{id}`.
 7. `salesOrderRepository.search(...)` → category "Sales Order", title = `orderNumber`, subtitle = customer name, path = `/sales/orders/{id}`.
 8. `purchaseOrderRepository.search(...)` → category "Purchase Order", title = `orderNumber`, subtitle = supplier name, path = `/purchases/orders/{id}`.
 9. `receiptRepository.search(...)` → category "Receipt", title = `receiptNumber`, subtitle = customer name, path = `/sales/receipts/{id}`.
 10. `paymentRepository.search(...)` → category "Payment", title = `paymentNumber`, subtitle = supplier name, path = `/purchases/payments/{id}`.
- No overall result cap across categories: worst case = 10 categories × 5 = 50 results in one response.
- Combination/ranking: results are **not** re-ranked or scored across categories — they are simply concatenated in the fixed order above, each category internally ordered however that module's own repository `search` query orders it (typically by relevance/recency of that module, not documented here since it belongs to those modules).

Repository (which repositories these services query into)

Neither `AlertService` nor `GlobalSearchService` owns any repository or entity of its own. They reuse repositories belonging to other modules:

- `InventoryRepository` (Inventory module) — `countOutOfStock`, `countLowStock`, `countReorderCandidates`.

- `OutstandingBillService` (Accounting module) — wraps its own repository queries for `customerOutstanding / supplierOutstanding` ageing buckets.
- `FinancialYearService` (Admin/Financial Year module) — `getCurrent()`.
- `AccountingHealthCheckService` (Accounting module) — `runHealthCheck()`.
- `SaleRepository`, `PurchaseRepository`, `ProductRepository`, `SupplierRepository`, `CreditNoteRepository`, `DebitNoteRepository`, `SalesOrderRepository`, `PurchaseOrderRepository`, `ReceiptRepository`, `PaymentRepository` — each module's own repository, called via its existing `search(...)` method.
- `StoreAccessService` — used only for store-scoping the search (`hasAllStoresAccess`, `getAccessibleStoreIds`).

5. Database

Tables

This module has **no tables of its own**. It only reads from other modules' tables, via the repositories/ services listed in §4:

- `inventory` (joined with `product`) — for stock counts.
- Whatever tables back `OutstandingBillService`'s ageing report (sales/purchases + payment/receipt allocations).
- `financial_year`.
- Whatever tables `AccountingHealthCheckService` inspects (journal entries, accounts, notes).
- `sale`, `purchase`, `product`, `supplier`, `credit_note`, `debit_note`, `sales_order`, `purchase_order`, `receipt`, `payment` — for Global Search.
- Dashboard reads from `product`, `customer`, `supplier`, `sale`, `purchase`, plus whatever Accounting's `/reports/dashboard` and GST's `/gst-reports/liability` endpoints read (control-account balances / GST return computations, owned by the Accounting/GST modules, not this one).

There is no `alert`, `notification`, or `search_index / search_log` table anywhere in the schema for this module — confirmed by `AlertService`'s own Javadoc ("no stored 'notification' rows") and by the absence of any repository field on either service besides the reused ones above.

Relationships

N/A at this module's own level — no owned entities means no owned foreign keys or relationships. All relationships (Inventory→Product, Sale→Customer, Purchase→Supplier, etc.) belong to and are documented by their owning modules.

6. Validation

- `GlobalSearchController / GlobalSearchService`: no `@Valid` / DTO validation annotations. The only "validation" is the service's own `q.length() < 2` short-circuit (returns an empty result set rather than a 400).
- `AlertController / AlertService`: no request body/params at all, so nothing to validate.
- Neither module defines its own DTO field-level `@NotNull` / `@Pattern` / etc. constraints (both DTOs above are outbound-only response shapes). No `fieldErrors` map is ever produced by either endpoint — this module's CLAUDE.md-referenced field-error UX pattern (`Register.tsx / UserFormModal.tsx`) does not apply here since neither endpoint accepts a validated request body.

7. Authentication / Authorization

- Both `GET /api/alerts` and `GET /api/search` fall under `SecurityConfig`'s catch-all `.anyRequest().authenticated()` rule (only `/api/auth/**` and `/actuator/health` are `permitAll()`). So both require a valid JWT, but **neither controller method carries a `@PreAuthorize` annotation** — any authenticated user of any role can call them, there is no permission-specific gate at the controller level.
- `GlobalSearchService` performs its own **data-level** authorization for store-sensitive categories only (Sale, Sales Order, Purchase, Purchase Order, Receipt, Payment, Credit Note, Debit Note all take a `storeId` filter) via `resolveSearchStoreIdOrNull()` — see §4. Product and Supplier are not store-filtered by Global Search's own query calls (both pass `null` for store where a store param exists, matching those repositories' own `search` signatures — Product/Supplier are not store-scoped entities in this codebase).
- `AlertService`'s inventory counts are explicitly aggregated across **all** stores (`storeId = null` is hardcoded, with a code comment: "Alert Center itself isn't store-scoped yet") — i.e. a store-scoped user still sees company-wide inventory alert counts, not just their own store's.
- Frontend routing: `/dashboard` and `/alerts` are both wrapped only in `<ProtectedRoute>` (requires user to be non-null, i.e. logged in) — not in `<AdminRoute>` or `<ManagerRoute>`, so every authenticated role can reach both pages. `Dashboard.tsx` gates only specific cards client-side via `canSeePurchases = user?.role === 'ADMIN' || user?.role === 'STORE_MANAGER'`; this is a **frontend-only presentation gate** — the underlying `purchaseApi`, `accountingReportApi`, `gstReportApi` calls it skips for other roles are still reachable directly by any authenticated user unless those endpoints enforce their own `@PreAuthorize` (governed by those modules, not this one).

8. Permissions (DASHBOARD_VIEW etc, from RolePermissions.java)

- `Permission.DASHBOARD_VIEW` exists and is granted to **every** role defined in `RolePermissions.build()`: ADMIN (via `EnumSet.allOf()`), STORE_MANAGER (inherits ADMIN's set minus the admin-tier exclusions, which don't include DASHBOARD_VIEW), ACCOUNTANT, SALES_USER, PURCHASE_USER, INVENTORY_USER, and STAFF all explicitly list `Permission.DASHBOARD_VIEW` in their `EnumSet.of(...)`.
- However, `DASHBOARD_VIEW` is **not actually enforced anywhere** in this module — grep of `AlertController`, `GlobalSearchController`, and the frontend route table shows no `@PreAuthorize("hasAuthority('PERM_DASHBOARD_VIEW')")` and no permission-gated `<Route>` for `/dashboard` or `/alerts`. It exists in the permission map (satisfying the "every role can see the dashboard" intent) but is not wired to a concrete check in this module's code.
- No `ALERT_VIEW` or `SEARCH_VIEW` (or similar) permission constant exists in `entity/Permission.java` — confirmed by grep; Alerts and Global Search have no permission of their own at all, only implicit "any authenticated user" access.

9. Transaction Handling

- `AlertService.getAlerts()` — `@Transactional(readOnly = true)`.
- `GlobalSearchService.search(query)` — `@Transactional(readOnly = true)`.
- Both are pure read/aggregation operations; neither writes to the database, so there is no commit/rollback behavior of interest — a single read-only transaction wraps each call for consistency across the several repository/service reads made within it.

- Dashboard, being frontend-composed, has no server-side transaction of its own; each of its underlying endpoint calls is transactional within its own owning module.

10. Error Handling

- Uncaught exceptions from either controller fall through to `GlobalExceptionHandler`'s generic `@ExceptionHandler(Exception.class)` → HTTP 500 with message "An unexpected error occurred", unless a more specific handled exception type is thrown.
- The one concrete failure path identified: if no `FinancialYear` is currently marked `current` in the database, `FinancialYearService.getCurrent()` throws `BadRequestException("No current financial year is set")`, which `GlobalExceptionHandler.handleBadRequest` maps to **HTTP 400** with that exact message. Since `AlertService.getAlerts()` calls this unconditionally as step 6 of its alert list, **the entire `/api/alerts` call fails with a 400 in that state** — not just the "financial year nearing close" alert being skipped. This would also break the Dashboard's Alerts card (its `alertApi.list().catch(() => null)` swallows the error and just shows zero alerts, no error surfaced) and break the standalone `AlertsCenter` page (shows a `toast.error('Failed to load alerts')` via `parseApiError`).
- `GlobalSearch.tsx`'s search call has no error toast at all — a failed `/api/search` request silently resets `results` to `[]`, so a broken search backend looks identical to "no results found" from the user's perspective.
- `AlertsCenter.tsx` surfaces load failures via `toast.error(parseApiError(err, 'Failed to load alerts').message)`.

11. Audit Flow

NOT FOUND IN CURRENT CODEBASE. Neither `AlertController/AlertService` nor `GlobalSearchController/GlobalSearchService` writes any audit-log entry, and searching a search performed or an alert viewed is not recorded anywhere (there is no `AuditService/AuditLog` call in either service file). The codebase does have an Admin "Audit Trail" module (`AuditController/AUDIT_VIEW` permission referenced in `RolePermissions.java`) but it is unrelated to and not invoked by this module.

12. Important Side Effects

- **Alert Center failing entirely on a missing current financial year** (see §10) — a single missing/unset "current" FY row breaks the whole alert feed, not just that one alert.
- **Two independent, inconsistent low-stock computations:** `AlertService.countLowStock/countOutOfStock` (server-side, compares `currentStock` against `minStockLevel`, aggregated across all stores) vs. Dashboard's own client-side `getStockStatus()` (compares `product.stockQuantity` against `product.minStockLevel`, computed only over the first 200 products fetched by `productApi.list({size:200})`, i.e. can be **incomplete** if the catalog has more than 200 products, and uses a different field name/source than Inventory's per-store `currentStock`). The Alert Center's "Low Stock" count and the Dashboard's "Low Stock Products" tile are **not guaranteed to agree** — different data sources, different field, different scope (store-aggregated inventory rows vs. one flat product page).
- Global Search silently returns fewer results than exist (capped at 5 per category, 10 categories max) with no "show more"/pagination — a user searching a common term only ever sees the first 5 matches per document type.
- Global Search for a store-scoped user with multiple accessible stores and no store currently selected silently returns **zero** results for every store-sensitive category (Sale, Purchase, Credit Note, Debit Note, Sales Order, Purchase Order, Receipt, Payment) via the `-1L` sentinel — this fails

silently/gracefully rather than erroring, but could confuse a user who doesn't realize their search coverage is incomplete.

- Dashboard's `Promise.allSettled` design means a failure in any one underlying call (e.g. GST liability failing) degrades that one card only (shows 0/-) without blocking the rest of the page — by design, not a bug, but worth noting as it means partial/incomplete dashboard data can be shown without any visible error to the user.

13. Dependencies on Other Modules

- **Inventory** — `InventoryRepository.countOutOfStock/countLowStock/countReorderCandidates` (Alert Center); Dashboard also independently reads `Product.stockQuantity/minStockLevel` via `productApi`.
- **Accounting** — `OutstandingBillService` (customer/supplier ageing → receivable/payable overdue alerts), `AccountingHealthCheckService` (health-check findings → alerts), `accountingReportApi.dashboard()` (cash/bank/receivable/payable balances on Dashboard).
- **Admin / Financial Year** — `FinancialYearService.getCurrent()` (FY-nearing-close alert).
- **Sales** — `SaleRepository.search` (Global Search "Sale" category); `saleApi.list` (Dashboard's today's-sales total, recent sales list, 7-day chart).
- **Purchases** — `PurchaseRepository.search` (Global Search "Purchase" category); `purchaseApi.list` (Dashboard's purchase count and 7-day chart, manager/admin only).
- **Products** — `ProductRepository.search` (Global Search "Product" category); `productApi.list` (Dashboard's product count and low-stock list).
- **Suppliers** — `SupplierRepository.search` (Global Search "Supplier" category); `supplierApi.list` (Dashboard's supplier count).
- **Customers** — `customerApi.list()` (Dashboard's customer count only; Global Search has no "Customer" category — see §15).
- **Credit/Debit Notes** — `CreditNoteRepository.search / DebitNoteRepository.search` (Global Search categories).
- **Sales/Purchase Orders** — `SalesOrderRepository.search / PurchaseOrderRepository.search` (Global Search categories).
- **Receipts/Payments** — `ReceiptRepository.search / PaymentRepository.search` (Global Search categories).
- **GST** — `gstReportApi.liability()` (Dashboard's GST Liability card, manager/admin only).
- **Multi-Store / StoreAccessService** — store-scoping of Global Search results.

14. Key Operation Flows

Loading the Alert Center

1. UI: `AlertsCenter` mounts → `load()`.
2. Function: `alertApi.list()`.
3. API: `GET /api/alerts` (no params).
4. Controller: `AlertController.getAlerts()`.
5. Service: `AlertService.getAlerts()` — runs the 7 checks in §4 in order (inventory out-of-stock/low-stock/reorder counts → receivable/payable overdue → FY nearing close → accounting health check findings), building a `List<AlertItem>`.
6. Repository/other services: `InventoryRepository`, `OutstandingBillService`, `FinancialYearService`, `AccountingHealthCheckService`.
7. Database: `inventory / product` tables, ageing report source tables, `financial_year`, accounting journal/ledger tables.

8. Response: `List<AlertItem>` JSON (`severity`, `category`, `message`, `path`).
9. UI: split into critical/warning counts for stat tiles, full list rendered with click-to-navigate via `path` .

Performing a Global Search

1. UI: user types in `GlobalSearch.tsx` 's input; `query` state updates.
2. Function: debounced (300ms) `globalSearchApi.search(trimmed)` , only once `trimmed.length >= 2` .
3. API: `GET /api/search?q={trimmed}` .
4. Controller: `GlobalSearchController.search(q)` .
5. Service: `GlobalSearchService.search(q)` — trims/length-checks `q` , resolves `storeId` via `resolveSearchStoreIdOrNull()` , then queries the 10 repositories in §4's fixed order, each capped to 5 results, building one flat `List<GlobalSearchResultItem>` .
6. Repository: `SaleRepository` , `PurchaseRepository` , `ProductRepository` , `SupplierRepository` , `CreditNoteRepository` , `DebitNoteRepository` , `SalesOrderRepository` , `PurchaseOrderRepository` , `ReceiptRepository` , `PaymentRepository` — each module's own existing `search(...)` .
7. Database: `sale` , `purchase` , `product` , `supplier` , `credit_note` , `debit_note` , `sales_order` , `purchase_order` , `receipt` , `payment` tables.
8. Response: `GlobalSearchResponse` (`query` , `results[]`).
9. UI: results shown grouped by category pill in the dropdown; click navigates to `item.path` .

Loading the Dashboard (fan-out to multiple backend endpoints — Dashboard has no backend of its own)

1. UI: Dashboard mounts (or `canSeePurchases` changes) → `load()` .
2. Functions/APIs fired in parallel via `Promise.allSettled` — batch 1 (all roles): `productApi.list({size: 200})` → `GET /api/products` ; `customerApi.list()` → `GET /api/customers` ; `supplierApi.list({size: 1, status: ACTIVE})` → `GET /api/suppliers` ; `saleApi.list(...)` ×3 → `GET /api/sales` (today's totals, recent 5, 7-day chart).
3. Batch 2 (manager/admin only, `canSeePurchases`): `purchaseApi.list(...)` ×2 → `GET /api/purchases` ; `accountingReportApi.dashboard()` → `GET /api/accounting/reports/dashboard` ; `gstReportApi.liability(period)` → `GET /api/gst-reports/liability` .
4. Independently: `alertApi.list().catch(() => null)` → `GET /api/alerts` .
5. Controllers: `ProductController` , `CustomerController` , `SupplierController` , `SaleController` , `PurchaseController` , `AccountingReportController` (or equivalent), `GstReportController` , `AlertController` — each belonging to its own module, each with its own service/repository/DB chain (not this module's).
6. Response: each endpoint's own response shape (paged lists, `AccountingDashboardResponse` , `GstLiabilityResponse` , `AlertItem[]`).
7. UI: each `Promise.allSettled` result is checked individually (`status === 'fulfilled'`) and mapped into its own piece of Dashboard state, feeding the stat cards, two bar charts, Alerts card, Recent Sales list, and Low Stock Products list described in §2.

15. Manual Changes

- **Adding a new alert type/condition:** add a new block inside `AlertService.getAlerts()` (backend/ `src/main/java/com/storehub/service/AlertService.java`), following the existing pattern — inject/ reuse the owning module's existing repository or service (do not write new persistence for the alert itself, per the class's own documented design), compute a count/condition, and conditionally `alerts.add(AlertItem.builder().severity(...).category(...).message(...).path(...).build())` . Choose `severity` as "CRITICAL" or "WARNING" (the only two values the frontend `AlertSeverity` type and both `AlertsCenter.tsx` / `Dashboard.tsx` UIs recognize — any other string would render with no badge styling since the frontend only special-cases those two). Set `path` to the frontend route the alert should deep-link to. No frontend change is needed unless a new severity value is introduced.

- **Adding a new entity to Global Search's scope:** (1) confirm/add a `search(...)` repository method on that entity's existing repository, matching the pattern in `SaleRepository / ProductRepository` /etc. (paged, filterable); (2) inject that repository into `GlobalSearchService` via constructor (`@RequiredArgsConstructor` — just add the field); (3) add a new loop block in `GlobalSearchService.search()` following the existing 10 blocks, choosing a new `category` string, mapping `title/subtitle/path` appropriately, and deciding whether the entity is store-sensitive (pass `storeId` if so, matching the `Sale/Purchase/etc.` pattern; omit if the entity isn't store-scoped, matching `Product/Supplier`). No DTO change needed (`GlobalSearchResultItem` is already generic). No frontend change needed — `GlobalSearch.tsx` renders any `category` string generically.
- **Adding a new Dashboard card:** edit `frontend/src/pages/Dashboard.tsx` only. If the data is already available from an existing owning-module endpoint (preferred, matches this module's established "compose, don't own" pattern), add that API call to one of the two `Promise.allSettled` batches (or a new independent call, per the `alertApi` pattern, if it should never block the rest of the page) and a new entry in the `stats/financialStats` array or a new `Card`. If no existing summary endpoint covers the needed data, a new endpoint would need to be added to the **owning module** (e.g. a new field on `AccountingDashboardResponse` if it's an accounting number, or a new endpoint on that module's controller) — this module's own `AlertController / GlobalSearchController` would only be the right place if the new card is itself an alert-like or search-like aggregate.
- **Modules affected by such changes:** any module whose entity/repository is touched by (1) or (2) above — e.g. adding a Sales Order alert would touch the Sales module's read path (via its repository) plus `AlertService`; adding a Customer category to Global Search would touch the Customer module's repository (it would need to gain a `search(...)` method, since `CustomerRepository` is not currently one of the 10 repositories queried) plus `GlobalSearchService`. Adding a new Dashboard card touches whichever module owns the underlying data plus `Dashboard.tsx` — never this module's own backend, since it has none.
